

MODERN WORKPLACE AUTOMATION

M365Ops

Guida alla configurazione iniziale — Intune, Entra ID, Exchange Online e motore AI con integrazione MCP (Lokka), pensata per essere estesa nel tempo.

Tenant di riferimento in questa guida: contoso.onmicrosoft.com (profilo "contoso-test")
Versione documento: 1.34 — 19 Agosto 2026

Indice

1. Cos'è M365Ops e principio di fondo
2. Architettura
3. Prerequisiti software
4. App Registration in Entra ID
5. Certificato ed Exchange.ManageAsApp
6. Ruoli RBAC su Exchange Online
7. Registrazione del tenant nel modulo
8. Motore AI (Claude / Azure OpenAI)
9. Connettori MCP (Lokka) — configurazione per-tenant
10. Autenticazione delegata (utente + MFA) — tenant senza App Registration
11. Le 50+ cmdlet Exchange Online e come le usa l'AI
12. Avvio dell'applicazione
13. Uso quotidiano
14. Sicurezza — principi non negoziabili
15. Stato attuale del progetto
16. Roadmap ed estensioni future
17. Script personalizzati (Scripts\Custom)
18. Portabilità — spostare l'ambiente su un altro PC
19. Log delle azioni — storico e debug
20. Stato e utilizzo del motore AI
21. Stato e reset MFA — dettagli tecnici
22. Report AI-orchestrati con grafici — dettagli tecnici
23. Memoria della conversazione — dettagli tecnici
24. Appendice A — struttura dei file
25. Appendice B — tutto avviene dalla GUI, non serve PowerShell

1. Cos'è M365Ops e principio di fondo

M365Ops è un modulo PowerShell + interfaccia web di chat per amministrare un tenant Microsoft 365 Modern Workplace (Intune, Entra ID, Exchange Online) combinando:

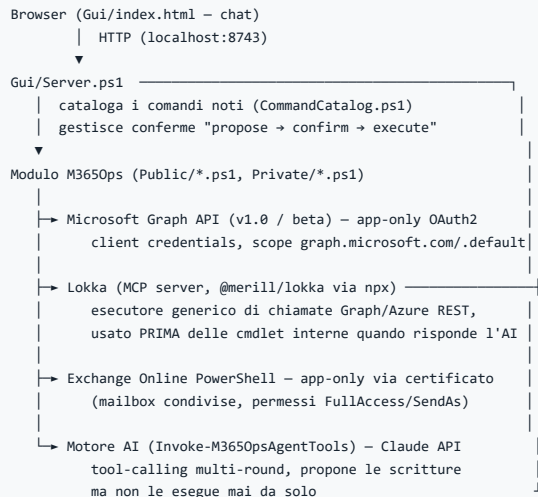
- un **catalogo comandi locale**, deterministico e a costo zero, per le richieste note e ripetitive;
- un **motore AI con accesso reale ai dati** (tool-calling), usato quando la richiesta non rientra nel catalogo;
- un principio di conferma umana obbligatoria per ogni azione di scrittura.

Principio di fondo. L'interpretazione dei dati (spiegare perché un device non è conforme, raggruppare pattern, decidere una causa probabile) vive nel motore AI, *non* in regole scritte a mano nel codice. Le cmdlet del modulo restituiscono solo dati grezzi o eseguono azioni — mai interpretazione cablata. Vedi `Get-M365OpsCompliancePatterns.ps1` come esempio di riferimento.

Perché non un semplice bot a regole fisse

Un sistema a sole regole hardcoded (if/else su ogni possibile domanda) non scala e non generalizza: funziona solo per le domande previste in anticipo. M365Ops usa le regole solo per instradare le richieste ovvie a costo zero (senza chiamare l'AI), e delega al modello AI — con accesso reale ai dati tramite tool — tutto ciò che richiede ragionamento o non è stato previsto esplicitamente.

2. Architettura



Flusso di una richiesta in chat

1. Il messaggio arriva a `Handle-ChatMessage` in `Gui/Server.ps1`.
2. Se c'è un'azione in sospeso in attesa di conferma, la gestisce prima di tutto.
3. Altrimenti prova il **catalogo comandi** (regex/trigger locali, zero costo AI).
4. Se nessun comando corrisponde, passa la richiesta a `Invoke-M365OpsAgentTools`, che chiama Claude con gli strumenti Lokka disponibili al primo giro, e le cmdlet interne come fallback nei giri successivi.
5. Se l'AI propone una scrittura (`propose_graph_write`), l'azione resta in sospeso finché l'utente non conferma esplicitamente in chat.

3. Prerequisiti software

Componente	Uso	Verifica	Installazione automatica
PowerShell 7+	esecuzione del modulo e del server web	<code>pwsh -v</code>	si — <code>M3650ps.bat</code> lo controlla PRIMA di avviare qualsiasi altra cosa e lo installa da solo via winget se manca, con una finestra grafica dedicata (sezione 3.1)
Node.js + npx	avvio del server MCP Lokka (<code>npx -y @merill/lokka</code>)	<code>npx -v</code>	si — stessa finestra grafica di PowerShell 7, oppure pulsante "Installa automaticamente" nel banner delle Impostazioni (sezione 3.1)
Modulo <code>ImportExcel</code>	export report in formato .xlsx	<code>Get-Module -ListAvailable ImportExcel</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata
Modulo <code>ExchangeOnlineManagement</code> — ⚠ VERSIONE FISSATA A 3.4.0	connessione a Exchange Online (app-only e delegata)	<code>Get-Module -ListAvailable ExchangeOnlineManagement</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata. Fissata in COPPIA con MicrosoftTeams 6.5.0 (vedi sotto): verificata dal vivo priva del conflitto di sezione 6.6 quando Exchange si connette PRIMA di Teams — M365Ops installa/ripara sempre la 3.4.0 e rimuove attivamente qualunque versione ≥3.10.0 trovata sul disco ad ogni connessione. Non aggiornarlo mai manualmente (<code>Update-Module</code> o simili) senza prima ripetere la verifica di sezione 6.6 su una copia separata — vedi il riquadro sotto.
Microsoft Edge	generazione PDF (report ed export, incl. questo documento) via <code>--headless --print-to-pdf</code>	presente di default su Windows 11	si — stessa finestra grafica di PowerShell 7/Node.js, utile solo su PC senza Edge preinstallato (raro su Windows 11)
Modulo <code>IntuneWin32App</code>	solo per <code>New-M3650psWin32App</code> (packaging app Win32)	<code>Get-Module -ListAvailable IntuneWin32App</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata
Modulo <code>MicrosoftTeams</code> — ⚠ VERSIONE FISSATA A 6.5.0	connessione a Microsoft Teams	<code>Get-Module -ListAvailable MicrosoftTeams</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata. Fissata in COPPIA con ExchangeOnlineManagement 3.4.0 (vedi sopra e sezione 6.6) — non aggiornarlo mai manualmente per lo stesso motivo.
Modulo <code>PnP.PowerShell</code>	connessione a SharePoint Online	<code>Get-Module -ListAvailable PnP.PowerShell</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata
Modulo <code>PdfLexer</code>	estrazione testo da PDF caricati nella Knowledge Base	<code>Get-Module -ListAvailable PdfLexer</code>	si — stessa finestra grafica (sezione 3.1), fallback al primo uso se saltata

Microsoft Graph NON compare in questa tabella: questo progetto non usa mai l'SDK ufficiale `Microsoft.Graph` (pesante, molte dipendenze) - ogni chiamata Graph passa da `Invoke-M3650psGraphRequest`, un wrapper REST puro (`Invoke-RestMethod` + bearer token), quindi non serve nessun modulo dedicato.

Tutti i moduli PowerShell mancanti vengono installati automaticamente al primo uso dalle rispettive cmdlet (`Install-Module ... -Scope CurrentUser`), non serve preinstallarli manualmente.

⚠ Procedura per cambiare la coppia di versioni fissate ExchangeOnlineManagement 3.4.0 / MicrosoftTeams 6.5.0 (leggere prima di toccare uno dei due pin): non aggiornare mai un pin per assunzione o perché "e' uscita una versione nuova" - i due moduli vanno sempre riverificati INSIEME, mai uno alla volta (sezione 6.6: nessuna coppia di versioni condivide la stessa build della libreria di autenticazione condivisa, quindi ogni combinazione va testata dal vivo, non dedotta dai numeri di versione). Prima di cambiare: (1) installare le nuove versioni IN AFFIANCAMENTO a quelle attuali su una macchina di test, mai sostituendole; (2) in un processo PowerShell isolato, importare `ExchangeOnlineManagement` (nuova versione) e chiamare `Connect-ExchangeOnline / Connect-IPPSession` con parametri anche finti (`-AppId / -CertificateThumbprint` e, separatamente, `-AccessToken` finto per il login delegato - l'eventuale conflitto di assembly si manifesta PRIMA di qualunque validazione di rete, quindi non serve un tenant vero); (3) SOLO se nessun errore "Could not load file or assembly"/"The given assembly name was invalid" compare in nessuno dei passaggi, importare `MicrosoftTeams` (nuova versione) DOPO Exchange e chiamare `Connect-MicrosoftTeams`, verificando che sia anch'esso pulito; (4) verificare anche che `-CertificateThumbprint / -Certificate, -AccessToken e -CertificateThumbprint` su `Connect-IPPSession` esistano ancora come parametri nella nuova versione Exchange (in 3.4.0, `Connect-IPPSession` non supporta ancora nessun flusso delegato device-code - verificare se questo e' cambiato); (5) solo a questo punto aggiornare i numeri di versione ovunque compaiano nel codice (`Assert-M3650psExoSafeVersion.ps1`, `Assert-M3650psTeamsSafeVersion.ps1`, `Install-M3650psPrerequisites.ps1`, `Show-M3650psPrereqInstaller.ps1` - e la soglia `3.10.0` nella logica di disinstallazione attiva di Exchange, se la nuova versione sicura non e' piu' quella immediatamente successiva). Richiesto esplicitamente dall'utente il 23/08/2026 e ribadito il 25/08/2026, dopo aver trovato lui stesso l'errore nella diagnosi due volte di seguito ("non e' che dipende da versioni diverse dei moduli?"; poi "trova una soluzione con moduli piu' vecchi"): un processo scritto una volta, da ripetere identico ogni volta, invece di rifidarsi della memoria.

3.1 Controllo e installazione automatica (17/08/2026, unificata in una GUI il 22/08/2026)

Su richiesta esplicita, l'app non si limita a segnalare i prerequisiti mancanti: li installa da sola dove tecnicamente possibile, senza che l'operatore debba far nulla — un doppio click su `M3650ps.bat` è sufficiente perché l'ambiente sia pronto all'uso.

Percorso comune (tutto già presente): nessuna finestra. `M3650ps.bat` controlla PowerShell 7, winget e Node.js con un semplice `where` prima di fare qualunque altra cosa - se ci sono già tutti (il caso normale su un PC dove l'app è già stata usata), avvia il server direttamente, esattamente come prima, senza alcun ritardo o finestra aggiuntiva.

Qualcosa manca: `Show-M3650psPrereqInstaller.ps1` . Se manca anche solo uno tra PowerShell 7, winget o Node.js, `M3650ps.bat` apre una finestra grafica dedicata (Windows Forms, eseguita in Windows PowerShell 5.1 - sempre presente, a differenza di PowerShell 7 stesso) che mostra passo per passo cosa sta verificando/installando: PowerShell 7, Node.js, Microsoft Edge, i tre moduli PowerShell (ExchangeOnlineManagement, ImportExcel, IntuneWin32App - dal 22/08/2026, vedi nota sotto), e - se manca anche winget stesso - il bootstrap di winget prima di tutto il resto (vedi sotto).

Ogni passaggio lungo (bootstrap di winget, singole installazioni winget o di modulo) mostra l'output del processo in tempo quasi reale riga per riga MENTRE gira (non solo alla fine) e un contatore di secondi nella barra di stato, così la finestra non sembra mai bloccata anche durante un passaggio silenzioso di 1-2 minuti - bug reale segnalato dal vivo il 22/08/2026 subito dopo il primo utilizzo pratico ("sembra stuck... le persone si rompono"): la prima versione leggeva l'output del processo figlio solo DOPO che finiva (bloccante), quindi durante un bootstrap lungo la finestra restava letteralmente immobile senza nessun segno di vita. La finestra si chiude da sola se tutto va a buon fine, altrimenti resta aperta con il log completo e un pulsante "Chiudi". Prima del 22/08/2026 questi tre passaggi erano divisi fra un file `.bat` di solo testo (PowerShell 7) e un secondo controllo invisibile minuti dopo (Node.js/Edge, dentro `Launch-M365Ops.ps1`) - ora sono un unico flusso visibile, richiesto esplicitamente dall'utente ("una gui che segua l'installazione dei moduli. aggrega dentro l'installazione dei prereq anche nodejs visto che cmq serve").

Bootstrap di winget stesso, se assente — raro (PC molto minimali/ datati o Windows Server), ma possibile proprio sul tipo di "PC pulito" dove serve di più. Un file a parte, `Bootstrap-Winget.ps1`, usa il modulo ufficiale Microsoft `Microsoft.WinGet.Client` (`Repair-WinGetPackageManager`) per installarlo da zero, con TLS 1.2 e il provider NuGet forzati esplicitamente prima di `Install-Module` (altrimenti, su un PC davvero vergine, il tentativo può fallire silenziosamente per un protocollo TLS troppo vecchio negoziato di default, o restare in attesa di un prompt del provider NuGet mai mostrato). Se anche questo bootstrap fallisce, la finestra mostra il link diretto per l'installazione manuale di PowerShell 7 invece di restare bloccata senza spiegazioni.

Il banner "Ambiente non completo" (tab Impostazioni) con il pulsante **"Installa automaticamente"** resta disponibile come ripetizione manuale dentro l'app stessa, per il caso in cui qualcosa cambi DOPO il primo avvio (es. Node.js disinstallato per sbaglio) - chiama `Install-M365OpsPrerequisites` (`POST /api/install-prerequisites`), che gira anche come seconda passata idempotente ad ogni avvio di `Launch-M365Ops.ps1`.

Al primo avvio, `Launch-M365Ops.ps1` crea anche un collegamento **M365Ops.lnk** sul Desktop con un'icona dedicata (`Assets\M365Ops.ico`) - un file `.bat` non può avere un'icona propria (limite di Windows: mostra sempre quella generica di `cmd.exe`), un collegamento sì. Operazione idempotente e non bloccante: se il collegamento esiste già e punta al `M365Ops.bat` corretto non viene ricreato, e un suo eventuale fallimento (es. cartella Desktop non scrivibile) è solo cosmetico, non impedisce mai l'avvio del server.

4. App Registration in Entra ID

Ogni tenant gestito da M365Ops richiede una propria App Registration, con autenticazione **app-only (client credentials)** — mai login interattivo ripetuto.

4.1 Creazione e client secret

- 1. portal.azure.com → Microsoft Entra ID → **App registrations** → New registration
- 2. Certificates & secrets → New client secret → copia subito il valore (non sarà più visibile)
- 3. Il valore del secret **non va mai salvato in un file**: si imposta una sola volta come variabile d'ambiente Windows (vedi sezione 7)

Alternativa: solo certificato, nessun secret (aggiunto il 21/08/2026) - richiesto esplicitamente dall'utente: se hai già creato il certificato per Exchange Online (sezione 6.1) puoi usare **lo stesso certificato** anche per Microsoft Graph, senza creare nessun client secret. M365Ops firma una richiesta di token con la chiave privata del certificato (JWT "client assertion", lo stesso meccanismo standard che `Connect-ExchangeOnline -CertificateThumbprint` usa per Exchange) invece di inviare un secret in chiaro - un solo materiale da gestire e rinnovare per tutta l'app, non due. Per usarlo: lascia vuoti sia "Nome variabile d'ambiente per il secret" sia "Valore client secret" nel tab Tenant, e compila solo il campo "Thumbprint certificato" (sezione 6.1). Se invece il secret È configurato e risolvibile, resta la scelta di default (nessun cambiamento per le installazioni esistenti) - il certificato viene usato solo quando il secret non è disponibile. Verificato dal vivo: token Graph ottenuto e usato con successo per una chiamata reale (`/organization`) con solo il certificato, nessun secret configurato.

4.2 Permessi Microsoft Graph (Application, non Delegated)

Permesso	Motivo
DeviceManagementManagedDevices.Read.All	lettura device Intune, stato compliance
DeviceManagementConfiguration.ReadWrite.All	creazione/lettura profili di configurazione
DeviceManagementApps.ReadWrite.All	creazione/assegnazione app Intune (Win32, Store), criteri di protezione app MAM (Android/iOS)
DeviceManagementRBAC.ReadWrite.All	Scope Tag, ruoli RBAC personalizzati, assegnazioni di ruolo - verificato dal vivo il 19/08/2026: senza questo permesso Graph risponde 403 su <code>/deviceManagement/roleScopeTags</code> e su <code>/deviceManagement/roleDefinitions</code>
DeviceManagementServiceConfig.ReadWrite.All	restrizioni/limiti di iscrizione dispositivi, Autopilot - verificato dal vivo il 19/08/2026: senza questo permesso Graph risponde 403 su <code>/deviceManagement/deviceEnrollmentConfigurations</code>
DeviceManagementScripts.ReadWrite.All	script di distribuzione Windows/macOS, Proactive Remediations - verificato dal vivo il 19/08/2026: senza questo permesso Graph risponde 403 su <code>/deviceManagement/deviceManagementScripts</code> e su <code>/deviceManagement/deviceHealthScripts</code>
Group.ReadWrite.All	creazione gruppi e gestione membri
User.ReadWrite.All	ricerca utenti per nome/UPN E creazione/modifica utenti - <code>User.Read.All</code> da solo NON basta per creare un utente (403 "Insufficient privileges", incontrato dal vivo il 17/08/2026 su un tenant che aveva solo il permesso di lettura)
Mail.Send	invio report generati (CSV/XLSX/PDF) via email
Team.ReadBasic.All	elenco Team del tenant (report Teams/canali) - non ancora aggiunto su tutti i tenant, verificare se un 403 compare su <code>/teams</code>
Channel.ReadBasic.All	elenco canali di un Team - richiesto insieme a <code>Team.ReadBasic.All</code> , uno dei due da solo non basta per un report completo
SecurityAlert.Read.All	lettura alert di sicurezza (Defender) - solo se si usa <code>graph_api_call</code> su <code>/security/alerts_v2</code> (sezione 17.7)
SecurityIncident.Read.All	lettura incidenti di sicurezza - solo per <code>/security/incidents</code> (sezione 17.7)
ThreatHunting.Read.All	query KQL avanzate (equivalente di "Explorer") - solo per lo strumento <code>security_hunting_query</code> (sezione 17.7); richiede ANCHE la licenza Microsoft Defender for Endpoint Plan 2, non solo il permesso - senza quella licenza Graph risponde comunque 403 anche a permesso concesso
Organization.Read.All	report licenze (<code>/organization</code> , <code>/subscribedSkus</code>) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, semplicemente mai documentato prima in questa tabella (era già in sezione 10.8 per la modalità Delegata, dimenticato qui)
AuditLog.Read.All	log di sign-in - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
UserAuthenticationMethod.ReadWrite.All	stato MFA e reset (sezione 21) - scope dedicato, <i>non</i> coperto da <code>User.ReadWrite.All</code> - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Sites.Read.All	lettura siti SharePoint via <code>graph_api_call</code> diretto (percorso Graph nativo, DIVERSO dal permesso <code>Sites.FullControl.All</code> sotto l'API separata "SharePoint" richiesto per <code>sharepoint_query</code> /PnP - sezione 4.3) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Application.Read.All	visibilità sulle App Registration del tenant (sola lettura) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Policy.Read.All	lettura policy del tenant (es. Conditional Access - sola lettura) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
RoleManagement.Read.Directory	lettura assegnazioni di ruolo Entra ID (sola lettura) - verificato dal vivo il 19/08/2026: già presente e funzionante su vnsys-test, stesso motivo sopra
Reports.Read.All	report di utilizzo Microsoft 365 (usage report) - anche i report di utilizzo Copilot (<code>Get-M365OpsCopilotUsageReport</code> / <code>Get-M365OpsCopilotUsageSummary</code> , sezione 24) - verificato dal vivo il 19/08/2026 (e riconfermato il 22/08/2026 per Copilot): NON ancora

	concesso su vnsys-test, Graph risponde <code>{ "code": "UnknownError", "message": "{ \"error\": { \"code\": \"S2SUnauthorized\", \"message\": \"Invalid permission.\" } }\" }</code>
<code>IdentityRiskyUser.Read.All</code>	segnali di rischio Identity Protection (<code>/identityProtection/riskyUsers</code>) - verificato dal vivo il 19/08/2026: NON ancora concesso su vnsys-test, Graph risponde 403 "required scopes are missing in the token"; permesso corretto confermato su Microsoft Learn il 19/08/2026 - non <code>SecurityEvents.Read.All</code> (nome usato per errore in una prima stesura di questa tabella e ancora presente, da correggere, in sezione 10.8)

Questa tabella e' stata verificata e completata dal vivo il 19/08/2026 dopo che l'utente ha chiesto esplicitamente conferma che l'elenco fosse completo per TUTTO il modulo, non solo per Intune: 9 permessi risultavano gia' documentati in sezione 10.8 (modalita' Delegata) per le stesse identiche funzionalita', ma mai aggiunti qui - un vero gap di documentazione, non funzionale (le funzionalita' corrispondenti funzionavano gia' su vnsys-test, semplicemente senza che il permesso necessario fosse elencato). Due di questi risultano invece davvero NON concessi su vnsys-test (`Reports.Read.All`, `IdentityRiskyUser.Read.All`), verificato con l'errore Graph reale.

Dopo aver aggiunto i permessi, cliccare sempre **"Grant admin consent"** — senza consenso admin le chiamate app-only falliscono con errore di autorizzazione anche se il permesso risulta "aggiunto" nella lista.

Un 403 "Insufficient privileges to complete the operation" / "Authorization_RequestDenied" su una scrittura significa quasi sempre questo: manca il permesso Application specifico per QUELL'azione (es. serve `User.ReadWrite.All` per creare un utente, non basta `User.Read.All`) oppure il permesso c'è ma manca ancora il "Grant admin consent" - mai un bug del codice, sempre da controllare prima nel portale.

4.3 Permessi SharePoint (API separata da Microsoft Graph)

Aggiunto il 17/08/2026 per il supporto SharePoint/OneDrive (sezione 17.9): a differenza di **tutti** gli altri permessi di questa pagina, che sono sotto l'API **Microsoft Graph**, SharePoint online (via PnP.PowerShell, usato per l'elenco siti/permessi/OneDrive) richiede un consenso separato sotto un'API diversa, chiamata semplicemente **"SharePoint"** nel selettore "APIs my organization uses" del portale.

- 1. App registration → API permissions → Add a permission → tab "APIs my organization uses" → cerca **"SharePoint"** (non "Microsoft Graph")
- 2. Application permissions → `Sites.FullControl.All` → Add permissions
- 3. **Grant admin consent** (esattamente come per Microsoft Graph, sezione 4.2)

Verificato dal vivo il 17/08/2026: senza questo permesso, l'autenticazione col certificato riesce comunque (nessun errore di login) ma **ogni** chiamata reale (anche solo leggere un sito) fallisce con "Unexpected response from the server... status code is Unauthorized" - un errore diverso nella forma dal classico 403 di Graph, ma stesso significato: permesso mancante, non un bug.

Questo permesso riguarda SOLO i tenant App-only. Su un tenant Delegato non esiste un'App Registration a cui concedere `Sites.FullControl.All` - la lettura SharePoint dipende invece dal **ruolo Entra ID dell'utente** con cui si fa login (tipicamente serve *SharePoint Administrator* o *Global Administrator*). Verificato dal vivo il 17/08/2026 su Fabrikam-Prod (Delegato): il login a codice dispositivo può completarsi correttamente (autenticazione riuscita) e le chiamate fallire comunque con "Attempted to perform an unauthorized operation." - un problema di ruolo, distinto e successivo rispetto al login stesso. Se capita, verifica il ruolo dell'utente in Entra ID → Roles and administrators, non l'App Registration.

Il ruolo va assegnato PRIMA di fare login, non dopo. Un ruolo Entra ID concesso mentre una sessione SharePoint delegata è già attiva non si applica al token già emesso (i ruoli sono valutati al momento del login, non rivalutati in tempo reale) - se dopo aver assegnato il ruolo l'errore "Attempted to perform an unauthorized operation." persiste identico, la causa più probabile non è che il ruolo non sia stato assegnato correttamente, ma che serva un **nuovo login**: pulsante "Connetti / Test connessione SharePoint" nel tab **Tenant** (sezione "Stato connessioni", non nel tab MCP/Connettori - vedi sezione 10.7), non semplicemente ripetere la domanda in chat.

Bug reale trovato e corretto il 17/08/2026 grazie al test dal vivo su Fabrikam-Prod: il polling del login a codice dispositivo (SharePoint, Teams, Exchange e il login Graph generico - stesso schema in tutti e 4) usava `setInterval` , che pianifica il giro successivo a orario fisso *indipendentemente* da quanto impiega quello precedente a rispondere. Se lo scambio del token era più lento del solito, due poll potevano finire in volo insieme: il primo completava con successo e ripuliva lo stato lato server, il secondo (già partito) trovava lo stato appena rimosso e mostrava "nessun login in corso" *sopra* il messaggio di successo appena scritto - un login riuscito che sembrava fallito. Prova diretta nei log: la chiave di sessione veniva salvata correttamente, poi risultava assente 22 secondi dopo con due tentativi di lettura falliti nello stesso secondo. Corretto sostituendo `setInterval` con un `setTimeout` che pianifica il giro successivo SOLO dopo aver ricevuto risposta da quello corrente, rendendo la sovrapposizione impossibile per costruzione.

4.4 Permessi Teams (API separata, terza diversa da Graph e da SharePoint)

Aggiunto il 17/08/2026 per il supporto Teams (sezione 17.11): l'elenco Team/canali/membri funziona SUBITO con lo stesso certificato di Exchange, verificato dal vivo, nessun permesso aggiuntivo. I **criteri** (riunione/chiamata/messaggistica) e la **configurazione di accesso esterno/ospiti** invece richiedono una terza API separata, chiamata **"Skype and Teams Tenant Admin API"**.

- 1. App registration → API permissions → Add a permission → tab "APIs my organization uses" → cerca **"Skype and Teams Tenant Admin API"**
- 2. Application permissions → `application_access` → Add permissions
- 3. **Grant admin consent**

Non ancora verificato se basta il permesso da solo o se serve ANCHE assegnare al service principal dell'App Registration un ruolo Entra ID (es. "Teams Administrator") - la documentazione Microsoft per l'autenticazione app-only di questo modulo menziona entrambi in scenari diversi. Se dopo aver concesso il permesso le cmdlet di policy continuano a fallire con lo stesso errore, il prossimo passo da provare è assegnare quel ruolo al service principal (Entra ID → Enterprise applications → cerca l'App Registration → Roles and administrators).

Verificato dal vivo il 17/08/2026 senza questo permesso: `Get-Team` / `Get-TeamChannel` / `Get-TeamUser` funzionano comunque, ma `Get-CsTeamsMeetingPolicy` / `Get-CsTeamsCallingPolicy` / `Get-CsTeamsMessagingPolicy` e le cmdlet di configurazione ospiti falliscono con "Access Denied. Provide different credential or request access." - un terzo errore diverso da "Unauthorized" (SharePoint) e da 403 (Graph), ma stesso significato: permesso mancante.

4.5 Ruolo Purview per le retention policy (RBAC nel portale Compliance, non un permesso App Registration)

Aggiunto il 18/08/2026 per il supporto retention policy (sezione 17.12). A differenza di Graph/SharePoint/Teams sopra, qui non basta un permesso API sull'App Registration: Microsoft Purview usa un sistema di **ruoli RBAC separato**, assegnato direttamente nel portale compliance, non in Entra ID.

Correzione del 18/08/2026 rispetto alla prima stesura di questa sezione: un service principal (l'App Registration usata in modalita' AppOnly) NON puo' essere aggiunto DIRETTAMENTE come membro di un gruppo di ruoli Purview dal portale - a differenza di un utente Delegato, per cui l'assegnazione diretta funziona davvero cosi' come descritto sotto. Per un service principal serve un passaggio intermedio, verificato sulla documentazione reale (non a memoria) dopo che l'utente ha chiesto esplicitamente conferma di questa procedura.

Tenant Delegato (login utente, nessuna App Registration): assegnazione diretta, un solo passaggio.

1. Vai su compliance.microsoft.com → **Ruoli e ambiti** → **Autorizzazioni**
2. Cerca il gruppo di ruoli **"Compliance Administrator"** (o un ruolo piu' mirato tipo "Records Management"/"Retention Management" se disponibile nel tenant)
3. Aggiungi come membro l'utente delegato (lo stesso UPN gia' usato per Exchange)

Tenant AppOnly (service principal/certificato): un service principal non e' assegnabile direttamente a un gruppo di ruoli Purview - serve un gruppo di sicurezza Entra ID come tramite:

1. Crea un gruppo di sicurezza Entra ID con l'opzione **"Azure AD roles can be assigned to this group" = Yes** attiva alla creazione (va impostata alla creazione, non modificabile dopo su un gruppo gia' esistente)
2. Aggiungi il service principal dell'App Registration (AppOnly) come membro di quel gruppo
3. Da Security & Compliance PowerShell, assegna il GRUPPO (non il service principal) al ruolo: `Add-RoleGroupMember -Identity "Compliance Administrator" -Member "<nome del gruppo di sicurezza>"`

Limite aggiuntivo per AppOnly, dalla stessa fonte: anche dopo aver assegnato il ruolo, l'autenticazione app-only (certificato) copre solo i permessi Application - non sufficiente per alcuni cmdlet di ricerca compliance specifici (es. `Start-ComplianceSearch`, `New-ComplianceSearchAction`), che richiederebbero permessi Delegated anche su un tenant configurato AppOnly. Non ancora verificato se questa limitazione tocca anche `Get-RetentionCompliancePolicy` (lettura semplice, non una ricerca) - da confermare quando il ruolo sara' assegnato sul tenant di test.

Verificato dal vivo il 18/08/2026 SENZA questo ruolo: `Connect-IPSSession` autentica correttamente (nessun errore di connessione) ma espone SOLO i cmdlet coperti dai ruoli gia' assegnati - sul tenant di test, solo quelli di Quarantena (`Get-QuarantineMessage` e affini, gia' funzionanti). `Get-RetentionCompliancePolicy` risulta quindi "term not recognized", lo STESSO messaggio che comparirebbe per un nome di comando sbagliato - a differenza di SharePoint/Teams, che rispondono con un errore di permesso esplicito ("Unauthorized"/"Access Denied"). Se la lettura fallisce con questo messaggio, la causa piu' probabile e' il ruolo mancante, non un errore di battitura nel nome del cmdlet.

5. Certificato ed Exchange.ManageAsApp

Exchange Online PowerShell in modalità app-only **non supporta i client secret**: richiede autenticazione a certificato.

5.1 Generazione del certificato (una tantum, sul PC amministrativo)

Dal 27/08/2026 questo passo si può fare anche dalla GUI: tab Tenant → box "Certificato applicativo" → "Genera nuovo certificato" - genera lo stesso identico certificato con gli stessi parametri qui sotto, salva da solo il thumbprint sul profilo attivo, e offre il `.cer` pubblico da scaricare. Resta comunque un solo passaggio manuale (sezione 5.2 sotto): caricare quel file su Entra ID. I comandi seguenti restano validi per chi preferisce farlo a mano o da un altro PC.

```
$cert = New-SelfSignedCertificate -Subject "CN=M365Ops-Exchange" `
    -CertStoreLocation "Cert:\CurrentUser\My" -KeyExportPolicy Exportable `
    -KeySpec Signature -KeyLength 2048 -KeyAlgorithm RSA -HashAlgorithm SHA256

Export-Certificate -Cert $cert -FilePath "Config\M365Ops-Exchange.cer"
$cert.Thumbprint # salva questo valore: serve al passo 5.3
```

Il file `.cer` (chiave pubblica, nessun segreto) va caricato sul portale; la chiave privata resta nel certificate store di Windows di questo PC e non va mai copiata altrove.

5.2 Caricamento su Entra ID

1. App registration → Certificates & secrets → tab **Certificates** → Upload certificate
2. Seleziona `Config\M365Ops-Exchange.cer`

5.3 Permessi Exchange.ManageAsApp

Non è un permesso Microsoft Graph — è un'API separata, si trova in un punto diverso del portale.

1. API permissions → **Add a permission**
2. Tab **"APIs my organization uses"** (non "Microsoft APIs") → cerca **"Office 365 Exchange Online"**
3. **Application permissions** (non Delegated) → spunta **Exchange.ManageAsApp** → Add permissions
4. **Grant admin consent**

6. Ruoli RBAC su Exchange Online

Avere il permesso `Exchange.ManageAsApp` non basta: il service principal dell'app deve anche essere **registrato dentro Exchange Online** e ricevere un ruolo RBAC. Questo passo richiede una sessione interattiva come Global/Exchange Admin (non può essere automatizzato dall'app stessa, che gira app-only).

```
# una tantum, sul PC amministrativo
Install-Module ExchangeOnlineManagement -Scope CurrentUser
Connect-ExchangeOnline -UserPrincipalName tuo-admin@contoso.onmicrosoft.com
```

6.1 Registrare il service principal in Exchange Online

Serve l'**Object ID della Enterprise Application** (Entra ID → Enterprise applications → cerca il nome dell'app → Overview → Object ID) — è diverso dall'Object ID della App Registration.

```
New-ServicePrincipal -AppId "00000000-0000-0000-0000-000000000001" `
-ObjectId "<Object-ID-della-Enterprise-Application>" `
-DisplayName "M365Ops"
```

6.2 Assegnazione del ruolo — due opzioni

Opzione A — Full admin (tenant di test)

usata su contoso-test

```
Add-RoleGroupMember "Organization Management" -Member "M365Ops"
```

Il ruolo più ampio disponibile. Comodo in un tenant di test dove non serve calibrare i permessi, ma va evitato su un tenant cliente reale in produzione.

Opzione B — Permessi minimi (consigliata per tenant di produzione)

```
New-RoleGroup -Name "M365Ops-ReadOnly" `
-Roles "View-Only Recipients","View-Only Configuration" `
-Members "M365Ops"
```

Sufficiente per le funzionalità attuali (lettura mailbox condivise e permessi FullAccess/SendAs/ SendOnBehalf), che sono di sola lettura.

6.3 Verifica

```
Get-ServicePrincipal | Where-Object DisplayName -eq "M365Ops"
Get-RoleGroupMember "Organization Management" # o "M365Ops-ReadOnly"
Get-ManagementRoleAssignment -RoleAssignee "M365Ops" # conferma i ruoli effettivi assegnati
Disconnect-ExchangeOnline -Confirm:$false
```

Poi, dall'app, tab Manutenzione/MCP → pulsante "Test connessione Exchange".

Attenzione ai tempi di propagazione. Anche se `Get-RoleGroupMember` mostra subito il service principal come membro, l'assegnazione RBAC può impiegare **fino a 60 minuti** (tipicamente 15-30) prima di diventare effettiva per l'autenticazione app-only/certificato. In questa finestra la connessione fallisce con:

```
Error: The user "<tenant>\<AppId>" isn't assigned to any management roles.
```

Non è un errore di configurazione: se `Get-ManagementRoleAssignment -RoleAssignee` restituisce righe, basta attendere e riprovare — non serve rifare i passi precedenti.

6.4 Alcuni cmdlet restano "term not recognized" anche con Organization Management (scoperto dal vivo il 20/08/2026)

Aggiornamento importante (21/08/2026): questa indagine si e' rivelata in parte una pista sbagliata. Il caso `Receive-/SendConnector` descritto qui sotto NON era affatto un problema di RBAC - erano semplicemente i cmdlet SBAGLIATI, esclusivi di Exchange on-premises e mai esistiti in Exchange Online (vedi il riquadro dedicato piu' sotto e la sezione 17.15). Il caso `New-M365OpsAcceptedDomain` invece resta un genuino limite RBAC/modalita' di autenticazione, ora CONFERMATO (non piu' un'ipotesi): funziona in modalita' Delegated, non in App-only. I due cmdlet producevano lo stesso identico messaggio di errore testuale ("term not recognized") per due cause completamente diverse - la lezione utile qui e' che un messaggio di errore identico non garantisce una causa comune, verificare sempre ogni caso separatamente prima di generalizzare una diagnosi.

Bug reale riscontrato testando `New-M365OpsAcceptedDomain` e le cmdlet `Receive-/SendConnector`: il service principal era già membro del role group **"Organization Management"** (sezione 6.2, opzione A) - eppure questi cmdlet restituivano comunque `"term not recognized"`, come se il ruolo non fosse affatto assegnato.

Verificato sulla documentazione ufficiale Microsoft ([app-only-auth-powershell-v2](#), non a memoria): per l'autenticazione app-only/certificato esistono **due meccanismi RBAC distinti**, non uno solo:

- **Opzione 2 (quella usata finora in questa guida, sezione 6.2):** assegnare il service principal a un role group Exchange interno (`Add-RoleGroupMember "Organization Management"`).
- **Opzione 1:** assegnare un vero **ruolo directory Microsoft Entra ID** all'app (Exchange Administrator, Global Administrator, ecc.).

Microsoft dichiara esplicitamente: *"Global Administrator e Exchange Administrator forniscono i permessi necessari per QUALSIASI task in Exchange Online PowerShell... Security Administrator NON ha i permessi necessari per gli stessi task."* I due meccanismi si **sommano** (RBAC combina i permessi da tutte le fonti) - aggiungere il ruolo directory non toglie nulla di già configurato con il role group.

Passi per assegnare il ruolo directory Entra ID

1. Vai su entra.microsoft.com (o portal.azure.com) → cerca **"Ruoli e amministratori"** ("Roles and administrators").
2. Cerca e clicca sul **nome** del ruolo **"Exchange Administrator"** (non la casella di spunta).
3. Nella pagina "Assegnazioni" → **"Aggiungi assegnazioni"**.
4. Cerca e seleziona l'App Registration di M365Ops (nome dell'app in Entra ID - può differire dal `DisplayName` con cui è registrata dentro Exchange via `New-ServicePrincipal`, sezione 6.1).
5. Conferma, poi attendi la propagazione (stessa finestra fino a 60 minuti di sopra).

ancora irrisolto Assegnato sul tenant di test (app registrata come "intune-ai-interface", ruolo directory Exchange Administrator). Nota onesta su un errore di questa stessa guida: una versione precedente di questa sezione affermava "verificato oltre 12 ore dopo l'assegnazione" - un tempo trascorso MAI verificato in modo indipendente (nessun log/timestamp consultato per calcolarlo), solo assunto in modo sbagliato leggendo il contesto della conversazione. Corretto dopo una segnalazione esplicita dell'utente ("il ruolo è stato aggiunto circa un'ora fa non 12 ore"): riprovato a ~1 ora dall'assegnazione reale (quindi entro o appena oltre la finestra di 60 minuti documentata sopra, non ben oltre come scritto prima) - `New-AcceptedDomain` e `Get-ReceiveConnector` restituiscono ancora **"term not recognized"**. A quell'orizzonte temporale la sola propagazione resta un'ipotesi plausibile (non ancora esclusa con certezza come lo sarebbe stata a 12 ore reali) - i due controlli sotto restano comunque validi ed erano già stati fatti indipendentemente dal tempo trascorso:

- Verificato via Graph (`GET /servicePrincipals/{id}/` e `roleManagement/directory/roleAssignments`) che il ruolo directory "Exchange Administrator" risulta assegnato al service principal **giusto** (stesso `AppId` configurato in `Config\tenants.json` per questo tenant - esclusa l'ipotesi "ruolo assegnato all'app sbagliata").
- Verificato lato Exchange con `Get-ManagementRoleAssignment -RoleAssignee <NomeVisualizzatoInExchange>` (attenzione: il nome da usare qui è quello mostrato da `Get-ServicePrincipal`, es. "Exchange-for-ia" in questo test - NON il `DisplayName` dell'App Registration in Entra ID, i due possono differire) che il ruolo **"Remote and Accepted Domains"** (che dovrebbe coprire proprio `New-AcceptedDomain`) risulta effettivamente assegnato via il role group "Organization Management".

Quindi: sia il ruolo directory Entra ID sia il ruolo RBAC Exchange specifico risultano correttamente assegnati all'app giusta, eppure il cmdlet resta non esposto nella sessione app-only/certificato.

CONFERMATO dal vivo il 21/08/2026 (utente: "puoi fare test in modalita' delegata?"): login interattivo reale completato con `diego@vnsys.it` (membro di "Organization Management", verificato con `Get-ManagementRoleAssignment` - decine di ruoli, incluso "Remote and Accepted Domains") - `Get-AcceptedDomain` funziona immediatamente in modalita' **Delegated**, restituendo i 5 domini reali del tenant. L'ipotesi delle sessioni app-only/CBA limitate a un sottoinsieme curato di cmdlet, indipendente dall'RBAC assegnato, e' quindi confermata per `New-/Set-/Remove-/Get-AcceptedDomain` : questi cmdlet richiedono una sessione Delegated, non sono raggiungibili in App-only indipendentemente da quanti ruoli si assegnano al service principal. **Stato finale:** in App-only resta bloccato (limite della piattaforma, non un bug di questo progetto); in Delegated funziona. Chi ha bisogno di gestire Accepted Domain da M365Ops deve usare un profilo tenant in modalita' Delegated (sezione 10) invece di App-only.

Il caso Receive-/SendConnector era tutt'altro: un cmdlet sbagliato, non un limite RBAC. Verificato dal vivo lo stesso giorno: lo stesso "term not recognized" su `Get-ReceiveConnector` compare ANCHE in modalita' Delegated con `diego@vnsys.it` (membro completo di Organization Management) - se fosse stato un limite RBAC o di sessione app-only, un utente con quel livello di privilegio avrebbe dovuto riuscire. Causa reale, confermata su documentazione ufficiale Microsoft (non a memoria): `Get-/New-/Set-/Remove-ReceiveConnector` e `...SendConnector` sono cmdlet **esclusivi di Exchange on-premises**, non sono MAI esistenti in Exchange Online - "this cmdlet is available only in on-premises Exchange" ([Get-ReceiveConnector, Microsoft Learn](#)). Il vero equivalente Exchange Online e' `Get-/New-/Set-/Remove-InboundConnector` e `...OutboundConnector` - nomi diversi, non alias, con parametri diversi (es. `SenderDomains` invece di `Bindings` / `RemoteIPRanges` , che sono concetti legati a un server fisico che non esiste in cloud). Corretto sostituendo tutti gli 8 wrapper del modulo (`Get-/New-/Set-/Remove-M365OpsInboundConnector` e `...OutboundConnector` , sezione 17.15) - testato dal vivo con un ciclo completo create/verifica/modifica/verifica/rimuovi/verifica per entrambi, IN APP-ONLY (nessun limite di sessione per questi, a differenza di Accepted Domain sopra): funzionano perfettamente.

6.5 Conflitto assembly .NET passando da App-only a Delegato nello stesso processo

Scoperto dal vivo il 21/08/2026: connettendosi a Exchange Online come Delegato (login interattivo) DOPO che lo stesso processo server aveva già usato una connessione App-only (certificato), la connessione fallisce con:

```
Errore: Token ottenuto ma la connessione a Exchange e' fallita: Could not load type
'Microsoft.IdentityModel.Telemetry.TelemetryClient' from assembly 'Microsoft.IdentityModel.Tokens,
Version=8.2.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35'.
```

Causa: conflitto di VERSIONE di una DLL condivisa, non un bug di questo progetto. I due percorsi di autenticazione (certificato vs interattivo/MFA) di [ExchangeOnlineManagement](#) caricano versioni diverse di [Microsoft.IdentityModel.Tokens](#) (una dipendenza condivisa con MSAL) - .NET non può avere due versioni della stessa DLL caricate insieme nello stesso processo. Una volta che una versione è stata caricata (dalla PRIMA connessione, di qualunque tipo), ogni tentativo successivo che ne richiederebbe una diversa fallisce con un errore di tipo/assembly poco chiaro come questo.

Unico rimedio reale: un processo NUOVO. Riavviare il server libera completamente il problema (nessuna DLL ancora caricata) - verificato dal vivo: lo stesso login Delegato che falliva ha funzionato subito dopo un riavvio pulito del server, PRIMA di qualunque altra connessione Exchange nello stesso nuovo processo.

Guardrail aggiunto lo stesso giorno (richiesto esplicitamente dall'utente: "quando c'è un'applicazione attiva e clicco su attiva la modalità delegata, abbi cura di disconnettere tutto quanto connesso prima"): [Connect-M365Ops](#) ora disconnette esplicitamente Teams ([Disconnect-M365OpsTeams](#), nuovo) e SharePoint ([Disconnect-M365OpsSharePoint](#), nuovo) ad ogni cambio di profilo tenant, oltre a Exchange e Lokka che già lo facevano.

Onestà tecnica su questo guardrail: chiudere una SESSIONE (Disconnect-ExchangeOnline ecc.) non annulla una DLL già caricata nel processo - il guardrail riduce il rischio di stato di sessione del tenant precedente rimasto attivo per sbaglio (comunque corretto di per sé), ma NON è una soluzione completa al conflitto di assembly sopra. Se il problema si ripresenta passando da un tenant all'altro con AuthMode diversa, il riavvio del server resta l'unico rimedio certo.

6.6 Conflitto DIVERSO: MicrosoftTeams e ExchangeOnlineManagement - RISOLTO DEFINITIVAMENTE con isolamento reattivo dei processi (26/08/2026, storia della diagnosi precedente lasciata sotto invariata)

Soluzione definitiva (v0.9.59, 26/08/2026), sostituisce il pin di versione sotto come rimedio primario: i pin di versione 3.4.0/6.5.0 documentati qui sotto restano installati per default (nessuna ragione per usare qualcos'altro), ma non sono più l'unico argine al conflitto - se scatta comunque (PC con versioni residue, o una futura combinazione mai testata), [Connect-M365OpsExchange](#) / [Connect-M365OpsTeams](#) / [Connect-M365OpsCompliance](#) ora tentano automaticamente un **isolamento reattivo**: la connessione che avrebbe generato il conflitto (Exchange o Teams, quale dei due arriva secondo) viene spostata in un sottoprocesso `pwh.exe` SEPARATO (`Private\Workers\M365OpsIsolatedWorker.ps1`), che importa SOLO quel modulo - i due moduli non condividono più mai lo stesso processo .NET, quindi il conflitto non può semplicemente più verificarsi. Da quel momento, ogni cmdlet del modulo isolato (Get-Mailbox, Get-DistributionGroup, Get-CASMailbox ecc. - verificato dal vivo un elenco di 835 cmdlet proxyati automaticamente sul tenant di test, non una lista scritta a mano) viene intercettato da una funzione proxy generata dinamicamente e inoltrato al sottoprocesso via JSON-RPC su stdio (stesso protocollo già usato per Lokka/CLI Microsoft 365), **senza che l'utente debba riavviare il server** - il messaggio "riavvia il server" di questa sezione resta solo per i casi non coperti (vedi limiti sotto). Attivazione sempre e solo REATTIVA, mai preventiva: un utente che usa un solo modulo per tutta la sessione non paga mai il costo di un secondo processo.

Limiti noti di questa prima versione, verificati dal vivo sul tenant "vnsys-test", non ipotizzati:

- **Solo App-only per ora.** Un tenant Delegato continua a vedere il classico messaggio "riavvia il server" - il token Exchange ottenuto via device-code non è oggi salvato in nessuna cache riutilizzabile dal sottoprocesso isolato; estenderlo richiede un login dedicato nel worker, non ancora costruito.
- **Cmdlet Purview "classici" (es. [Get-RetentionCompliancePolicy](#)) possono risultare assenti anche quando [Connect-IPPSession](#) non lancia nessun errore** - scoperto dal vivo durante la verifica di questa stessa funzionalità, e confermato con un test MINIMALE, del tutto fuori dall'architettura di isolamento (solo `Import-Module` + `Connect-ExchangeOnline` + `Connect-IPPSession` puri): su questo tenant/ questa versione del modulo, [Connect-IPPSession](#) riporta successo ma [Get-IPSession](#) subito dopo non mostra nessuna sessione remota attiva - la sessione di implicit remoting "classica" da cui derivano quei nomi di cmdlet non viene proprio stabilita. Non è un bug di questo progetto né dell'isolamento (riprodotto identico SENZA isolamento): è coerente con quanto già documentato più sotto in questa stessa sezione, che [ExchangeOnlineManagement 3.2.0+](#) ha spostato Security & Compliance sulle REST API - i vecchi nomi cmdlet legati al protocollo di remoting dismesso semplicemente non vengono più iniettati allo stesso modo. I cmdlet mailbox (Get-Mailbox, Get-CASMailbox, Get-DistributionGroup ecc., che NON dipendono da questa sessione) restano invece pienamente funzionanti - verificato con chiamate ripetute (12 invocazioni consecutive, incluse chiamate interlacciate a più cmdlet diversi) senza nessun errore.
- **Connessione al worker isolato più lenta della connessione diretta** (tipicamente 1-2 minuti la prima volta per tenant/modulo: avvio del processo + `Connect-ExchangeOnline` + `Connect-IPPSession` + attesa di stabilizzazione dell'elenco comandi) - accettabile perché succede una sola volta per tenant per la vita del processo server, non ad ogni comando.
- **CORRETTO IL 26/08/2026 (v0.9.60), bug reale trovato dal vivo durante un bug-hunt dedicato di 3 ore:** il meccanismo sopra scattava SOLO quando il conflitto si manifestava durante la CONNESSIONE stessa (dentro il blocco catch di `Connect-M365OpsExchange/Teams/Compliance`) - ma Teams ed Exchange possono connettersi ENTRAMBI con successo nello stesso processo (nessun errore in quel momento) e il conflitto manifestarsi solo DOPO, su una scrittura specifica: riprodotto dal vivo con `Set-Team`, fallito con `"Method not found: 'Void Microsoft.Identity.Client.ITokenCache.Deserialize(Byte[])'"` - una TERZA variante del messaggio .NET per lo stesso conflitto (le prime due, "Could not load file or assembly" e "The given assembly name is invalid", sono fallimenti di CARICAMENTO assembly; questa è un `MissingMethodException` - l'assembly incompatibile è già caricato, solo quel metodo specifico non esiste più in quella versione). `Get-M365OpsModuleConflictHint` (Private) ora riconosce anche questo pattern ("Method not found" + "Microsoft.Identity.Client" nello stesso messaggio, non un singolo pattern fisso). Più in profondità: `Execute-PendingAction` (`Gui\Server.ps1`), per i casi `ExoWrite / TeamsWrite`, ora passa dalla nuova `Invoke-M365OpsWriteWithIsolationRecovery` (Public) - se la scrittura fallisce con un conflitto riconosciuto, tenta l'isolamento reattivo ADESSO (anche se il connect era già riuscito prima) e RIPROVA la stessa scrittura una sola volta attraverso il processo isolato, invece di limitarsi a mostrare l'errore .NET grezzo. **Onestà tecnica su questo fix:** il percorso "nessun conflitto, tutto ok al primo tentativo" e la logica di riconoscimento pattern sono verificati dal vivo (test unitari mirati + una scrittura Teams reale completata con successo attraverso il wrapper); il percorso di RETRY specifico (fallimento reale seguito da isolamento-e-risuscita-al-secondo-tentativo) non è stato ri-riprodotto forzatamente in questo giro, perché il conflitto stesso è intrinsecamente non deterministico (dipende dalle versioni esatte installate in quel momento, stessa causa già documentata sopra per il conflitto originale) - il meccanismo di retry riusa però ESATTAMENTE lo stesso `Connect-M365OpsIsolatedModule` già verificato a fondo poco sopra in questa sezione, quindi il rischio residuo è basso, non assente. **AGGIORNAMENTO, scoperto ancora dal vivo pochi minuti dopo aver scritto il paragrafo sopra, nella STESSA sessione di test:** il conflitto di questo tipo, una volta scattato una prima volta nel processo, ha lasciato un effetto collaterale più ampio di quanto creduto inizialmente - non solo la singola scrittura che lo ha fatto emergere (recuperata con successo dal meccanismo sopra), ma ANCHE letture Exchange completamente indipendenti, eseguite diversi minuti dopo, hanno cominciato a fallire con un errore DIVERSO ma correlato (`IDX12729`, un errore di decodifica token JWT di `Microsoft.IdentityModel` - stessa famiglia di librerie di autenticazione condivisa, ennesima variante dello stesso problema di fondo). Confermato con un confronto diretto: la STESSA query, in un processo PowerShell completamente nuovo (fuori dal server), ha funzionato subito senza alcun errore - la degradazione è quindi specifica del processo server già "toccato" dal conflitto, non un problema di dati o di rete. Un riavvio completo del server ha risolto tutto immediatamente (stessa query, stesso processo dopo il riavvio: risultato corretto). **Conclusione onesta:** il meccanismo di recupero sopra riduce concretamente l'impatto immediato (l'azione confermata dall'utente va comunque a buon fine, verificato dal vivo), ma NON è una cura

completa per il processo nel suo complesso - un conflitto reale, una volta scattato, può lasciare un'instabilità residua su chiamate successive non correlate. La guida raccomanda quindi ancora, come indicazione prudente dopo aver visto comparire QUALUNQUE errore di questa famiglia (assembly/versione/token JWT legati a Microsoft.Identity.Client o Microsoft.IdentityModel), un riavvio completo del server come passo successivo, anche se l'azione specifica che ha innescato l'errore sembra essere andata a buon fine grazie al recupero automatico. Rimane inoltre un limite strutturale non ancora coperto: cmdlet di SOLA LETTURA che incontrano lo stesso conflitto dopo un connect riuscito (non solo le scritture) non passano ancora dal meccanismo di recupero con retry - se capita, il messaggio resta quello grezzo (o la variante IDX12729 sopra) finché non viene esteso anche a quel percorso.

Verificato dal vivo end-to-end, sul tenant reale "vnsys-test", non con credenziali finte: connesso Teams per primo (direzione nota per rompersi), poi Exchange - isolamento attivato automaticamente, confermato nei log ("Isolamento reattivo attivato per Exchange - 835 cmdlet ora eseguiti in un processo separato"); Get-Mailbox / Get-DistributionGroup / Get-CASMailbox hanno restituito dati reali attraverso il proxy; Teams è rimasto funzionante in-process per tutta la durata (mai isolato, non necessario); il worker isolato si chiude correttamente alla disconnessione/cambio tenant.

Un secondo conflitto di assembly, distinto da 6.5 (quello sopra e' fra due MODALITA' di autenticazione dello STESSO modulo Exchange; questo e' fra DUE MODULI diversi, indipendente dalla modalita' Delegata/App-only): connettersi a Microsoft Teams e poi a Exchange Online nello STESSO processo server (o viceversa) poteva fallire con un errore analogo - Microsoft.IdentityModel.Abstractions se Teams e' stato connesso per primo, Microsoft.Identity.Client se Exchange e' stato connesso per primo.

Storia completa della diagnosi, lasciata visibile perche' istruttiva - il percorso e' stato lungo e con piu' false piste corrette man mano dall'utente: (1) creduto inizialmente un conflitto Microsoft inevitabile, in ENTRAMBE le direzioni, con qualunque versione; (2) l'utente ha corretto questa diagnosi ("ieri ho connesso tutto senza errori, non dipende dalla versione dei moduli?") - confermato che dalla versione **3.10.0** di ExchangeOnlineManagement in poi il conflitto e' effettivamente SEMPRE presente, in entrambe le direzioni, qualunque versione di Teams - quella soglia resta vera; (3) un primo pin a 3.9.0 (senza pin su Teams) e' sembrato risolvere il problema su due versioni Teams testate, poi smentito da una matrice piu' ampia (Teams 7.1.0/7.3.1/7.9.0) - nessun ordine era affidabilmente sicuro con quel pin; (4) rimossa ogni guardia preventiva su richiesta dell'utente, sostituita con un messaggio reattivo; (5) l'utente ha insistito, correttamente, che "fino all'altro giorno" tutto funzionava in delegato/device-code, e ha guidato la ricerca verso versioni piu' vecchie di ENTRAMBI i moduli - analizzata la libreria di autenticazione condivisa (Microsoft.Identity.Client.dll) bundled in OGNI versione rilasciata di entrambi i moduli (16 versioni Exchange, 16 versioni Teams): nessuna coppia condivide la build esatta in nessun punto della storia di nessuno dei due moduli, ma la coppia **ExchangeOnlineManagement 3.4.0 + MicrosoftTeams 6.5.0**, con Exchange connesso PRIMA di Teams, e' risultata verificata come sicura in un test dal vivo completo di tutti i flussi del progetto (App-only, login delegato device-code via -AccessToken , Purview App-only, poi Teams App-only connesso dopo tutti e tre) - nessun conflitto in nessuno dei quattro passaggi, ripetuto due volte.

Fissata la coppia di versioni, non solo una delle due (v0.9.47): Assert-M365OpsExoSafeVersion (ExchangeOnlineManagement 3.4.0) e Assert-M365OpsTeamsSafeVersion (MicrosoftTeams 6.5.0, nuova funzione) garantiscono entrambe che la propria versione sia installata E integra (stesso controllo di integrita' dei file, non solo del manifest, introdotto dopo il bug "The given assembly name was invalid" - vedi sotto), la importano e riparano da sole un'installazione danneggiata quando e' ancora possibile farlo (PRIMA di qualunque Import-Module). Assert-M365OpsExoSafeVersion disinstalla anche attivamente qualunque versione ≥3.10.0 trovata sul disco - quella soglia resta l'unica certezza universale di questo conflitto, indipendente dalla versione di Teams. 3.4.0 e' stata scelta deliberatamente come la piu' vecchia versione ExchangeOnlineManagement che supporta TUTTO cio' che serve al progetto (-CertificateThumbprint per App-only, -AccessToken per il login delegato device-code, -CertificateThumbprint su Connect-IPPSession per Purview App-only) - non e' stata scelta a caso, e' la piu' lontana possibile (in termini di eta'/dipendenze) dalla soglia 3.10.0. 6.5.0 e' stata preferita a 5.0.0 (anch'essa verificata sicura nella stessa coppia) perche' piu' recente a parita' di Microsoft.Identity.Client bundled (condiviso da tutte le versioni Teams 4.0.0-6.5.0) - piu' bugfix, e predata anche il passaggio di Connect-MicrosoftTeams a WAM come broker di default (sezione 10.6/17.14, introdotto in 7.8.1-preview), quindi non ne soffre per costruzione. La guardia preventiva resta rimossa (decisione dell'utente: "non voglio il safe guard, nelle versioni vecchie funzionava") - Get-M365OpsModuleConflictHint (Private) resta come rete di sicurezza reattiva per l'ordine inverso o per PC con versioni residue diverse. **Verifica finale (v0.9.48) fatta con il tenant di test REALE attraverso la GUI vera**, non solo con credenziali finte in script isolati: cliccati in sequenza i pulsanti Purview, Exchange e Teams nel tab Tenant sul tenant vnsys-test - tutti e tre connessi con successo nello stesso processo server, con dati reali restituiti (mailbox condivise, Team elencati), non solo "nessun errore".

PERCHE' 3.4.0 e non 3.1.0 (che sulla carta sarebbe la piu' vecchia con tutto cio' che serve) - scoperto testando con CREDENZIALI REALI, non finte: un primo pin a 3.1.0 e' stato scelto e poi scartato lo stesso giorno dopo un test dal vivo sul tenant vnsys-test reale - Connect-IPPSession in App-only su 3.1.0 falliva con un errore WS-Management/WinRM ("Connecting to remote server ps.compliance.protection.outlook.com failed..."), non un conflitto di assembly: 3.1.0 usa ancora il vecchio protocollo Remote PowerShell (RPS) per Security & Compliance, che Microsoft ha dismesso lato server dal 15 luglio 2023 (banner ufficiale mostrato dal modulo stesso alla connessione) - un endpoint che non esiste piu', nessuna correzione lato nostro puo' farlo funzionare. I test con credenziali FINTE fatti fino a quel momento non lo avevano mai scoperto: fallivano prima, sul controllo locale del certificato, senza mai arrivare alla vera chiamata di rete - lezione diretta: un test con credenziali finte verifica l'assenza del conflitto di assembly, non che la funzionalita' sia davvero utilizzabile. ExchangeOnlineManagement 3.2.0 ha spostato Security & Compliance sulle REST API (stesso banner del modulo) - 3.4.0 (ultima versione della fascia 3.1.0-3.4.0 che condivide la stessa build di Microsoft.Identity.Client , quindi eredita la stessa garanzia di non conflitto con Teams senza dover essere riverificata da zero) e' stata poi riverificata con SUCCESSO REALE: connessione Purview autentica sul tenant di test, non solo "nessun errore".

Login delegato a Purview: implementato via popup interattivo, non device-code (v0.9.48) - Connect-IPPSession su questa versione fissata non supporta ALCUN flusso device-code/token (ne' -Device ne' -AccessToken , solo -UserPrincipalName / -Credential o i parametri App-only), a differenza di Connect-ExchangeOnline che sulla stessa versione supporta gia' -AccessToken perfettamente - limite reale del modulo su questa versione, non un bug. -UserPrincipalName da solo pero' attiva un vero prompt di autenticazione moderna (confermato dalla guida integrata del modulo: "allows you to skip entering a username in the modern authentication credentials prompt") - un popup bloccante, stesso principio gia' usato per Teams/SharePoint delegati (gated da -AllowInteractive , mai avviato da solo). Sicuro dal punto di vista del conflitto Teams perche' passa dalla STESSA libreria di autenticazione condivisa gia' verificata pulita per Purview App-only nella stessa coppia di versioni - non e' un percorso di codice diverso, solo un modo diverso di popolare le credenziali. Connect-M365OpsCompliance.ps1 lo sceglie dinamicamente (controlla se -Device esiste prima di usarlo, altrimenti usa solo -UserPrincipalName). **Novita' collaterale scoperta nello stesso giro:** Purview in modalita' Delegata non aveva NESSUN punto di ingresso in GUI prima di questo fix (Get-M365OpsRetentionCompliancePolicies chiamava Connect-M365OpsCompliance senza -AllowInteractive , quindi falliva sempre con "vai a connetterlo" senza che esistesse davvero un modo di farlo) - aggiunto un riquadro "Purview / Compliance" nel tab Tenant, stesso schema di Exchange/SharePoint/Teams, con nuovo endpoint POST /api/compliance-test .

Regola pratica: connetti sempre Exchange (o Purview) PRIMA di Teams nella stessa sessione del server - in questo ordine, con questa coppia di versioni, funziona senza conflitti in ogni modalita' testata. Se hai gia' installato questo progetto e aggiorni da una versione precedente, la prima connessione Exchange/Teams dopo l'aggiornamento allinea da sola i moduli sul disco (installa/ripara le versioni corrette, rimuove una eventuale ≥3.10.0) - **ma serve un riavvio COMPLETO del processo server, non solo un refresh del browser:** la pagina index.html (compresi questi avvisi) viene letta una sola volta all'avvio del server e tenuta in memoria per tutta la sua vita (sezione 12.2).

BUG DISTINTO scoperto durante questa indagine, gia' corretto: un errore separato, "The given assembly name was invalid.", poteva comparire anche SENZA alcun coinvolgimento di Teams - un'installazione locale di ExchangeOnlineManagement danneggiata (Get-Module -ListAvailable legge solo il manifest, non verifica che ogni DLL referenziata esista e sia integra). Entrambe le funzioni Assert-M365Ops*SafeVersion ora verificano l'integrita' dei file critici PRIMA di importare, quando e' ancora possibile ripararla - una volta che Import-Module ha caricato l'assembly nel processo, Windows blocca quei file a livello di file system e nemmeno Remove-Item puo' piu' toccarli, verificato dal vivo.

7. Registrazione del tenant nel modulo

Il file `Config\tenants.json` è la fonte di verità per ogni tenant configurato. **Non contiene mai segreti**: solo ID, thumbprint del certificato (non è un segreto) e il NOME della variabile d'ambiente che contiene il client secret. Tutta questa sezione si fa dalla GUI, tab **Tenant** → riquadro "Aggiungi / aggiorna profilo" — non serve aprire un terminale.

Messaggio di benvenuto al primo avvio (aggiunto il 21/08/2026): finché `Config\tenants.json` non contiene nessun profilo, la chat mostra un messaggio di onboarding invece del messaggio normale - indirizza al tab Tenant e riassume le due modalità di autenticazione (Delegata vs App Registration), con un cenno ai permessi da assegnare per App Registration e al fatto che Exchange Online richiede in più il certificato. Sparisce da solo non appena si salva il primo profilo, nessuna azione manuale da fare.

1. **Nome profilo** — un nome a scelta per riconoscerlo (es. `contoso-test`).
2. **Tenant ID** — dominio `....onmicrosoft.com` o GUID del tenant, indifferentemente.
3. **Modalità di autenticazione** — App-only (richiede App Registration, sezioni 4-6) o Delegata (login col tuo utente, sezione 10).
4. Se App-only: **Client ID, nome variabile d'ambiente per il secret** (es. `M365_CLIENT_SECRET`) e — nel campo separato subito sotto — **il valore del secret stesso**, copiato dal portale Entra ID: il campo lo scrive direttamente nella variabile d'ambiente del tuo utente Windows (mai in un file), lascialo vuoto per non toccare un valore già impostato in precedenza.
5. **Thumbprint certificato Exchange** (solo App-only, facoltativo se non usi Exchange).
6. **Salva profilo** → il tenant appare nell'elenco sopra: clicca **Attiva** per connetterti (equivalente a `Connect-M365Ops`).

Il mittente email (per l'invio report) e la chiave del motore AI (`ANTHROPIC_API_KEY` o le variabili Azure OpenAI) si impostano allo stesso modo, ciascuno dal proprio tab (rispettivamente "Email" e "Motore AI") — sempre un campo di testo, mai una variabile d'ambiente da impostare a mano.

Aggiungere un secondo tenant (in parallelo, senza toccare il primo)

Stesso riquadro, di nuovo: compila i campi con i dati del nuovo tenant (nome profilo diverso, propria App Registration/secret) e Salva — il profilo precedente resta intatto e selezionabile in qualsiasi momento dall'elenco.

Ogni tenant ha la propria App Registration, il proprio certificato Exchange, il proprio mittente email e i propri connettori MCP (sezione 9) — tutto annidato nel *proprio* oggetto dentro `tenants.json`, senza condivisione con altri profili.

8. Motore AI (Claude / Azure OpenAI)

Provider	Variabili richieste	Tool-calling reale (chat con dati)
Claude	ANTHROPIC_API_KEY	implementato
Azure OpenAI	AZURE_OPENAI_KEY , AZURE_OPENAI_ENDPOINT , AZURE_OPENAI_DEPLOYMENT	implementato

Il provider si sceglie dal tab "Motore AI" delle impostazioni dell'app (persistito su `Config\ai-provider.json` — sopravvive al riavvio del server) e vale per **tutto**: sia la chat libera con accesso reale ai dati (`Invoke-M365OpsAgentTools`) sia le chiamate singole senza strumenti (analisi pattern, diagnosi/correzione errori). Non è più vero, dal 15/08/2026, che la chat richieda sempre Claude a prescindere dal selettore. Claude e Azure OpenAI sono due alternative complete e alla pari: nessuno dei due è un "default" consigliato, la scelta è sempre e solo dell'utente (vedi anche v0.9.26 in changelog).

8.1 Due API diverse, stesso set di strumenti

Claude (Anthropic Messages API) e Azure OpenAI (Chat Completions API, parametro `tools / tool_choice`) hanno una forma di richiesta e risposta strutturalmente diversa — verificato sulla documentazione Microsoft Learn reale, non assunto per analogia. `Invoke-M365OpsAgentTools` gestisce questa differenza ramificando **solo** la parte che parla con l'API (costruzione della richiesta, lettura della risposta); le **definizioni degli strumenti** (quali dati leggono/scrivono) e la logica che li esegue restano un unico blocco condiviso tra i due provider — ogni chiamata a uno strumento, da qualunque provider arrivi, viene prima ricondotta alla stessa forma interna prima di essere eseguita, così aggiungere uno strumento nuovo richiede una sola modifica, non due.

8.2 Un limite reale di Azure OpenAI: 1024 caratteri per descrizione strumento

Azure OpenAI rifiuta (o tronca, a seconda del client) una descrizione di funzione oltre i 1024 caratteri — Claude non ha questo limite. Due strumenti di questo modulo (`exo_query` e `propose_exo_write` , che elencano l'intero catalogo delle 50+ cmdlet Exchange nella propria descrizione) superano abbondantemente quella soglia. Invece di troncarli a metà elenco perdendo cmdlet in modo silenzioso, per il ramo Azure la descrizione lunga viene accorciata nella definizione dello strumento e il contenuto integrale spostato nel messaggio di sistema della conversazione — il modello vede comunque l'elenco completo, solo in una posizione diversa della richiesta.

Bug reale corretto lo stesso giorno. La prima versione del supporto Azure OpenAI aveva un difetto di scoping identico a un altro già risolto in precedenza: `Invoke-M365OpsAgentTools` vive nel modulo, quindi il suo `$script:` è lo scope del *modulo*, non quello di `Server.ps1` — un valore di default basato su `$script:ActiveIPProvider` dentro la funzione vedeva sempre `$null` e cadeva silenziosamente su Claude, a prescindere dalla scelta reale in GUI. Corretto obbligando `Server.ps1` a passare `-Provider` in modo esplicito ad ogni chiamata, come già avviene per ogni altra funzione AI del modulo.

8.3 Configurare un deployment Azure OpenAI in Azure AI Foundry

Serve un **deployment** (non basta avere una sottoscrizione Azure) — un modello scelto e attivato dentro una risorsa Azure OpenAI/progetto Azure AI Foundry, con un nome che poi va inserito nell'app. Percorso completo, testato dal vivo il 15/08/2026:

1. Vai su ai.azure.com (Azure AI Foundry) → apri o crea un progetto.
2. **Deployments** → **Deploy a base model**.
3. Scegli un modello — vedi 8.3.1 per quale.
4. Dai un nome al deployment (può essere uguale al nome del modello, es. `gpt-5-mini`) → conferma.
5. A deployment completato ("Succeeded"), nel pannello di destra trovi i tre valori che servono all'app: **Project endpoint**, **API Key** (icona occhio per rivelarla), e il nome del deployment appena scelto.
6. Nell'app, tab **Motore AI** → provider Azure OpenAI → incolla i tre valori (endpoint, chiave, nome deployment esatto — case-sensitive) → Salva → **Testa connessione**.

8.3.1 Quale modello scegliere

Nel catalogo modelli di Azure AI Foundry, evita:

- **varianti "codex"** — specializzate per generazione di codice, non per ragionamento/uso di strumenti generico: non è il compito di questa app;
- **tier "nano"** — il più economico, ma anche il meno affidabile su orchestrazione multi-passo di strumenti (esattamente il compito di `Invoke-M365OpsAgentTools`);
- **tier "pro"** — capacità più alta ma latenza/costo maggiori, non necessari qui: la chat fa già fino a 8 giri di chiamate per una singola domanda.

Il **tier "mini"** del modello più recente disponibile nel catalogo è il compromesso giusto tra costo, velocità e affidabilità sul tool-calling — usato e verificato dal vivo con `gpt-5-mini` .

8.3.2 Due incidenti reali, entrambi corretti nel codice (non serve più pensarci)

1. Due formati di endpoint diversi. Il portale Azure mostra l'endpoint in due forme diverse a seconda di dove lo copi: quella "classica" della risorsa Azure OpenAI (`https://risorsa.openai.azure.com/`) e quella più recente "Project endpoint" di Azure AI Foundry (`https://risorsa.services.ai.azure.com/openai/v1`). Il modulo costruisce sempre la stessa chiamata REST classica (`/openai/deployments/{deployment}/chat/completions?api-version=...`), che va applicata alla *radice* della risorsa — incollare la seconda forma senza normalizzare produceva un URL con `/openai/` duplicato e un 404. Corretto: l'endpoint viene normalizzato automaticamente (rimosso un eventuale `/openai/v1` o `/openai` finale) prima di costruire la chiamata, qualunque delle due forme incolli.

2. max_tokens rifiutato dai modelli "reasoning". I modelli più recenti (serie o1/o3/o4 e famiglia gpt-5.x, incluso `gpt-5-mini`) rispondono **400 Bad Request** al parametro `max_tokens` , e vogliono `max_completion_tokens` al suo posto — non deducibile in anticipo dal solo nome del deployment. Corretto: il modulo prova prima con `max_tokens` (compatibile con i modelli più vecchi) e, solo se Azure risponde con quell'errore specifico, riprova automaticamente con `max_completion_tokens` — nessuna azione manuale richiesta, la scelta giusta viene anche ricordata tra un giro di tool-calling e il successivo nella stessa conversazione.

Testato dal vivo. Dopo questi due fix, una domanda reale ("elenco licenze del tenant") ha attraversato l'intero ciclo nuovo — costruzione della richiesta, chiamata a `graph_api_call/subscribedSkus`, e risposta finale — con Azure OpenAI (`gpt-5-mini`) come provider attivo, in circa 15 secondi.

9. Connettori MCP (Lokka) — configurazione per-tenant

Lokka è un server MCP open-source che esegue chiamate Graph/Azure REST generiche. È lo strumento **primario** usato dall'AI per leggere e proporre scritture — le cmdlet interne del modulo restano come fallback, usate solo se Lokka non copre il caso.

Sui tenant **Delegati**, Lokka (client credentials) non è mai disponibile — ma gli strumenti `graph_api_call` / `propose_graph_write` restano comunque offerti all'AI: a runtime scelgono da soli se passare da Lokka o da una chiamata diretta con `Invoke-M365OpsGraphRequest` usando lo stesso token delegato già in uso per le altre funzioni. Bug reale corretto: in una prima versione questi strumenti venivano esclusi del tutto sui tenant Delegati, impedendo di leggere dati Graph generici (es. licenze utente via `/users/{id}/licenseDetails`) nonostante il token valido fosse già disponibile.

La configurazione (comando, argomenti, mapping variabili) vive dentro la sotto-sezione `McpServers` del profilo tenant attivo in `tenants.json`, modificabile anche da GUI (tab MCP-Connettori):

```
{
  "contoso-test": {
    ...
    "McpServers": {
      "lokka": { "command": "npx", "args": "-y @merill/lokka", "envMapping": "none" }
    }
  }
}
```

Perché tenant-scoped: due tenant diversi possono avere connettori, credenziali o versioni del server MCP diverse in parallelo, senza che la configurazione dell'uno interferisca con quella dell'altro.

Anche la gestione a runtime è interamente da GUI, tab MCP-Connettori: il pulsante "🔄 **Riconnetti con la nuova configurazione**" chiama `Connect-M365OpsLokka -Force` dopo ogni modifica ai campi comando/argomenti. Ogni server elencato sotto "Altri server MCP" ha un proprio pulsante "Rimuovi" (con conferma). Lokka non è rimovibile da lì (è builtin): "rimuoverlo" significherebbe solo azzerare una personalizzazione, quindi si gestisce modificando i campi comando/argomenti invece che con una rimozione vera e propria.

10. Autenticazione delegata (utente + MFA) — tenant senza App Registration

Tutto il modulo, finora, presume che tu possa creare un'App Registration sul tenant target (sezione 4). Nella pratica capita spesso di gestire tenant dove hai ruoli admin delegati (Exchange Administrator, User Administrator, Teams Administrator...) ma **non** i permessi per registrare un'app (serve Global Administrator, Application Administrator o Cloud Application Administrator). Per questi casi esiste una seconda modalità, per-tenant, che non richiede alcuna App Registration.

10.1 Come funziona

Il profilo tenant si imposta con `AuthMode: Delegated` invece di `AppOnly`. In questa modalità il modulo non usa client credentials: usa un **login interattivo con il tuo utente** (username + password + MFA), tramite il client pubblico multi-tenant di Microsoft "Microsoft Graph Command Line Tools" (id `14d82eec-204b-4c2f-b7e8-296a70dab67e`) — un'app di Microsoft stessa, già presente in ogni tenant, che non richiede nessuna registrazione da parte tua. Il token risultante riflette esattamente i ruoli RBAC già assegnati al tuo utente su quel tenant: se hai Exchange Administrator, puoi leggere/scrivere ciò che quel ruolo permette; se non hai un ruolo su una risorsa, quella chiamata fallirà — esattamente come nel portale.

10.2 Isolamento tra tenant — non negoziabile

Un tenant AppOnly e un tenant Delegated (o due tenant Delegated diversi) non condividono MAI stato. Ad ogni cambio tenant (`Connect-M365Ops -TenantProfile ...`), il modulo:

- chiude esplicitamente la sessione Exchange Online precedente, se attiva;
- ferma il sottoprocesso Lokka precedente, se attivo.

Senza questo, Lokka (un unico sottoprocesso globale) e la sessione Exchange Online (remoting implicito, anch'esso globale) potrebbero restare agganciati al tenant precedente e far eseguire un comando pensato per il tenant appena attivato contro quello sbagliato. Il cambio tenant è sempre un'azione esplicita tua — non avviene mai automaticamente in mezzo a una conversazione.

10.3 Configurazione

Dalla GUI, tab Tenant → riquadro "Aggiungi / aggiorna profilo": "Modalità di autenticazione" → Delegata → inserisci il tuo UPN su quel tenant (es. `mario.rossi@clientey.onmicrosoft.com`) → Salva profilo → Attiva. Compare un riquadro "Login delegato" con un pulsante **"Accedi con il mio utente"**.

10.4 Primo login (device code flow)

1. Clic su "Accedi con il mio utente": l'app mostra un URL (`microsoft.com/devicelogin`) e un codice a 9 caratteri.
2. Apri l'URL in un browser (anche su un altro dispositivo), inserisci il codice, accedi con il tuo utente e completa l'MFA.
3. La GUI verifica in background (polling automatico) e conferma appena il login è completato — nessuna azione ulteriore da parte tua.

Il token viene tenuto in cache **solo in memoria** per la durata del processo (mai scritto su disco, stesso principio di zero-segreti-in-chiaro delle altre credenziali) e si auto-rinnova finché resta usato regolarmente; se il server si riavvia, va rifatto il login.

10.5 Limiti noti di questa modalità

Aspetto	AppOnly	Delegated
Serve App Registration	Sì	No
Esecuzione unattended	Sì, sempre	No — richiede un login umano ogni volta che la sessione scade
Permessi effettivi	Quelli concessi all'app (admin consent)	Quelli del ruolo RBAC del tuo utente su quel tenant
Lokka (MCP)	Disponibile	Non disponibile — richiede client credentials. L'AI usa solo <code>exo_query</code> /le cmdlet dirette
Connessione Exchange	Silenziosa (certificato)	Apri una finestra di login interattiva — blocca il server finché non completi

In pratica: usa AppOnly ovunque puoi (è il modello con cui è stato pensato tutto il resto del sistema); usa Delegated solo sui tenant dove non hai alternativa, sapendo che quel tenant specifico richiederà un tuo login occasionale invece di girare 24/7 senza interazione.

10.6 Rischio di blocco — perché la connessione Exchange delegata non è mai automatica

Il server di questo modulo gestisce le richieste HTTP **una alla volta** (single-threaded): se una richiesta si blocca in attesa di qualcosa, **l'intera app resta ferma per chiunque**, non solo per chi ha fatto quella richiesta specifica.

Incidente reale. Su un tenant Delegated appena attivato (login Graph già completato, ma sessione Exchange non ancora stabilita), una domanda in chat ("elenco licenze") ha fatto sì che l'AI provasse una cmdlet Exchange (`exo_query`). Ogni cmdlet Exchange chiama `Connect-M365OpsExchange` come primo passo, che in modalità Delegated avrebbe aperto un login interattivo — bloccando il server *per tutti*, per minuti, senza alcun feedback, finché il processo non è stato terminato manualmente da riga di comando.

Prima correzione (fail-fast): `Connect-M365OpsExchange` su un tenant Delegato senza sessione già attiva fallisce **immediatamente** con un messaggio chiaro invece di tentare un login bloccante, quando la connessione scatterebbe come effetto collaterale implicito di un tool AI (es. `exo_query`).

Seconda correzione (device code non bloccante) — la connessione esplicita stessa non blocca più: il primo tentativo di connessione esplicita (pulsante "Connetti Exchange") usava `Connect-ExchangeOnline -Device`, che stampa il codice solo nella console del processo — invisibile per un server lanciato in background — e blocca comunque l'intera app fino al completamento. Ora il pulsante usa lo stesso principio già collaudato per il login Graph (sezione 10.4): richiesta del codice device separata e non bloccante (client id `fb78d390-0c51-40cd-8e17-fdbfab77341b`, estratto e verificato empiricamente dal .dll del modulo `ExchangeOnlineManagement` — il client "legacy" `a0c73c16-...` talvolta citato online risulta disabilitato sui

tenant moderni), polling non bloccante per il completamento, e solo alla fine `Connect-ExchangeOnline -AccessToken` (rapido, nessuna interazione umana necessaria a quel punto) per stabilire la sessione vera e propria. Il codice compare direttamente nella GUI, non serve più che qualcuno lo legga dalla console per te.

Testato dal vivo, incidente reale: prima il server è rimasto bloccato senza feedback finché non è stato terminato a mano da riga di comando; il codice generato era visibile solo nella console del processo (`To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code ...`). Con la correzione, lo stesso flusso ora restituisce il codice immediatamente nella risposta HTTP, senza mai bloccare il server.

10.7 GUI riorganizzata — stato connessioni annidato nella riga del tenant attivo

Il tab **Tenant** mostra un indicatore per ciascun servizio (pallino verde/giallo): modalità (App-only/Delegata), Microsoft Graph, Exchange Online, e Lokka (solo App-only, non pertinente in modalità Delegata). La sezione **Exchange Online** (pulsante di connessione/test) vive qui, non in MCP/Connettori, dato che è dal tab Tenant che si gestisce tutto il resto del tenant attivo — il tab MCP/Connettori contiene solo Lokka e altri server MCP. Dal 18/08/2026 questo pannello non è più un blocco fisso sotto l'elenco profili: ogni riga tenant ha un proprio chevron espandi/collassa, e solo il tenant **attivo** mostra il pannello reale (espanso di default) — le righe degli altri profili, quando espanso, mostrano solo un avviso che non c'è nulla di reale da verificare finché non li attivi. Attivare un tenant diverso sposta il pannello reale sulla sua riga automaticamente.

Il provider AI selezionato nel tab "Motore AI" è persistito su `Config\ai-provider.json` (solo il nome, non è un segreto) — un riavvio del server lo ricarica automaticamente, non torna più a Claude come default se stavi usando Azure OpenAI.

10.8 Permessi Microsoft Graph richiesti in modalità Delegata

A differenza di AppOnly (dove i permessi si scelgono e si concedono manualmente in fase di App Registration, sezione 4.2), in modalità Delegata i permessi sono uno **scope OAuth richiesto al login**: il token contiene solo ciò che è stato esplicitamente domandato in `Private\Invoke-M365OpsDeviceCodeFlow.ps1 ( $script:M365OpsDeviceCodeScopes )`, non ciò che l'utente avrebbe "in teoria" tramite i propri ruoli RBAC. Uno scope non richiesto qui produce un 403 che sembra un problema di consenso admin per un'app estranea, ma è quasi sempre solo che il token non l'ha mai chiesto — incontrato due volte dal vivo (`Organization.Read.All` per il report licenze, `UserAuthenticationMethod.ReadWrite.All` per lo stato/reset MFA) prima di arrivare all'elenco completo sotto.

Permesso	Motivo
<code>DeviceManagementManagedDevices.Read.All</code>	lettura device Intune, stato compliance
<code>DeviceManagementConfiguration.ReadWrite.All</code>	profili di configurazione
<code>DeviceManagementApps.ReadWrite.All</code>	app Intune (Win32, Store), assegnazioni, criteri di protezione app MAM
<code>DeviceManagementRBAC.ReadWrite.All</code>	Scope Tag, ruoli RBAC personalizzati, assegnazioni di ruolo
<code>DeviceManagementServiceConfig.ReadWrite.All</code>	restrizioni/limiti di iscrizione dispositivi, Autopilot
<code>DeviceManagementScripts.ReadWrite.All</code>	script di distribuzione Windows/macOS, Proactive Remediations
<code>Group.ReadWrite.All</code>	creazione gruppi, gestione membri
<code>User.ReadWrite.All</code>	lettura e scrittura su utenti (es. disabilitare un account)
<code>UserAuthenticationMethod.ReadWrite.All</code>	stato MFA e reset (sezione 21) — scope dedicato, <i>non</i> coperto da <code>User.ReadWrite.All</code>
<code>Directory.Read.All</code>	lettura oggetti directory generici oltre a utenti/gruppi
<code>Organization.Read.All</code>	informazioni tenant, licenze (<code>/subscribedSkus</code> , <code>/organization</code>)
<code>AuditLog.Read.All</code>	log di sign-in
<code>RoleManagement.Read.Directory</code>	lettura assegnazioni di ruolo (sola lettura — vedi nota sotto)
<code>Reports.Read.All</code>	report di utilizzo (usage report Microsoft 365)
<code>Sites.Read.All</code>	lettura siti SharePoint
<code>Application.Read.All</code>	visibilità sulle App Registration del tenant (sola lettura)
<code>IdentityRiskyUser.Read.All</code>	segnali di rischio Identity Protection (<code>/identityProtection/riskyUsers</code>) - corretto il 19/08/2026: questa riga elencava per errore <code>SecurityEvents.Read.All</code> (permesso per un'API diversa, gli alert Defender), mai verificato dal vivo fino a quel momento - nome corretto confermato su Microsoft Learn
<code>Policy.Read.All</code>	lettura policy del tenant (es. Conditional Access — sola lettura)
<code>Team.ReadBasic.All</code> / <code>Channel.ReadBasic.All</code>	dati Teams di base
<code>Mail.Send</code>	invio report generati via email
<code>offline_access</code>	refresh token — senza, la sessione non si rinnova da sola

Scritture ad alto raggio d'azione escluse di proposito. `RoleManagement.ReadWrite.Directory` (assegna/revoca ruoli admin), `Policy.ReadWrite.ConditionalAccess` (un Conditional Access sbagliato può bloccare fuori l'intero tenant) e `Application.ReadWrite.All` (può creare credenziali su un'app) non sono nello scope richiesto — vanno aggiunti solo su richiesta esplicita per una funzionalità reale già costruita, mai "perché potrebbe servire".

Ogni volta che si aggiunge un nuovo scope, serve un **nuovo login** (tab Tenant → "Accedi con il mio utente") — un token già in cache non guadagna retroattivamente un permesso non richiesto al momento in cui è stato emesso. La prima volta che un nuovo scope viene effettivamente usato può comparire una schermata di consenso durante il login stesso, normale per un permesso mai richiesto prima su quel tenant.

10.9 Connessione Intune in modalità Delegata — un limite del modulo di terze parti, non nostro

La pacchettizzazione app Win32 (`New-M365OpsWin32App`) passa dal modulo di terze parti `IntuneWin32App` , che gestisce la propria autenticazione in modo indipendente dal resto di questo progetto — non condivide il token Graph già in uso per tutto il resto.

Incidente reale. Il flusso a codice dispositivo (`Connect-MSIntuneGraph -DeviceCode`) di `IntuneWin32App` ha un bug reale su PowerShell 7: il gestore d'errore del ciclo di polling chiama `$ErrorResponse.GetResponseStream()` , un metodo che esiste solo su `System.Net.WebResponse` (.NET Framework), non su `System.Net.Http.HttpResponseMessage` (le eccezioni di PS7) — crolla sul primissimo "in attesa che l'utente completi il login", cioè sempre, a prescindere da quanto in fretta si inserisca il codice. Il flusso a codice dispositivo per Intune quindi **non può funzionare** su questo sistema, indipendentemente da qualunque cosa faccia questo modulo.

Corretto passando a `-Interactive` (`Connect-MSIntuneGraph -Interactive`), un flusso PKCE con redirect su `localhost` scritto da zero dal modulo stesso, che non condivide quel bug — apre il browser di sistema con **selezione account esplicita** (`prompt=select_account`), utile anche per essere certi di quale utenza si stia usando.

Connessione esplicita, mai implicita

Come per Exchange (sezione 10.6), la connessione Intune su un tenant Delegato non parte mai da sola dentro un'azione automatica: `Connect-M365OpsIntune` fallisce subito con un messaggio chiaro se non c'è già una sessione valida, a meno di essere chiamata esplicitamente con `-AllowInteractive` — riservato al pulsante dedicato **"Connetti Intune (browser)"** nel tab Tenant. Il pulsante avvisa esplicitamente che il server resterà bloccato (a thread singolo, sezione 10.6) finché non completi il login nel browser appena aperto — va quindi cliccato PRIMA di chiedere in chat di pacchettizzare un'app, mai durante una richiesta composta dove bloccherebbe l'app senza preavviso.

Verifica dal vivo, non solo un flag

La connessione non si considera valida solo perché `Connect-MSIntuneGraph` non ha sollevato un errore: `Test-M365OpsIntuneAuthValid` chiama Graph con il token appena ottenuto (`/organization`) e confronta il tenant risolto con quello attivo — se non combaciano (token/cache riusato da un'altra sessione) o la chiamata fallisce, la connessione viene rifiutata e va rifatta, invece di scoprire il problema più tardi dentro `New-M365OpsWin32App` con un errore criptico del modulo. La GUI mostra l'utenza e il tenant effettivamente verificati, non solo un'icona verde presunta.

10.10 Prompt di admin consent al login Graph delegato — e perché CLI Microsoft 365 può risolverlo (27/08/2026)

Domanda reale dell'utente: "mi loggo nel tenant e mi chiede Graph come admin consent anche se mi loggo con il mio account? questo CLI 365 risolve?" — risposta completa, non ipotetica.

Il login delegato descritto in questa sezione (10.4) usa il client pubblico Microsoft **"Microsoft Graph Command Line Tools"** (`14d82eec-...`). Il login delegato di **CLI Microsoft 365** (`Connect-M365OpsCLIMicrosoft365 -AllowInteractive` , tab Tenant → riquadro "CLI Microsoft 365") usa invece un client pubblico Microsoft **diverso**, di proprietà del progetto PnP/CLI Microsoft 365. Sono due app Microsoft first-party SEPARATE, ciascuna con il proprio stato di consenso INDIPENDENTE su un dato tenant — è quindi del tutto normale che una richieda ancora un consenso (o lo richieda "admin", vedi sotto) mentre l'altra sia già pronta all'uso, anche stesso account, stesso tenant.

Perché normalmente NON serve un admin che clicchi "Grant admin consent": un login DELEGATO (con la tua utenza reale, non un'App Registration Application) chiede consenso solo per gli scope DELEGATI necessari — per la stragrande maggioranza di questi scope, in un tenant con le impostazioni di default, il consenso è **auto-consenso dell'utente** al primo utilizzo (un semplice popup "Questa app vuole accedere a... Accetta"), non un passaggio riservato a un amministratore. Il permesso EFFETTIVO che ottieni resta comunque limitato al tuo ruolo RBAC reale su quel tenant (sezione 10.1) — l'auto-consenso concede solo la possibilità tecnica di chiedere quello scope, non un privilegio aggiuntivo.

L'eccezione che spiega un prompt "admin consent" anche loggandoti con il tuo utente: se il tenant ha la policy Entra ID *"Users can consent to apps"* disabilitata (impostazione di sicurezza comune in ambienti aziendali/hardened), OGNI app — inclusi entrambi i client pubblici Microsoft sopra — richiede un consenso amministratore una tantum, a prescindere da quale dei due stai usando. In quel caso passare a CLI Microsoft 365 NON risolve da solo: risolve solo se quel client specifico ha già ricevuto il consenso admin in passato (es. da un utente Global Admin che lo ha usato prima), mentre l'altro client (Graph CLI Tools) non lo ha mai ricevuto — è quindi possibile che "risolva" per puro caso di quale dei due è già stato consentito su quel tenant, non per un motivo strutturale che lo garantisca sempre.

Come verificare qual è il caso reale, invece di indovinare: se il prompt che compare menziona esplicitamente "chiedi al tuo amministratore di concedere il consenso" (`AADSTS65001` con indicazione admin-only, o la UI mostra "Serve l'approvazione di un amministratore" invece del normale pulsante "Accetta"), è la policy tenant-wide — un amministratore deve concedere il consenso una tantum per QUEL client specifico (Entra ID → App aziendali → cerca il nome del client → Autorizzazioni → "Concedi consenso amministratore"), dopo di che qualunque utente può accedere normalmente. Se invece il prompt è il normale popup "Accetta" che chiunque può cliccare da solo, non è affatto una policy di blocco — è solo il primo utilizzo di quel client su quel tenant, e basta cliccare Accetta una volta.

Conclusione pratica: sì, tenere CLI Microsoft 365 come connettore alternativo ha senso proprio per questo scenario — se il client Graph CLI Tools risulta bloccato/ non ancora consentito su un tenant specifico, il login delegato di CLI Microsoft 365 (client diverso) vale la pena provarlo come seconda strada, prima di dover coinvolgere un amministratore del tenant per un consenso esplicito. Non è garantito che eviti SEMPRE il problema (la policy tenant-wide, se presente, blocca comunque entrambi finché un admin non consente il client specifico in uso), ma nella pratica risolve la maggioranza dei casi in cui il blocco non è una policy ma solo "questo client non è ancora stato usato/consentito qui".

11. Le 50+ cmdlet Exchange Online e come le usa l'AI

Oltre alle due cmdlet iniziali (mailbox condivise e relativi permessi), il modulo espone oggi oltre 50 cmdlet Exchange Online, organizzate per area. Tutte richiedono `Connect-M365OpsExchange` (chiamata automaticamente al loro interno) e rispettano lo stesso principio di conferma umana per le scritture.

Area	Esempi di cmdlet
Mailbox condivise	<code>New-M365OpsSharedMailbox</code> , <code>Grant-/Revoke-M365OpsMailboxPermission</code> , <code>Add-/Remove-M365OpsSendOnBehalf</code> , <code>Get-M365OpsSharedMailboxReport</code>
Gruppi Exchange	Distribution list, mail-security group, dynamic distribution group — <code>Get/New/Remove</code> , gestione membri, <code>Get-M365OpsGroupsOverviewReport</code>
Mail flow	<code>Get-/New-/Set-/Remove-M365OpsTransportRule</code> , <code>Get-M365OpsMailFlowReport</code>
Tracking	<code>Get-M365OpsMessageTrace</code> , <code>Get-M365OpsMessageTraceDetail</code>
Domini	<code>Get-M365OpsAcceptedDomains</code>
Contatti	<code>Get-/New-/Remove-M365OpsMailContact</code> , <code>Get-M365OpsMailUsers</code>
Mailbox risorsa	<code>Get-M365OpsResourceMailboxes</code> , <code>New-M365OpsRoomMailbox</code> / <code>EquipmentMailbox</code> , policy di prenotazione
Migrazioni	<code>Get-M365OpsMigrationBatches</code> / <code>MigrationUserStatus</code> / <code>MigrationEndpoints</code> (lettura); <code>New-M365OpsMigrationBatch</code> — SOLO su un endpoint già configurato (mai credenziali nuove da chat), creato sempre non avviato; <code>Start-M365OpsMigrationBatch</code> per avviarlo, passo separato
Report mailbox	Utilizzo/quota, mailbox inattive, inoltri automatici, risposte automatiche, delegati aggregati, regole sospette, litigation hold
Sicurezza	<code>Get-M365OpsInboxRulesReport</code> (regole sospette), <code>Get-M365OpsSharedMailboxSignInStatus</code> (mailbox condivise con login abilitato — rischio)
Calendario / Cartelle pubbliche	<code>Get-/Set-M365OpsCalendarPermission</code> , <code>Get-M365OpsPublicFolders</code>

11.1 Come l'AI le usa

L'AI non ha 50 strumenti separati da scegliere (costerebbe troppo token e confonderebbe il modello): ha due strumenti generici, sullo stesso principio di `graph_api_call` / `propose_graph_write` usati per Lokka:

- **exo_query** — sola lettura, il modello specifica quale cmdlet (dalla lista consentita) e con quali parametri, viene eseguita subito;
- **propose_exo_write** — come `propose_graph_write` : il modello propone cmdlet + parametri + motivazione, ma non esegue mai nulla. La proposta torna in chat con "Cmdlet: ... Parametri: ... Motivo: ...", in attesa di conferma esplicita.

Entrambi gli strumenti validano il nome della cmdlet contro una lista consentita (non è possibile far eseguire all'AI una cmdlet PowerShell arbitraria).

11.2 Report via catalogo (senza costo AI)

I report più richiesti sono anche comandi diretti del catalogo, zero costo AI: "esporta report utilizzo mailbox in excel", "esporta report mailbox condivise in pdf", "esporta report mailbox totali", "esporta report mail flow", "esporta report inoltri", "esporta report regole sospette".

Nota sul routing: un trigger generico come "permessi mailbox" senza specificare "condivise" passa volutamente all'AI (non al comando dedicato alle sole mailbox condivise) — così può usare `Get-M365OpsMailboxDelegatesReport` , che copre *tutte* le mailbox, non solo quelle condivise, ed essere esplicita sui limiti invece di rispondere con uno scope ristretto senza dirlo.

11.3 Migrazioni — solo su endpoint già esistenti

La creazione di un **nuovo** endpoint di migrazione (che richiede le credenziali del sistema sorgente esterno — server IMAP, Exchange on-premises, ecc.) resta volutamente fuori da questo modulo: va fatta manualmente dal centro amministrazione Exchange o con `New-MigrationEndpoint` diretto, con supervisione — inserire credenziali di un sistema esterno tramite la chat non è un caso d'uso che vogliamo abilitare.

Quello che l'AI *può* fare è creare un **batch** su un endpoint *già configurato*: prima cerca gli endpoint disponibili con `exo_query` su `Get-M365OpsMigrationEndpoints` , poi propone `New-M365OpsMigrationBatch` con quell'identity esatta — `New-M365OpsMigrationBatch` rifiuta qualunque `SourceEndpoint` che non corrisponda esattamente a uno di quelli configurati, invece di lasciar passare un nome indovinato. Il batch viene sempre creato **non avviato**: avviarlo è un passo separato ed esplicito (`Start-M365OpsMigrationBatch`), con la sua stessa conferma umana di ogni altra scrittura.

Opzioni reali supportate

Opzione	Cmdlet	Effetto
<code>-Users</code>	<code>New-M365OpsMigrationBatch</code>	Elenco preciso di indirizzi da migrare (batch mirato) — via CSV caricato o elencati in chat
<code>-Cutover</code>	<code>New-M365OpsMigrationBatch</code>	Batch sull'intera organizzazione, nessun elenco
<code>-AutoComplete</code>	<code>New-M365OpsMigrationBatch</code>	Ogni mailbox si finalizza da sola non appena sincronizzata; di default assente — resta in sospeso, serve conferma manuale
<code>-CompleteAfter <data/ora></code>	<code>New-M365OpsMigrationBatch</code>	Programma la finalizzazione a un momento preciso nel futuro (es. fuori orario lavorativo) invece che subito
<code>-StartAfter <data/ora></code>	<code>Start-M365OpsMigrationBatch</code>	Programma l'inizio della sincronizzazione invece di farla partire immediatamente all'avvio

Caricare un elenco di indirizzi da CSV

Sopra la casella di chat c'è un pulsante **"Carica CSV migrazione..."** (prima colonna del file: `EmailAddress`, un indirizzo per riga). Una volta caricato, il contenuto viene fornito automaticamente all'AI quando chiedi di creare un batch — non serve incollare gli indirizzi a mano in chat.

Basta un solo prompt in chat — esempio verificato dal vivo

Sì: dopo aver caricato il CSV, un singolo messaggio in linguaggio naturale è sufficiente. L'AI cerca da sola l'endpoint disponibile via `exo_query`, legge gli indirizzi dal CSV caricato, e costruisce la proposta completa:

```
Utente: crea un batch di migrazione chiamato TestBatch1 con gli indirizzi
del csv caricato, usando l'endpoint disponibile, senza completamento automatico

Risposta (proposta, non ancora eseguita):
Cmdlet: New-M365OpsMigrationBatch
Parametri: {"SourceEndpoint":"Hybrid Migration Endpoint - EWS (Default Web Site)",
           "Name":"TestBatch1",
           "Users":["mario.rossi@contoso.com","anna.bianchi@contoso.com"]},
           "AutoComplete":false}
Motivo: [spiegazione in italiano di cosa fa, mostrata prima della conferma]

(rispondi 'si' per eseguirla davvero, 'no' per annullare)
```

Per programmare una finalizzazione futura basta aggiungerlo alla stessa frase, es. "...e completala automaticamente il 1 settembre alle 22" — l'AI traduce la data in `CompleteAfter` senza bisogno di comandi PowerShell separati.

11.4 Copertura completa dei parametri — mai a memoria, sempre da Microsoft Learn

Le cmdlet di scrittura di questo modulo non limitano a priori quali parametri nativi puoi usare: ognuna espone anche `ExtraParams` (un oggetto libero) per qualunque parametro reale della cmdlet Exchange/Graph sottostante non già previsto esplicitamente — es. `Alias`, `RetentionPolicy`, `ModeratedBy` su `New-Mailbox`, e così per ogni altra cmdlet.

Per evitare che l'AI inventi un nome di parametro plausibile-ma-sbagliato (un rischio reale con la sola conoscenza pregressa del modello, che può essere incompleta o superata), ha accesso a uno strumento aggiuntivo:

- **lookup_ms_docs** — consulta la pagina reale e aggiornata di Microsoft Learn per una cmdlet Exchange (URL diretto alla pagina di riferimento, con sintassi completa e tipo di ogni parametro) o, in fallback, la ricerca Microsoft Learn per argomenti Graph. Il modello è istruito a usarlo *prima* di includere in `ExtraParams` un parametro non già verificato in conversazione, e quando l'utente chiede esplicitamente "quali altre opzioni ci sono per X".

Testato dal vivo. Alla domanda "quali altre opzioni ha New-Mailbox oltre a quelle che usi di solito per creare una mailbox condivisa? dammene 5", l'AI ha consultato `lookup_ms_docs` "New-Mailbox" e risposto con 5 parametri reali (`RetentionPolicy`, `AddressBookPolicy`, `ModeratedBy` / `ModerationEnabled`, `Archive`, `ThrottlingPolicy`), spiegando anche come usarli via `ExtraParams` in `propose_exo_write`.

Questo principio è scritto anche nel `README.md` del modulo e in `Scripts\Custom\README.md` come convenzione permanente per chi (umano o AI) scrive o corregge codice in questo progetto: mai una chiamata a una cmdlet nativa costruita a memoria, sempre verificata contro la documentazione reale prima di proporla.

12. Avvio dell'applicazione

12.1 Avvio con un click — M365Ops.bat

Nella cartella radice del progetto, `M365Ops.bat` (che richiama `Launch-M365Ops.ps1`) è il modo consigliato per l'uso quotidiano: un doppio click avvia il server e apre il browser sulla chat, senza passare dal terminale.

Non duplica mai la sessione. Prima di avviare qualunque cosa, controlla con una vera chiamata HTTP a `/api/status` se un'istanza è già attiva sulla porta: se sì, si limita ad aprire il browser sulla sessione già in corso (stesso tenant, stessa cronologia chat) invece di avviarne una seconda in conflitto sulla stessa porta o lasciarne una orfana in background. Legge inoltre `Config\last-active-tenant.txt` (scritto da `Connect-M365Ops` ad ogni cambio tenant) per far ripartire il server esattamente sul tenant su cui stavi lavorando l'ultima volta, non su un default fisso.

12.2 Avvio manuale da terminale (facoltativo — non serve per l'uso normale)

Il doppio click su `M365Ops.bat` (12.1) è sufficiente in ogni caso normale: questa sezione esiste solo per chi preferisce vedere l'output in un terminale invece che in una finestra nascosta, o per diagnosticare un avvio che fallisce silenziosamente in modo "con un click".

```
cd <percorso-della-cartella-M365Ops>
.\Launch-M365Ops.ps1
```

È lo stesso identico script richiamato da `M365Ops.bat` (resume-session, rilevamento porta, controllo prerequisiti inclusi) — la sola differenza è che l'output resta visibile nella finestra invece che nascosto. Il server web parte sulla porta configurata (8743 di default, sezione "Aggiornamenti" nel tab Manutenzione per cambiarla), carica automaticamente l'ultimo tenant attivo e avvia la connessione a Lokka. Dal tab Manutenzione della GUI è comunque disponibile un pulsante di riavvio che non richiede di tornare al terminale.

Il riavvio (pulsante GUI o `POST /api/restart`) è l'unico modo sicuro di far ripartire il server: essendo a thread singolo, la richiesta di riavvio viene "eseguita" solo subito dopo aver chiuso la risposta HTTP corrente, quindi non può mai interrompere una scrittura confermata a metà. Terminare il processo dall'esterno (es. Task Manager, `Stop-Process -Force`) non ha questa garanzia: se c'è una richiesta bloccante in corso (es. una scrittura Graph appena confermata con "sì"), un kill esterno può interromperla a metà - un incidente reale del 15/08/2026, poi verificato senza conseguenze solo controllando a mano lo stato del tenant dopo un nuovo login.

12.3 Se il server non risponde affatto — "Termina e riavvia" (22/08/2026)

Il pulsante di riavvio del tab Manutenzione (12.2) funziona solo se il server può ancora rispondere a una richiesta HTTP - se è davvero bloccato (es. un login interattivo Teams/ SharePoint rimasto in attesa, sezione 10.6), nemmeno quel pulsante risponde, perché il server è a thread singolo: una richiesta bloccata blocca anche tutte le altre, incluso il tab Log stesso. Per questo caso specifico, un collegamento **"M365Ops - Termina e riavvia"** viene creato in automatico sul Desktop (stessa icona di M365Ops, accanto al collegamento normale) - termina forzatamente il processo del server (anche se bloccato) e lo riavvia da `M365Ops.bat`, tutto da un doppio click, senza dover aprire Task Manager a mano.

Usalo SOLO come ultima risorsa, quando il pulsante di riavvio normale (12.2) non risponde affatto - non ha la stessa garanzia di "mai a metà di una scrittura" descritta sopra (un kill esterno, con qualunque strumento, può in teoria interrompere una scrittura Graph appena confermata). Se il server risponde ancora anche solo lentamente, preferisci sempre il pulsante di riavvio normale.

Tecnicamente: `Kill-And-Restart-M365Ops.ps1 / .bat` nella cartella dell'app, Windows PowerShell 5.1 (non richiede che `pwsh.exe` funzioni - deve poter terminare `pwsh.exe` anche quando è lui stesso ad essere bloccato). Termina SOLO i processi `pwsh.exe` che eseguono `Gui\Server.ps1` di QUESTA installazione specifica (confronto esplicito del percorso completo) - mai un generico "tutti i pwsh.exe di sistema", che potrebbero appartenere a tutt'altro.

13. Uso quotidiano

Il modulo non copre solo Intune: gestisce Intune/Entra ID (via Graph, direttamente o tramite Lokka) ed Exchange Online (tramite le 50+ cmdlet della sezione 11) nello stesso strumento di chat. Ecco esempi reali per ciascuna area.

13.1 Intune (device e compliance)

- "elenca i dispositivi" / "dispositivi non conformi"
- "analizza i pattern di non conformità" (usa l'AI)
- "esporta dispositivi in excel"

13.2 Entra ID (utenti e gruppi)

- "panoramica utente mario.rossi@contoso.com" — gruppi, dispositivi, app e policy assegnate
- "panoramica gruppo Marketing" — membri, app e policy assegnate al gruppo
- "crea gruppo Vendite-Test con membro mario.rossi@contoso.com" (richiede conferma)
- "trova l'utente Mario Rossi" — risoluzione nome → UPN
- qualunque altra domanda su utenti/gruppi/licenze non coperta sopra passa all'AI, che interroga Microsoft Graph direttamente tramite Lokka (sezione 9)

13.3 Exchange Online

- "permessi delle mailbox condivise" — solo condivise, comando diretto da catalogo
- "tira fuori tutti i permessi delle mailbox" — generico, passa all'AI che usa `Get-M365OpsMailboxDelegatesReport` e copre *tutte* le mailbox
- "esporta report mailbox condivise in pdf" / "esporta report utilizzo mailbox in excel"
- "quali mailbox hanno l'inoltro automatico configurato?" (report sicurezza)
- "crea una mailbox condivisa fatture@contoso.com" (richiede conferma)
- "dai FullAccess su fatture@contoso.com a mario.rossi@contoso.com" (richiede conferma)
- "elenca le regole di trasporto" / "crea una regola che blocca gli allegati .exe" (creata sempre disabilitata, richiede conferma per l'attivazione)
- "quali domini sono accettati dal tenant?"

13.4 Report ed export (trasversali)

Qualunque area sopra supporta l'export diretto in CSV/Excel/PDF e l'invio per email dell'ultimo report generato ("invio per email a nome@dominio.it").

Per richieste generiche ("fammi un report licenze", "report caselle di posta con grafico per dimensione") l'AI usa lo strumento `generate_report`: raccoglie prima i dati REALI del tenant (mai inventati/riassunti a memoria) con `graph_api_call` / `exo_query`, poi genera un file Excel (dati completi) e un PDF con grafici a barre/torta calcolati dal codice via `Group-Object` - mai chiedendo il conteggio all'AI, per evitare che un conteggio sbagliato su tanti record diventi un errore silenzioso in un report. I grafici sono SVG generati internamente, nessuna libreria esterna o dipendenza da internet in fase di stampa. Entrambi i file compaiono in chat come pulsanti di download diretti - non serve più andare a cercarli sul disco, e il testo di risposta non mostra mai il percorso assoluto del file.

13.5 Stato e reset MFA di un utente

- "stato mfa di mario.rossi@contoso.com" / "che metodi di autenticazione ha registrato mario.rossi?" — sola lettura, elenca i metodi registrati (Authenticator, telefono, FIDO2, Windows Hello, app OATH, Temporary Access Pass), mai la password
- "resetta l'mfa di mario.rossi@contoso.com" / "fallo reregistrare l'mfa" — **richiede sempre conferma esplicita** prima di rimuovere qualunque metodo: e' un'azione con impatto immediato (l'utente perde l'accesso ai metodi rimossi finché non li riconfigura). La password non viene mai toccata.

Non esiste in Microsoft Graph v1.0 un singolo flag "MFA abilitata" per utente (dipende da Conditional Access, non da questa API) - lo stato si deduce da quali metodi risultano registrati oltre alla password (verificato su Microsoft Learn, sezione 21). Serve lo scope `UserAuthenticationMethod.ReadWrite.All` **più** un ruolo Entra (Global Reader per la sola lettura; Authentication Administrator o Privileged Authentication Administrator per il reset) - un requisito di ruolo separato dallo scope, un 403 con lo scope corretto ma senza il ruolo è normale, non un bug.

13.6 Memoria della conversazione

Dal 15/08/2026 l'AI ricorda gli scambi precedenti con lo stesso tenant: gli ultimi turni della conversazione sono salvati in locale (`Config\ChatHistory-<tenant>.json`), con le eventuali password redatte prima del salvataggio) e rimandati come contesto ad ogni nuova domanda - una domanda di follow-up ("e per quell'utente creato prima?") funziona senza dover ripetere il contesto da zero. Un refresh della pagina ricarica lo stesso storico (e un'eventuale proposta di scrittura ancora in sospeso), invece di mostrare una chat vuota. Il pulsante **"Nuova conversazione"** (o scrivere "nuova conversazione"/"dimentica tutto" in chat) azzerà lo storico per il tenant attivo, per ripartire puliti su un argomento non correlato.

13.7 Conferma umana obbligatoria — vale sempre, senza eccezioni

Ogni azione di scrittura (creazione, modifica, eliminazione) segue sempre il flusso *proponi* → *conferma* → *esegui*, indipendentemente da:

- **dove avviene la scrittura** — Microsoft Graph (`propose_graph_write`, via Lokka) o Exchange Online (`propose_exo_write`, sezione 11.1): stesso principio, stesso meccanismo di proposta;
- **la modalità di autenticazione del tenant** — AppOnly o Delegated (sezione 10): il gate di conferma vive nel livello chat/AI, non nel livello di autenticazione, quindi si applica identicamente in entrambi i casi.

Le due liste di cmdlet consentite all'AI (lettura ed effettivamente eseguibili subito, vs. scrittura e solo proponibili) sono statiche e senza sovrapposizioni: una cmdlet di scrittura non può mai finire nel percorso di sola lettura per errore. Verificato anche con un test dal vivo: una richiesta di scrittura (creazione di un contatto) si è fermata alla proposta, un messaggio successivo inviato con la proposta ancora in sospeso non l'ha bypassata, e il rifiuto esplicito ("no") l'ha annullata senza mai chiamare la cmdlet reale.

Una sola proposta di scrittura per risposta. Bug reale osservato il 15/08/2026: l'AI aveva proposto "crea utente" e, nella stessa risposta, subito dopo "assegna licenza" - la seconda proposta sovrascriveva silenziosamente la prima, l'utente confermava pensando di approvare entrambe ma solo l'ultima era davvero in sospeso. Ora una seconda proposta nella stessa risposta viene rifiutata dallo strumento stesso: per un piano a più passaggi (es. crea utente POI assegna licenza), l'AI propone solo il primo passo e si ferma, mostrando un indicatore **"Passo 1 di 2"** nel testo della conferma - il passo successivo viene proposto solo in un messaggio separato, dopo che il primo è stato confermato ed eseguito.

13.8 Quando manca uno strumento: l'AI può proporre di crearne uno

Se chiedi qualcosa di ricorrente che nessuno strumento esistente copre (un report specifico di questo tenant, un'estrazione ad hoc), l'AI può proporre di scrivere un nuovo script permanente invece di limitarsi a spiegarti come farlo a mano — vedi sezione 17.6 per i dettagli completi (conferma sempre col codice per intero, validazioni automatiche, riavvio per caricarlo).

14. Sicurezza — principi non negoziabili

1. **Zero segreti su disco.** `tenants.json` contiene solo ID, thumbprint (pubblico) e nomi di variabili d'ambiente — mai client secret o API key in chiaro.
2. **Propose → confirm → execute.** Nessuna scrittura contro Graph o Exchange avviene senza conferma umana esplicita in chat.
3. **Le cmdlet di scrittura creano senza assegnare/attivare per default** — l'assegnazione è sempre un passo separato ed esplicito.
4. **Le cmdlet di lettura non interpretano mai i dati** — l'interpretazione passa sempre dal motore AI, mai da regole cablate nel codice.
5. **Isolamento multi-tenant** — credenziali, certificati e connettori MCP di un tenant non sono mai visibili o raggiungibili da un altro profilo.

15. Stato attuale del progetto

Area	Stato	Note
Graph app-only (Intune/Entra ID)	funzionante	Device, gruppi, app, profili di configurazione
Lokka MCP	funzionante	Tool primario per l'AI, tenant-scoped
Export report CSV/XLSX/PDF	funzionante	PDF via Microsoft Edge headless
Invio report via email	funzionante	Graph Mail.Send , testato end-to-end
Exchange Online — connessione app-only (certificato)	funzionante	Verificato dal vivo dopo completamento RBAC (sezione 6)
Exchange Online — 50+ cmdlet (sezione 11)	funzionante	Mailbox, gruppi, mail flow, tracking, contatti, risorse, report — testate dal vivo sul tenant
Autenticazione delegata (utente + MFA)	funzionante	Device code flow verificato dal vivo; per tenant senza App Registration (sezione 10)
Tool-calling AI — Claude	funzionante	Loop tool-calling completo, con <code>exo_query / propose_exo_write</code>
Tool-calling AI — Azure OpenAI	funzionante	Stesso loop/stessi strumenti di Claude su Chat Completions API (sezione 8.1/8.2), scelta in GUI persisti
Persistenza scelta provider AI	funzionante	<code>Config\ai-provider.json</code> , sopravvive al riavvio (sezione 10.8)
Server MCP CLI Microsoft 365	collegato	Copre Entra/Outlook/Planner/OneDrive/Purview (audit/compliance search)/Teams lato Graph, PIU' Sh Exchange profondo, Intune, criteri Teams. Prerequisito (<code>npm install -g @pnp/cli-microsoft365</code> (certificato se configurato, altrimenti client secret) sia Delegato (login a browser bloccante, un click esp solo configurazione). Processo isolato per ogni tenant, resta vivo in background al cambio profilo
Altri server MCP oltre Lokka e CLI Microsoft 365	salvabili ma non collegati	La GUI permette di registrarli/rimuoverli; l'infrastruttura di connessione (Connect-M365OpsMcpServer per CLI Microsoft 365
Rimozione server MCP da GUI	funzionante	Pulsante "Rimuovi" per riga, testato dal vivo (aggiunto e rimosso)
Creazione batch di migrazione	funzionante	Solo su endpoint già configurati (sezione 11.3), creato sempre non avviato — testato dal vivo, endpoir
Script personalizzati (Scripts\Custom)	funzionante	Scoperta automatica, testato dal vivo con uno script reale (sezione 17)
Controllo stato ambiente / portabilità	funzionante	<code>Get-M365OpsSetupStatus</code> + banner GUI, testato dal vivo (sezione 18)
Log persistente delle azioni	funzionante	Un file al giorno in <code>Logs\</code> + visualizzatore nel tab Manutenzione (sezione 19)
Blocco server su login Exchange delegato implicito	corretto	Bug reale riscontrato in uso e risolto stesso giorno — vedi sezione 10.6
Copertura parametri Exchange/Graph (ExtraParams + lookup_ms_docs)	funzionante	Ogni cmdlet di scrittura accetta parametri nativi aggiuntivi verificati dal vivo su Microsoft Learn (sezior
Login Exchange delegato non bloccante (device code)	corretto	Sostituito il flusso bloccante con lo stesso schema del login Graph — codice mostrato in GUI, mai più
graph_api_call su tenant delegati	corretto	Bug reale: era escluso del tutto, ora chiama Graph direttamente quando Lokka non è disponibile (sezi
Pannello stato connessioni + Exchange/Intune sotto Tenant	funzionante	Indicatori chiari Graph/Exchange/Intune/Lokka per entrambe le modalità (sezione 10.7)
Stato/contatore chiamate motore AI	funzionante	Contatore per provider + test di connessione dal vivo (sezione 20)
Connessione Intune su tenant Delegati	funzionante	<code>-Interactive</code> (non <code>-DeviceCode</code> , bug reale nel modulo terzo), verifica dal vivo del token (sezion
Stato/reset MFA utente	funzionante	Reset sempre con conferma esplicita, testato dal vivo (sezione 13.5, 21)
Report AI-orchestrati con grafici (Excel + PDF)	funzionante	<code>generate_report</code> , grafici SVG generati internamente, download diretto da GUI (sezione 13.4, 22)
Avvio con un click (M365Ops.bat)	funzionante	Resume-session su porta già attiva, tenant ripreso da <code>last-active-tenant.txt</code> (sezione 12.1)
Indicatore di avanzamento onesto in GUI	corretto	Bug reale: messaggio statico "sto elaborando" indistinguibile da un blocco reale — ora un contatore d
Memoria della conversazione	funzionante	Storico persistito per tenant, ricaricato al refresh pagina, azzerabile a comando (sezione 13.6, 23)
Una sola proposta di scrittura per risposta + indicatore "passo X di N"	corretto	Bug reale: due proposte nella stessa risposta si sovrascrivevano in silenzio (sezione 13.7)
Message trace su Get-MessageTraceV2	corretto	Le cmdlet classiche sono in dismissione dal 1/09/2025, verificato su Microsoft Learn (sezione 11)
Ricerca registro di controllo unificato (Purview <code>Search-UnifiedAuditLog</code>)	non costruito	Tecnicamente fattibile (cmdlet reale, richiede licenza/ruolo Audit Log) ma non ancora implementato —
Creazione di nuovi script da parte dell'AI	funzionante	<code>propose_new_custom_script</code> , conferma con codice completo sempre obbligatoria, validazione sin
Bug: risposta AI vuota su richieste complesse	corretto	Budget token 2000→4000 + rilevamento e messaggio esplicito invece di un messaggio bianco silenzic
Bug: crash su array vuoto serializzato in JSON	corretto	<code>ConvertTo-Json</code> pipelato su un array vuoto restituisce <code>\$null</code> vero, non <code>[]</code> - causava un errore su
Sicurezza email (antispam, threat policy, allow/block list, quarantena+rilascio)	funzionante	Nuove cmdlet lette/scritte via <code>exo_query / propose_exo_write</code> , stessa connessione Exchange già
Message trace oltre i 10 giorni	funzionante	Incatenamento automatico delle finestre da 10 giorni (sezione 17.7) - bug reale scoperto e corretto lo

Export diretto senza passare dati grezzi nella conversazione	funzionante	<code>generate_raw_export</code> : query+scrittura file in un solo passaggio lato server, per volumi che altrimenti (sezione 17.7)
Controllo prerequisiti con installazione automatica	funzionante	PowerShell 7 controllato/installato da <code>M365Ops.bat</code> via winget prima dell'avvio; Node.js/Edge installati
Invio report via email dalla conversazione libera	funzionante	<code>propose_send_report_email</code> , conferma sempre richiesta, testo email scritto dall'IA con overview r
SharePoint/OneDrive (siti, condivisione esterna, storage, permessi, account orfani)	funzionante	Verificato dal vivo il 17/08/2026 dopo la concessione del permesso SharePoint (sezione 4.3) - 44 siti, 1
Bug: URL centro amministrazione SharePoint non risolvibile su TenantId a GUID	corretto	Derivazione assumeva sempre il formato dominio - ora risolve il dominio iniziale verificato via Graph
Microsoft Teams (elenco/canali/membri, criteri, accesso esterno)	funzionante	Elenco/canali/membri verificati dal vivo (14 Team reali); criteri e accesso esterno costruiti, in attesa del
Bug: richiesta AI rifiutata per un messaggio con 'role' nullo in un punto imprecisato	mitigato	Filtro difensivo sull'array messages appena prima di ogni chiamata (sia Azure sia Claude) - una voce m
Bug: login a codice dispositivo (Exchange/Graph) mostrava "nessun login in corso" dopo un successo reale	corretto	Race condition da <code>setInterval</code> - due poll potevano sovrapporsi, il secondo trovava lo stato appen
Bug: login Delegato SharePoint/Teams falliva sempre (AADSTS700016, client_id indovinato)	corretto	Il flusso a codice dispositivo custom per SharePoint/Teams richiedeva un client_id OAuth verificato cor
Bug: pulsante "Connetti SharePoint/Teams" continuava a rifiutare il login anche dopo il fix precedente	corretto	<code>\$script:M365OpsContext</code> letto direttamente in Server.ps1 (fuori dal modulo) è sempre vuoto - ste
Testo: riferimento al tab sbagliato ("MCP/Connettori" invece di "Tenant") in più messaggi di errore	corretto	La sezione "Stato connessioni" (Exchange/SharePoint/Teams/Intune) è nel tab Tenant dalla riorganizza
App Registration dedicata per il login SharePoint interattivo (discovery automatico)	funzionante	<code>Sync-M365OpsSharePointAppRegistration</code> : scopre via Graph se l'app esiste già nel tenant e ne s
Bug: catalogo comandi deterministico intercettava richieste piene ("dispositivi"/"non conform" come sottostringa)	corretto	Trigger troppo generici (<code>ListDevices</code> , <code>ListNonCompliant</code> in <code>CommandCatalog.ps1</code>) rispondev
Scrittura SharePoint (creazione sito, membri, ereditarietà permessi, sharing, quota)	funzionante	5 cmdlet nuove via PnP.PowerShell, stesso meccanismo di proposta/conferma del resto del modulo - v
Scrittura Teams (crea/modifica/rimuovi Team, assegna policy)	funzionante	4 cmdlet nuove - creazione Team verificata dal vivo (via Graph); assegnazione policy verificata fino al li
Bug: catalogo intercettava anche richieste composte di export ("scarica...dispositivi...e invialo a...")	corretto	Stesso schema del bug sopra su <code>ExportDevices</code> : un report con colonne personalizzate + invio ema
Bug: report generato dal catalogo non trovato da un invio email richiesto in un turno successivo	corretto	<code>\$script:LastReportPath</code> impostato da Server.ps1 (catalogo) non è visibile al modulo (dove vive
Bug: trigger catalogo troppo generici trovati in un secondo batch di test (MfaStatus, assegnazione app)	corretto	<code>MfaStatus</code> ('metodi di autenticazione' da solo) e il rilevamento 'assegna...gruppo' in Server.ps1 (per
Bug: due trigger latenti del catalogo trovati per ispezione proattiva (mai innescati dal vivo)	corretto	<code>CompliancePatterns</code> ('pattern' da solo) ed <code>ExportCompliancePatterns</code> ('esporta.*analisi' senz
Packaging app custom da script (.ps1/.bat/.cmd), oltre a .exe/.msi	funzionante	<code>New-M365OpsWin32App</code> era già agnostico al tipo di file (nessuna modifica al packaging .intunewin ir
BUG SERIO: <code>Get-M365OpsManagedDevices -NonCompliantOnly</code> restituiva i dispositivi SBAGLIATI	corretto	Il filtro server-side OData <code>\$filter=complianceState ne 'compliant'</code> su <code>/deviceManagement/</code>
Bug: 5 ulteriori trigger del catalogo troppo generici, trovati con un giro di stress-test mirato	corretto	Stesso schema ricorrente (18/08/2026): <code>ExportMailFlowReport</code> / <code>ExportForwardingReport</code> / <code>Ex</code>
Bug: <code>Set-PnPWeb -BreakRoleInheritance</code> / <code>-ResetRoleInheritance</code> non esistono in PnP.PowerShell 3.4.0	corretto	Rimossi rispetto a versioni precedenti della libreria. Sostituiti con la REST API diretta (<code>Invoke-PnPSP</code>
Nota: il bug di serializzazione MicrosoftTeams (sezione 17.11.1) colpisce anche <code>Set-Team</code> , non solo l'assegnazione policy	limite noto	Trovato durante il bug-hunting del 18/08/2026: lo stesso errore "Dictionary...Keys must be strings" di
Nuovo: <code>Invoke-M365OpsProvisioningRecipientDiagnostic</code> - diagnosi provisioning mailbox	funzionante	Diagnostica in un colpo solo i problemi dietro NDR 550 5.1.10/mailbox introvabile/mailbox che non si

		ExchangeGuid di default - senza -Properties ExchangeGuid esplicito, il confronto ExchangeGu nessun errore. Verificato dal vivo tramite chat sia su una mailbox reale coerente (nessuna anomalia) sic
Nuovo: Get-M365OpsMoveRequestDiagnostic - diagnosi dettagliata migrazione mailbox	funzionante	Stesso livello di dettaglio di Get-MoveRequestStatistics -IncludeReport -DiagnosticInfo " MigrationUserStatistics (stessi parametri) per gli utenti di un batch di onboarding creato con N l'oggetto diagnostico ORIGINALE completo (non solo i campi riassunti), utile per un ticket Microsoft S
Nuovo: workaround OneDrive "accesso negato nonostante lo sharing sia corretto"	funzionante	3 cmdlet che coprono i passi automatizzabili del workaround noto Microsoft (concedi/revoca accesso M365OpsOneDriveDelegateAccess , Remove-M365OpsOneDriveSharingRecipient , Revoke-M36 MANUALE per design (richiede un browser interattivo) - il primo cmdlet restituisce l'URL diretto pront
BUG SERIO trovato dal vivo nel primo test end-to-end del workaround sopra	corretto	Remove-M365OpsOneDriveSharingRecipient non distingueva un permesso "owner" (che rappres sono sempre "read"/"write") da una vera condivisione. Se il destinatario da rimuovere coincideva con l collection admin - la cancellazione non aveva pero' alcun effetto reale (e' un permesso calcolato, non senza aver rimosso nulla di vero. Corretto escludendo sempre i permessi "owner" dalla scansione/rimc
Nuovo: retention policy Purview (Get-M365OpsRetentionCompliancePolicies)	in attesa di permesso	Connect-M365opsCompliance (Connect-IPPSession, stesso certificato di Exchange) verificato dal v tenant di test - Connect-IPPSession espone solo i cmdlet coperti dai ruoli gia' assegnati (oggi so esplicito. L'AI spiega gia' correttamente la causa e rimanda alla sezione 4.5 - verificato dal vivo tramite
Nuovo: Knowledge Base per tenant (tab "Documentazione")	funzionante	Upload .txt/.md/.docx/.xlsx/.xls/.pdf (Pdflexer per i PDF, installato automaticamente al primo uso) dal 1 leggero) resta sempre nel prompt di sistema senza chiamate AI aggiuntive; il testo completo si recupe contoso-test con un "codice segreto" di prova, l'AI lo ha recuperato e citato correttamente SOLO su q di confondere i due tenant. kb_query non accetta ne' espone un parametro tenant: legge sempre e clienti, non solo una convenzione
Bug reale trovato dal vivo testando la cancellazione di un documento dalla Knowledge Base	corretto	Stessa classe di bug ConvertTo-Json gia' vista altrove nel progetto (array a un elemento appiattito " \$catalogVuoto ConvertTo-Json Set-Content " non emetteva alcun oggetto in pipeline - errori. Corretto usando ConvertTo-Json -InputObject (mai in pipeline) in Add-/Remove-M365Op
Nuovo: pubblicazione su GitHub e aggiornamento in-app (canale Stable/Testing)	funzionante	Repository pubblicato (sorgente sanificato, nessun dato del tenant reale), tab Manutenzione mostra la versione da M3650ps .psd1 , cambio canale persistito, chiamata reale a api.github.com (nessuna
BUG reale: repository pubblicato ma lasciato Privato per errore	corretto	Con un repository Privato, GitHub risponde 404 (non un errore di permesso esplicito) a QUALUNQUE l'app, non solo per il proprietario del repo). L'app mostrava quindi "nessuna Release pubblicata" anche (stesso modo in cui lo interroga l'app) - risolto rendendo il repository Pubblico da Settings → Danger
Nuovo: prima Release pubblicata (v0.1.0)	funzionante	Baseline pubblica, non pre-release (canale Stable). Verificato dal vivo che l'app la riconosca come versi
Nuovo: Get-M365OpsLogHistory - ricerca log su piu' giorni dalla GUI	funzionante	Il tab Manutenzione mostrava solo il log DI OGGI (ultime 200 righe) nonostante lo storico su disco no gia' oggi ben oltre 6 mesi). Aggiunta una vera ricerca multi-giorno con filtri periodo (oggi/7/30/183/3 (sezione 17.11.1), ricerca testo "kb_query" trova le 4 righe corrette
Nuovo: timestamp sui messaggi in chat	funzionante	Ogni messaggio mostra ora data/ora (formato it-IT) - salvato in Add-M3650psChatHistoryTurn , m
Nuovo: Remove-M3650psTenant - rimuovi un profilo tenant dal tab Tenant	funzionante	Rimuove SOLO la registrazione in tenants.json (Tenant ID, Client ID, riferimento al secret) - non to rimozione del tenant ATTUALMENTE ATTIVO (pulsante disabilitato in GUI, controllo ripetuto lato serve (nessuna scrittura), rimozione di un profilo non attivo confermata, integrita' degli altri profili reali verif
Giro di test dal vivo su tutto il lavoro della giornata (18/08/2026)	completato	Install-M3650psUpdate eseguito per la PRIMA volta per davvero (mai testato prima): copia isolata: nuovi siano corretti e che nessuna cartella dati (Config/Logs/Uploads/Reports/Testing) sia stata toccat attivo, rimozione pulita, integrita' dei profili reali confermata). Ricerca log testata su tutti i filtri incrocia
BUG reale trovato dal vivo: Get-M3650psLogHistory , periodo "Oggi" includeva anche ieri	corretto	Off-by-one nel calcolo della soglia: oggi.AddDays(-Days) con Days=1 produceva la mezzanotte (verificato dal vivo: la ricerca con livello Error e periodo "Oggi" restituiva errori del 17/08, non solo del Corretto e riverificato dal vivo su tutti i periodi (oggi/7/30 giorni)
BUG reale trovato dal vivo: timestamp chat, lo storico precedente alla funzione mostrava un orario falso	corretto	Segnalato dall'utente con uno screenshot: messaggi con orario 15:53 comparivano PRIMA di un mess; storico salvate PRIMA dell'introduzione del timestamp (senza quel campo) usavano "adesso" come rip distinguendo undefined (messaggio live, "adesso" e' corretto) da null esplicito (voce di storico s
Giro di stress test "hard" su tutti i servizi (18/08/2026): 45 prompt reali su Exchange/Entra ID/Intune/Teams/SharePoint/Purview	completato	Batch di richieste realistiche (dirette, composte, con typo, ambigue) eseguito dal vivo sul tenant di test nel livello AI (sotto) + verifica reale di scritture (mailbox condivisa, gruppo di distribuzione, creati e rip
BUG SERIO trovato dal vivo: EmailLastReport inviava il report SBAGLIATO se il messaggio nominava un argomento diverso dall'ultimo generato	corretto	"mandami via mail un report dei dispositivi non conformi a X" dopo aver appena generato un report E \$script:LastReportPath . Corretto deviando all'AI (che ha il contesto per generare il report giuste riferimenti generici ("inviolato", "mandalo"). Riverificato dal vivo: ora genera e propone il report corretto
BUG SERIO trovato dal vivo, stessa sessione di test: ListNonCompliant ignorava silenziosamente "mandalo via mail" in una richiesta composta	corretto	Corretto il bug sopra, la stessa richiesta cadeva SUBITO dopo su questa voce del catalogo, che mostra DeferWords: una richiesta che nomina anche l'invio passa ora all'AI, che gestisce l'intera richiesta comp
BUG trovato dal vivo: trigger "Help" non deviava un problema reale con verbi coniugati diversamente dai DeferWords letterali	corretto	"aiuto, un dipendente non riesce piu' ad ACCEDERE alla mail" (terza persona, verbo "accedere") non ve l'elenco comandi invece di una risposta AI al problema reale. Corretto con pattern piu' ampi ('access\w
BUG SERIO trovato dal vivo: teams_query non aveva MAI avuto un case nello switch di dispatch	corretto	Lo strumento e' dichiarato nello schema mostrato all'AI (che lo chiama per davvero, log "Tool AI chiar riportava all'utente un fuorviante "strumento non disponibile" per Get-M3650psTeamsPolicies / G (list/canali/membri restano invisibili al problema perche' l'AI le recupera quasi sempre via graph_api dal vivo. Aggiunto il case mancante (stesso pattern di sharepoint_query/compliance_query) - verificatc
BUG trovato in combinazione col precedente: Get-M3650psTeamsExternalAccessConfig senza -ErrorAction Stop	corretto	Stessa classe di bug gia' corretta il 17/08/2026 in Get-M3650psTeamsPolicies per lo stesso identic che produce un array vuoto (\$null) invece di un'eccezione catturabile. Corretto sui 4 cmdlet Cs* de

file:///C:/Users/diego_o6s3y1g/Documents/claude/M365Ops/docs/Guida-Configurazione.html 31/82

Limite ambientale confermato (non nuovo, ri-osservato 2 volte in piu' con parametri diversi): <code>New-M365OpsTenantAllowBlockListSpoofItem</code> con <code>SpoofType: External</code> fallisce sempre con "A server side error has occurred" su <code>vnsys-test</code>	limite noto	Riprodotta altre due volte con valori diversi (indirizzo email completo e poi solo dominio come <code>Spoo</code> Sommato al primo caso già documentato (18/08/2026), questo cmdlet non ha mai completato con su copiato dalla documentazione ufficiale fallisce), probabile gate di licenza/servizio lato Microsoft (es. <code>pi</code> <code>Stop</code> continua a impedire correttamente un falso successo
Nota informativa (non un bug): il tenant <code>vnsys-test</code> risulta rinominato in passato da "vnsysit" a "vnsys"	solo informativo	Trovato per caso verificando un'apparente discrepanza: <code>Get-M365OpsAcceptedDomains</code> (Exchange) comunque un indirizzo di posta valido (usato con successo in questo stesso giro di test) perché è il d <code>vnsys.it</code> . <code>Get-M365OpsAcceptedDomains</code> non ha un bug: riflette fedelmente l'elenco Exchange
Giro di stress test "hard" su Intune (19/08/2026), stesso metodo già usato su Exchange: prompt reali su dispositivi/compliance/pacchettizzazione app, più verifica di coerenza sui flussi di collegamento	completato	10 richieste reali su <code>vnsys-test</code> (letture + scritture con conferma reale, oggetti <code>StressTest-/StressTest2-</code> dall'utente durante lo stesso giro di lavoro (sotto) + una nuova funzionalità richiesta a seguito di quel "errore non terminante ignorato in silenzio" già visto su Exchange: <code>Invoke-M365OpsGraphRequest</code>
BUG SERIO x4 trovati dal vivo: trigger del catalogo <code>ListDevices / CompliancePatterns / UserOverview / GroupOverview</code> ignoravano silenziosamente la seconda metà di una richiesta composta	corretto	Stesso schema già visto 4 volte in questo progetto (EmailLastReport/ListNonCompliant/Help/MfaStat "panoramica utente X e poi elenca i gruppi di distribuzione" eseguiva solo la prima parte. <code>GroupOver</code> come nome del gruppo invece di fermarsi al nome vero. Aggiunti gli stessi <code>DeferWords</code> a tutti e 4; rive
BUG SERIO trovato IN PRODUZIONE dall'utente durante questo stesso giro di lavoro: "pacchettizza e distribuisci l'app GIT, crea un gruppo X e assegnalo" ha eseguito SOLO la creazione del gruppo, fallendo poi in modo confuso all'assegnazione ("nessuna app disponibile")	corretto	Causa root radicalmente diversa da quanto sembrava all'inizio (non un problema di tool AI ne' di stat sottostringa - "pacchett" (doppia T) - per riconoscere l'intento di pacchettizzazione, ma il messagg l'intento, <code>\$hasPackageIntent</code> è risultato falso, e il passo di pacchettizzazione è stato interamente Corretto rendendo tollerante il conteggio delle T (<code>pacchet{1,2}izz / impacchet{1,2}</code>) in entram generale, come rete di sicurezza per frasi che non contengono nessuna di queste sottostringhe: se le v parti della richiesta. Riverificato dal vivo end-to-end con la stessa identica grafia ("pacchettizza"): seque
Nuovo, richiesto dall'utente a seguito del bug sopra: detection di uno script (<code>ps1/.bat/.cmd</code>) dedotta dall'AI dal contenuto, prima di chiedere sempre all'operatore	funzionante	<code>Get-M365OpsScriptDetectionSuggestion</code> (nuovo) legge il contenuto dello script caricato e chie mai al posto di chiederglielo: se l'AI non è ragionevolmente sicura, si torna a chiedere come sempre f nota "dedotto dall'AI dal contenuto dello script... verificalo", stesso principio già in uso per la detecti correttamente il percorso esatto, mostrato per conferma) e uno script senza alcun segnale verificabile
BUG SERIO trovato IN PRODUZIONE dall'utente lo stesso giorno: "pacchettizza e distribuisci l'app GIT, crea un gruppo X e assegnalo" ha eseguito SOLO la creazione del gruppo, fallendo poi all'assegnazione ("nessuna app disponibile")	corretto	Causa root: il rilevatore di richieste composte in <code>Gui\Server.ps1</code> riconosceva l'intento di pacchettiz battitura comune in italiano) - <code>\$hasPackageIntent</code> risultava falso e il passo di pacchettizzazione ve (<code>pacchet{1,2}izz / impacchet{1,2}</code>) in entrambi i punti del codice che replicavano lo stesso cor nessuna di queste sottostringhe. Riverificato dal vivo: sequenza a 3 passi (gruppo+pacchettizzazione+
Nuovo: assegnazione app Intune con parametri avanzati (filtro assegnazione, notifiche, riavvio, disponibilità/scadenza, priorità banda) - richiesto esplicitamente dall'utente	funzionante	<code>Set-M365OpsAppAssignment</code> gestiva prima SOLO intent+target (2 valori su 4 di intent reali: manca <code>NotificationMode</code> , <code>RestartGracePeriodMinutes / RestartCountdownDisplayMinutes / Res</code> <code>Graph (win32LobAppAssignmentSettings , deviceAndAppManagementAssignmentFilterType)</code> <code>propose_graph_write</code> diretto, non passa da questa cmdlet) ha ricevuto lo schema JSON verificato
Nuovo: personalizzazione avanzata del packaging Win32 (requisiti hardware, install experience, comportamento riavvio, tempo massimo, disinstallazione da available, return code, scope tag, dipendenze, supersedence) - richiesto esplicitamente dall'utente	funzionante	<code>New-M365OpsWin32App</code> esteso con tutti questi parametri (schema verificato dal vivo sia lato Graph s <code>IntuneWin32AppReturnCode / Add-IntuneWin32AppDependency / Add-IntuneWin32AppSuperse</code> lettura Graph in <code>Execute-PendingAction</code> , con errore chiaro se il nome non trova esattamente 1 co libero dell'utente (mai inventate se non menzionate) - se l'utente non ne menziona nessuna, il testo di personalizzazioni (promemoria mostrato correttamente), caso con 5 personalizzazioni diverse in una f
BUG SERIO trovato dal vivo verificando la funzionalità sopra: un'app Intune REALE (quella "Git" dell'utente) è stata pacchettizzata con il payload SBAGLIATO (Audacity al posto di Git), in silenzio, senza alcun errore	corretto	Causa: <code>New-M365OpsWin32App</code> usava cartelle di lavoro FISSE e mai svuotate (<code>%TEMP%\M365OpsPack</code> <code>.intunewin</code> accumulati in <code>output</code> da run precedenti (risalenti al 14/08/2026), <code>Get-ChildItem \$</code> <code>*.intunewin</code>), MAI necessariamente quello appena generato. Scoperto rileggendo <code>setUpFilePath</code> c reale confermato: l'app "Git" creata in precedenza dall'utente stesso nella tenant di test aveva daver Corretto rendendo la cartella di lavoro univoca per OGNI chiamata (sottocartella con un GUID, elimina strutturalmente impossibile. Riverificato dal vivo dopo il fix, con i file "trappola" ancora presenti in <code>ou</code>
BUG trovato dal vivo nello stesso giro di verifica: il nome app richiesto esplicitamente ("chiamalo X") veniva sempre ignorato, l'app prendeva sempre il nome dai metadati dell'installer o dal nome del file	corretto	Nessuno dei due punti del codice che calcolano il nome dell'app (flusso a coda e flusso singolo) legge nome"/"con nome" ora hanno priorità sui metadati quando presenti nel messaggio. Riverificato dal vi
Nuovo: implementate TUTTE le aree Intune segnalate come "non implementato" nel censimento del 19/08/2026 (sezione 17.16) - Settings Catalog, Endpoint Security, Security Baseline, Autopilot, script Windows/macOS, Proactive Remediation, App Protection/MAM, anelli di aggiornamento, Modelli amministrativi, Scope Tag, restrizioni di iscrizione, modelli di notifica, ruoli RBAC personalizzati - richiesto esplicitamente dall'utente ("implementa tutta questa roba")	funzionante	~60 nuovi cmdlet in <code>Public\</code> , ognuno con schema Graph verificato dal vivo su Microsoft Learn prim <code>groupPolicyConfiguration</code> esistono solo in beta - i cmdlet corrispondenti usano <code>-Beta</code> di cons meccanismo query/proposta-scrittura di <code>exo_query / propose_exo_write</code> , con allowlist dedicate <code>Gui\Server.ps1</code> - percorso SEPARATO dal flusso dedicato <code>PackageApp/AssignApp</code> già esistente pe import del modulo): lettura Settings Catalog riuscita con dati reali (2 criteri restituiti correttamente), le configurato, risposta corretta senza errori), e l'intera pipeline di scrittura proposta→conferma→esecuzi mancante (non un bug), riportato all'utente in modo chiaro e distinto da un errore di codice. Tre aree <code>DeviceManagementRBAC.ReadWrite.All / DeviceManagementServiceConfig.ReadWrite.All</code>
BUG trovato durante il syntax-check del nuovo codice: <code>"\$Identity:"</code> dentro una stringa fra doppi apici viene interpretato da PowerShell come variabile con scope/drive (<code>\$Identity:</code>), non come variabile seguita da due punti letterali	corretto	Errore di parsing ("': was not followed by a valid variable name character") su due dei nuovi cmdlet (<code>\$</code> "Configurazione <code>\$Identity: priorit</code> ..." in un messaggio di conferma - corretto con <code>`\${I</code> (nessun'altra occorrenza trovata) prima di considerare il fix completo
Stress test massiccio pre-commit (19/08/2026) sulla seconda ondata Intune - prompt composti, casi limite, cicli scrittura reali con conferma, richiesta esplicita dell'utente	completato	~15 richieste reali su <code>vnsys-test</code> : letture composte multi-area, un piano a 2 scritture con fallimento par stessa risposta (correttamente rifiutato). Ha portato a 3 correzioni, la terza delle quali seria - vedi le 3 v
BUG SERIO trovato dal vivo: <code>ListNonCompliant</code> (catalogo deterministico) troncava silenziosamente la seconda metà di una richiesta composta nonostante contenesse "e poi"/"e anche"	corretto	Stesso schema già corretto 8 volte in questo progetto (EmailLastReport/ListNonCompliant/Help/MfaStat erano già state estese con parole SPECIFICHE (piano/remediation/mail) in un fix precedente del 18/08 anche"/quindi) aggiunte alle voci gemelle lo stesso giorno. "controlla i dispositivi non conformi e poi dispositivi. Corretto aggiungendo le 3 parole generiche; riverificato dal vivo con la stessa identica richi

BUG reale trovato dal vivo: <code>New-M365OpsNotificationTemplate</code> falliva sempre con 400 "An error has occurred" (nessun dettaglio dal servizio Intune) alla creazione	corretto	La diagnosi automatica dell'app (Invoke-M365OpsErrorTriage) ha proposto DUE correzioni sbagliate d lo schema Graph già fetchato dal vivo (che smentiva chiaramente entrambe le ipotesi). Isolata la causa: non è scrivibile in creazione a QUALUNQUE valore/formato (provato "en-us"/"en-US"/"it-IT", falliti ide <code>isDefault=true</code> (già) il comportamento della funzione per il messaggio stesso, semplicemente non rilettura con defaultLocale popolato correttamente, eliminazione) - lezione: non fidarsi della diagnosi umano
BUG GRAVE trovato dal vivo: l'AI ha dichiarato di aver proposto ed eseguito con successo l'eliminazione di un oggetto Intune reale, inventando un JSON di risultato plausibile, SENZA MAI chiamare alcuno strumento - l'oggetto non è mai stato toccato	mitigato	Al messaggio "elimina il modello di notifica X" l'AI ha risposto "Ho proposto l'eliminazione... in attesa di risultato inventato ma identico nella forma a un successo reale, ancora senza alcuna chiamata. Il codice azione era davvero in sospeso, quindi nessuna esecuzione è avvenuta" - il bug è stato puramente nel proporre/conferma reali senza davvero richiamare gli strumenti. Confermato l'impatto rileggendo l'oggi CRITICA ASSOLUTA nel prompt di sistema (generica, vale per ogni <code>propose_*</code> , non solo Intune): viet ("si") ma nel contesto non risulta una vera chiamata <code>propose_*</code> nel turno immediatamente precedente: fix: l'AI ha ri-verificato l'oggetto su Graph, registrato una proposta REALE (confermata da <code>/api/status</code> comportamento probabilistico del modello (non un bug deterministico nel codice), la mitigazione riduce distruttiva importante con una lettura indipendente quando il dubbio è lecito
Audit completo delle tabelle permessi (19/08/2026), richiesto esplicitamente dall'utente dopo aver notato che la verifica precedente copriva solo Intune ("parlo di tutto non solo di Intune")	corretto	La tabella 4.2 (permessi AppOnly, quella seguita da chi configura il modulo) mancava 9 righe già prese MFA, lettura siti SharePoint via Graph diretto, elenco App Registration, Conditional Access, ruoli Entra (<code>Reports.Read.All</code> per i report di utilizzo Microsoft 365, <code>IdentityRiskUser.Read.All</code> per i siti la tabella 10.8 elencava per questo secondo permesso il nome sbagliato (<code>SecurityEvents.Read.All</code> Microsoft Learn. Aggiunte le 9 righe mancanti a 4.2 e corretto il nome in 10.8
BUG reale trovato dal vivo dall'utente: una domanda di audit sicurezza ("ci sono stati accessi sospetti nelle ultime settimane?") ha ricevuto un'analisi basata SOLO sui primi 100 sign-in, con <code>@odata.nextLink</code> presente ma mai seguito - risposta presentata come valida invece che come campione parziale	corretto	Nessuno strumento/istruzione diceva mai all'AI di seguire la paginazione Graph quando la domanda è Verificato dal vivo prima di scrivere il fix che la capacità di base funziona già: passare il valore di <code>@odata.nextLink</code> completo di <code>\$skiptoken</code> , nessuna modifica necessaria). Aggiunta una regola esplicita alla descrizione <code>@odata.nextLink</code> fino a un tetto di sicurezza di 10 pagine, dichiarando esplicitamente un eventuale message trace Exchange. Riverificato dal vivo con la stessa identica domanda dell'utente: 2 chiamate completamente... 2 pagine, senza ulteriori nextLink" invece di analizzare solo un campione parziale
Nuovo (v0.9.3, 19/08/2026): avvio (<code>Launch-M365Ops.ps1</code>) reso robusto - i fallimenti ora sono visibili invece di silenziosi	funzionante	Lo script gira sempre <code>-WindowStyle Hidden</code> (lanciato da <code>M365Ops.bat</code>): fino ad ora un fallimento doppio click sia per un'automazione che lo invoca senza attendere il completamento - osservato dal background tramite <code>cmd /c M365Ops.bat</code> non ha mai avviato davvero il server (nessun errore visit al suo completamento. Aggiunto <code>Config\last-start-status.txt</code> (esito+timestamp scritto ad ogni fallimento reali (<code>pwsh</code> non trovato, server non pronto entro il timeout) - il controllo prerequisiti (Node invece di sparire in una finestra nascosta. Verificato dal vivo: percorso di riuso di un'istanza già attiva
Bug-hunt mirato (v0.9.4-0.9.5, 19/08/2026): 21 cmdlet di scrittura senza <code>-ErrorAction Stop</code> su Exchange/SharePoint/Teams/Intune, stesso schema già visto altrove nel progetto ma mai propagato	corretto	Un errore reale e non terminante del cmdlet nativo sottostante veniva ignorato in silenzio: la funzione tentata aggiunta di un membro inesistente: prima del fix "Added: True" con 0 membri reali, dopo il fix <code>[Forbidden] Access Denied</code> catturato e diagnosticato invece di un falso successo). Su SharePoint difensiva/di consistenza, dichiarato onestamente invece di spacciarla per una conferma dal vivo. Fix su con un test rapido)
Bug-hunt mirato sul catalogo comandi (v0.9.6-0.9.7, 19/08/2026): risposte locali sbagliate su richieste scritte in modo diverso dal previsto, mai arrivate all'AI	corretto	Richiesto esplicitamente dall'utente dopo aver chiesto "siamo sicuri che l'AI intervenga sempre, o il catalogo verifica sistematicamente, per un messaggio realistico, se il catalogo risponde con la voce ATTESA, con <code>ListNonCompliant</code> facendo rispondere <code>ListDevices</code> con TUTTI i dispositivi invece dei soli non conformi" (forma grammaticale normale, non un refuso) non riconosciuto da nessun trigger; 6 voci di meccanismo <code>DeferWords</code> stesso (usato da tutte le 18 voci): il confine di parola <code>\b</code> finale faceva fallire (<code>'consiglio' -match '\bconsigli\b'</code> è <code>\$false</code>) e corretto in un solo punto in <code>Server.ps1</code> , "nn"/"sn"/"cmq") prima del matching - i refusi VERI dentro una parola (es. "quta" per "questa") restano
Causa radice trovata (v0.9.8, 19/08/2026) del bug "role mancante" - prima solo mitigato, sezione 20.5	corretto	Ripreso su richiesta esplicita dell'utente dopo averlo rivisto ricomparire durante il bug-hunt sul catalogo verificate empiricamente in isolamento: <code>Get-M365OpsChatHistory</code> restituisce un array vuoto che comunque esegue zero iterazioni come una collezione vuota, ne esegue una con <code>\$_ = \$null</code> , producendo la v sull'input in <code>Invoke-M365OpsAgentTools.ps1</code> . Verificato dal vivo: lo stesso identico scenario che p
Diagnosi/auto-correzione di una scrittura fallita estesa con ricerca documentale reale (v0.9.9-0.9.11, 19/08/2026)	corretto	Richiesto dall'utente dopo un caso reale (regola di un Assignment Filter Intune rifiutata da Graph, per <code>M365OpsWriteRecovery</code> (propone il retry immediato con parametri corretti) ragionavano SOLO sul tentativo una chiamata AI breve per capire cosa cercare, poi consultano Microsoft Learn reale (<code>Invoke-M365Ops</code> nuovo meccanismo: nessuna scrittura pericolosa mai proposta, un tentativo bloccato dal filtro nativo con Assignment Filter direttamente nel prompt di sistema (<code>operatingSystemVersion</code> vs <code>osVersion</code> , PRIMO tentativo, non solo dopo un fallimento
BUG SERIO: array a un elemento collassato a scalare in <code>ConvertTo-M365OpsHashtable</code> (v0.9.12, 19/08/2026)	corretto	Trovato durante uno stress test multi-round richiesto dall'utente: <code>"groupTypes":["DynamicMembers"</code> attraversando questa funzione - usata su OGNI parametro proveniente da un tool AI prima di quasi ogni meccanismo di fondo del bug "role mancante" sopra (un array restituito da una funzione PowerShell v funzione ricorsiva generica. Corretto con la virgola unaria (<code>return ,@(...</code>), verificata empiricamente identificate correttamente per un dubbio non correlato su un campo diverso - corretto chiarendo che
Nuovo: <code>generate_raw_graph_export</code> - i report su dati Graph sono sempre righe reali, mai aggregate (v0.9.13, 19/08/2026)	corretto	"fai un report dei sign-in log degli ultimi 7 giorni di diego@vnsys.it" ha prodotto un file Excel con UN problema ma SOLO per le cmdlet Exchange (<code>exo_query</code>) - i dati Graph non avevano equivalente, quindi proprio budget di token, nonostante il divieto esplicito nella descrizione dello strumento. Aggiunto lo deliberatamente generale (non specifico ai sign-in) su richiesta esplicita dell'utente. Verificato dal vivo UX nel testo del comando locale "aiuto": la Description di <code>SharedMailboxPermissions</code> conteneva
BUG STRUTTURALE: <code>-ErrorAction Stop</code> non protegge da un cmdlet nativo IRRISOLVIBILE, solo da un cmdlet che restituisce un errore morbido	corretto	Trovato testando <code>New-M365OpsAcceptedDomain</code> (area "implementata ma mai testata dal vivo", sezione Receive/SendConnector, sezione 6.4 - verificato con una lettura indipendente). Causa verificata empiricamente

(v0.9.14, 20/08/2026)		argomento viene ignorato (PowerShell lo lega solo DOPO aver risolto il cmdlet) - l'esecuzione proseguì. <code>ErrorAction Stop</code> cmdlet-per-cmdlet dei bug-hunt precedenti restano vulnerabili esattamente a q Fix strutturale, non piu' per-cmdlet: <code>\$ErrorActionPreference = 'Stop'</code> a livello di SCOPE DEL M che <code>Gui\Server.ps1</code> non chiama mai cmdlet native direttamente (solo funzioni del modulo), quindi testate (creazione/eliminazione reale di un gruppo, fallimento intenzionale ancora onesto) - nessuna r
Nuovo: Knowledge Base GLOBALE + diagnosi errori che consulta anche la KB (v0.9.15, 20/08/2026)	corretto	Richiesto esplicitamente dall'utente: la guida di configurazione ora e' caricabile come documento in u configuro l'app per Exchange Online?" in qualunque momento. <code>Invoke-M365OpsErrorTriage</code> e <code>I</code> setup/configurazione di questa app. Bug reale trovato caricando la guida (52 pagine): <code>PdfLexer 's (</code> <code>.Length</code> contava le pagine (52) invece dei caratteri, scartando il testo vero e mandando in errore la <code>M365OpsExtractedFileText.ps1</code> - stessa famiglia "array trattato come scalare" gia' vista piu' volte effettiva a <code>kb_query</code>); lo stesso errore RBAC reale della sezione 6.4 passato a <code>Invoke-M365OpsErr</code> del solo match per parola chiave (bypassando l'AI)
Nuovo: la guida nella Knowledge Base globale si autoaggiorna, messaggio di benvenuto accorciato (v0.9.16, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente dopo aver notato che il documento appena caricato nella KB glob manuale facile da dimenticare. <code>Gui\Server.ps1</code> ora confronta ad ogni avvio l'hash SHA256 di <code>doc</code> inutile quando la guida e' invariata) - vedi sezione 17.13. Verificato dal vivo con due riavvii consecutivi chat all'avvio (<code>Gui\index.html</code>): prima elencava per esteso ogni area gestita, ora rimanda direttam qualunque momento
Nuovo: autenticazione Graph via certificato (senza secret) + messaggio di onboarding al primo avvio (v0.9.17, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente. Prima domanda: "possiamo usare il certificato anziche' il secret?" firmato dal certificato al posto del client secret (RFC 7523), stesso meccanismo gia' usato da <code>Connect</code> (cerca il certificato in <code>Cert:\CurrentUser\My</code> poi <code>Cert:\LocalMachine\My</code> , come fa gia' <code>Connect-Exchang</code> esistenti), altrimenti il certificato se configurato - mai una sostituzione silenziosa di un secret gia' funz token Graph ottenuto e usato con successo per una chiamata reale a <code>/organization</code> . Rinominato c facoltativo)" - non e' piu' specifico di Exchange - e aggiunta una nota che i due campi (secret/certifica Tenant e riassume le due modalita' di autenticazione con una panoramica dei permessi per App Regis messaggio di benvenuto normale finche' non si salva il primo profilo. Verificato dal vivo con un test di false
Nuovo: check permessi app in tempo reale, pulsante "🔒 Stato permessi" (v0.9.18, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente: "vorrei fare una sezione check permessi app dove... l'interfaccia sc vivere (sotto-sezione statica nel tab Tenant vs pulsante in chat) - scelto il pulsante in chat, coerente co assegnazioni REALI del service principal (<code>GET /servicePrincipals/{id}/appRoleAssignments</code>), guida (mai a memoria) per dare uno stato lettura/scrittura per Intune, Entra ID, Report, MFA, Sicurezza Exchange Online. Scoperta scritta in codice DURANTE lo sviluppo, prima di consegnare: il nome risorsa: tentativo con quel nome produceva sempre "nessun accesso" anche a permesso presente, corretto co (<code>CommandCatalog.ps1</code> , <code>RequiresAI=\$false</code> - dati gia' precisi, un riassunto AI aggiungerebbe so nel browser: risultato consistente con le assegnazioni grezze verificate a mano (3 permessi Intune scrit
Tema chiaro/scuro, pannello Upload a scomparsa, pulsanti header unificati (v0.9.19, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente, in due parti. Tema scuro: pulsante nell'header, scelta persistita in I con una sola nuova regola CSS perche' tutta la GUI usava gia' variabili CSS per i colori, nessuna riscritt pannello a scomparsa (pulsante "🔒 Upload" nell'header), stesso pattern toggle del pannello Imposta secondo, aggiunto in v0.9.18, non aveva nessuna regola CSS dedicata) - unificati sotto una sola classe <code>Gui\index.html</code> viene tenuto in memoria dal server all'avvio (mai riletto ad ogni richiesta, a differe con un riavvio del server, poi verificato dal vivo nel browser (toggle upload, toggle tema con persisten
Testato dal vivo: Set-M365OpsRemoteDomain e ciclo completo gruppi dinamici; corretta un'informazione sbagliata sulla sezione 6.4 (21/08/2026)	corretto	Ripreso il compito rimasto sospeso dall'inizio della sessione ("copiamo cio' che e' implementato ma n indipendentemente, ripristinato al valore originale. <code>New-/Set-/Remove-M365OpsDynamicDistribut</code> passaggio confermato con una lettura indipendente, ambiente di test tornato pulito. <code>New-/Set-/Rem</code> limite RBAC di Receive/SendConnector). Corretta anche un'informazione sbagliata scritta in questa ste Administrator" - un tempo trascorso MAI verificato in modo indipendente, solo assunto in modo sbag Retestato a ~1 ora reale dall'assegnazione: <code>New-AcceptedDomain/Get-ReceiveConnector</code> falliscono ar stata riscritta per riflettere onestamente questa incertezza invece di un tempo trascorso inventato
BUG: check permessi non riconosceva un permesso Graph piu' ampio come sufficiente per uno piu' stretto (v0.9.21, 21/08/2026)	corretto	Trovato dal vivo dall'utente, non durante lo sviluppo: "l'app ha <code>Sites.FullControl.All</code> che dovrebbe esser stringa esatta, quindi <code>Sites.FullControl.All</code> concesso non soddisfaceva un requisito di <code>Sites.</code> <code>FullControl.All</code> , stesso prefisso): un permesso piu' ampio ora soddisfa automaticamente un requi SharePoint via Graph e' passata da "nessun accesso" a "lettura". Chiarito in parallelo (stessa conversazi (<code>DeviceManagementScripts/ServiceConfig/RBAC.ReadWrite.All</code>) risultavano ancora genuinamente asse un permesso nella lista "API permissions" non crea un'assegnazione finche' non si preme esplicitamen
BUG STRUTTURALE: Receive-/SendConnector erano cmdlet on-premises-only, mai esistiti in Exchange Online - risolto con InboundConnector/OutboundConnector (v0.9.22, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente: "puoi fare test in modalita' delegata?" sul limite RBAC apparente con <code>Get-ManagementRoleAssignment</code> : <code>Get-AcceptedDomain</code> ha funzionato subito (5 domini reali) <code>Get-ReceiveConnector</code> ha dato lo STESSO "term not recognized" anche con quell'utente (nome memoria): <code>Get-/New-/Set-/Remove-ReceiveConnector</code> e <code>...SendConnector</code> sono cmdlet ESC mese di indagine RBAC (sezione 6.4) era in parte una pista sbagliata per questo caso specifico: stesso <code>M365OpsReceiveConnector/SendConnector</code> rimossi, sostituiti da <code>.../InboundConnector/OutboundCon</code> <code>RecipientDomains/SmartHosts</code> per Outbound - diversi da <code>Bindings/RemoteIPRanges/AddressSpaces d</code> <code>allowlist exo_query/propose_exo_write</code> (<code>Invoke-M365OpsAgentTools.ps1</code>). Verificato dal vivo con un ci zero problemi di permessi - conferma che non era mai stato un limite RBAC per questi due cmdlet
Anti-spam/anti-phishing/malware in scrittura, check permessi Delegated con ruoli directory, guardrail disconnect, fix login Teams (v0.9.23, 21/08/2026)	corretto	Round esteso, richiesto esplicitamente dall'utente. (1) Implementate le 3 aree anti-spam/anti-phishing aree), nessun nuovo ruolo RBAC richiesto (Transport Hygiene gia' assegnato copre tutto, verificato dal con <code>Get-Command</code> , non dalla documentazione che suggeriva il contrario) - aggiunti 6 cmdlet <code>Enable-/</code> <code>M365OpsDelegatedPermissionsCheck</code> legge i ruoli RBAC Exchange REALI dell'utente (<code>Get-Managemer</code> registrata" su un tenant Delegato, e mancanza dei ruoli DIRECTORY Entra ID (Global Admin ecc, sistem gia' ottenuto). (3) Guardrail: <code>Connect-M365Ops</code> ora disconnette anche Teams/SharePoint (prima solo E: (<code>Microsoft.IdentityModel.Tokens</code>) passando da un tenant App-only a uno Delegato nello stesso proces

		STRUTTURALE trovato dal vivo (successo due volte all'utente): il login Teams Delegato (-UseDeviceAut del processo) - corretto rimuovendo il parametro, cosi' il modulo usa il proprio flusso interattivo di de
Stress test feroce post-v0.9.23: tutto verificato via AI/chat reale, 1 bug reale trovato e corretto (v0.9.24, 21/08/2026)	corretto	Richiesto esplicitamente dall'utente: "esegui un feroce stress test... con prove reali di utilizzo... non tras chat/AI reale (non chiamate dirette PowerShell): ciclo completo criterio+regola anti-spam creato/verifi M365OpsAntiSpamRule (non Set- con -Enabled, il bug di v0.9.23); lettura anti-phishing e filtro malwar round di modifiche); creazione/verifica indipendente/rimozione di un gruppo di distribuzione (area co verificato sul caso limite (riattivazione tenant senza nulla da disconnettere, nessun errore); 2 tentativi d rivendicasse un'approvazione gia' data in una chat precedente). Bug reale trovato: la domanda inform (letteralmente presente nella frase) ed ESEGUIVA il controllo vero invece di rispondere con l'AI/la guid AppPermissionsCheck del catalogo, verificato dal vivo che la stessa domanda ora risponde corrett
Audit del codice preesistente su richiesta esplicita ("mai dare per scontato"): 3 bug reali trovati e corretti + 5 criticita' da un test su PC pulito (v0.9.25, 21/08/2026)	corretto	Audit sistematico del codice preesistente (mai toccato in questa sessione fino ad ora), richiesto esp script preesistenti? mai assumere che non possano esistere bug ancora non scoperti". Grep sistematic live reale, non a memoria - 18 "non risolvibili" trovati, la maggior parte falsi positivi (funzioni locali con funzioni interne di ALTRI moduli). 3 bug reali confermati: (1) <code>Get-M365OpsCompliancePatterns.p</code> "\$enriched = foreach(...)" collassa a scalare con esattamente 1 dispositivo non conforme, E anche un v empiricamente con un dispositivo sintetico prima di correggere entrambi i livelli. (2)(3) <code>Enable-/Dis cmdlet ESCLUSIVI</code> di Exchange on-premises, mai esistiti in Exchange Online, confermato su document alcun equivalente cloud da sostituire (concetto legato alla sincronizzazione AD on-premise): corretto f non e' disponibile.
Stesso commit: 5 criticita' reali segnalate dall'utente durante un test end-to-end su un PC pulito (nessuna delle 5 era mai stata provata dal vivo su un ambiente davvero vergine prima d'ora): 1. Bug di regressione critico: il salvataggio di un profilo tenant richiedeva ANCORA -SecretEnvVar obbligatorio (sia lato JS sia lato <code>Set-M365OpsTenant.ps1</code>) anche dopo che il certificato era diver funzionalita' era pensata per abilitare. Corretto in entrambi i punti: serve ClientId sempre, e SecretEnvVar O ExchangeCertThumbprint (non necessariamente entrambi). 2. winget bootstrap automatico: su un PC molto minimale winget stesso puo' mancare, bloccando l'intera catena di auto-installazione. Aggiunto un tentativo di bootstrap automatico (modulo uffici VCLibs/UI.Xaml, niente URL di pacchetti fissati a mano) prima di arrendersi con istruzioni manuali. 3. Bug reale nell'installazione Node.js: <code>winget install</code> senza <code>--source winget</code> esplicito interrogava ANCHE la sorgente msstore - un errore del backend Microsoft Store ("0x8a15003b Rest A specificando sempre <code>--source winget</code> . 4. Messaggio fuorviante sulla chiave AI mancante: citava solo la variabile ANTHROPIC_API_KEY, dando l'impressione sbagliata che fosse l'unico provider supportato. Prima correzione (questa riga, v 5. Problema architetturale reale, non solo di messaggistica: senza nessuna chiave AI configurata, ANCHE la guida stessa era irraggiungibile (leggibile solo tramite <code>kb_query</code> dentro il motore AI) letta. Aggiunta una route statica <code>GET /guida</code> che serve <code>docs\Guida-Configurazione.pdf</code> direttamente, senza passare mai dal motore AI - raggiungiibile a zero configurazione, zero chiavi, ze Tutti e 5 verificati dal vivo dopo la correzione: profilo solo-certificato salvato con successo via API reale, <code>Install-M365OpsPrerequisites</code> testato senza regressioni con winget gia' presente, <code>GET /guic</code>		
BUG: la v0.9.25 aveva corretto solo il TESTO del messaggio "chiave AI mancante", non la logica - nessun provider e' un default (v0.9.26, 22/08/2026)	corretto	Correzione richiesta esplicitamente dall'utente dopo aver riletto la riga del changelog v0.9.25: "manca default". Aveva ragione: <code>Get-M365OpsSetupStatus.ps1</code> continuava a controllare UNA SOLA variab prima, causa vera del comportamento originale: <code>\$script:ActiveAIProvider</code> e' una variabile di <code>Gu</code> SEMPRE <code>\$null</code> , e il controllo finiva SEMPRE nel ramo ANTHROPIC_API_KEY indipendentemente da c l'interpolazione di quella variabile risultava vuota). Non un problema di testo quindi, ma la causa strut <code>AZURE_OPENAI_*</code> in modo indipendente e alla pari, senza piu' fare riferimento alla variabile fuori scv vivo via <code>GET /api/setup-status</code> su un server reale riavviato, con entrambi i provider configurati: s
BUG STRUTTURALE: "ps7 assente e winget non disponibile" ricomparso identico su un secondo PC pulito con la v0.9.26 gia' installata - GUI di installazione prerequisiti unificata, icona e collegamento Desktop (v0.9.27, 22/08/2026)	corretto	Segnalato dal vivo dall'utente dopo aver riprovato l'installazione: "provata la 0.9.26 avviata installazion bootstrap winget aggiunto in v0.9.25 viveva in <code>Install-M365OpsPrerequisites.ps1</code> , dentro il MC PowerShell 7 (il modulo richiede un ambiente PowerShell gia' funzionante per essere importato). Corra differenza di <code>pwsh.exe</code>) - stesso bootstrap ufficiale Microsoft (<code>Microsoft.WinGet.Client / Rep</code> comune di fallimenti silenziosi di <code>Install-Module</code> su un PC vergine, applicato anche al bootstrap g invece di "goto" dentro blocchi <code>if()</code> (problema noto di parsing di cmd.exe: l'intero blocco tra paren bootstrap fallito, bootstrap riuscito via fallback WindowsApps) prima di toccare il file reale, oltre al nor bellina per il bat e una gui che segua l'installazione dei moduli. aggrega dentro l'installazione dei prer blu/turche, generata via System.Drawing, nessun logo Microsoft riprodotto), assegnata a un nuovo c Windows, un collegamento si'; (2) <code>Show-M365OpsPrereqInstaller.ps1</code> , finestra Windows Forms c <code>M365Ops.bat</code> solo se manca davvero qualcosa - il percorso comune con tutto gia' presente resta ist <code>M365Ops.ps1</code> , e' ora aggregato nello stesso flusso visibile.
BUG UX: la GUI di installazione (v0.9.27) sembrava bloccata durante il bootstrap di winget - output del processo figlio mostrato solo alla fine (v0.9.28, 22/08/2026)	corretto	Segnalato dal vivo dall'utente al primo utilizzo pratico della GUI v0.9.27, con screenshot: la finestra res rompono". Causa reale: la funzione che aspettava il processo figlio (bootstrap di winget o singola insta fine) - durante <code>Install-Module / Repair-WinGetPackageManager</code> , che puo' durare 1-2 minuti su lavorando normalmente. Corretto reindirizzando l'output del processo figlio su un file temporaneo e " figlio e la sua comparsa nel file, dovuto al buffering interno dell'host PowerShell su stdout rediretto - z ogni giro di polling (ogni 200ms), indipendente dal buffering dell'output, che da' un segno di vita con winget non ha nascosto correttamente pwsh/winget dal processo di test, facendo scattare per errore l <code>version / npx --version</code> confermati funzionanti dopo) che durante quell'installazione reale la bar sono comparse nel log MENTRE il processo era ancora in corso, non solo alla fine.
BUG (diagnosi da codice, in attesa di conferma dal vivo): winget appena installato risultava "non trovato" a un riavvio successivo, PATH figlio ereditato dal genitore prima del bootstrap (v0.9.29, 22/08/2026)	corretto	Segnalato dal vivo dall'utente su Windows Sandbox (utente <code>WDAGUtilityAccount</code> nello screenshot, server non ha risposto entro 10s" e' comparso, e rilanciando <code>M365Ops.bat</code> winget sembrava di nuov verifica pulita... fammi sapere"): un processo figlio Windows eredita il <code>%PATH%</code> del genitore AL MOMI rileggeva mai il proprio <code>%PATH%</code> prima di avviare <code>Launch-M365Ops.ps1</code> (pwsh.exe), che quindi ere ritrovato winget "assente" (controllava solo <code>Get-Command</code> , nessun fallback sul percorso diretto Wind moduli diverso da Windows PowerShell 5.1, quindi nessuna cache riusata dal primo tentativo) - proba sola a ridurre il rischio: (1) <code>M365Ops.bat</code> ora rilegge il PATH vero dal registro (Machine+User) subito rompono il parsing del blocco <code>if()</code> che lo contiene, stesso tipo di rischio gia' visto con "goto" in v0 winget manchi, e usa il percorso risolto (non il nome nudo "winget") per la chiamata di installazione ve allo stesso modo; (3) il timeout di attesa del server in <code>Launch-M365Ops.ps1</code> alzato da 10 a 30 secon

		mentre il server sta ancora, semplicemente, lavorando. Verificato dal vivo su questa macchina di svilu aggiunta in v0.9.30: questa diagnosi NON era la causa reale dell'errore segnalato - le tre correzi
BUG CONFERMATO (dal log reale): il server non partiva affatto su un'installazione DAVVERO vergine, zero profili tenant salvati - crash immediato, non lentezza (v0.9.30, 22/08/2026)	corretto	L'utente ha condiviso il contenuto reale di <code>Logs\server-console-error.log</code> su richiesta esplicita rivelatasi sbagliata) della v0.9.29: <code>Connect-M365Ops.ps1:16</code> lanciava "Nessun profilo salvato. Usa p try/catch, in cima allo script, PRIMA che il listener HTTP venga mai creato molto piu' sotto - su un'instz 7/winget/Node.js, ma anche senza NESSUNA configurazione applicativa), l'intero processo del server r "lento a rispondere" (lo stesso messaggio generico "non ha risposto entro N secondi"), ma la causa ve dal log reale) aveva puntato nella direzione sbagliata. Il meccanismo di onboarding "firstRun" (sezione gestiva gia' correttamente un <code>tenants.json</code> assente) - ma non poteva mai essere raggiunto, perch sessione: ogni test precedente partiva gia' da un PC con almeno un profilo tenant esistente (tipicamen configurazione non era mai stata provata dal vivo fino a questo test su Windows Sandbox. Corretto av attivo (stato gia' previsto e gestito correttamente altrove nel codice) e l'utente vede la schermata di or isolata completa del progetto, <code>Config\tenants.json</code> rimosso, server avviato con il profilo di defau <code>/api/setup-status</code> e la pagina principale serviti correttamente senza nessun tenant attivo.
BUG: login Exchange delegato riuscito ma poi fallito per un modulo mancante mai auto-installato in quel percorso; progress bar Marquee invisibile su Windows Sandbox (v0.9.31, 22/08/2026)	corretto	Segnalato dal vivo dall'utente su Windows Sandbox: login a Microsoft Graph delegato riuscito, ma Exc 'ExchangeOnlineManagement' was not loaded because no valid module file was found in any module <code>ExchangeOnlineManagement</code> senza NESSUN fallback di auto-installazione, a differenza di <code>Connect</code> solo dei due gestiva l'assenza. Stesso buco trovato anche in <code>New-M365OpsWin32App.ps1</code> per il mod esplicitamente, con enfasi, di risolvere alla radice invece di rincorrere un fallback alla volta: "ti avevo de <code>M365OpsPrerequisites.ps1</code> ora installa in anticipo anche i tre moduli PowerShell (<code>ExchangeOnli</code> irrobustimento TLS 1.2/provider NuGet gia' usato per winget - i fallback "al primo uso" nelle singole c l'installazione degli stessi tre moduli dentro <code>Show-M365OpsPrereqInstaller.ps1</code> (visibile, non sol processo <code>pwsh.exe (PowerShell 7) figlio</code> , non nello scope di Windows PowerShell 5.1 in cui gira ques (<code>Documents\WindowsPowerShell\Modules</code> vs <code>Documents\PowerShell\Modules</code>) - installarli ne "mancante" dal vivo - causa reale: mai chiamato <code>[System.Windows.Forms.Application]::Enable</code> spazio vuoto invece che con un aspetto solo meno moderno - un problema noto, non un'ipotesi) - ag passi (0-4, un incremento per sezione completata: PowerShell 7, Node.js/Edge, moduli PowerShell, fine invece del solo "sta succedendo qualcosa". Verificato dal vivo su questa macchina di sviluppo: <code>Insta</code> comune, nessun rallentamento aggiunto).
Completato l'elenco moduli prerequisito: mancavano MicrosoftTeams, PnP.PowerShell, PdfLexer (v0.9.32, 22/08/2026)	corretto	L'utente ha notato subito dopo la v0.9.31 che l'elenco copriva solo Exchange/Excel/Intune: "ma i moduli segnalato (Exchange/Intune) senza fare il giro completo. Rifatto lo stesso grep sistematico su tutto il c uso" (<code>MicrosoftTeams</code> in <code>Connect-M365OpsTeams.ps1</code> , <code>PnP.PowerShell</code> in <code>Connect-M365Op</code> Knowledge Base, <code>Get-M365OpsExtractedFileText.ps1</code>) - tutti e tre avevano gia' un fallback local tutti e tre sia a <code>Install-M365OpsPrerequisites.ps1</code> sia a <code>Show-M365OpsPrereqInstaller.ps1</code> esplicitamente in guida come intenzionalmente escluso dall'elenco: il progetto non usa mai l'SDK offic bearer token), quindi non serve nessun modulo dedicato per Graph. Verificato dal vivo su questa mac nessuna regressione.
BUG: tutti e 6 i moduli PowerShell fallivano l'installazione nella GUI con "PackageManagement... currently in use" (v0.9.33, 22/08/2026)	corretto	Segnalato dal vivo dall'utente subito dopo la v0.9.32, log incollato in chat: ogni singolo modulo falliva <code>currently in use. Retry the operation after closing the applications.</code> , con "(exit cod SEPARATO per ciascuno dei 6 moduli, e ciascuno ripeteva da zero il bootstrap TLS1.2/provider NuGet processo poteva partire prima che il primo avesse rilasciato del tutto i file di PackageManagement/Po sintomo dello stesso problema: il processo falliva in un modo che lasciava <code>\$proc.ExitCode</code> non po SOLO processo pwsh.exe (bootstrap TLS1.2/NuGet fatto una volta sola, poi un ciclo che installa ogni r che toccano quei file insieme. Stessa ottimizzazione applicata anche al controllo di presenza (prima 6 <code>Install-Module</code> del progetto (GUI e <code>Install-M365OpsPrerequisites.ps1</code>) per evitare un'altra REALE (non simulata): un modulo di test innocuo (<code>PSScriptAnalyzer</code> , mai usato da questo progett inesistente, che ha riportato il proprio errore specifico invece di un "(exit code)" vuoto - nessuna trac
Diagnostica aggiunta (non ancora un fix confermato): login Teams delegato segnalato dal vivo come bloccato dopo il popup su Windows Sandbox (v0.9.34, 22/08/2026)	diagnostica, non ancora risolto	Segnalato dal vivo dall'utente: "exch sharepoint si sono connessi, teams mi ha chiesto login l'ho fatta e Delegato, il ramo interessato di <code>Connect-M365OpsTeams.ps1</code> non e' esercitabile). Non essendoci ur devono mai passare da qui), invece di tentare un fix alla cieca e' stata aggiunta diagnostica prima/dop su <code>Logs\m365ops-YYYYMMDD.log</code> (persistente, recuperabile anche a processo terminato - a differen questa sessione, vedi il bug del device-code gia' risolto in sezione 17.14) con i secondi trascorsi sia in (bloccata per sempre, es. componente mancante nell'ambiente Sandbox come WebView2 Runtime o t richiedere 10-30s legittimi) - invece di doverlo dedurre da un'osservazione soggettiva ("sembra blocca modifica) continua a funzionare correttamente via <code>POST /api/teams-test</code> reale, nessuna riga di di
BUG STRUTTURALE trovato: login Teams delegato bloccato per sempre da WAM (Web Account Manager), il broker diventato default nel modulo MicrosoftTeams (v0.9.35, 22/08/2026)	corretto	Causa vera del blocco Teams delegato, trovata dopo che l'utente ha confermato dal vivo un blocco CC "non puoi implementarlo come in Intune, che funziona?" Confrontando <code>Connect-M365OpsIntune.p</code> <code>Connect-MicrosoftTeams</code> usa di DEFAULT Web Account Manager (WAM) come broker di autenticazione interattiva con una UI reale a cui agganciarsi; in un contesto senza finestra visibile come questo server sempre in attesa di una risposta che non arrivera' mai. Stessa CLASSE di bug del device-code gia' risol recente: WAM e' diventato il default DOPO che quel primo fix era stato scritto, il modulo si e' aggiorn con il parametro ufficiale <code>-DisableWAM</code> , introdotto proprio in v7.8.1-preview "per scenari come cont modo CONDIZIONALE, non incondizionato: verificato che su questa stessa macchina di sviluppo il mo caso con un errore di parameter binding, un bug diverso ma reale introdotto per evitarne un altro. La <code>[version]'7.8.1'</code>) prima di aggiungere lo switch. Aggiunta anche una riga di diagnostica prima/dc utile a isolare in futuro se un blocco fosse nell'import del modulo invece che nella connessione. Verific continua a funzionare correttamente via <code>POST /api/teams-test</code> reale. Il ramo Delegato con <code>-Dis</code>
-DisableWAM confermato funzionante (il popup c'era, era solo nascosto dietro il browser) + irrobustimento sistematico di tutti i fallback Install-	corretto	L'utente ha riprovato il login Teams delegato sulla v0.9.35 e ha segnalato di nuovo un blocco, stavolta diagnostica aggiunta, PRIMA del punto dove interviene <code>-DisableWAM</code>) - prova che il blocco (se reale

Module + nuovo strumento "Termina e riavvia" (v0.9.36, 22/08/2026)		l'irrobustimento TLS1.2/provider NuGet/ <code>-SkipPublisherCheck</code> gia' applicato altrove nel progetto (ne' agli altri identici sparsi nel codice piu' vecchio - un buco reale, potenzialmente capace dello stesso capito!! c'era il popup ma era sotto il browser!!!" - <code>-DisableWAM</code> aveva funzionato correttamente notato. Nessun vero blocco in questo episodio specifico - ma l'irrobustimento trovato nel frattempo riaggiunto a TUTTI i fallback <code>Install-Module</code> rimasti scoperti (<code>Connect-M365OpsTeams.ps1</code> , <code>Connect-M365OpsExchangeDelegatedLogin.ps1</code> , <code>New-M365OpsWin32App.ps1</code> , <code>Get-M365OpsExtractedFiles.ps1</code> v0.9.31/32, applicato stavolta alla robustezza dell'installazione invece che al suo elenco. Nuovo strumento powershell e riavvii il .bat?"): un vero pulsante DENTRO la pagina web non avrebbe funzionato per il client (stesso motivo per cui, durante un blocco reale, anche il tab Log smette di rispondere). Aggiunti <code>Kill-Process</code> anche se e' lui stesso ad essere bloccato: termina SOLO i processi <code>pwsh.exe</code> che eseguono <code>Gui\Server\Start-Server.ps1</code> rilancia via <code>M365Ops.bat</code> . Aggiunto un secondo collegamento sul Desktop ("M365Ops - Termina e riavvia") e i collegamenti, idempotente per ciascuno indipendentemente). Verificato dal vivo su questa macchina con <code>M365Ops.bat</code> , tornato in linea con il tenant corretto ripristinato (<code>last-active-tenant.txt</code>) in
BUG DI TERZE PARTI trovato e riprodotto: MicrosoftTeams e ExchangeOnlineManagement non possono coesistere nello stesso processo server - conflitto noto di Microsoft, guardia aggiunta con messaggio chiaro invece di un crash apparente (v0.9.36, 22/08/2026)	mitigato (nessun fix possibile lato Microsoft)	Segnalato dal vivo dall'utente SUL SUO PC REALE (non sul Sandbox - dettaglio confermato esplicitamente "Could not load file or assembly '...ExchangeOnlineManagement\3.10.1\NetCore\Microsoft.IdentityModel.Clients.OAuth2\BrokerPlugin.dll' or one of its dependencies. The system cannot find the file specified."), un vero blocco del client indipendente sia dal Sandbox sia dalla modalita' Delegata: connettere Teams poi Exchange nello stesso processo: fallisce ANCH'ESSO, stavolta su <code>Microsoft.Identity.Client</code> - cambiare l'ordine non risolto documentato da anni, non specifico di questo progetto: i moduli <code>MicrosoftTeams</code> e <code>ExchangeOnlineManagement</code> (<code>Microsoft.Identity.Client</code> / <code>Microsoft.IdentityModel.*</code>) - una volta che un modulo le cariche "ExchangeOnline", office365itpros.com, 2023). Un fix vero (isolare uno dei due moduli in un processo/job separato <code>M365OpsTeams</code> / <code>Connect-M365OpsExchange</code> / <code>Connect-M365OpsCompliance</code> / <code>Complete-M365OpsCompliance</code>) gia' stato caricato in questo processo - se si', restituiscono subito un errore chiaro che spiega il conflitto apparente. Verificato dal vivo su questa macchina di sviluppo in entrambe le direzioni: il messaggio che ora è comprensibile.
Nuovo: report di utilizzo Microsoft 365 Copilot - primo passo del supporto Copilot, ambito volutamente ristretto ai soli report (v0.9.37, 22/08/2026)	implementato	Richiesto esplicitamente dall'utente: "dobbiamo aggiungere tutto quello che riguarda il supporto copilot per qualunque cosa - risultato: solo i report di utilizzo sono affidabili e senza limitazioni App-only/Delegated documentazione ufficiale, solo Delegato - l'utente ha confermato esplicitamente di NON voler procedere report"). Catalogo agenti/app Copilot ancora in preview con 403 documentati anche a permessi corretti significato shippare qualcosa gia' morto in pochi giorni. Cronologia interazioni reali (<code>AIEnterpriseInsights</code> senza una richiesta esplicita separata. Aggiunte <code>Get-M365OpsCopilotUsageReport</code> (per-utente) e <code>Set-M365OpsCopilotUsageReport</code> (per-utente) e permesso gia' documentato come mancante su vnys-test da un giro precedente (19/08/2026) per altro CSV (comportamento documentato, non un bug) - entrambe le cmdlet la convertono con <code>ConvertFrom-Json</code> (vengo concesso). Confermato inoltre esplicitamente all'utente, su sua richiesta di chiarimento: M365Ops esistente) puo' gia' rispondere a domande concettuali su Copilot senza bisogno di nuova integrazione funzionalita' nuova da costruire per quello. Vedi sezione 24 per il dettaglio completo.
BUG GRAVISSIMO trovato: l'auto-update non aveva MAI applicato nessuna modifica di codice da settimane, nonostante ogni giro si dichiarasse riuscito e il numero di versione salisse correttamente (v0.9.38, 22/08/2026)	corretto	Spiega retroattivamente buona parte della difficolta' di questa intera sessione di debug: dopo che l'utente <code>M365OpsTeams.ps1</code> dalla sua macchina si e' scoperto che era ancora la versione di settimane prima (<code>aggiornato da 0.9.33 a 0.9.34</code> , poi a 0.9.35, 0.9.36, 0.9.37, ognuno senza errori. Causa reale, riprodotto - <code>Force</code> , quando <code>dest\Public</code> ESISTE GIA' (sempre vero per un aggiornamento, mai per copia l'intera cartella sorgente DENTRO quella di destinazione, producendo <code>dest\Public\Public\</code> . soggetto a questo problema) si aggiornava correttamente ad ogni giro, dando la falsa impressione che il primo download, mai piu' toccato da nessun aggiornamento successivo. Corretto con una copia che potrebbe gia' esistere, solo singoli file copiati con <code>Copy-Item -Force</code> dopo aver creato le cartelle di destinazione (script personalizzati dell'utente) all'elenco delle cartelle mai toccate dall'update, protezione mancante finiva in <code>dest\Public\Public\NuovoFile.ps1</code> con la vecchia logica; con la nuova, lo stesso scenario al livello corretto, nessuna cartella <code>Public\Public</code> creata, il file estraneo intatto.
Azione necessaria per chi ha gia' installato una versione precedente: questo fix corregge solo gli aggiornamenti FUTURI - non ripulisce l'annidamento gia' eventualmente presente su disco da aggiornare cartella <code>Public\</code> (e probabilmente <code>Private\</code> , <code>Gui\</code> , <code>Tools\</code>) potrebbe contenere sottocartelle annidate con lo stesso nome (es. <code>Public\Public\</code>) piene di file di versioni intermedie mai applicate cartella NUOVA (mai sovrascrivere quella vecchia) - <code>Config\</code> / <code>Logs\</code> / <code>Uploads\</code> / <code>Reports\</code> restano comunque sulla vecchia cartella, quindi se servono i dati di un tenant gia' configurato vanno copiati altrove		
Avviso diretto in GUI vicino a Exchange Online e Teams sul conflitto 6.6 - confermato empiricamente che Disconnect/Remove-Module NON liberano le assembly caricate (v0.9.39, 22/08/2026)	implementato	Richiesto esplicitamente dall'utente: prima "c'e" modo da GUI di smontare il modulo... un disconnect e' (non per ipotesi) se <code>Disconnect-ExchangeOnline</code> seguito da <code>Remove-Module ExchangeOnlineManagement</code> errore - conferma che una volta che una DLL e' caricata in un processo .NET, ne resta parte per tutta la vita (rimuovono solo i comandi dalla sessione corrente, mai le librerie gia' caricate in memoria). Aggiunto il comando <code>Remove-Module ExchangeOnlineManagement</code> "Stato connessioni": "NB: i moduli Exchange e Teams non convivono nello stesso processo server (come per l'altra", con rimando alla sezione 6.6. Verificato dal vivo nel browser (non solo per lettura del codice) una sola volta all'avvio, sezione 12.2).
Due bug reali segnalati dal vivo su PC di test: tab Manutenzione mostrava "Cannot bind argument to parameter 'Name' because it is an empty string" nel controllo stato; auto-sync guida-> Knowledge Base falliva con "Could not find a part of the path '...Config\KnowledgeBase_global.json'" (v0.9.40, 23/08/2026)	corretto	Primo bug: <code>Get-M365OpsSetupStatus.ps1</code> chiamava <code>Get-M365OpsSecret -Name \$ctx.SecretName</code> SOLO con certificato (<code>Set-M365OpsTenant</code> accetta <code>-SecretEnvVar</code> O <code>-ExchangeCertificateThumbprint</code> parametro Mandatory come "valore mancante", e in un host non interattivo come questo processo se <code>M365OpsToken.ps1</code> e <code>Connect-M365OpsIntune.ps1</code> proteggevano gia' la stessa identica chiamata prima della chiamata. Secondo bug: <code>Add-M365OpsKnowledgeDocument.ps1</code> creava la cartella <code>Uploads\</code> <code>Config\</code> non era ancora mai stata creata (nessun tenant salvato, nessuna porta/canale di aggiornamento (che gira ad ogni avvio del server, quindi puo' essere la primissima cosa scritta su disco) falliva silenzio scrittura, stesso pattern gia' usato per <code>Uploads\kb\</code> nella riga sopra. Entrambi verificati con controlli <code>0.9.38\M365Ops-0.9.38\Config\...</code> annidato - probabile estrazione manuale dello zip GitHub cc segnalato all'utente (reinstallazione pulita in una cartella vuota, non innestata).
Il conflitto MicrosoftTeams/ExchangeOnlineManagement (sezione 6.6) NON e' universale: risolto fissando ExchangeOnlineManagement alla versione 3.9.0, 23/08/2026	corretto	L'utente ha correttamente messo in dubbio la diagnosi precedente ("mitigato, nessun fix possibile") con il messaggio "Verificato dal vivo (non a memoria, con <code>Connect-ExchangeOnline</code> REALE e credenziali di test) che il conflitto persiste".

l'unica verificata dal vivo come conflict-free (v0.9.41, 23/08/2026)		conflitto NON dipende dalla versione di <code>MicrosoftTeams</code> installata (riprodotto identico con 7.3.1 e (testato con entrambe le versioni Teams), 3.10.0 e superiori vanno SEMPRE in conflitto (riprodotto i Teams 7.9.0 + Exchange 3.10.1 - probabilmente 3.10.0 ha aggiornato la libreria di autenticazione cond supporta comunque tutto cio' che serve al progetto: <code>-CertificateThumbprint</code> / <code>-Certificate</code> (versione (<code>-RequiredVersion '3.9.0'</code> , mai piu' "l'ultima disponibile") in tutti i punti che installano <code>M365OpsExchangeDelegatedLogin.ps1</code> , <code>Install-M365OpsPrerequisites.ps1</code> , <code>Show-M365Ops qualcosa</code> ": senza questo, <code>Import-Module</code> senza <code>-RequiredVersion</code> avrebbe comunque caricato l rimuove altre versioni gia' installate (es. una 3.10.x installata prima di questo fix) - le affianca nella prop piedi per difesa in profondita', ma con questo fix non dovrebbe piu' scattare nell'uso normale. Un agg vivo con <code>Connect-ExchangeOnline</code> reale, mai per assunzione) prima di cambiare questo numero.
Rinforzo del fix precedente: disinstallazione ATTIVA di ExchangeOnlineManagement >= 3.10.0 se trovata sul disco (non solo pin passivo), avviso reso molto piu' visibile in GUI e guida, procedura scritta per testare le prossime versioni (v0.9.42, 23/08/2026)	implementato	Richiesto esplicitamente dall'utente subito dopo il fix precedente: "aggiungi un check che se e' installa cosi non la aggiornano per sbaglio, e la prossima volta che esce una nuova versione facciamo test spe sbagliata gia' presente sul disco - sufficiente per QUESTO progetto (che chiama sempre <code>-RequiredV Assert-M365OpsExoSafeVersion</code> (nuovo file <code>Private/Assert-M365OpsExoSafeVersion.ps1</code>) <code>RequiredVersion</code> , best-effort - un fallimento e' solo loggato, il pin nei chiamanti resta comunque ef <code>M365OpsExchange.ps1</code> , <code>Connect-M365OpsCompliance.ps1</code> , <code>Complete-M365OpsExchangeDeleg M365OpsPrerequisites.ps1</code> . Stessa logica duplicata (necessariamente, come testo generato) dentri anche il controllo di presenza li' e' stato corretto per trattare come "MISSING" (quindi da rimediare) il reinstallata la 3.10.1 su questa macchina di sviluppo appositamente per il test, poi eseguita <code>Assert-M365Ops</code> presente; ripetuto lo stesso test end-to-end anche sul percorso stringa-generata di <code>Show-M365OpsPr</code> di sviluppo allineata alla versione corretta (solo 3.9.0 rimasta per ExchangeOnlineManagement, oltre a aggiunta : avvisi in GUI (riquadri Exchange Online e Microsoft Teams, tab Tenant) riscritti per nominare ExchangeOnlineManagement evidenziata con l'avviso di versione fissata; nuovo riquadro in sezione 3 affiancamento su una macchina di test, mai sostituire; test con <code>Connect-ExchangeOnline</code> reale e c ovunque compaia, elencato esplicitamente file per file).
BUG REALE trovato dopo il fix precedente: la guardia anti-crash del 22/08/2026 era rimasta attiva anche nella direzione ormai sicura (Teams poi Exchange), bloccando esattamente quello che il pin 3.9.0 aveva appena risolto; scoperto anche che l'ordine INVERSO (Exchange poi Teams) resta rotto anche con la 3.9.0 (v0.9.43, 24/08/2026)	corretto	L'utente ha segnalato dal vivo su v0.9.42: "sono alla 0.9.42 ma non vedo nella GUI l'avviso che dici e i c INVERSO rispetto a tutti i test precedenti - Exchange 3.9.0 importato PRIMA, poi <code>Import-Module M365OpsExchange.ps1</code> / <code>Connect-M365OpsCompliance.ps1</code> / <code>Complete-M365OpsExchangeDeleg QUALUNQUE</code> direzione) era rimasta li' intatta anche dopo aver fissato la versione - bloccava quindi AN piu' vero per quella direzione. Root cause reale del sintomo segnalato: non un problema di versione r i punti che importano ExchangeOnlineManagement (verificato dal vivo, non solo letto: chiamate reali i fallisce pulito solo sul certificato finto). La guardia RESTA invece, aggiornata con un messaggio piu' azi e' gia' stato caricato prima (verificato che continua a bloccare correttamente in quel caso). Avvisi in GL un riavvio. Sulla domanda "non vedo l'avviso in GUI" : <code>index.html</code> viene letto una sola volta all'av basta, serve un riavvio COMPLETO del processo server (tab Manutenzione o "M365Ops - Termina e ria
CORREZIONE della correzione precedente, stesso giorno: nessun ordine di connessione si e' rivelato affidabilmente sicuro - rimossa OGNI guardia preventiva, sostituita con un messaggio chiaro SOLO se il conflitto scatta davvero (v0.9.44, 24/08/2026)	corretto	L'utente ha messo in dubbio la v0.9.43 subito dopo averla vista: "non c'e' magari una versione un po' prima era inverso (Exchange-poi-Teams funzionava, Teams-poi-Exchange no)". Prima di rispondere, es 7.9.0) × 2 ordini = 12 combinazioni, ciascuna in un processo pwsh.exe isolato separato, con chiamate Exchange 3.9.0, ANCHE l'ordine Teams-poi-Exchange (quello su cui si basava tutta la v0.9.43) va in con prima. Scoperta anche una causa di disturbo concreta sulla macchina di sviluppo: un'installazione AIIU per permessi insufficienti, mescolata con le installazioni CurrentUser fatte durante i test - un fattore an quando ha visto la guardia bidirezionale ripristinata: "non voglio il safe guard, nelle versioni vecchie fu sempre rotto, bloccare A PRIORI un tentativo che su un PC specifico potrebbe funzionare e' un danno gia' caricato, blocco senza nemmeno provare") da tutti e quattro i punti di connessione (<code>Connect-M3 M365OpsTeams.ps1</code>) - si tenta sempre la connessione vera, in qualunque ordine. Nuova funzione con not load file or assembly" nel messaggio di un'eccezione REALE (non prevista in anticipo) e lo riveste c (incluso il successo) passa inalterato. La versione FISSATA a 3.9.0 resta invariata (unica certezza solida: ≥3.10.0 gia' presenti. Verificato dal vivo: chiamate reali a <code>Connect-M365OpsTeams</code> poi <code>Connect-M3 M365OpsExoSafeVersion</code> (chiamata SEMPRE da tutti i punti di connessione PRIMA di qualunque <code>Im Microsoft.Identity.Client.dll</code> , non solo la presenza del manifest) - se mancano o sono vuoti, nuovo PRIMA che qualunque <code>Import-Module</code> possa bloccarla. Nuova funzione <code>Private/Invoke-ExchangeOnline / Connect-IPPSession</code>) per il caso raro in cui il danno avvenga DOPO che il mo segnala chiaramente il problema e la necessita' di un riavvio invece del criptico errore .NET nativo. Bug sovrascritto da <code>Get-M365OpsModuleConflictHint</code> (perche' conteneva anch'esso la frase-innesco c codice , con un ciclo di test completo: cancellato il file critico, chiamata la funzione vera <code>Connect-M365Ops</code> connessione prosegue fino al normale errore di certificato finto invece del criptico errore di assembly
BUG DISTINTO trovato lo stesso giorno: "The given assembly name was invalid." su Exchange dopo un riavvio pulito, SENZA Teams coinvolto - non era il conflitto Teams/Exchange ma un'installazione locale di ExchangeOnlineManagement danneggiata, ora rilevata e riparata PRIMA che diventi irreparabile (v0.9.46, 25/08/2026)	corretto	L'utente ha segnalato dal vivo: "connesso teams, vado su exch e ho questo errore: The given assembly dimostrando che NON era (sempre) il conflitto di sezione 6.6. Riprodotto dal vivo su questa macchina ExchangeOnlineManagement 3.9.0: <code>Get-Module -ListAvailable</code> continuava a segnalare la versio riusciva SENZA errori (non carica ogni DLL referenziata, solo l'assembly principale) - l'errore compariva l'autenticazione. Un primo tentativo di riparazione "in-process" (rilevare l'errore dentro il blocco catch che <code>Import-Module</code> ha caricato l'assembly principale nel processo, Windows blocca quei file a livello un assembly caricato non si puo' piu' scaricare, ma qui il blocco e' ancora piu' basso, a livello di file, nc <code>M365OpsExoSafeVersion</code> (chiamata SEMPRE da tutti i punti di connessione PRIMA di qualunque <code>Im Microsoft.Identity.Client.dll</code> , non solo la presenza del manifest) - se mancano o sono vuoti, nuovo PRIMA che qualunque <code>Import-Module</code> possa bloccarla. Nuova funzione <code>Private/Invoke-ExchangeOnline / Connect-IPPSession</code>) per il caso raro in cui il danno avvenga DOPO che il mo segnala chiaramente il problema e la necessita' di un riavvio invece del criptico errore .NET nativo. Bug sovrascritto da <code>Get-M365OpsModuleConflictHint</code> (perche' conteneva anch'esso la frase-innesco c codice , con un ciclo di test completo: cancellato il file critico, chiamata la funzione vera <code>Connect-M365Ops</code> connessione prosegue fino al normale errore di certificato finto invece del criptico errore di assembly
RISOLTO DAVVERO il conflitto MicrosoftTeams/ExchangeOnlineManagement: fissata la coppia ExchangeOnlineManagement 3.1.0 + MicrosoftTeams 6.5.0 (Exchange connesso prima), verificata dal vivo priva del conflitto in ogni flusso del progetto incluso il login delegato device-code (v0.9.47, 25/08/2026 - versione Exchange poi corretta a 3.4.0 in v0.9.48, vedi riga sotto)	corretto	L'utente ha rifiutato la conclusione precedente ("nessuna coppia e' affidabile, si gestisce solo l'errore q eseguito test reali con moduli exchange 3 e teams leggermente piu' vecchi?" Aveva ragione a insistere piu' recente. Prima di implementare, una domanda intermedia dell'utente ("puo' essere un problema c Exchange poi Teams, ma CONNECT in ordine inverso (Teams prima) - fallisce comunque, confermando IMPORTA il modulo. Trovato poi che ExchangeOnlineManagement 3.1.0 e' la piu' vecchia versione con

		<code>Connect-IPSSession</code> per Purview, presente da 3.0.0) - testata dal vivo in coppia con Teams 5.0.0 e PRIMA, tutti e quattro i passaggi del progetto (App-only certificato, delegato device-code via <code>-Access</code> risultavano puliti, senza alcun conflitto - ripetuto due volte per escludere un caso fortuito. Fissata la C <code>Assert-M365OpsExoSafeVersion.ps1</code>, incluso il controllo di integrità dei file introdotto nel fix per <code>Connect-M365OpsTeams.ps1</code> e, in modalità solo-installazione, da <code>Install-M365OpsPrerequisites</code> su 3.1.0 non supporta ALCUN flusso delegato device-code/token (solo <code>-UserPrincipalName</code> / <code>-Create</code> invece del criptico errore di parametro mancante. Bug collaterale corretto nello stesso giro: <code>-ShowBanner</code> dinamicamente come già fatto per <code>-DisableWAM</code> in <code>Connect-M365OpsTeams.ps1</code>. Verificato dal v0.9.48, ha scoperto perché questo non bastava): <code>Connect-M365OpsExchange</code>, <code>Connect-M365Ops</code> ogni chiamata falliva pulita solo sulle credenziali finte usate per il test.
CORREZIONE il giorno stesso: 3.1.0 sembrava a posto con credenziali finte ma era in realtà INUTILIZZABILE per Purview con credenziali reali (protocollo dismesso da Microsoft) - corretto il pin a 3.4.0, implementato il login delegato interattivo a Purview, aggiunto il relativo pulsante in GUI (mai esistito prima) (v0.9.48, 25/08/2026)	corretto	Dopo aver implementato v0.9.47, l'utente ha chiesto "esiste un'altra via per Purview [delegato]?" - indi: <code>M365OpsCompliance</code> dal vivo nella GUI vera, sul tenant REALE <code>vnsys-test</code> (non con credenziali fin failed... WS-Management protocol" - un errore di rete/protocollo, non un conflitto di assembly. Causa che Microsoft ha dismesso LATO SERVER dal 15 luglio 2023 (confermato dal banner stesso del modulo test precedenti con credenziali finte non lo avevano mai scoperto perché fallivano PRIMA, sul controllo verifica l'assenza di un conflitto di assembly, non che una funzionalità sia davvero utilizzabile in produzione, stesso annuncio) - dato che 3.1.0-3.4.0 condividono TUTTE la stessa build di <code>Microsoft.IdentityModel.Clients.ActiveDirectory</code> (che non è stata mai aggiornata, stesso comportamento di caricamento) - riverificare "nessun errore". Pin aggiornato ovunque nel codice da 3.1.0 a 3.4.0. Login delegato a Purview implementato <code>AccessToken</code>, ma <code>-UserPrincipalName</code> da solo attiva un vero prompt di autenticazione moderna sicuro dal punto di vista del conflitto Teams perché passa dalla stessa libreria condivisa già verificata e il solo <code>-UserPrincipalName</code>. Scoperto nello stesso giro che Purview Delegata non aveva NEEDED <code>AllowInteractive</code>, quindi falliva sempre con "vai a connetterlo" senza che esistesse davvero con nuovo endpoint <code>POST /api/compliance-test</code> e stato esposto in <code>Get-M365OpsActiveTenant</code> sequenza Purview → Exchange → Teams nel tab Tenant - tutti e tre connessi con successo nello stesso riallineata alla coppia corretta 3.4.0/6.5.0 (rimossa ogni altra versione installata durante l'indagine).
BUG REALE trovato su una macchina di test nuova: Intune App-only rifiutava un profilo configurato SOLO con certificato ("non ha un SecretEnvVar configurato"), anche se lo stesso certificato funzionava già per Exchange sullo stesso profilo (v0.9.49, 25/08/2026)	corretto	Segnalato dal vivo: "da macchina nuova di test, provo connessione ad Intune con app e ottengo Error progetto a richiedere sempre e solo il secret in App-only, senza mai accettare il certificato come altern (vedi <code>Set-M365OpsTenant</code>). Verificato dal vivo sul modulo <code>IntuneWin32App</code> installato che <code>Connect-M365OpsIntune</code> esisteva da sempre, semplicemente non veniva mai usato. Corretto aggiungendo lo stesso fallback cerca il certificato in <code>Cert:\CurrentUser\My</code> poi <code>Cert:\LocalMachine\My</code> (stessi due store già configurati, il messaggio d'errore ora lo dice esplicitamente invece di nominare solo il secret. Verificato usato per Exchange sul tenant di test, nessun secret) e chiamata <code>Connect-M365OpsIntune</code> per davvero.
TRE bug reali segnalati dal vivo dopo un test pulito (nessun riavvio server nel mezzo) del login delegato Exchange: (1) poll "fantasma" mostrava "Nessun login Exchange in corso" a connessione già riuscita; (2) voce "Sessione delegata" in Ambiente non completo confondeva la sessione Graph generica con quella Exchange; (3) salvare il profilo tenant GIA' ATTIVO (es. Intune restava "connesso" dopo aver modificato/riattivato il profilo) non ne resettava mai lo stato di connessione (v0.9.50, 26/08/2026)	corretto	L'utente ha incollato il codice dispositivo, atteso il completamento e ricevuto "Errore: Nessun login Exchange connessione risultava GIA' attiva, provando che non era un fallimento reale. Causa radice: <code>Gui\Serve</code> davvero il login impiega qualche secondo in più (dentro <code>Complete-M365OpsExchangeDelegatedLogin</code> documentato nel suo stesso <code>.SYNOPSIS</code>) e il <code>fetch()</code> lato browser si interrompe prima (rete/proxy finito il primo, ormai riuscito e con la voce "pending" già rimossa: il poll "in più" trova il dizionario vuoto <code>M365OpsExchangeDelegatedLogin.ps1</code>, prima di dichiarare errore per "nessuna voce pending", ora un errore fantasma, stesso principio già usato per Intune in <code>Test-M365OpsIntuneAuthValid</code> (mai che prima lasciava il pannello di stato bloccato sul messaggio vecchio finché l'utente non cambiava tab autocorregge subito invece di restare visibile. Analizzando lo stesso report, la voce "Sessione delegata sessione attiva" a login Exchange già riuscito) si è rivelata corretta ma ambigua: controlla la sessione Exchange PowerShell appena stabilita - rinominata in "Sessione Microsoft Graph delegata (generale)" Investigando infine la segnalazione separata "torno sul tenant delegato e Intune risulta ancora connesso dentro <code>Connect-M365Ops</code>, chiamato da <code>POST /api/tenants/activate</code> - ma il gemello "Salva profilo (autenticazione) non lo richiamava mai dopo <code>Set-M365OpsTenant</code>, a differenza di <code>POST /api/emails</code> fermi ai valori di prima della modifica. Corretto richiamando <code>Connect-M365Ops -TenantProfile \$ (\$script:ActiveTenantProfile -eq \$body.name)</code> - un salvataggio di un profilo NON attivo rest non riproducibili in isolamento senza un tenant Delegato reale e una finestra di rete instabile - il raggio invitato a confermare col prossimo test naturale.
GUI Tenant: sezione "Aggiungi/aggiorna profilo" ora crollata di default con un pulsante "Modifica" per riga che la apre precompilata; corretto anche il glitch grafico gemello Intune (messaggio "Connesso" rimasto accanto al pulsante dopo un vero disconnect) (v0.9.51, 26/08/2026)	implementato	Richiesto esplicitamente dall'utente: "la sezione aggiungi/aggiorna profilo deve essere crollata e non si capisce se per aggiornare vnys debba scrivere il nome o che altro fare". La sezione (tab Tenant) e' crollata. Ogni riga di <code>#profile-list</code> ha un nuovo pulsante "Modifica" che apre la sezione GIA' precompilata (thumbprint certificato) - MAI il valore del secret, che non e' salvato in nessun file (solo il suo nome var campi aggiuntivi (prima esprimeva solo nome/tenantid/authMode, insufficiente per precompilare). Un restare aperta col form "consumato". Salvare con lo stesso nome resta un aggiornamento (upsert), no vivo nel browser (server locale avviato apposta sul profilo reale <code>vnsys-test</code> di questa macchina di "Annulla" - sezione chiusa, form svuotato; creato un profilo di prova, salvato (sezione chiusa da sola con <code>Set-M365OpsTenant</code> stesso giro, segnalato dal vivo dall'utente mentre testava il fix del 21/08/2026 sul reset dei flag di connessione correttamente rosso ("Non connesso") ma il testo accanto al pulsante "Connetti Intune" restava fermo Exchange/SharePoint/Teams/Purview (messaggio accanto al pulsante mai ripulito da un refresh di statistiche) altri quattro riquadri in <code>loadConnectionStatus()</code>.
ROOT CAUSE REALE del "Nessun login Exchange in corso" trovata riverificando dal vivo il fix v0.9.50: un pipeline leak trasformava la risposta del login riuscito in un array malformato (Status sempre undefined lato client), non un poll duplicato/ritardato come ipotizzato - scoperto un BUG DI SICUREZZA nello stesso punto, <code>AccessToken/RefreshToken</code> veri arrivavano fino al browser via JSON (v0.9.52, 26/08/2026)	corretto	Su richiesta esplicita dell'utente ("fai questo prima" - riverificare il fix v0.9.50 con un riavvio pulito prima "vnys delegata"), non solo per lettura del codice. Il primo poll dopo il login mostrava un risultato inatteso (corretta a meta) in v0.9.50: dentro <code>Complete-M365OpsExchangeDelegatedLogin.ps1</code>, la chiamata soppressa ne' catturata - il suo valore di ritorno finiva quindi ANCH'ESSO sull'output della funzione, in <code>undefined</code>: ne' il ramo "Completed" ne' quello "Error" scattavano mai su un login riuscito, il poll corretto (consumato dal primo poll, quello riuscito ma malformato): QUESTO e' il momento in cui compariva di <code>\$script:M365OpsExchangeConnected</code> prima di dichiarare errore) restava corretto e utile come rete per <code>M365OpsExoSafeVersion</code> (Out-Null) e riscrivendo la funzione per costruire sempre esplicitamente

		momento un problema piu' serio, di sicurezza: nel ramo di successo, la vecchia funzione restituiva VERI (servono per passarli a <code>Connect-ExchangeOnline -AccessToken</code> subito dopo) - <code>Gui\Serve</code> ottenere nuovi access token senza rifare login) arrivava quindi fino al browser/JS senza che servisse m esplicitamente, mai un oggetto intermedio grezzo, in ogni ramo (<code>Completed/Pending/Error</code>). Fix gem <code>M3650psExchange.ps1</code> e <code>Connect-M3650psCompliance.ps1</code>) eseguiva <code>& \$Action</code> senza soppr <code>Connect-ExchangeOnline / Connect-IPPSession</code> avrebbe potuto propagarsi con lo stesso ider to-end due volte, non per lettura del codice: prima il bug riprodotto tale e quale (array malformato c secondo login delegato reale da zero - risposta del poll un singolo oggetto pulito <code>{Status:"Comple</code> (<code>exchangeConnected:true</code>).
Nuovo connettore AI: server MCP CLI Microsoft 365 (@pnp/cli-microsoft365-mcp-server) collegato al motore di ragionamento in chat, con lo stesso livello di sicurezza (proposta+conferma) gia' in uso per Lokka; livello di connessione MCP reso generico (Lokka non e' piu' l'unico server tracciabile); parecchi dettagli reali del pacchetto risultati diversi dalla sua documentazione pubblica, corretti dopo test dal vivo (v0.9.53, 26/08/2026)	implementato	Richiesto esplicitamente dall'utente: valutare se CLI Microsoft 365 (pnp.github.io/cli-microsoft365) pot implementare: gruppi di comandi confermati sulla documentazione ufficiale - spo, entra, outlook, pla Platform. Il gruppo 'exo' esiste ma e' stretto (solo application role assignment, non gestione mailbox/E Admin Center" mai esposta da questa CLI). Conclusione: NON puo' sostituire i moduli PS nelle due are complemento per SharePoint/Entra/Outlook/Planner/Purview-audit, mai un fallback generale. Archite <code>\$script:M3650psLokkaProcess / ...LokkaTools</code>) - nuove funzioni <code>Connect-M3650psMcpServ</code> Public), stato in due dizionari per nome (<code>\$script:M3650psMcpProcesses / ...McpTools</code>) cosi' pit altro codice esistente toccato. (2) <code>Connect-M3650psCliMicrosoft365.ps1</code> (nuovo): sincronizza l'a l'utente Windows, non per sessione) col tenant M365Ops attivo usando connessioni nominate (<code>m365</code> per evitare che lo stato di un tenant precedente sopravviva a un cambio profilo (Intune, sezione 6.4). S solo a certificato o un tenant Delegato vengono rifiutati con un messaggio chiaro invece di un tentativ (<code>cli_m365_run_command / cli_m365_search_commands / cli_m365_get_command_docs / prop propose_cli_m365_command</code> (stesso schema <code>\$pendingWrite / \$script:PendingAction / Exe</code> distingue lettura da scrittura al suo interno (un solo tool generico esegue qualunque comando) - nuov ~400 comandi a mano: <code>get/list/search/export/status</code> = lettura, ogni altro verbo (incluso uno reali diversi dalla documentazione pubblica, trovati SOLO chiamando davvero il server MCP inst terze parti che lo elencano); ogni comando deve includere il prefisso letterale <code>"m365 "</code> (rifiutato altr automaticamente, un suggerimento embedded nella risposta di <code>m365_search_commands</code> lo dice es <code>m365_search_commands</code> , non un solo parametro come indicato altrove. Limite reale piu' importar "SharePoint does not support authentication using client ID and secret" - verificato con un comando recuperati con successo). SharePoint via questa integrazione resta quindi utilizzabile solo con un login <code>microsoft365</code> (il server MCP lo richiede installato globalmente, non lo scarica da solo) - un'installazi non si propaga a processi gia' avviati), quindi il solo pulsante "Riavvia server" non basta la prima volta criptico <code>"m365' is not recognized..."</code>. Verificato dal vivo end-to-end, non per lettura del codice: sul connessione gia' esistente (<code>connection use</code>) verificata separatamente, lettura reale <code>entra user</code> : ferma il processo, <code>McpServerStatus</code> torna a "non connesso"), e il messaggio d'errore azionabile rip Riquadro GUI "Altri server MCP" (tab MCP/Connettori) esteso con un pulsante "Connetti/Test" e uno s mostrato l'esito cancellava il messaggio all'istante, corretto con un ritardo breve.
Ogni risposta dell'AI in chat (e ogni riga di log per tool chiamato) mostra ora la SORGENTE reale usata - Lokka (MCP), CLI Microsoft 365 (MCP), Graph diretto (Delegato) o moduli PowerShell interni; aggiunto avviso "server a thread singolo" per CLI Microsoft 365, analizzato un errore ricorrente nei log all'avvio (v0.9.54, 26/08/2026)	implementato	Richiesto esplicitamente dall'utente dopo l'introduzione di CLI Microsoft 365: sapere sempre quale ba dice se e' passato da Lokka o da una chiamata diretta. Nuova <code>Get-M3650psToolSourceLabel</code> (Priv passa mai da Lokka ma da Graph diretto con token delegato - etichetta diversa in quel caso). Ogni rig fondo una riga <i>Fonte dati:</i> ... con le sorgenti usate (deduplicate), SOLO se almeno un tool e' stato davv reale ("quanti utenti ci sono nel tenant?"): risposta "Nel tenant ci sono 55 utenti... Fonte dati: Lokka (M analizzato un errore visto ripetersi nei log dell'utente ad ogni avvio del server ("Errore interno: Excepti ambiente di sviluppo, tracciato a una connessione TCP speculativa/interrotta del browser subito dopo in modo sicuro dalla protezione del 18/08/2026 (il server non si interrompe mai per questo, loggato d questo (tutte 200 OK) - non e' un bug funzionante, resta solo rumore nei log, non e' stato modificato "Connetti/Test" puo' richiedere parecchi secondi al primo utilizzo (scaricamento npx) e blocca il resto c questo nuovo connettore.
BUG REALE trovato dal vivo dallo screenshot dell'utente: tab MCP/Connettori mostrava "Errore nel caricare i server MCP" quando nessun tenant e' attivo, invece di degradare come le altre sezioni della GUI (v0.9.55, 26/08/2026)	corretto	<code>Get-M3650psMcpServers</code> lanciava un'eccezione quando <code>\$script:M3650psContext</code> e' vuoto (ne: l'eccezione arrivava al gestore d'errore generico (risposta 500 con <code>{role;text}</code> , non un array), che solito banner "Attiva un profilo dal tab Tenant" gia' mostrato altrove. Nessun altro chiamante dipende che gia' fa lo stesso quando non c'e' nessun profilo salvato. Corretto nello stesso giro un bug gemello <code>/api/mcp-status</code> : quando nessun tenant e' attivo, <code>Get-M3650psActiveTenantInfo</code> restituisce <code>\$</code> con una voce fantasma (<code>{name:null,...}</code>) - stesso identico bug di scope gia' documentato e corre <code>ForEach-Object</code> . Verificato dal vivo, non per lettura del codice: modulo importato SENZA mai chiar
CLI Microsoft 365 esteso ai tenant Delegati (login a browser bloccante) e collegato anche nel tab Tenant, sezione "Stato connessioni" - prima disponibile solo App-only e solo dal tab MCP/Connettori (v0.9.56, 26/08/2026)	implementato	Due richieste esplicite dell'utente sullo stesso giro. (1) "la CLI supporta anche il login a browser, perche deviceCode' e' bloccante e quindi "non ancora implementato" - ragionamento incompleto, corretto qu stesso progetto per Teams/SharePoint/Intune su tenant Delegati (parametro <code>-AllowInteractive</code> e flusso composto). <code>Connect-M3650psCliMicrosoft365.ps1</code> ora accetta <code>-AllowInteractive</code> : se con, esegue <code>m365 login --authType browser --tenant ... --connectionName ...</code> (scelto t deviceCode e' anch'esso bloccante sulla CLI, nessun flusso a due passi su nessuno dei due). <code>Connect: MCP/Connettori</code> e il nuovo pulsante nel tab Tenant mostrano lo stesso avviso di conferma gia' usato f riquadro "CLI Microsoft 365" nella sezione "Stato connessioni" del tab Tenant (accanto a Exchange/Shi generico <code>data.mcpServers</code> - stesso backend del pulsante gia' esistente nel tab MCP/Connettori, so vivo: sul tenant App-only "vnsys-test" il pulsante nel tab Tenant connette correttamente (stesso comp processo); sul tenant Delegato "vnsys delegata", confermato che il pulsante richiede la conferma esplic "serve -AllowInteractive") - il completamento REALE del login a browser (che richiede un'interazione u
CLI Microsoft 365: installazione automatica del prerequisito mancante (npm install -g @pnp/cli-microsoft365), invece di chiedere solo all'utente di farlo a mano (v0.9.57, 26/08/2026)	implementato	Segnalato dal vivo dall'utente: dopo un riavvio completo, stesso errore "comando 'm365' non trovato" imperfetta, chiarita e poi colmata: Lokka e' completamente autosufficiente (<code>npx -y</code> scarica ed esegue sottile che invoca internamente il comando <code>m365</code> come processo ESTERNO gia' installato - <code>npx scari M3650psCliMicrosoft365Installed</code> (Private, stesso principio di <code>Assert-M3650psExoSafeVersi</code>

		subito invece di far partire un npx cold-start inutile), e se manca esegue da sola <code>npm install -g @m365ops/m365ops</code> (ma spiegato chiaramente invece di lasciarlo scoprire per errore: anche con l'installazione automatica r ATTUALE non vedra' comunque 'm365' finche' non viene riavviato per davvero (chiudere tutto e riaprir dell'installazione) - la funzione lo dice esplicitamente nel messaggio finale invece di un generico "insta di dire solo "esegui questo comando a mano". Verificato dal vivo che il percorso "gia' installato" non t reale <code>Connect-M365OpsMcpServer</code> .
Processi MCP isolati per TENANT (non piu' terminati ad ogni cambio profilo); CLI Microsoft 365 spostata dal tab MCP/Connettori al tab Tenant; installazione automatica out-of-the-box del prerequisito npm; rimossi i pin fissi di versione ExchangeOnlineManagement/MicrosoftTeams (ora sempre l'ultima disponibile) come primo passo verso l'isolamento reattivo dei processi Teams/Exchange (v0.9.58, 26/08/2026)	implementato	Tre richieste esplicite dell'utente sullo stesso giro. (1) "rendi ogni processo MCP isolato da altri tenant, condivisi tra TUTTI i tenant) - un solo processo alla volta, terminato e riavviato ad ogni cambio profilo processo OS distinto per ogni coppia, mai condiviso; il cambio tenant non termina piu' nulla, i process vivo (non per assunzione) un rischio reale nel farlo: l'identita' "attiva" di CLI Microsoft 365 vive in un fil di due tenant diversi potrebbero quindi in teoria interferire tramite quel file condiviso. Senza poter ver cache al login, la scelta e' stata quella piu' prudente su un'operazione che tocca dati reali di un tenant comando reale (mai sulle sole chiamate di ricerca/documentazione, che non toccano dati di un tenant verificabile. <code>Disconnect-M365OpsAllMcpServers</code> resta per una pulizia totale (riavvio vero del serve cli"; il pulsante "Connetti/Test" di CLI Microsoft 365 e' stato tolto dal tab MCP/Connettori (che ora mo: insieme a Exchange/SharePoint/Teams/Purview/Intune, coerente col fatto che tutti sono specifici del t <code>install -g @m365ops/cli-microsoft365</code> aggiunto sia a <code>Show-M365OpsPrereqInstaller.ps1</code> (l'ins PATH, niente piu' "serve un riavvio completo" per il caso comune) sia a <code>Install-M365OpsPrerequis</code> sicurezza per chi salta entrambi questi percorsi. Nello stesso giro, primo passo verso l'isolamento rea <code>M365OpsExoSafeVersion / Assert-M365OpsTeamsSafeVersion</code> non pinnano piu' una versione fis disco, senza contattare PSGallery ad ogni connessione (solo quando serve davvero installare) per non cura una coppia di versioni - non regge piu' una volta che il conflitto viene gestito isolando la connes questa sessione, v0.9.50 e v0.9.52). Aggiornati in coerenza <code>Invoke-M365OpsWithExoRepairRetry.p</code> <code>M365OpsPrerequisites.ps1</code> (nessun pin, nessuna disinstallazione mirata per soglia). Verificato con switch su un secondo tenant senza terminare il primo, switch di ritorno con riuso confermato dai log, c
Isolamento reattivo dei processi Teams/Exchange: il conflitto .NET di sezione 6.6 viene ora risolto DAVVERO nello stesso turno (sottoprocesso separato per il modulo in conflitto), senza più richiedere un riavvio del server - due bug reali trovati e corretti durante la verifica dal vivo (v0.9.59, 26/08/2026)	implementato	Completamento del lavoro iniziato in v0.9.58 (rimozione dei pin di versione come base per questo). N importa SOLO <code>ExchangeOnlineManagement</code> OPPURE SOLO <code>MicrosoftTeams</code> (mai entrambi), cor <code>M365OpsExchange.ps1 / Connect-M365OpsTeams.ps1 / Connect-M365OpsCompliance.ps1</code> tent: <code>M365OpsModuleConflictHint</code> riconosce davvero il pattern del conflitto in un'eccezione reale già av worker, alla connessione, riporta al processo padre l'elenco COMPLETO dei comandi che è diventato d ciascuno (uno SCRIPTBLOCK condiviso installato sotto ~800 nomi diversi via <code>Set-Item function:g</code> verificato dal vivo che <code>\$MyInvocation.MyCommand.Name</code> riporta correttamente il nome con cui il sir quei cmdlet da QUALUNQUE punto del modulo M365Ops viene automaticamente intercettata e inoltr la prima verifica end-to-end (non teorico): un primo tentativo filtrava i comandi resi disponibili con <code>Get-Mailbox, Get-DistributionGroup</code> compresi, entrambi mancanti) - causa reale, non un problema di <code>ExchangeOnline</code> inietta la stragrande maggioranza dei comandi tramite implicit remoting in un moc modulo li manca quasi tutti per costruzione. Corretto con un diff PRIMA/DOPO sui nomi di TUTTI i co: automaticamente qualunque comando il connect abbia reso disponibile, ovunque viva; verificato dal v attraverso il proxy. SECONDO bug reale trovato nello stesso giro di verifica: un fallimento di <code>Conn</code> SOLO sullo stderr del worker, che nessun chiamante legge mai quando il "connect" nel suo complesso Corretto restituendo esplicitamente <code>purviewConnected / purviewError</code> nel risultato del "connect davvero. Anche con questo fix, <code>Get-RetentionCompliancePolicy</code> è risultato assente sul tenant di architettura (vedi sezione 6.6 per il dettaglio completo): limite pre-esistente del tenant/versione modu sostituita una singola attesa fissa di 2s con un polling che si ferma solo quando il conteggio totale dei errore isolato di conflitto assembly su <code>Get-CASMailbox</code> visto UNA SOLA volta durante i test dal vivc anomalia rara, non riprodotta, monitorabile ora più facilmente grazie al logging aggiunto in questo st poi Exchange - isolamento attivato automaticamente e confermato nei log: <code>Get-Mailbox/Get-Distribut</code> tutta la durata; disconnessione del worker isolato confermata pulita a fine sessione/cambio tenant.
Bug-hunt dedicato di 3 ore (mandato esplicito dell'utente): trovati e corretti un pipeline leak sistemico in tre funzioni di connessione condivise, 15 cmdlet di scrittura Exchange senza protezione da errori non terminanti, un limite CLI Microsoft 365 evitabile, e un vero buco nell'isolamento reattivo di sezione 6.6 (il conflitto può manifestarsi anche DOPO un connect riuscito, non solo durante) - tutti riprodotti e verificati dal vivo sul tenant reale "vnsys-test", non ipotizzati (v0.9.60, 26/08/2026)	corretto	Sessione di test strutturata come richiesto esplicitamente dall'utente: (1) verifica del fresh-install (revis obsoleti - un messaggio d'errore che citava ancora "10s" invece del budget reale di 30s alzato in una s browser (tutti i tab, form, pulsanti, dark mode, edge case come submit con campi vuoti, click ripetuti, t non ipotetico) - 96 riusciti, i 6 "falliti" tutti triagati singolarmente: 2 limiti di permesso Graph reali e già mancanti/sbagliati passati apposta per zero-parametri, non bug del prodotto); (4) audit reale (non teo (dettaglio in fondo a questa voce). BUG PIÙ SIGNIFICATIVO: pipeline leak in <code>Connect-M365OpsExchange.ps1 / Connect-M365OpsT</code> <code>M365OpsExchangeDelegatedLogin.ps1</code> (v0.9.52: <code>Assert-M365OpsExoSafeVersion</code> non soppres chiamate CONDIVISE a <code>Assert-M365OpsExoSafeVersion / Assert-M365OpsTeamsSafeVersion</code> , Exchange/Teams/Purview come primo passo di un processo server appena avviato ereditava lo stesso mostrata all'utente conteneva un array con l'oggetto di diagnostica versione ANCHE insieme al risulta dal vivo PRIMA (bug riprodotto) e DOPO (risposta pulita, un solo oggetto) con due cicli identici di crea 15 cmdlet di scrittura senza <code>-ErrorAction Stop</code> - stessa classe di bug già corretta UNA VOLTA p errori NON terminanti, che senza <code>-ErrorAction Stop</code> vengono ignorati in silenzio - la funzione dic scansione automatica mirata ai verbi New/Set/Remove/Enable/Disable ha trovato e corretto 12 file (<code>New-M365OpsEquipmentMailbox</code> , <code>New-M365OpsMailContact</code> , <code>New-M365OpsMigrationBatch</code> , <code>New-M365OpsMailContact</code> , <code>Remove-M365OpsTransportRule</code> , <code>Set-M365OpsTransportRuleState</code>); 3 file sfuggiti alla prima perché usano verbi nativi meno comuni non coperti dal primo filtro (<code>Release-M365OpsMigrationBatch</code> , verbo "Start" già nel filtro ma il file era stato inizialmente saltato) - lezione dal vivo, non solo per lettura del codice: creato un contatto di posta reale con <code>New-M365OpsMailC</code> genere di fallimento sarebbe passato silenziosamente come successo; DOPO il fix, l'eccezione reale di Graph (<code>Invoke-M365OpsGraphRequest</code> , già con <code>-ErrorAction Stop</code> al suo interno), <code>SharePoint/</code>

		correttamente) - il problema era specifico delle chiamate dirette a cmdlet nativi Exchange/Teams spar CLI Microsoft 365: eliminato un riavvio non più necessario - <code>Assert-M365OpsCliMicrosoft365</code> sempre un errore "riavvia tutto", credendo la limitazione strutturale (Windows non propaga un PATH c spawna il sottoprocesso <code>npx</code> del server MCP CLI-Microsoft365 (<code>Connect-M365OpsMcpServer.ps1</code> altrove nel progetto per lo stesso scenario, <code>Install-M365OpsPrerequisites.ps1</code> / <code>Show-M365Ops</code> Verificato dal vivo end-to-end attraverso il flusso reale: una domanda in chat sui piani Planner ha i completo del PC) ha connesso CLI Microsoft 365 con successo e usato realmente lo strumento (corret strumento stesso, non di M365Ops, riportato onestamente invece di essere nascosto o inventato). Buco reale nell'isolamento reattivo di sezione 6.6: vedi quella sezione per il dettaglio completo (nu <code>M365OpsModuleConflictHint</code>) - scoperto durante un test dal vivo apparentemente innocuo (rinomi lasciato un'instabilità residua nel processo oltre alla singola operazione che lo ha fatto emergere - lett token JWT); confermato con un confronto diretto (stessa query in un processo nuovo: nessun errore) c totale - dettaglio e raccomandazione onesta in sezione 6.6. Altri fix minori trovati durante il giro GUI: (a) i pulsanti Si/No di una proposta di scrittura risolta scr non riesegue nulla (il server ignora correttamente un 'si'/no' orfano, verificato dal vivo), ma restava co messaggio, di qualunque tipo (<code>Gui\index.html</code>); (b) il riquadro "Stato connessioni" del tab Tenant, senza menzionare l'isolamento reattivo introdotto in v0.9.59 - aggiornato per riflettere la soluzione re Audit reale CLI Microsoft 365 vs Lokka vs script PS1 interni (richiesto esplicitamente, verificato cor Lokka (Graph diretto, dati di gruppo M365 di base); creazione/rimozione di uno Scope Tag Intune → n regola scritta nel prompt di sistema); creazione/rimozione di una distribution list e di un gruppo di sic query sui siti SharePoint con più storage → moduli interni (PnP, mai Graph/CLI M365 per SharePoint, c Planner → prima Lokka (403, permesso mancante), poi CLI Microsoft 365 come fallback dichiarato (lim confermato nella pratica non solo sulla carta.
Log scritture separato + trasparenza completa su fonte/comando eseguito per ogni scrittura confermata (anche nel turno di esecuzione); isolamento reattivo Teams/Exchange/Purview esteso ai tenant Delegati; corretto un glitch reale del modello AI che dichiarava una scrittura già' eseguita (con un ID inventato) nella stessa risposta in cui la proponeva soltanto; guida aggiornata su CLI Microsoft 365 (v0.9.61, 27/08/2026)	implementato	Quattro richieste/scoperte sullo stesso giro, in ordine di priorita' dichiarata dall'utente. (1) "una finestr (Public) scrive in <code>Logs\writes-YYYYMMDD.log</code> - deliberatamente FUORI dal pattern glob <code>m365ops</code> esteso con <code>-LogFilePrefix</code> (default invariato <code>'m365ops'</code>) per riusare la stessa logica di scansion Manutenzione, Cronologia log) ha ora un selettore "Log generale / Solo scritture" prima del periodo, s il comando ps che sta per lanciare, sia che sia Lokka, CLI 365 o qualunque cosa": la proposta PRIMA di non lo mostrava mai, ne' la sorgente - <code>Execute-PendingAction</code> non passa mai dall'AI (che e' l'unic <code>M365OpsWriteResponse</code> (helper condiviso in <code>Gui\Server.ps1</code>) applicato a TUTTI i case di scrittura: (ExoWrite/TeamsWrite/SharePointWrite/IntuneWrite/CustomWrite/LokkaWrite/CliM365Write/CreateG Comando eseguito: <code>...-</code> al messaggio "Fatto." mostrato in chat. Nuovo <code>Format-M365OpsComma</code> annidati); nuovo <code>Get-M365OpsWriteActionSourceLabel</code> (Public) mappa il <code>Type</code> di un'azione con Delegato). Anche i FALLIMENTI vengono loggati nel nuovo log scritture, in UN SOLO punto centrale (i ricostruito e il messaggio d'errore reale come dettaglio. Guida: nuova sezione 10.10, risposta a una dc delegato Graph e quello di CLI Microsoft 365 usano DUE client pubblici Microsoft separati con stato d l'eccezione reale (policy tenant "Users can consent to apps" disabilitata, che blocca comunque entram dal vivo: controllo sintassi su tutti i file toccati e reimport pulito del modulo (funzioni nuove visibili); u 10.10 non richiuso correttamente (chiudeva con <code></p></code> invece di <code></div></code>), trovato con un controllo Isolamento reattivo esteso ai tenant Delegati (priorita' massima, richiesta urgente dal vivo): sub connessi, poi Purview fallito con lo stesso errore "Microsoft.Identity.Client.dll... manifest definition doe lanciava un errore esplicito su Delegato, quindi il tentativo di isolamento falliva a sua volta e l'utente v Delegati (UserPrincipalName per Exchange, TenantId+DisableWAM per Teams) invece di lanciare un e ExchangeOnline -UserPrincipalName (popup interattivo) poi tenta <code>Connect-IPPSession</code> nell deliberatamente NIENTE device-code (<code>-Device</code>) in questo worker, a differenza del processo princ umano e la richiesta resterebbe bloccata fino al timeout (stessa classe di bug già' vista con Teams dev alzato da 60s a 240s quando il tenant e' Delegato (un popup richiede un'azione umana, non solo calcc TUTTO SUO (un popup in piu' da completare) la prima volta che serve per quel tenant in quel process test funzionale diretto di <code>Get-M365OpsIsolatedConnectParams</code> (parametri corretti per Exchange/I proprio richiede un login umano reale con MFA che non e' stato possibile eseguire in autonomia in qu Glitch reale trovato e corretto durante il test dal vivo del log scritture: creando un modello di not risposta ha ANCHE scritto un testo tipo "La creazione risulta confermata ed eseguita correttamente... e diverso. La "REGOLA CRITICA ASSOLUTA anti-fabbricazione" già' nel system prompt (aggiunta il 19/08 (1) system prompt rinforzato con l'esempio negativo esatto appena osservato, incluso l'avvertimento e <code>Gui\Server.ps1</code> - quando una scrittura risulta genuinamente proposta (<code>PendingWrite</code> valorizza' già' avvenuto ("risulta eseguita", "e' stato creato", "ho eliminato", ecc.); se corrisponde, si antepone un Warn per telemetria. Verificato dal vivo: riprodotto il bug originale (ID inventato confermato diverso tentativo successivo ("Non e' ancora stato creato: la proposta e' in attesa di conferma"), la rete di sicur
Maratona di bug-hunt (16 ore, richiesta esplicita) - 11 bug reali trovati e corretti su Intune/Exchange/GUI/report/console, logo animato in header, corretto un errore criptico ricorrente nei log (v0.9.62, 23/08/2026)	implementato	Prima meta' della maratona di stress-test/bug-hunt di almeno 16 ore richiesta esplicitamente dall'uter SharePoint/Entra/report/MCP e la GUI/installer, incrociando ogni allowlist/descrizione strumento AI in naturale (incluso lo stile "utente confuso" richiesto). Trovati e corretti: Intune - 7 disallineamenti reali tra la descrizione mostrata all'AI e i parametri veri (<code>Invoke-M36 Technologies</code> come opzionale quando e' MANDATORY - un parametro mandatory mancante in un (stessa classe di bug già' vista per WAM/device-code); <code>New-M365OpsDeviceScript</code> documentava ' <code>M365OpsAppProtectionPolicy</code> , <code>New-M365OpsEnrollmentPlatformRestriction</code> , <code>New-M365Ops cmdlet Exchange); <code>Get-M365OpsConfigurationSettingDefinitions</code> documentava <code>SearchText</code> DOPO che l'utente aveva già' confermato la proposta (l'esecuzione usa uno splat diretto <code>& \$cmdlet</code></code>

		nel suo docstring): l'azione Graph "assign" SOSTITUISCE sempre l'intera collezione di assegnazioni del - una seconda chiamata su un gruppo diverso cancella silenziosamente la prima. Exchange - fallimento totale riportato per un successo parziale: <code>New-M365OpsDistributionGro</code> l'eccezione di <code>Add-DistributionGroupMember</code> (con <code>-ErrorAction Stop</code>, corretto in un giro pre esisteva già' sul tenant. Ora ogni membro viene aggiunto con il proprio try/catch: il gruppo conta sem Fuga di dati tra tenant: <code>\$script:LastReportPath</code> (l'ultimo report generato, usato da <code>propose_s</code> tenant (Intune/Exchange/Teams/SharePoint) - un report del Tenant A poteva finire allegato a un'email GUI - sicurezza sulla conferma "sì" (il più serio): il controllo era <code>'^(si ok conferma procedi va</code> di backup?"), quindi una domanda di follow-up con un'azione distruttiva già' in sospeso (es. un reset M (ripulito da punteggiatura finale) sia una parola di conferma nota, non solo la prima - riprodotto dal vi separata. GUI - altri 3 bug reali: (1) il catalogo comandi locale intercettava "resetta l'mfa di X" (contiene "mfa d verbi di reset/rimozione alle DeferWords che fanno passare il messaggio all'AI; (2) "panoramica gruppi corretta per fermarsi al primo segnale di frase successiva (virgola o "e" + verbo di continuazione); (3) u richiesta composta di ripacchettizzazione + nuovo gruppo - corretto escludendo il participio passato c Console/log - errore criptico ricorrente: "Errore interno: Exception calling 'Write' with '3' argument(s dall'utente - causa reale: un client disconnesso (tab chiusa, fetch interrotto) esattamente durante l'invi già' protetta da un fix dell'18/08/2026) - ora isolata in un try/catch dedicato, classificata correttamente Logo animato (richiesto dall'utente durante la maratona): SVG inline di una rete neurale (2-3-2 nodi) nell'header - nessun asset esterno, colori legati a <code>var(--accent) / var(--accent-strong)</code> quind Verificato dal vivo: sintassi su tutti i 382 file + reimport pulito del modulo dopo ogni gruppo di modi (creazione/eliminazione di due modelli di notifica Intune di test, riproduzione dal vivo del bug "sì" seg disallineamenti Intune e la fuga di report tra tenant sono stati verificati per lettura diretta dei <code>param(</code> verificare sempre" già' in uso in questo progetto. Non ancora verificato dal vivo con un login umano r <code>M365OpsAdminTemplateSetting</code> segnalata come sospetta ma non confermata (nota aggiunta alla d
Maratona di bug-hunt (16 ore, seconda parte) - 9 bug reali in più nel catalogo comandi locale di chat (Gui\CommandCatalog.ps1), 1 sistemico su 6 voci (v0.9.63, 23/08/2026)	implementato	Un secondo agente di audit, dedicato stavolta esclusivamente a <code>Gui\CommandCatalog.ps1</code> (dopo c locale (a costo zero, bypassa l'AI) che intercetta un messaggio il cui VERO intento è diverso da quello ListNonCompliant/UserOverview/MfaStatus/GroupOverview/CompliancePatterns/ListDevices): l'account" (virgola invece di "e poi") non veniva deviato, la seconda parte spariva in silenzio. Aggiunte riferito a tutt'altro (phishing/attacchi/naming) - aggiunte DeferWords dedicate (impossibile richiedere non aveva alcun controllo sul verbo - "aggiungi il permesso FullAccess sulla mailbox condivisa X" veni stessa sessione). <code>ExportMailboxUsageReport/ExportForwardingReport</code> avevano un'alternativa di trigge con la risposta alla domanda). <code>ExportAllMailboxesReport</code> non gestiva la negazione ("non tutte le mailb su 5 pattern (la voce gemella <code>ExportCompliancePatterns</code> era già' stata corretta a '0,30') il 18/08/2026, export dispositivi grezzo. Help usava la stringa letterale 'bloccat' (solo participio passato) invece di 'blc sintassi ok; ogni singolo fix testato con un mini-harness PowerShell che riproduce esattamente la logic dall'agente, confermando che ognuno ora devia correttamente all'AI; il caso "stato mfa di X, poi dimm catalogo locale, mostrando solo lo stato MFA - dopo il fix, la richiesta passa correttamente all'AI e risp
Maratona di bug-hunt (16 ore, terza parte) - fuga di dati tra tenant su nomi profilo simili, crash che perdeva un Excel già' generato, log scritture incompleto, packaging .msi rotto, glitch AI su risultato vuoto (v0.9.64, 23/08/2026)	implementato	Ripresa della maratona dopo una pausa forzata dal limite di utilizzo (4 ore, come previsto dall'utente s Win32/Autopilot/Installer Insight; generazione report e persistenza chat/Knowledge Base) più' un bug Fuga di dati tra tenant (il più serio, riprodotto dal vivo): storico chat e Knowledge Base sono salva regex NON è iniettiva ("North West" e "North_West", o "Contoso.Test" e "Contoso Test", si riducono a finivano a condividere lo stesso file di storico/KB - conversazioni e documenti di un tenant visibili nell' messaggio che spiega perché'. Crash che perdeva un Excel già' generato (riprodotto dal vivo): <code>Export-M365OpsDataReport</code> cc - un campo con un oggetto Graph nidificato singolo (es. <code>signInActivity</code>, comune) fa lanciare a <code>G</code> dall'intera funzione perdendo anche l'xlsx già' scritto con successo poco sopra. Corretto alla radice in come prima - questo elimina anche la divergenza silenziosa già' esistente tra Excel, che scriveva una c all'intero blocco di costruzione PDF, così' un problema imprevisto futuro degrada sempre a un avviso, genera senza avvisi. Log scritture incompleto: il blocco catch generico di <code>Execute-PendingAction</code> era l'UNICO punto come risultato normale invece che come eccezione, <code>Clm365Write</code> con lo stesso schema, <code>PackageApp/</code> assenti dal log nonostante fossero fallimenti di scrittura reali e quotidiani. Nuovo <code>Write-M365OpsWri</code> futuro che dimenticasse di loggare da solo. Packaging .msi rotto: un file .msi caricato produceva un <code>InstallCommandLine</code> tipo "App.msi /S" - costruisce <code>msiexec /i "file.msi" /quiet /norestart</code>; l'uninstall (serve la ProductCode, non di riconosciuta. Guardia mancante su AssignApp standalone: il ramo singolo di assegnazione app usava lo stesso tr assegnato l'app come available a tutti ma alcuni utenti non riescono a installarla, sai perché'?" propon Glitch AI su risultato vuoto (trovato e diagnosticato dal vivo): alla domanda "mostrami i modelli a ma la risposta finale è stata "Il report Excel dei dispositivi Autopilot è pronto" - il testo di uno scambi

		Aggiunta una nuova regola al prompt che impone di dichiarare esplicitamente un risultato vuoto per l corretta ("Non risultano modelli amministrativi configurati: 0 modelli").
Maratona di bug-hunt (16 ore, quarta parte) - retry via isolamento reattivo anche sui dati Teams (non solo sulla connessione), colonna Teams sempre vuota nelle retention policy (v0.9.65, 23/08/2026)	implementato	Un agente di audit dedicato a Purview/retention e alle policy Teams (aree toccate solo di striscio dagli semplice script pwsh isolato - serve lo stato accumulato di un processo server lungo, coerente con la r Isolamento reattivo incompleto sui dati Teams: <code>Connect-M365OpsTeams</code> ha già una logica di rec chiamate dirette ai cmdlet Cs* nativi dentro <code>Get-M365OpsTeamsExternalAccessConfig</code> / <code>Get-M36</code> processo in quel momento, non solo dal momento della connessione), l'eccezione risaliva senza alcun mostrato "riavvia M365Ops" nonostante il log del server mostrasse che l'isolamento reattivo si era GIÀ dopo che l'isolamento era pronto. Corretto in entrambe le funzioni: il corpo vero e proprio ora vive in corrisponde al pattern di conflitto noto). Nota onesta: durante la riverifica dal vivo di questo stesso fi Teams", invece del solito "Could not load file or assembly") - non riproducibile a comando con uno scr riconosciuti da <code>Get-M365OpsModuleConflictHint</code> - lasciato come apertura nota invece di un fix al Colonna Teams sempre vuota nelle retention policy: <code>Get-M365OpsRetentionCompliancePolicy</code> <code>RetentionCompliancePolicy</code> - Teams espone due proprietà separate (<code>TeamsChannelLocation</code> , <code>PSCustomObject</code> non lancia un errore in PowerShell, restituisce silenziosamente <code>\$null</code> - la colonna e nessuna policy copre Teams. Corretto separando le due proprietà reali.
Maratona di bug-hunt (16 ore, quinta parte) - autoreview: 4 regressioni introdotte dai fix precedenti di questa stessa maratona, trovate da un secondo agente prima che diventassero un problema reale (v0.9.66, 23/08/2026)	implementato	Diverso dagli altri giri di questa maratona: qui non si cercano bug nuovi, si rilegge <code>git diff</code> dell'int volte" applicata alla propria stessa maratona, non solo al codice preesistente. Trovate 4 regressioni rea 1) 'poi'/'dopo' matchava dentro "poiché": il fix del giro precedente (<code>CommandCatalog.ps1</code>) aggiun dove serve apposta) - ma 'poi'/'dopo' sono parole COMPLETE, non radici, quindi senza un confine anc Corretto a <code>'poi\b'</code> / <code>'dopo\b'</code> - verificato dal vivo che "poiché" ora non matcha piu' mentre "X, p 2) "si, esegui pure" non veniva piu' riconosciuta come conferma: il fix di sicurezza sulla conferma " dopo "si" (frase di conferma naturalissima) restava attaccata alla parola, che quindi non risultava mai u da ogni singola parola dopo lo split - verificato dal vivo end-to-end: creato un gruppo di test risponde 3) Fallimenti con eccezione loggati DUE VOLTE nel log scritte: <code>Write-M365OpsWriteFailureI</code> fallimenti CON eccezione - che però il blocco catch generico di <code>Execute-PendingAction</code> registrav due punti che invocano <code>Execute-PendingAction</code> , e' ora l'UNICO punto che scrive nel log scritte, 4) Il fix per gli Hashtable non veniva mai raggiunto: il ramo dedicato aggiunto in <code>ConvertTo-M36</code> implementa anche <code>IEnumerable</code> e finiva sempre intercettato prima dal ramo piu' generico per gli array ramo generico - verificato dal vivo che ora produce "chiave=valore" invece del testo grezzo del tipo .N
CLI Microsoft 365 sempre pronto su ogni tenant + login delegato realmente funzionante (era rotto) + pulsanti di login coerenti (v0.9.67, 23/08/2026)	corretto	Tre problemi reali segnalati dal vivo dall'utente durante un test su un nuovo tenant Delegato ("AlePira 1) CLI-Microsoft365 richiedeva configurazione manuale per ogni singolo tenant: a differenza di L nuovo tenant falliva con "Server MCP 'CLI-Microsoft365' non configurato per questo tenant". L'utente Microsoft365 e' ora un secondo default incorporato, sempre presente per ogni tenant (nuovo o già es 2) Login delegato di CLI Microsoft 365 completamente rotto (URGENTE, riprodotto dal vivo): cli prompt interattivo invisibile ("?: appld:") e falliva dopo un timeout con un errore criptico contenente te v11.10.0): <code>--appId</code> e' ora "cruciale" per QUALUNQUE tipo di login, incluso <code>--authType browser</code> pubblico integrato senza bisogno di specificarlo. Senza <code>--appId</code> , il comando entrava in un prompt c vero terminale (stessa classe di bug già vista e corretta il 21/08/2026 su Teams con <code>-UseDeviceAuth</code> tenant Microsoft "Microsoft Graph Command Line Tools" già usato per il device-code Graph in quest coerente con l'obiettivo originale di "Delegato = nessuna App Registration". Verificato dal vivo con un web browser that just has been opened", nessun prompt. 3) Pulsante di login "sparisce" per ~15 minuti se il popup viene chiuso senza completare il login dentro la pagina Microsoft non produce alcun errore OAuth - il device code resta "authorization_pend 'Error', lasciando il pulsante disabilitato fino alla scadenza naturale del codice. Corretto aggiungendo u sul pulsante principale - coerenza di stile richiesta esplicitamente dal vivo): un click ferma solo il pollin un flow precedente ancora pendente al riavvio. Per Teams/SharePoint/Purview/Intune/CLI-Microsoft36 finisce, non solo il pulsante) un "Annulla" lato client sarebbe stato un fix finto - mostrerebbe il pulsant 25 secondi, spiegando che il server e' probabilmente bloccato per davvero e l'unico modo per sblocca
L'AI chiedeva un secondo login inutile quando CLI Microsoft 365 poteva già rispondere (v0.9.68, 23/08/2026)	corretto	Bug reale segnalato dal vivo su un tenant Delegato ("AlePiras") con SOLO CLI Microsoft 365 connesso "Nessuna sessione delegata attiva" - un errore di CONNESSIONE mancante (la sessione Graph generic login separato, ignorando che CLI Microsoft 365 era già connesso e avrebbe potuto rispondere subito diceva già di usare <code>cli_m365_*</code> come fallback quando <code>graph_api_call</code> "non ha dato il dato cercato", ma aggiungendo un'eccezione esplicita per i tenant Delegati: se <code>graph_api_call</code> fallisce per sessione/conne (Entra/Outlook/Planner/OneDrive/Purview-ricerca/Teams-app-chat), il modello deve provarlo PRIMA c
CLI Microsoft 365 trattato come sostituto proattivo di Graph, esteso a scritture e alla scoperta app SharePoint (v0.9.69, 23/08/2026)	corretto	Estensione richiesta esplicitamente dal vivo dopo v0.9.68 ("senza graph attivo bisogna usare il cli per c 1) Preferenza proattiva, non solo reattiva, e anche sulle scritture: v0.9.68 correggeva solo <code>graph_</code> (scrittura, es. "crea un account"): stesso fallimento "Nessuna sessione delegata attiva", nessun fallback delegata generica risulta attiva - se NO e CLI Microsoft 365 e' configurato, il modello viene istruito esp fallimento) per qualunque lettura O scrittura nel dominio di CLI365 (Entra/Outlook/Planner/OneDrive/ 2) SharePoint bloccato dallo stesso problema per un motivo nascosto: segnalato dal vivo, "perche

		riusciva a connettere SharePoint, con lo stesso errore "Nessuna sessione delegata attiva". Causa reale: TANTUM per tenant) usava Graph per quella ricerca - un dettaglio implementativo interno, non un ver principio: se la sessione Graph generica non e' attiva e CLI Microsoft 365 e' configurato, prova prima inesistente) - solo se anche questo fallisce arriva l'errore originale che chiede il login Graph. Una volta
Messaggi onesti durante l'attesa dei login interattivi (v0.9.70, 23/08/2026, maratona di debug)	corretto	Segnalato dal vivo: "da che clicco connetti a che apre il browser passa un botto, a volte si apre a volte (SharePoint/Teams/Purview/Intune/CLI365-delegato): causa principale confermata NON e' WAM (le ve precedenti a quando Microsoft ha reso WAM il broker predefinito - verificato dal vivo con <code>Get-Modul</code>: appariva identico sia che il server stesse davvero aprendo il browser sia che stesse ancora verificando/ <code>M365OpsSharePoint.ps1</code> L60-71 e gli <code>Assert-M365Ops*SafeVersion</code>) - lo stesso identico testo s iniziale che avvisa esplicitamente della possibile attesa per l'installazione moduli PRIMA di menzionare legittimamente in corso) e riscritto per coprire onestamente entrambe le cause plausibili (modulo in in
4 bug reali trovati e corretti dall'agente di audit sulle cmdlet Exchange (maratona di debug, v0.9.71, 23/08/2026)	corretto	Primo lotto dell'agente dedicato al test dal vivo (non solo lettura di codice) delle cmdlet Get-M365Op 1-3) Colonna data creazione sempre vuota (<code>Get-M365OpsAllMailboxes</code> , <code>Get-M365OpsSharedM</code> <code>WhenMailboxCreated</code> non ne fa parte, quindi selezionarla dopo con <code>Select-Object</code> produceva s <code>WhenMailboxCreated</code> esplicitamente a ciascuna chiamata - verificato dal vivo: tutte le 25+7+7 mailb 4) Il fallback del report mail flow falliva sempre, proprio quando serviva (<code>Get-M365OpsMailFlo</code> <code>PageSize</code> (V1) - cmdlet in dismissione dal 1/9/2025, che su questo tenant fallisce ormai con un error sempre a sua volta. Corretto passando a <code>Get-MessageTraceV2 -ResultSize</code> (stesso fix gia' applic reali per stato (Failed/Expanded/Delivered). Confermati NON bug (dati mancanti legittimi sul tenant di test, non fallimenti silenziosi): retention pol AntiPhishRules/AutopilotDevices/UpdateRings/AppProtectionPolicies/ResourceMailboxes/AdminTemp
Ciclo completo scrittura Intune verificato dal vivo con pulizia garantita + documentazione Settings corretta (maratona di debug, v0.9.72, 23/08/2026)	corretto	Secondo agente della maratona: ciclo crea→verifica→elimina→verifica su cmdlet di scrittura Intune rea <code>marathon-*</code> , pulizia finale confermata con una scansione dedicata (0 oggetti di test residui). Nessun Bug di documentazione trovato e corretto: <code>New-M365OpsConfigurationPolicy</code> dichiarava che i template stesso fornisce spesso dei default" - verificato dal vivo che questo e' FALSO in ogni caso pro 400 <code>"dCV2Policy.Settings : The Settings field is required."</code> . Corretta la documentazio una voce Settings reale. Lecture Entra aggiuntive verificate senza problemi: conditional access policies, riepilogo stato conform
Autoreview della maratona: 1 regressione reale trovata e corretta nel fix CLI365/SharePoint di poco prima (v0.9.72, 23/08/2026)	corretto	Terzo agente della maratona, con lo stesso compito gia' collaudato nelle maratone precedenti: non ce introdotte dai fix stessi. Trovata una regressione reale in <code>Sync-M365OpsSharePointAppRegistratic</code> correttamente "l'app non esiste" (testo semplice, non JSON valido - <code>"Error: App with name 'X'</code> tentativo Graph successivo - che pero' fallisce SEMPRE in questo scenario specifico (nessuna sessione invece del corretto "NotFound" con il comando di registrazione gia' pronto - esattamente il caso PIU' found" nella risposta CLI365 e restituendo subito lo stato corretto, verificato dal vivo contro l'output re
La GUI era illeggibile su mobile - nessun tag viewport (maratona di debug, v0.9.73, 23/08/2026)	corretto	Trovato testando dal vivo la GUI a larghezza mobile (375px, mai fatto prima in nessuna maratona prec larghezza fisica dello schermo e renderizza la pagina su una viewport virtuale desktop (tipicamente 98 (<code>window.visualViewport.width</code> restituiva 980 invece di 375). Aggiunto <code><meta name="viewport</code> corretto. Trovato un secondo problema minore nello stesso giro: con la viewport corretta, il gruppo di comunque un piccolo overflow orizzontale (435px di contenuto su 375px di schermo, misurato via <code>ge</code> capo correttamente, zero overflow orizzontale (<code>body.scrollWidth</code> == <code>window.innerWidth</code>), ne
2 bug reali su Teams trovati dall'agente di audit lettura Teams/SharePoint (maratona di debug, v0.9.74, 23/08/2026)	corretto	Quarto agente della maratona: tutte le Get-M365Ops* di Teams e SharePoint/OneDrive invocate per d 1) Riga fantasma nei risultati Teams alla prima connessione (<code>Connect-M365OpsTeams.ps1</code>): Co quell'oggetto finiva sulla pipeline di output della funzione e, a cascata, su quella di ogni chiamante (es riuscita per sessione (le successive fanno return anticipato), motivo per cui non era mai stato notato p (Delegato e AppOnly). 2) Dati reali spariti silenziosamente dagli export CSV/tabella (<code>Get-M365OpsTeamsExternalAcce</code> colonne invece di uno schema unificato, a differenza del gemello <code>Get-M365OpsTeamsPolicies</code> (che oggetto per l'intera tabella - risultato verificato dal vivo: valori reali come <code>AllowUserChat=True</code> spa riga, stesso schema del gemello. Nota di documentazione aggiornata nello stesso giro: <code>Get-M365OpsTeamsPolicies</code> dichiarava anc righe reali restituite. Aggiornata la nota.
Bug reale su aggiornamento messaggi notifica localizzati + gap scoperto su rimozione siti SharePoint (maratona di debug, v0.9.75, 23/08/2026)	corretto	Quinto agente della maratona: ciclo completo scrittura su Exchange (gruppi distribuzione, mailbox cor Bug reale trovato e corretto: <code>Set-M365OpsNotificationTemplateMessage</code> - aggiornare un mes: del PATCH includeva ancora <code>locale</code> (ereditato dal body condiviso col ramo POST, dove invece serve funzionava correttamente, solo l'aggiornamento di uno esistente era rotto. Corretto separando i due k Gap reale scoperto (non ancora colmato): il modulo non ha NESSUN wrapper <code>Remove-M365OpsSh</code> ripetutamente (4 tentativi dell'agente + 3 miei successivi, incluso un tentativo alternativo via Graph DE non un bug del modulo.

		⚠ Residuo reale lasciato sul tenant "vnsys-test": il sito di test https://vnsysit.sharepoint. (probabile restrizione temporale della piattaforma su siti appena creati) con Remove-PnPTenantSite amministrazione SharePoint.
Un "crash" segnalato dal vivo su Teams era in realta' un errore di rete client dopo un successo lato server + rete di sicurezza sul loop principale (v0.9.76, 23/08/2026)	corretto	Segnalazione dal vivo: "provando Teams delegato, dopo il login ho ricevuto Errore di comunicazione: 6.6) era scattato, l'isolamento reattivo aveva riconnesso con successo tramite un SECONDO popup di login (spesso lunga, specialmente con un secondo popup mai segnalato prima), la connessione TCP d indistinguibile per l'utente da un fallimento vero, anche quando il server ha gia' completato con succe Corretto su due fronti: (1) tutti e 5 i pulsanti di connessione bloccanti (SharePoint/Teams/Purview/Intu l'utente con solo il messaggio d'errore grezzo - se l'operazione e' comunque riuscita (come in questo comparire un SECONDO popup separato se scatta il conflitto noto, invece di lasciare l'utente a chiede Aggiunta anche una rete di sicurezza indipendente, non legata a questo episodio specifico: il ciclo prir eccezione sfuggita al gestore per-richiasta interno terminerebbe l'intero processo senza lasciare tracci davvero - non impedisce un crash genuino (es. un fault nativo non catturabile da .NET), ma garantisce
Domande in prima persona su un tenant Delegato ricevevano un rifiuto invece di una risposta (maratona di debug, ambiente Delegato, v0.9.77, 23/08/2026)	corretto	Primo test dal vivo sul tenant Delegato "AlePiras" dopo che l'utente ha completato tutti i login: "mostr un rifiuto onesto (nessuna invenzione), ma senza che il modello avesse MAI tentato nessuna chiamata <code>/me/...</code> si risolve automaticamente all'identita' di chi ha fatto il login delegato - il modello non sap dell'utente. Corretto aggiungendo una regola esplicita: su un tenant Delegato con sessione Graph atti <code>/me/joinedTeams</code> / <code>/me/mailboxSettings</code> / <code>/me/managedDevices</code> ecc. invece di rifiutarsi o chie esplicitamente a chi si riferisce la domanda.
Conteggio MFA tenant-wide: il modello a volte rispondeva senza tentare nessuna chiamata (maratona di debug, ambiente Delegato, v0.9.78, 23/08/2026)	corretto	Secondo test dal vivo sul tenant Delegato "AlePiras": "quanti utenti hanno mfa non configurata?" ha ri nel log della Cronologia (Manutenzione): tra "Avvio Invoke-M365OpsAgentTools" e "completato" sono differenza di altre domande nella STESSA conversazione (es. "ci sono piani planner attivi?" poco dopo, funzionano, il modello ha scelto specificamente di non provare per questa domanda. Causa: nessun to utente specifico) - il modello doveva improvvisare l'endpoint Graph corretto (<code>/reports/authenticat</code> funzionato su un altro tenant in precedenza nella stessa maratona, qui no). Verificato dal vivo che l'enr esplicita nella descrizione dello strumento <code>get_user_mfa_status</code>, cosi' il modello non deve piu' inc
"Stato permessi" mostrava note di sviluppo interne all'utente finale (v0.9.79, 23/08/2026)	corretto	Segnalato dal vivo: il pulsante "Stato permessi" mostrava righe come "nota: Verificato dal vivo il 21/08 verifica, riferimenti a questo stesso progetto) finito per errore nel campo <code>Note</code> restituito all'utente fir <code>M3650psDelegatedPermissionsCheck.ps1</code> e <code>Get-M3650psAppPermissionsCheck.ps1</code>: le date i campo <code>Note</code> mostrato all'utente ora dice solo cio' che gli serve sapere ("dedotto dal nome del ruolo
2 bug reali su assegnazione ruoli RBAC Intune trovati dall'agente di audit scritture (maratona di debug, v0.9.80, 23/08/2026)	corretto	Sesto agente della maratona, ciclo crea→verifica→elimina→verifica su ruoli custom Intune e relative as <code>/deviceManagement/roleDefinitions/{id}/roleAssignments</code>, la rotta documentata per il tipo usato davvero da Intune (<code>deviceAndAppManagementRoleAssignment</code>) si crea invece sulla collezion <code>roleDefinition@odata.bind</code> - corretto e verificato dal vivo, propagata la stessa correzione alla ro successo). Corretto anche il campo "chi puo' usare il ruolo": non <code>scopeMembers</code> (nome fuorviante) n Bug 2: con un solo gruppo in <code>-ScopeGroupIds</code>, il body inviato aveva <code>resourceScopes</code> come strir <code>\$ScopeGroupIds } else { @() }</code> spacchetta un array a un solo elemento passandolo attraverso l' all'intero if/else, verificato confrontando l'output JSON prima/dopo.
Login Teams Delegato non piu' bloccante per il server + 3 bug reali trovati durante la verifica dal vivo del refactor (v0.9.81, 23/08/2026)	implementato	Richiesta esplicita dal vivo dopo l'episodio v0.9.76 ("rendi la questione multi thread cosiv' che queste o e' stato scartato di proposito (rischio concreto di bug di concorrenza su stato condiviso critico per la s che risolverebbe). Implementata invece l'alternativa mirata concordata: il login Teams Delegato ora pa reale) tramite due nuove funzioni non bloccanti, <code>Start-/Get-M3650psIsolatedModuleConnectAsy</code> uso per il codice dispositivo Graph/Exchange, pulsante "Annulla" dedicato incluso) invece di restare bl anche quando nessun conflitto .NET si sarebbe mai verificato - scambio deliberato per un server che n Durante la verifica dal vivo del refactor (necessaria perche' la logica di completamento connessione, g identica dal nuovo percorso asincrono) sono emersi 2 bug reali, entrambi corretti e riverificati dal <code>PSObject.Properties.Name</code> contiene UN elemento <code>\$null</code> invece di zero elementi - un quirk Poi quel <code>\$null</code> finiva come chiave di un hashtable, causando "Index operation failed; the array index evi comandi da proxare con un diff prima/dopo su <code>Get-Command -CommandType Function, Cmdlet</code> <code>ExchangeOnline</code>, mai catalogabili in anticipo) ma SEMPRE a zero per Teams, verificato dal vivo: 'Ge completamente pulito, PRIMA di qualunque <code>Import-Module</code> - PowerShell cataloga i nomi esportati senza il fix: l'isolamento Teams "riusciva" (nessuna eccezione, log di successo) ma installava ZERO prox crash che ha permesso di scoprirlo. Corretto includendo anche i comandi il cui <code>ModuleName</code> corrisp quel <code>ModuleName</code>). Verificato dal vivo dopo il fix: isolamento Teams App-only forzato manualmente, 3) Un terzo bug reale, trovato SOLO testando il nuovo percorso asincrono end-to-end (mai emerso su falliva con "Errore MCP: The type '...Dictionary'2[[...AzureAccount+Property...],[...String...]]' is not suppo <code>Connect-MicrosoftTeams</code>) solo se <code>\$script:M3650psTeamsConnected</code> e' gia' vero - un flag imp mai passarci). Risultato: <code>Get-M3650psTeamsList</code> richiamava <code>Connect-M3650psTeams</code>, che provav globale (installata per ogni comando del modulo appena connesso, <code>Connect-MicrosoftTeams</code> con davvero un secondo <code>Connect-MicrosoftTeams</code> ridondante e provava a restituirne il risultato (oggi Corretto impostando il flag <code>\$script:M3650psTeamsConnected</code> / <code>\$script:M3650psExchangeConn</code> e asincrono, dopo un connect riuscito) invece che nei soli corpi di <code>Connect-M3650psTeams</code> / <code>Conne</code> subito dopo restituisce correttamente le 15 righe reali senza errori.

CLI Microsoft 365 verificato per la prima volta con comandi reali - trovato e corretto un falso "installazione fallita" su Windows PowerShell 5.1 (agente 9 della maratona, v0.9.82, 23/08/2026)	corretto	Nono agente della maratona: primo test dal vivo di CLI Microsoft 365 con comandi REALI (<code>entra / s</code> segnalata "mai toccata" nella maratona, il login era stato riparato in v0.9.67 ma nessun comando reale Bug reale trovato e corretto: <code>Assert-M365OpsCliMicrosoft365Installed.ps1</code> cattura l'output <code>allow-scripts...</code> per una dipendenza, presente su ogni installazione recente del pacchetto). Sotto <code>\$Err</code> di quegli avvisi a eccezione terminante non appena attraversa la pipeline, ben prima del controllo esp veniva segnalata come fallita, con l'avviso npm scambiato per l'errore vero. Verificato dal vivo che Pow mano su Windows PowerShell 5.1, stessa classe di gap "solo 5.1" già nota per la lettura file senza <code>-Er</code> in un blocco <code>finally</code>) - il controllo su <code>\$LASTEXITCODE</code> resta l'unica fonte di verità su successo/fa prima del fix, riesce dopo. Risultato del test end-to-end: CLI Microsoft 365 e' genuinamente utilizzabile in chat oggi per Entra/f su SharePoint (limite documentato dell'autenticazione a client secret) e su Planner/Outlook (permessi ripetizione del blocco di ~3 minuti segnalato in precedenza durante questo giro (ogni comando testat connessione resta confermato, pagato una sola volta per processo server.
Script personalizzati (Scripts\Custom), testati dal vivo per la prima volta - 2 bug reali trovati e corretti (agente 10 della maratona, v0.9.83, 23/08/2026)	corretto	Decimo agente della maratona: area segnalata "mai stress-testata" - verificato dal vivo che il caricame funzionano davvero end-to-end per l'unico script reale presente (<code>Get-M365OpsOneDriveSharingRej</code> silenzioso: il tenant di test semplicemente non ha link di condivisione attivi in questo momento, verific Bug 1: <code>Get-M365OpsCustomScriptCatalog.ps1</code> - la lista di esclusione dei parametri comuni non i esposto all'AI mostrava un parametro FASULLO (<code>Upn</code>, <code>ProgressAction</code> invece del solo <code>Upn</code> reale, <code>custom_script_query</code> / <code>propose_custom_script_write</code> . Corretto aggiungendo <code>ProgressAct:</code> Bug 2: <code>M365Ops.psm1</code> , il loader degli script personalizzati - il blocco <code>catch</code> usava <code>\$_Name</code> per r l'oggetto file della pipeline - l'avviso non diceva MAI quale script fosse rotto ("Script personalizzato " r Corretto catturando l'oggetto file in una variabile dedicata PRIMA del <code>try</code> - verificato dal vivo che l'i <code>.NOTES Mode:</code> si carica come funzione PowerShell ma risulta correttamente <code>Valid=\$false</code> nel ca
Chiuso l'item prioritario sul fallback CLI365 per le scritture Entra - guard deterministico, non traduzione automatica (agente 11 della maratona, v0.9.84, 23/08/2026)	implementato	Undicesimo agente della maratona: risolto l'item lasciato deliberatamente aperto dal v0.9.68/69 ("una su "vnsys-test": creazione+eliminazione di un gruppo Entra sia via POST Graph diretto (<code>New-M365Ops</code> Evidenza reale raccolta, non ipotetica: <code>m365 entra group add</code> RIFIUTA il comando senza un <code>-- mailEnabled / securityEnabled</code> booleani); anche passando <code>--type</code> corretto, il gruppo risultan <code>visibility="Public"</code> invece di <code>null</code> - normalizzazione e default REALMENTE diversi tra i due p stata implementata, ne' lo sarà senza una revisione dedicata per ogni singolo endpoint. Implementato invece un guard deterministico in <code>Invoke-M365OpsAgentTools.ps1</code> : quando la se percorso Entra ID (<code>/users</code> , <code>/groups</code> , <code>/devices</code> , <code>/directoryRoles</code> , <code>/organization</code> , <code>/dom</code> veda anche solo una richiesta di conferma, rimandando l'AI a costruire da sola un <code>propose_cli_m36</code> ristretto a Entra ID - Exchange/Intune/Teams/SharePoint via Graph restano sempre e solo su <code>propose</code> Verificato dal vivo attraverso il percorso reale del guard (stato Delegato+sessione-mancante simu <code>propose_graph_write</code> - guard confermato funzionante - ma ha inizialmente costruito il comando rifiutato il comando con un errore chiaro, nessun oggetto creato. Confermato che il guard resta sicuro di qualunque <code>propose_cli_m365_command</code> scritto male, non introdotto da questo fix. Pulizia finale
Autoreview del blocco v0.9.81-v0.9.84: gemello del fix installazione CLI365 mai propagato (agente 12 della maratona, v0.9.85, 24/08/2026)	corretto	Dodicesimo agente della maratona, con lo stesso compito già collaudato: non cercare bug nuovi, rilec scritture Entra→CLI365) cercando regressioni introdotte dai fix stessi. Un primo tentativo era stato inte Trovato e corretto: non una vera regressione, ma un gemello del bug v0.9.82 mai propagato. <code>Publi</code> corretta in v0.9.82) faceva la STESSA identica chiamata <code>npm install -g @pnp/cli-microsoft365</code> "changed 207 packages", exit 0) veniva comunque segnalata come fallita per lo stesso motivo già' dia prima del controllo su <code>\$LASTEXITCODE</code>). Corretto propagando lo stesso identico fix (EAP locale <code>'Cc</code> riuscita. Altre aree controllate, nessuna regressione trovata: il guard scritture Entra non puo' mai scattare su preferenza proattiva CLI365 già' esistente nello stesso file; il login Teams asincrono ririverificato end-to- Scripts\Custom ririprodotti dal vivo con un nuovo file dalla sintassi rotta, entrambi confermati.
La nota finale in chat ora dice sempre quale motore IA ha risposto, non solo quali dati sono stati consultati (v0.9.86, 24/08/2026)	implementato	Richiesto esplicitamente dall'utente durante un'analisi sui token inviati alle API IA: la nota "Fonte dati: reali del tenant tramite uno strumento (Lokka/CLI365/moduli interni) - una risposta puramente conver scriverla senza mai dirlo esplicitamente. Aggiunta un'etichetta del motore IA usato (<code>Claude Sonnet</code> <code>M365OpsAgentTools</code> (i tre punti di uscita della funzione: risposta con dati, risposta puramente conve Elaborata da IA: Y.", altrimenti "Risposta generata da IA (Y) senza consultare dati del tenant." Verificato
La nota sorgente estesa a OGNI interazione in chat, non solo al loop AI con tool-calling (v0.9.87, 24/08/2026)	implementato	Seguito diretto del v0.9.86, segnalato dal vivo dall'utente: "dammi i fattori mfa di X" non mostrava nes <code>RequiresAI=\$false</code>), un percorso completamente diverso da <code>Invoke-M365OpsAgentTools</code> (dov utente/gruppo, ecc.) non ci sono mai passati vicino. Estratta la mappa nomi motore IA in una funzione <code>Gui/Server.ps1</code> finora scoperti: (1) il dispatch del catalogo comandi locali - "Fonte: comando local Elaborata da IA: Y." per le voci <code>RequiresAI=\$true</code> (es. <code>CompliancePatterns</code> , che chiama davvero prima di completare); (2) il percorso di diagnosi/correzione di una scrittura fallita (<code>Invoke-M365OpsI</code> fallback quando <code>Invoke-M365OpsAgentTools</code> stessa lancia un'eccezione e si ripiega su <code>Invoke-M:</code> mostrava nulla ora chiude con " _Fonte: comando locale 'MfaStatus', nessuna IA usata_"; verificata anc

Autoreview del v0.9.87: la nota d'errore diceva sempre "nessuna IA usata", anche per le voci che usano davvero l'IA (agente 14 della maratona, v0.9.88, 24/08/2026)	corretto	Quattordicesimo agente della maratona, su richiesta esplicita dell'utente subito dopo il v0.9.87 ("doipx fix stessi). Trovato e corretto: il ramo di successo del dispatch catalogo distingue correttamente "nessuna IA usata" (fallito prima di completare)", anche per le voci <code>RequiresAI=\$true</code> (<code>Compliance</code> risposta vuota) - contraddicendo lo scopo stesso della trasparenza appena introdotta. Riprodotto dal v0.9.88 "nessuna IA usata" nonostante la voce chiami davvero l'IA. Corretto branchando il blocco <code>catch</code> sullo stesso ramo. Il caso di controllo non-AI (invariato, ancora corretto). Altre aree controllate, nessun problema trovato: tutti i ~19 Formatter del catalogo restituiscono se di stringhe pulite; nessuna interferenza con <code>\$script:LastReportPath</code> /allegati; nessuna collisione con <code>markdown/HTML</code>; <code>Get-M365OpsAiProviderLabel</code> confermata raggiungibile da una sessione reale allegato, <code>GroupOverview</code> , piu' i due casi sintetici di fallimento sopra) - tutte corrette.
Cache-hit sulle chiamate IA da 12,9% a ~99% - elenco strumenti reso identico su ogni round (v0.9.89, 24/08/2026)	implementato	Seguito diretto di un'indagine sui token spediti alle API IA, richiesta esplicitamente dall'utente dopo aver visto <code>list_noncompliant_devices</code> , <code>get_device_compliance_reasons</code> , <code>get_user_overview</code> , <code>get_device_compliance_reasons</code> come primo tentativo (fix storico contro un comportamento reale osservato in passato). Questo rende (<code>cache_control</code> , mai attivato in questo progetto) sia quello automatico di Azure OpenAI (sempre a richiesta precedente: un elenco strumenti diverso rompe il confronto proprio nel punto piu' pesante del round, 12,9% di cache-hit sul secondo round di una conversazione Azure, ben lontano dal quasi-100% atteso). Corretto: l'elenco strumenti e' ora IDENTICO su ogni round (nessuna esclusione per round). La preferita frase esplicita nel prompt di sistema (round-invariante per costruzione, non rompe mai il prefisso con <code>cache_control</code>). Verificato dal vivo su "vnsys-test": cache-hit sul round 1 passato da 12,9% (prima) a ~99% (dopo, con <code>cache_control</code> pensati apposta per tentare il modello verso una delle 6 scorciatoie escluse in precedenza ("panoramica evita deliberatamente le parole-trigger, un riepilogo ambiguo tra utente/gruppo) - in tutti e sei i casi i round di conversazione. Verifica indipendente aggiuntiva tramite un agente dedicato di regression-review: nessun provider (il beneficio di costo resta pero' solo per Azure, che ha il caching automatico - Claude non ne ha).
Nuova funzionalita': analisi intestazioni email/NDR con spiegazione IA, tab "Intestazioni email" (v0.9.90, 25/08/2026)	nuovo	Richiesta esplicitamente dall'utente durante un caso reale di troubleshooting su un NDR ("550 5.4.1 RequiredAuthAs=Anonymous (Message Header Analyzer of Microsoft), senza mai mandare nulla. 5322 complete, da "Visualizza origine messaggio" in Outlook, OPPURE un blocco di diagnostica NDR/ subito: riepilogo (oggetto/mittente/destinatario/data), tabella degli hop di consegna con ritardo calcolato leggibili per i campi piu' comuni), e per un NDR il codice SMTP/esteso scomposto con una spiegazione di destinatario, non un blocco di sicurezza/relay come il nome farebbe pensare - verificato sulla documentazione incollata PIU' l'analisi gia' calcolata in locale all'IA configurata (stesso principio di trasparenza del troubleshooting - mai eseguito automaticamente, un click esplicito e separato dal solo parsing locale). Nuovo file <code>Private/Get-M365OpsMessageHeaderAnalysis.ps1</code> (parser, nessuna dipendenza esterna) per il parsing, <code>/api/analyze-headers-ai</code> per la spiegazione IA). Verificato incollato dall'utente (non un caso costruito a mano) restituiva i codici SMTP/esteso vuoti nonostante i parentesi quadre va spesso a capo da solo) e <code>Get-M365OpsMessageHeaderAnalysis</code> non matcha mai un a-capo in .NET regex senza l'opz <code>hop</code>, ritardi 0/1/0/4 secondi, verificato identico all'output di riferimento di Microsoft Message Header (l'analisi locale), sia tramite chiamata diretta alle nuove route sia dalla GUI vera e propria nel browser.
Analizzatore intestazioni: pulsante combinato copia+apri MHA, riferimento ai dati grezzi, 3 bug reali corretti da 2 agenti dedicati (v0.9.91, 25/08/2026)	corretto	Seguito diretto del v0.9.90, su richiesta esplicita dell'utente ("fai partire gli agenti per controllare tutto" <code>mha.azurewebsites.net</code> senza un tasto Copia separato). Verificato PRIMA di costruire: se lo strumento Microsoft supportasse un modo per pre-compilarsi via un finto "auto-fill" sarebbe stato disonesto (il link aprirebbe il tool vuoto); implementato invece un pulsante copia separato. Due agenti dedicati, in parallelo: uno di regression-review sul commit v0.9.90, uno di stress-test del tool. 2 bug reali nel parser (<code>Private/Get-M365OpsMessageHeaderAnalysis.ps1</code>), entrambi su varianti (<code>Mon, 24 Aug 2026 01:23:39 -0700 (PDT)</code>) facevano fallire silenziosamente il calcolo del ritardo della settimana (facoltativo per RFC 5322, omissso da gateway come Cisco IronPort/ESA, es. <code>24 Aug 2026 01:23:39 -0700 (PDT)</code>) corretto rendendo il giorno della settimana facoltativo nella regex. Generalizzato nello stesso giro il ricambio del mittente Gmail senza alcun mittente esplicito. 1 bug reale nella GUI (<code>Gui/index.html</code>), trovato dall'agente di regression-review: la nuova sezione "Intestazioni email" (ramo piu' comune (intestazioni normali, non un rimbalzo) non la mostrava mai, senza nessun errore in browser dopo ogni correzione.
Conteggio token spediti/ricevuti nella nota "Elaborata da IA" della spiegazione intestazioni (v0.9.92, 25/08/2026)	implementato	Richiesta esplicitamente dall'utente per l'interazione IA dell'analizzatore intestazioni ("in questo caso di debug percepibili. Verifica fatta: il conteggio token arriva GRATIS nello stesso campo "usage" gia' presente in struttura reale della risposta prima di implementare, non assunto). Aggiunto un parametro opzionale <code>-ReturnUsage</code> a <code>Invoke-M365OpsAgent</code> (Public): se presente e' presente e non tocca il contratto delle altre ~5 funzioni che gia' chiamano questa attendendosi una stringa senza predefinito resta identico). Usato per ora solo nella route <code>/api/analyze-headers-ai</code> : la nota finale effettivamente maggiore di zero. Verificato dal vivo: nota reale mostrata correttamente ("1095 token in risposta di funzione senza il nuovo parametro - continua a restituire una stringa semplice come prima).

Risposte lunghe in chat non piu' tagliate a meta' frase dopo un ricaricamento pagina + toggle "Mostra tutto" (v0.9.93, 25/08/2026)	corretto	Bug reale segnalato dal vivo dall'utente con uno screenshot: un'analisi dettagliata (pattern di non conf appena ricevuta, ma dopo un ricaricamento della pagina. Causa reale: <code>Add-M365OpsChatHistoryTurn.ps1</code> troncava il testo a 3000 caratteri PRIMA di salvar <code>/api/chat/history</code>), dove va mostrato il testo COMPLETO; (2) fornire contesto conversazionale all'I reinviando un report lungo per intero ad ogni chiamata. Applicare lo stesso taglio a entrambi gli scop Corretto separando i due scopi: <code>Add-M365OpsChatHistoryTurn.ps1</code> ora salva SEMPRE il testo cc contesto per l'IA (<code>Invoke-M365OpsAgentTools.ps1</code> , nuova variabile <code>\$maxHistoryCharsPerTurn</code> turno successivo che usa quello storico come contesto continua a funzionare correttamente (riassunto Aggiunto anche (richiesto esplicitamente, "aggiungere un click per mostrare tutto su richiesta"): un se collassato con un'ellissi e un pulsante "Mostra tutto"/"Mostra meno" per espandere/ricomprimere il te lunghi, espansione e ricompressione funzionanti in entrambe le direzioni.
Autoreview del v0.9.93: un taglio a 3000/1500 caratteri poteva spezzare un'emoji, corrompendo silenziosamente il testo (agente 17 della maratona, v0.9.94, 25/08/2026)	corretto	Diciassettesimo agente della maratona, su richiesta esplicita dell'utente ("a valle rifai tutte le verifiche c Trovato e corretto: sia <code>.Substring()</code> lato server (PowerShell) sia <code>.slice()</code> lato client (JavaScript) surrogata (es. un'emoji, U+10000 e oltre), si perde il surrogato basso lasciando un surrogato alto orfar converte in byte UTF-8 per la chiamata HTTP reale verso l'IA, il surrogato orfano non e' rappresentabil nessun errore visibile. Lato client lo stesso problema e' solo estetico (il testo completo resta comunqu taglio di una posizione quando cade su un surrogato alto, cosi' la coppia resta sempre intera. Altre aree controllate, nessun problema trovato: nessun altro punto del codice (10 file con riferime <code>\$maxPairs = 8</code> (ultimi 16 scambi) confermato intatto; testo <code>\$null</code> /vuoto e il taglio esatto al conf ridisegnati dallo storico al caricamento pagina. Verificato dal vivo end-to-end su "vnysys-test": risposta reale in entrambe le direzioni dopo il fix.
Analizzatore intestazioni spostato dalle Impostazioni a un pulsante in primo piano nella barra strumenti + reindirizzamento se incollato per errore in chat + conteggio token esteso a TUTTA la chat, non solo l'analizzatore (v0.9.95, 25/08/2026)	corretto	Tre correzioni dal vivo richieste esplicitamente dall'utente dopo aver provato la funzionalita' reale nel p 1) Riposizionamento GUI: la tab "Intestazioni email" viveva dentro il pannello "⊗ Impostazioni tenan un posto adatto per una funzionalita' pensata per l'uso estemporaneo e reattivo ("ho appena ricevuto livello proprio (<code>#headers-pane1</code>), con un pulsante dedicato "🔍 Intestazioni email" nella barra strur funzionalita' persa nello spostamento (Analizza, Chiedi all'IA, Pulisci, pulsante combinato copia+apri V 2) Reindirizzamento se incollato nel posto sbagliato: segnalato dal vivo che incollare delle intestazi tabella hop, niente verifica SPF/DKIM/DMARC, niente link a Microsoft Message Header Analyzer, nient nel catalogo comandi locali (<code>Gui/CommandCatalog.ps1</code> , <code>PastedEmailHeadersRedirect</code> , contro di autorilevamento gia' usata dal parser: righe <code>Received: from , RecipientStatus : , o {LED=</code> interamente dal catalogo locale. 3) Conteggio token esteso alla chat generale: il conteggio token (introdotto in v0.9.92) copriva solo risposta della chat principale, non solo li'. Aggiunta in <code>Invoke-M365OpsAgentTools.ps1</code> una somrr puo' attraversare piu' round di andata/ritorno con l'IA, uno per ogni chiamata a uno strumento) e ripo conversazionale, risposta interrotta per limite di passaggi) sia per il ramo Claude sia per quello Azure (Verificato dal vivo: una domanda senza strumenti ("ciao, come stai?") mostra "31254 token inviati, 21 round reali (30514+30738+32675 token in ingresso, log per round verificato) e la nota finale riporta c round). Pannello riposizionato testato end-to-end nel browser (apertura/chiusura, Analizza su un'intes positivo (blocco intestazioni incollato in chat, nessuna chiamata IA, redirect corretto) sia sul caso nega
Autoreview del v0.9.95: falso positivo del redirect intestazioni su una normale frase inglese (agente di regression-review dedicato, v0.9.96, 25/08/2026)	corretto	Rilette SOLO le modifiche del commit v0.9.95 (pannello intestazioni riposizionato, redirect catalogo, cc maratona. Trovato e corretto: il trigger <code>received:\s*from\s</code> della nuova voce <code>PastedEmailHeadersRedir</code> sottostringa a meta' frase la faceva scattare, anche senza nessuna intestazione email incollata davvero <code>invoices to billing@contoso.com</code>" veniva intercettato e redirezionato al pannello intestazioni invece di r sulla propria riga nel testo grezzo incollato, mai a meta' di un'altra frase - corretto ancorando il trigge vivo dopo il fix, tramite chiamate dirette a <code>/api/chat</code> : la stessa frase inglese ora riceve correttament continuano a scatenare il redirect come previsto - confermato anche nella cronologia chat persistita, c Altre aree controllate, nessun problema trovato: nessun riferimento residuo alla vecchia posizione contemporaneamente (si impilano in verticale nel normale flusso del layout, nessuna sovrapposizione, orizzontale); tutte le funzionalita' del pannello (Analizza, Pulisci, pulsante IA, pulsante combinato copia <code>\$response.usage.input_tokens</code> e affini restano sicuri anche con <code>\$response.usage</code> assente/ <code>\$r</code> un test isolato su PowerShell 5.1); l'accumulo su piu' round riverificato dal vivo confrontando i log per- 25401+25624+27561=78586 token in ingresso e 0+24832+24832=49664 dalla cache, entrambi coinc "Risposta generata da IA (Y, ...) senza consultare dati del tenant." confermati invariati nella struttura, sc Nota fuori ambito, non corretta in questo giro (pre-esistente, non introdotta da v0.9.95): durante il rimasta bloccata oltre 3 minuti senza mai rispondere, e - poiche' il listener HTTP del server e' sincrono irraggiungibile finche' il processo non e' stato terminato e riavviato manualmente. Non riprodotto in n architetturale e' reale e indipendente dall'ambiente: qualunque chiamata IA senza timeout esplicito pu della sola review del v0.9.95.

Una chiamata IA appesa non blocca piu' l'intero server a tempo indeterminato - timeout esplicito su tutte le chiamate a Claude/Azure OpenAI (v0.9.97, 25/08/2026)	corretto	Seguito diretto del rischio architetturale segnalato (non corretto) dall'agente di regression-review del Causa reale: il server GUI (<code>Gui/Server.ps1</code>) usa un <code>System.Net.HttpListener</code> con un unico ciclo di richiesta alla volta. Nessuna delle 6 chiamate <code>Invoke-RestMethod</code> verso le API IA (3 in <code>Invoke-M365OpsAgentTools.ps1</code> , 3 in <code>Invoke-M365OpsAgent.ps1</code>) specificava <code>-TimeoutSec</code> - senza, <code>Invoke-RestMethod</code> non applica NESSUN limite proprio (il default per QUALUNQUE altra richiesta nel frattempo, non solo per chi ha fatto quella domanda - riprodotto come irraggiungibile fino al riavvio manuale del processo). Corretto: aggiunto <code>-TimeoutSec 120</code> a tutte e 6 le chiamate. 120 secondi e' abbondante per una sessione di lavoro. <code>M365OpsAgentTools.ps1</code> hanno ciascuno il proprio budget separato, non condiviso con gli altri rou. Una riscrittura del server per essere davvero multi-thread e' stata scartata di proposito per lo stesso motivo di sicurezza delle scritture (<code>\$pendingWrite</code> , <code>\$script:M365OpsContext</code> , i dizionari dei processi MCF architetturali gia' fatte in questo progetto). Un timeout scaduto viene lanciato come <code>System.Threading.Tasks.TaskCanceledException</code> (ve attorno a queste chiamate gestiscono da tempo (nessuna gestione nuova necessaria); dentro <code>Invoke-M365OpsAgent</code> senza strumenti e poi su un messaggio d'errore pulito; dentro la route <code>/api/analyze</code>). Verificato dal vivo: 387 file <code>.ps1</code> sintatticamente puliti dopo la modifica; una domanda normale in meccanismo di timeout confermato isolatamente con <code>Invoke-RestMethod -TimeoutSec 2</code> contro previsto, stesso pattern usato nelle 6 chiamate reali.
Nuova funzionalita': editor grafico "Infrastruttura tenant" - diagramma on-prem/ibrido/multi-cloud disegnato in GUI, leggibile dall'IA su richiesta (v0.9.98, 25/08/2026)	nuovo	Richiesto esplicitamente dall'utente: una sezione GUI a parte, raggiungibile con un pulsante, per disegno grafica che entri anche nella Knowledge Base fruibile dall'IA - esteso poi esplicitamente dall'utente anche Editor: nuovo pulsante "  Infrastruttura" nella toolbar principale (accanto a Upload/Intestazioni e ma un nodo trascinabile con un pannello proprieta' (Tipo, Nome, Ruolo, IP, Dominio/regione/account, No on-prem, ibrido e multi-cloud: Domain Controller, Entra Connect, Tenant M365, Identity Provider (Okt Container/Kubernetes, Database, Storage/Bucket, Cloud provider (account/subscription AWS/Azure/G <tenant>.json , stesso schema di isolamento gia' in uso per storico chat e Knowledge Base) - nessuno Integrazione IA/KB: stesso principio gia' collaudato per la Knowledge Base (v0.9.86 e successivi) - un (nomi/ruoli/IP/domini/collegamenti) si legge on-demand con un nuovo tool <code>get_tenant_infrastru</code> parametro tenant esposto all'IA. La nota "Fonte dati" ora distingue correttamente <code>kb_query</code> e <code>get_tenant</code> - prima ricadevano entrambi nell'etichetta generica "moduli PowerShell interni di M365Ops", fu Nuovi file: <code>Public\Get-M365OpsInfraDiagram.ps1</code> , <code>Public\Set-M365OpsInfraDiagram.ps1</code> (<code>M365OpsInfraDiagramNarrative.ps1</code> (riassunto + testo completo per l'IA). Due nuove route in <code>Gu</code> Verificato dal vivo end-to-end: 390 file <code>.ps1</code> sintatticamente puliti; creazione di due nodi (Domain min") tramite la sequenza reale di eventi puntatore (non solo lettura del codice); salvataggio e persiste confermata rimuovere a cascata anche il collegamento associato; ricaricamento dopo un riavvio del se sincronizzato con il cloud?) ha correttamente richiamato <code>get_tenant_infrastructure</code> e risposto <code>"._Fonte dati: diagramma infrastruttura (disegnato dall'operatore, non verificato dal vivo). Elaborata da</code>
Stress-test dell'editor "Infrastruttura tenant": casella nodo invisibile in tema chiaro, collegamento fantasma se il nodo di partenza sparisce a meta' gesto (agente dedicato della maratona, v0.9.99, 25/08/2026)	corretto	Agente dedicato allo stress-test del solo editor "Infrastruttura" spedito in v0.9.98 (interazione pesante scope deliberatamente non sovrapposto). Test dal vivo con vere sequenze <code>PointerEvent</code> dispatcha della stessa maratona. 2 bug reali trovati e corretti: (1) la casella <code><rect></code> di ogni nodo non aveva ne' un attributo <code>fill</code> tema. In tema scuro il nero passava quasi inosservato (testo chiaro sopra, sembrava voluto), ma in tem illeggibili, confermato leggendo il colore calcolato reale nel DOM (<code>rgb(0,0,0)</code> di fill contro <code>rgb(2 per le altre superfici "carta" dell'app) - riverificato dal vivo in entrambi i temi, la casella ora prende il cc gesto di trascinamento termini (nodo di partenza sparito a meta' collegamento), il gestore <code>pointeru salvataggio lato server lo scarterebbe comunque), ma un oggetto fantasma restava comunque in men Corretto aggiungendo il controllo mancante (<code>infraNodes.some(n => n.id === fromId)</code>) prima</code></code> Altre 7 aree testate dal vivo, nessun problema trovato: 60 nodi + 55 collegamenti creati/salvati/rici nome/ruolo/etichetta collegamento (<code><script>...</script></code>) se testo letterale inerte) sia impostando il modello direttamente sia passando dai veri campi del pannello due collegamenti identici tra la stessa coppia di nodi coesistono senza problemi, selezionabili/elimin <code>M365OpsInfraDiagram.ps1</code>); il pulsante "Ricarica" scarta correttamente le modifiche non salvate, rip overflow orizzontale di pagina (limite di design v1 gia' noto, non un difetto: niente pan/zoom, si scorre Nota fuori ambito, non corretta in questo giro: un nome/nota da 250+ caratteri sborda visivament estremo) - segnalato come possibile miglioramento futuro (troncamento con ellissi sul canvas), non af
Documentazione e diagramma infrastruttura condivisi tra profili sullo stesso tenant reale, tipi Internet/Intranet, export/import cifrato del diagramma (v0.10.0, 25/08/2026)	nuovo	Quattro richieste esplicite dell'utente in seguito, tutte legate al diagramma infrastruttura del v0.9.98. 1) Storage condiviso per TENANT REALE, non per profilo: l'utente ha notato che i profili "vnsys-tes "vnsysit.onmicrosoft.com") ma prima vedevano due Knowledge Base/diagrammi separati e vuoti. Corr (<code>login.microsoftonline.com/<id>/v2.0/.well-known/openid-configuration</code> , nessuna aute dello stesso tenant come identiche: l'endpoint restituisce sempre lo stesso GUID indipendentemente c processo, fallback al valore grezzo se la rete non risponde - non blocca mai, la condivisione semplicerr nome profilo). Documentazione e Diagramma infrastruttura ora usano questa chiave (<code>Private\Get-</code>

		per PROFILO (una conversazione e' legata alla sessione/modalita' attiva, non al tenant in astratto). Mig sovrascritti) nel nuovo file condiviso, poi il file legacy viene rinominato con un suffisso <code>.migrated-cc</code> 2) Rimando dalla tab "Documentazione": il diagramma e' concettualmente documentazione del ten impostazioni) - aggiunto un box con pulsante "Apri Infrastruttura" nella tab Documentazione, che 3) Tipi di nodo Internet/Intranet: richiesto esplicitamente ("non fermarti solo all'ibrido, metti dentro solo-cloud, portando il totale a 19 tipi di nodo. Il campo IP esistente accetta gia' liberamente anche ur 4) Export/import del diagramma, cifrato "se possibile": nuovi pulsanti "Esporta..."/"Importa..." nel p 1) insieme al diagramma - all'import, se non coincide col tenant attivo in quel momento, un avviso no lasciato vuoto = export in chiaro): AES-256-CBC + HMAC-SHA256 (cifra-poi-autentica, chiavi derivate <code>System.Security.Cryptography.AesGcm</code> non e' disponibile in modo affidabile su Windows Powe 7 (il runtime reale della GUI). Verifica HMAC SEMPRES prima di decifrare (mai il contrario, per non espo anch'essa assente su .NET Framework). Nuovi file <code>Private\Protect-/Unprotect-M365OpsInfraEx</code> Verificato dal vivo end-to-end: 396 file <code>.ps1</code> sintatticamente puliti. Risoluzione Tenant ID conferma: tramite una vera chiamata di rete all'endpoint Microsoft. Migrazione confermata sui dati reali gia' pres file condiviso (da 6 + 1 nodi separati a 7 nodi condivisi), file legacy rinominati con successo, "AlePiras" ripetuto con successo per la Knowledge Base (documento caricato su "vnsys-test" immediatamente le Domanda IA reale dopo la migrazione ("quanti nodi ci sono nel diagramma infrastruttura di questo te nuova chiave. Export/import testati dal vivo nel browser con veri <code>PointerEvent</code> /eventi file: round-tr corrotto), avviso di tenant diverso mostrato correttamente su un import cross-tenant deliberato, e con "AlePiras" su disco rimasto invariato dopo l'import di prova).
Stress-test della migrazione/risoluzione Tenant ID: un file legacy Knowledge Base bloccato durante la copia poteva restare orfano per sempre (agente dedicato della maratona, v0.10.1, 25/08/2026)	corretto	Agente dedicato allo stress-test della sola risoluzione/migrazione Tenant ID introdotta in v0.10.0 (casi <code>id</code> tra nodi durante un merge, copia dei file caricati durante la migrazione Knowledge Base) - scope export/import). 5 casi limite verificati dal vivo senza problemi: profilo senza campo <code>TenantId</code> (fallback silenzioso valore grezzo, riga <code>Warn</code> confermata nel log); <code>Config\tenants.json</code> temporaneamente assente (corrotto (<code>catch</code> gestito con una riga <code>Warn</code> , nessun crash, nessun file canonico fantasma creato, il f di nodo tra due sorgenti legacy con lo stesso Tenant ID risolto (<code>Sort-Object -Property id -Uniq</code> accettato per costruzione, gli id reali sono timestamp+sequenza, una collisione vera e' astronomicame 1 bug reale trovato e corretto in <code>Private\Get-M365OpsKnowledgeBasePaths.ps1</code> , diverso e piu anche i FILE caricati fisicamente (<code>Uploads\kb\<legacy>\</code> - documento + sidecar <code>.extracted.txt</code> bloccato (lock esclusivo, permessi, qualunque causa transitoria) interrompeva l'INTERO ciclo, non solo INCONDIZIONATAMENTE subito dopo, anche in caso di eccezione catturata dal <code>catch</code> esterno) proc (non solo quello bloccato), catalogo canonico scritto con voci "fantasma" che puntavano a documenti migrazione - risultava gia' rinominato via. <code>Get-M365OpsKnowledgeDocumentText</code> su quei documen Corretto con due modifiche mirate: (1) ogni singola <code>Copy-Item</code> ha ora il proprio <code>try/catch</code> - un f rinominare-come-migrato del catalogo legacy scatta ora SOLO se nessuna copia e' fallita in questo pa ridenta automaticamente la copia dei soli file ancora mancanti (quelli gia' arrivati a destinazione vengo dal vivo con lo stesso identico scenario: primo tentativo (file ancora bloccato) - gli altri file del batch secondo tentativo (nessuna azione manuale, solo un nuovo accesso al profilo) - il file rimasto indietro il testo completo del documento in precedenza bloccato senza alcun errore. Profili/dati temporanei di stress-test (<code>zzstress-*</code>) rimossi da <code>Config\tenants.json</code> e dai relativ "vnsys delegata", <code>AlePiras</code>) e i loro dati non sono mai stati toccati.
Review sicurezza crypto export/import diagramma: un envelope malevolo/corrotto poteva bloccare l'intero server (agente dedicato della maratona, v0.10.2, 25/08/2026)	corretto	Agente dedicato alla review della cifratura del diagramma infrastruttura (<code>Private\Protect-/Unpro diagram/export</code> , <code>/import</code>) - scope deliberatamente non sovrapposto con gli altri due agenti para Review crypto, verificata dal vivo non solo letta: salt (16 byte) e IV generati freschi ad ogni chiama salt/IV/ciphertext tutti e tre diversi tra le due chiamate. Le due chiavi da 32 byte (AES, HMAC) sono me padding-oracle) - confermato leggendo il codice: il <code>throw</code> sul confronto HMAC precede <code>CreateDei</code> differenza; il controllo di lunghezza diversa avviene PRIMA del loop, non durante - accademico in que facoltativo (nota per il futuro, non un difetto: alzarle non guasterebbe). Passphrase errata, ciphertext n generico, nessun dettaglio interno che trapeli su quale parte sia sbagliata, nessuno stack trace verso il 2 bug reali trovati e corretti, entrambi nella gestione di un envelope di import non fidato (mai nella <code>iterations</code> dall'envelope importato senza alcun limite - un envelope con <code>iterations: 2000000</code> single-threaded, questo bloccava l'INTERO server per QUALUNQUE richiesta successiva, persino per / timeout dopo 10s, servito un riavvio manuale del processo per riprendersi. Corretto validando <code>itera</code> fuori da un intervallo ragionevole (1..2.000.000) viene ora rifiutato immediatamente con lo stesso erro bloccare tutto. (2) Canvas rotto su un import senza il campo <code>diagram</code> : un envelope non cifrato cc PowerShell <code>@(\$null)</code> produce un array con UN elemento <code>null</code> dentro, non un array vuoto) - que un'eccezione JavaScript non gestita. Corretto filtrando i valori nulli lato server prima di rispondere - riv Edge case delle route verificati dal vivo senza problemi: envelope vuoto/ <code>{}</code> /senza il marcatore <code>r</code> omessa che stringa vuota esplicita) - messaggio dedicato "serve la passphrase"; tetto dimensionale (5f nodi/collegamenti per il canvas), il tetto vive nel percorso di salvataggio condiviso <code>Set-M365OpsInfr</code> sintetici (troncati correttamente a 500/998); passphrase con caratteri unicode accentati, emoji, e fino a

Editor "Infrastruttura tenant" meno scomodo su schermo portatile: pannello proprieta' a fianco del canvas invece che sotto, altezza responsive (v0.10.3, 26/08/2026)	corretto	Segnalato dal vivo dall'utente: "in uno schermo portatile l'editing della infrastruttura appare un po' co Causa: il pannello proprieta' appariva SOTTO il canvas - selezionare un nodo lo faceva comparire in fo disegnare. Il canvas aveva inoltre un'altezza FISSA di 420px indipendente dallo spazio realmente dispo Corretto con tre interventi concordati con l'utente prima di implementare: (1) nuovo layout a due colc 900px), eliminando lo scorrimento verticale per raggiungere le proprieta' di un nodo appena selezionc compresse; (2) altezza del canvas resa proporzionale allo spazio disponibile (<code>height: min(420px, !</code> , comunque 420px; (3) testo esplicativo sopra la toolbar accorciato (da un paragrafo lungo a una frase, Bug reale trovato durante la verifica dal vivo dello stesso cambiamento, corretto nello stesso gi con <code>flex:1</code> in un contenitore <code>flex-direction: column</code> non si restringe automaticamente sulla larghezza il contenitore del canvas restava largo 1802px, sfiorando il viewport con uno scorrimento ori vivo che il layout a due colonne (largo) non ne risente (la larghezza resta comunque governata da <code>fl</code> Verificato dal vivo a tre larghezze: 1366×768 (laptop tipico) - due colonne fianco a fianco, nessuno s scorrimento di pagina ridotto ma non necessariamente azzerato (accettabile, il resto dell'intestazione/ orizzontale dopo il fix. Riverificate anche le funzionalita' esistenti con il nuovo layout: trascinamento re tema chiaro/scuro applicato correttamente (colore della casella nodo mai nero). 396 file <code>.ps1</code> sintatt
Stress-test approfondito del layout Infrastruttura v0.10.3: canvas che non rispettava l'altezza sul layout impilato (agente dedicato della maratona, v0.10.4, 26/08/2026)	corretto	Agente dedicato a stress-testare oltre le tre larghezze gia' verificate a mano in v0.10.3 - viewport table secondo agente in parallelo copriva invece una ricerca bug generale non legata a questa feature. Bug reale trovato dal vivo, stessa causa di quello gia' corretto in v0.10.3 ma sull'asse perpendic PRINCIPALE del contenitore flessibile - orizzontale nel layout a due colonne (dove infatti <code>height: mi</code> impilato sotto i 900px - esattamente dove quell'altezza avrebbe dovuto contare di piu'. Risultato: <code>fle</code> Riprodotto dal vivo su un tablet verticale (768×1024, layout impilato): altezza reale del canvas 1017px stessa esperienza scomoda (seleziona un nodo, scorri per trovare le proprieta') che l'intera modifica di pattern gia' usato per <code>#infra-props-panel</code> nella stessa media query: <code>flex: none !important; flex-basis ad auto</code> , cosi' l'altezza dichiarata torna a governare davvero la dimensione. Riverifica raggiungibile. Resto della checklist verificato dal vivo senza altri problemi: soglia dei 900px collaudata a 899/900 impilati, coerente con <code>max-width: 900px</code> inclusivo; 901px passa a due colonne); tablet 768×1024 (o largo 1600×900/1920×1080 - il canvas riempie correttamente lo spazio restante via <code>flex:1</code> orizzon non un difetto funzionale, lasciata cosi' com'era); trascinamento reale (sequenza <code>PointerEvent</code> auto confermato (delta schermo e delta coordinate salvate identici); creazione di un collegamento trascin layout - nessun collasso, scorrimento interno del canvas ancora disponibile; pannello proprieta' con al raggiungibili tramite lo scorrimento interno gia' presente del pannello. 396 file <code>.ps1</code> sintatticamente
Stress-test generale GUI/codice: 4 voci del catalogo comandi locale ignoravano in silenzio "invia/manda/spedisci ... a indirizzo@email" (v0.10.5, 26/08/2026)	corretto	Agente dedicato a un giro ampio non legato a una singola feature recente (percorsi utente end-to-en Infrastruttura v0.10.3/v0.10.4. Bug reale trovato dal vivo: <code>Gui\CommandCatalog.ps1</code> ha gia' un pattern consolidato (dal 19/08/2 <code>ExportDevices / ExportMailboxUsageReport / ExportSharedMailboxReport / ExportAllMai</code> si fanno da parte (passano all'IA) quando il messaggio contiene un verbo di invio ('invia/'manda/'spe meta' dal solo export locale. Questo pattern non era mai stato propagato a quattro voci gemelle - <code>Li</code> come "invia il report dei dispositivi non conformi a mario@contoso.com" o "manda la panoramica ute ignorando in silenzio la parte "invia/manda a" - nessun errore, nessuna proposta di invio, l'utente non con "Fonte: comando locale 'ListNonCompliant'"/ecc., mai "Elaborata da IA"). Corretto aggiungendo ai <code>DeferWords</code> delle quattro voci lo stesso segnale di invio gia' in uso altrove messaggio e' GIA' un segnale forte di intento di invio), <code>MfaStatus / UserOverview</code> richiedono SEV avrebbe fatto deviare all'IA anche l'uso normale ("stato mfa di mario@contoso.com"), aumentando in (<code>invia manda spedisci</code>)\w*.*@\S+\.\S+ - sicuro perche' l'uso normale non contiene mai un ve lo stesso pattern basta da solo, con il vantaggio aggiuntivo di evitare anche un secondo problema cor "panoramica gruppo IT e mandala a capo@contoso.com" avrebbe comunque catturato "IT e mandala Riverificato dal vivo dopo il fix: tutti e quattro i casi riprodotti in precedenza ora rispondono tramite delegata non e' attiva... Non e' stata inviata alcuna email a mario@contoso.com" - riconosce l'intento invariato, ancora gestito localmente a costo zero IA ("stato mfa di X", "panoramica utente X", "panorar oltre 30 casi ambigui gia' collaudati nei giri precedenti (17-23/08/2026): nessuna variazione di compor HTML bilanciati, 2 blocchi <code><script></code> inline sintatticamente validi. Altre aree stress-testate nello stesso giro, nessun problema trovato: coesistenza dei quattro pannr (sono siblings in flusso normale, non posizionati assolutamente, si limitano a impilarsi verticalmente), correttamente in sequenza dal server single-threaded, storico chat coerente, nessuna risposta incrocia esecuzione - script creato, riavvio automatico del server, nuovo strumento immediatamente utilizzabil (<code>/api/chat</code> con JSON malformato o corpo vuoto, <code>/api/tenants/activate</code> su un profilo inesiste <code>/api/kb/upload</code> con base64 non valido o nome file mancante, <code>/api/ai-status/test</code> con un pi
"Stato permessi" su tenant Delegato: un Exchange non connesso nascondeva anche il controllo Intune/Entra ID/Teams/SharePoint, completamente indipendente (v0.10.6, 26/08/2026)	corretto	Segnalato dal vivo dall'utente: cliccando "Stato permessi" su un tenant Delegato con Exchange non an corretto nel dire che serve connettere Exchange, ma l'utente ha giustamente notato: "perche' cita solo Causa reale: <code>Get-M365OpsDelegatedPermissionsCheck.ps1</code> (usata per i tenant Delegato, dove n <code>M365OpsExchange</code> SENZA protezione come primissimo passo. Su un tenant Delegato senza sessione

		thread singolo, un login bloccante congelerebbe l'intera app per tutti - comportamento corretto e vol il blocco - gia' esistente e completamente indipendente da Exchange - che controlla i ruoli DIRECTOR' mancante (Exchange) nascondeva quindi il risultato di un controllo del tutto slegato che avrebbe potu Corretto racchiudendo l'intero blocco Exchange (connessione + lettura ruoli RBAC) in un <code>try/catch</code> con lo stesso messaggio di <code>Connect-M365OpsExchange</code> - cosi' l'utente sa esattamente cosa fare) inve comportamento di <code>Connect-M365OpsExchange</code> stessa (il rifiuto del login automatico resta corretto Verificato dal vivo: riprodotto lo scenario esatto segnalato dall'utente su un tenant Delegato ("AlePir nello stesso messaggio: "Ruoli RBAC Exchange Online... non verificabile" (con l'istruzione su come con tab Tenant, 'Accedi a Microsoft Graph con il mio utente)" - il controllo raggiunge ora davvero l'area G <code>.ps1</code> sintatticamente puliti.
Stress-test Exchange/mail-flow su tutto il codice: 4 report/controlli morivano a meta' ciclo per una singola mailbox/risorsa problematica, analisi intestazioni email falliva su un giorno-della-settimana sbagliato (agente dedicato della maratona, v0.10.7, 26/08/2026)	corretto	Agente dedicato allo stress-test dell'intera area Exchange/mail-flow (mailbox, gruppi di distribuzione, Exchange, permessi app/delegati), non limitato alle feature spedite di recente - in parallelo ad altri qu ID/sicurezza, Teams/SharePoint/report/script personalizzati, GUI ampio). 4 bug reali trovati dal vivo, tutti la stessa causa radice gia' vista in v0.10.1/v0.10.2/v0.10.6 (una t altre pur non dipendendo affatto dalla prima): (1) <code>Get-M365OpsMailboxDelegatesReport.ps1</code> - il singola mailbox su cui quella lettura lancia un errore (throttling momentaneo, un tipo di destinatario r <code>M365OpsSharedMailboxReport.ps1</code> - stesso schema: le chiamate <code>Get-EXOMailboxPermission / statistiche (-ErrorAction SilentlyContinue). (3)</code> <code>Get-M365OpsMailboxStatistics.ps1</code> - ste <code>M365OpsInactiveMailboxes</code> , <code>Get-M365OpsMailboxUsageReport</code>) ma non qui: chiamata SENZA <code>M365OpsAppPermissionsCheck.ps1</code> - il ciclo che risolve gli appRoles per ogni RISORSA Graph distrir altre) non aveva nessuna protezione per risorsa: un errore Graph su una sola risorsa uccideva l'intera f limitarsi a mostrare "nessun accesso" solo per l'area davvero coinvolta. Corretti tutti e quattro isolando l'unita' di lavoro fallita (<code>try/catch</code> o <code>-ErrorAction SilentlyCo</code> produce ora una riga/nota dedicata per quell'elemento invece di interrompere l'intero comando - veri nessuna riga d'errore, tenant sano), <code>Get-M365OpsMailboxStatistics</code> (25 righe complete), report r sul percorso normale, il fix cambia comportamento solo quando un elemento specifico fallisce davvero 5° bug reale, area diversa: <code>Get-M365OpsMessageHeaderAnalysis.ps1</code> (analizzatore locale di inte "Tue,") viene VALIDATO da .NET contro la data numerica che segue, non letto come semplice testo: un composti a mano da script di terze parti) faceva fallire ENTRAMBI i tentativi di parsing gia' presenti ne because the day of week was incorrect" - non un formato non standard (gia' gestito con cura da codic l'orario grezzo non interpretato al posto di quello formattato e nessun calcolo di ritardo tra hop, esatt cadeva davvero di mercoledi': <code>DateTimeOffset.Parse</code> falliva puntualmente con l'errore sopra. Corri numerica, che e' quella davvero usata per il calcolo) - riverificato dal vivo via <code>POST /api/analyze-h</code> ricadere sul testo grezzo. Altre aree Exchange coperte senza trovare problemi: 105 file <code>.ps1</code> dell'area Exchange/mail-flow (<code>M365OpsIsolatedModule</code> , <code>Complete-M365OpsIsolatedModuleConnect</code> , percorso asincrono <code>Sta</code> nessun problema nuovo trovato; sorella App-only di <code>Get-M365OpsDelegatedPermissionsCheck</code> (radice, punto diverso del ciclo; forwarding report, mail flow report, litigation hold report, message trac modifica necessaria.
Stress-test Teams/SharePoint/OneDrive/report/script personalizzati su tutto il codice: PDF multi-sezione perdeva TUTTE le sezioni per un errore in una sola, un errore intermittente di Edge headless faceva riportare "PDF non generato" nonostante il file fosse gia' sul disco (agente dedicato della maratona, v0.10.14, 26/08/2026)	corretto	Agente dedicato allo stress-test dell'intera area Teams/SharePoint/OneDrive/generazione report/catal (stesso pattern "un passo fallito blocca i passi fratelli indipendenti" gia' cercato su tutto il codice in v0. 5 bug reali trovati e corretti, quattro della stessa famiglia gia' nota: (1) <code>Export-M365OpsDataRepo</code> che uno per sezione: un errore imprevisto in una sola sezione (es. un campo con dati anomali) faceva prima produceva un PDF con ZERO sezioni invece di 5 + una nota d'errore sulla sesta. Il percorso <code>xlsx</code> l'equivalente - ora ogni sezione ha il proprio <code>try/catch</code>, verificato dal vivo forzando un errore in una se: <code>M365OpsOneDriveSharingRecipient.ps1</code> (terzo passo del workaround OneDrive "accesso negato un fallimento su UN elemento (throttling, permesso gia' rimosso nel frattempo) lanciava un'eccezione ritorno pur essendo davvero rimossi su SharePoint) sia gli elementi restanti mai raggiunti - il chiamant in un nuovo campo <code>Failed / FailedCount</code> e il ciclo prosegue sugli altri. (3) <code>Get-M365OpsShareP</code> isolamento reciproco: un fallimento su UNO dei quattro controlli (es. throttling sul gruppo "Visitors") f reale (<code>vnysit.sharepoint.com/sites/test</code>): tutte e quattro le aree risolte correttamente. (4)/(5) <code>Get-M365OpsGet-CsTeams*Policy / Get-CsTeams*Configuration</code> indipendenti (Meeting/Calling/Messaging isolando ciascuna nel proprio <code>try/catch</code>, MA preservando invariato il meccanismo di retry via isolament esterno continua a scattare come prima) - verificato dal vivo: 19 criteri Teams (10 Meeting, 5 Calling, 4 6° bug reale, causa diversa: <code>Export-M365OpsReport.ps1</code> (ramo 'pdf') - dopo il lancio di Edge hea riprodotto dal vivo chiedendo "genera un report PDF" via <code>/api/chat</code> sui Team del tenant - la rispos (verificato confrontando timestamp e contenuto). Il processo <code>msedge.exe</code> avviato con <code>&</code> non garanti: sostituendo l'attesa fissa con un polling fino a 20 secondi (dimensione del file stabile per due controlli questa maratona. Riverificato dal vivo lo stesso identico scenario dopo il fix: PDF generato e offerto co Aree coperte senza trovare problemi: creazione/gestione Team e siti SharePoint, membership siti, er correttamente isolati), catalogo script personalizzati (verificato dal vivo con uno script deliberatamente <code>.ps1</code> sintatticamente puliti dopo le modifiche.
Stress-test GUI ampio su tutta l'app: upload mostrava "caricato con successo" anche quando falliva davvero, due punti della Settings panel diventavano	corretto	Agente dedicato allo stress-test dell'intera superficie GUI/UX (<code>Gui/Index.html</code> + <code>Gui/Server.ps</code>: fallito blocca i passi fratelli indipendenti" gia' cercato su tutto il codice in v0.10.1/v0.10.2/v0.10.6/v0.10

davvero inutilizzabili a larghezza mobile (agente dedicato della maratona, v0.10.15, 26/08/2026)		CRUD profili tenant + toggle campi AuthMode, avvio del login Delegato a codice dispositivo (percorsi 3 bug reali trovati e corretti: (1) <code>POST /api/upload</code> (file principale/icona/CSV migrazione) non aveva <code>FromBase64String / WriteAllBytes</code>, percorso non scrivibile...) cadeva solo nel catch generico esistente per uno status HTTP di errore, e nessuno dei tre handler client per questa rotta controllava <code>res.ok</code> / stato caricato davvero e il testo dell'errore come un normale messaggio di sistema invece che un errore "caricato" e il messaggio appariva come testo normale; dopo il fix l'etichetta resta "Nessun file caricato" / <code>api/kb/upload</code>) sia lato client (controllo esplicito su tutti e tre gli handler upload). (2)/(3) A 375px Infrastruttura già corretta in v0.9.73: la riga delle 6 schede Settings (<code>.settings-tabs</code>) e la riga di ogni bug header già noto - ma qui più grave: misurato dal vivo che i pulsanti "Modifica"/"X" di un profilo disponibili), non semplicemente "poco estetico" - un profilo diverso da quello attivo non era più modificabile per l'header), verificato dal vivo con screenshot e misure DOM dirette prima/dopo: nessun elemento schermo (375px), nessun impatto sul layout desktop. Aree coperte senza trovare problemi: cambio provider AI (Claude/Azure OpenAI, campi mostrati/nascondi "Controlla aggiornamenti" (canale Stabile), banner aggiornamento e il suo pulsante "Vedi in Manutenzione" già corretta: rifiuta un profilo App-only senza secret né certificato), avvio login Delegato a codice dispositivo più lento del solito per la contesa sul server a singolo thread condiviso con altri quattro agenti), Invio dentro un contenitore <code>display:flex</code> (verificato visivamente che il browser lo rispetta comunque, non è questo giro (rischio concreto su stato condiviso con altri quattro agenti in esecuzione sullo stesso server <code>FIRST_RUN_TEXT</code> coerente con i campi form reali, nessuna incongruenza trovata), completamento r
Stress-test Intune/Entra ID/MFA/Conditional Access su tutto il codice: l'assegnazione a gruppi e le app di destinazione dei criteri di protezione app (MAM) non hanno mai funzionato, un errore su un'area/dispositivo/piattaforma indipendente ne nascondeva altre già risolte con successo (agente dedicato della maratona, v0.10.16, 26/08/2026)	corretto	Agente dedicato allo stress-test dell'intera area Intune/Entra ID/MFA/Conditional Access (dispositivi gruppi app), non limitato alle feature spedite di recente - in parallelo ad altri quattro agenti sulle rispettive aree v0.10.1/v0.10.2/v0.10.6/v0.10.7/v0.10.14/v0.10.15). 2 bug reali più gravi, rotta OData sbagliata mai riuscita nemmeno una volta: creato un criterio di <code>M365OpsAppProtectionAssignment.ps1</code> e (2) <code>Set-M365OpsAppProtectionTargetApps.ps1</code> fa in GET da <code>Get-M365OpsAppProtectionPolicies</code>, mai scrivibili in POST; Graph rifiutava SEMPRE la scrittura non hanno mai funzionato una sola volta da quando esistono. Corretto individuando dal vivo risultato anch'esso sbagliato con un errore diverso, "Resource not found for the segment 'assign'" la riga (<code>/deviceAppManagement/{android ios}ManagedAppProtections/{id}/assign</code> <code>targetApps</code>, codice precedente: per mantenere il comportamento "AGGIUNGE" già promesso da queste funzioni, l'aggiunta (dopo la correzione della rotta): la GET di rilettura non include un <code>@odata.type</code> a livello di singolo e reinvia solo i campi che la action richiede davvero (<code>target</code> per le assegnazioni, <code>mobileAppDeployment</code> per entrambi gli elementi precedenti restavano intatti (comportamento "aggiunge" confermato, non sostituisce). 6 istanze dello stesso pattern "un passo fallito blocca i passi fratelli indipendenti" già cercato in riga/nota/campo d'errore dedicato invece di interrompere l'intero comando): (3) <code>Get-M365OpsCompliance</code> dispositivo problematico (rimosso nel frattempo, 404/429 transitorio) faceva sparire dall'analisi AI anche su UNA sola piattaforma (Android o iOS) nascondeva anche i criteri dell'altra già recuperati. (5)/(6) Le compliance sono tre chiamate Graph indipendenti fra loro, un errore su una sola faceva fallire l'intera categoria riuscita). Verificato dal vivo su vnsys-test dopo ogni fix: "verifica permessi app" (già corretto non conformità dispositivi" (5 dispositivi, correlazione VM non cifrate identificata correttamente dall'URL sul percorso normale. Aree coperte senza trovare problemi: RBAC Intune (creazione/rimozione ruoli custom e assegnazioni Template, Enrollment Configuration, Update Ring, Autopilot Deployment Profile, Proactive Remediation per gruppo come le due App Protection sopra), reset MFA (già correttamente isolato per metodo), stesura (correggere), directory role assignments/Entra ID admin roles (sorella App-only <code>Get-M365OpsAppPermissions</code> Autopilot; 401 file <code>.ps1</code> sintatticamente puliti dopo le modifiche.
Ultimi 3 casi dello stesso pattern "un passo fallito blocca i passi fratelli indipendenti": report membership/overview gruppi Exchange, criteri Safe Links/Safe Attachments (agente dedicato della maratona, v0.10.17, 26/08/2026)	corretto	Agente dedicato a un audit MIRATO su tutto il codice (non un'area funzionale specifica) alla ricerca dei gruppi v0.10.1/v0.10.2/v0.10.6/v0.10.7/v0.10.14/v0.10.15/v0.10.16), in parallelo ad altri quattro agenti sulle rispettive funzioni diverse), questi tre restavano scoperti. 3 bug reali corretti: (1) <code>Get-M365OpsGroupMembershipReport.ps1</code> - <code>Get-DistributionGroupMembers</code> throttling momentaneo Exchange Online) faceva sparire dal report anche i membri dei gruppi sani già presenti anche per i gruppi sani prima del fix. Corretto isolando la chiamata per gruppo (try/catch), un fallimento schema sul conteggio membri per gruppo statico, stessa correzione. (3) <code>Get-M365OpsThreatPolicy</code> (non la sola lista vuota per licenza mancante, già gestita) sulla prima chiamata (<code>Get-SafeLinksPolicy</code> indipendente. Corretto isolando le due chiamate, un fallimento su un tipo di criterio non nasconde più). Verificato dal vivo dopo il fix (server riavviato, non solo lettura del codice): tutti e tre isolati correttamente sintatticamente puliti su tutto il repository dopo l'unione dei fix di tutti e cinque gli agenti di questo gruppo. Nota di processo: questo giro di 5 agenti paralleli è partito da una correzione esplicita dell'utente, che ha richiesto immediata - la regola #2 della maratona ("verificare TUTTO il codice, non solo le aree toccate di recente") quella regola questa volta: oltre 20 bug reali trovati e corretti in un solo giro (somma dei cinque agenti funzionato una sola volta dalla loro creazione.
Intestazioni incollate in chat: non più un redirect che blocca il messaggio, ora l'AI risponde normalmente E si apre in più il pannello di analisi (v0.10.18, 26/08/2026)	nuovo	Cambio di approccio richiesto esplicitamente dall'utente sul comportamento introdotto in v0.9.95: prima di fare la domanda che vuole, e in più si apre una finestra separata con l'analisi dell'header". Corretto riprendendo tutto quanto già costruito, cambiando solo DOVE scatta l'effetto: rimossa la voce

		alla domanda reale posta insieme al testo incollato, esattamente come qualunque altro messaggio). Il interamente lato client in <code>Gui/index.html</code> - un puro effetto collaterale sulla UI, non piu' una decisio automaticamente il pannello "Intestazioni email" (se non gia' aperto), il testo viene copiato nella sua c dell'IA alla propria domanda E la tabella hop/verifica SPF-DKIM-DMARC/decodifica antispam/pulsante toccato - riusa integralmente quanto gia' costruito e verificato in v0.9.90-94. Verificato dal vivo end-to-end: lato server, lo stesso testo che prima veniva intercettato ora produce analizza SPF/DKIM/DMARC disponibili e chiede gli header mancanti piu' utili, non piu' un messaggio c solo, si popola col testo appena inviato, e mostra gia' riepilogo/tabella hop entro mezzo secondo, PRI falso positivo storico ("Your payment has been received: from now on please send invoices to billing@ puliti, tag HTML bilanciati, 2 blocchi <code><script></code> inline sintatticamente validi.
Fallback a CLI Microsoft 365 quando manca il consenso admin per il login Graph delegato (v0.10.19, 26/08/2026)	nuovo	Richiesto esplicitamente dall'utente: sui tenant Delegati il login Graph puo' fallire per consenso ammir quel tenant) - un errore di AUTORIZZAZIONE, non di credenziali sbagliate. CLI Microsoft 365 si autentico concesso il consenso all'una e non all'altra, quindi un fallback verso CLI365 al momento giusto e' conc Riconoscimento dell'errore specifico: <code>Public/Complete-M365OpsDelegatedLogin.ps1</code> ora rico polling del login a codice dispositivo, e lo segnala alla GUI con un nuovo campo <code>NeedsAdminConsen</code> GUI: quando <code>NeedsAdminConsent</code> e' vero, sotto il messaggio d'errore compare un box dedicato con "MCP/Connettori" e aziona il pulsante "Connetti CLI Microsoft 365" gia' esistente (nessuna logica dup Nessuna nuova logica di priorita' IA necessaria: una volta che il login CLI365 riesce dove Graph ha l app) attraverso il meccanismo gia' esistente in <code>Invoke-M365OpsAgentTools.ps1</code> (<code>\$graphDelegat</code> resta assente sia che il login non sia mai stato tentato sia che sia fallito per consenso, quindi la prefere CLI365 copre davvero - Intune, Exchange profondo, criteri Teams e SharePoint via CLI365 restano fuor Verificato dal vivo: la regex di riconoscimento testata contro un vero messaggio AADSTS65001 (ricor l'intera funzione <code>Complete-M365OpsDelegatedLogin</code> testata end-to-end con una chiamata reale si sul messaggio di consenso reale, <code>False</code> su un vero errore Azure AD diverso (device_code non valido pulsante passa davvero alla tab MCP/Connettori e aziona davvero il pulsante "Connetti CLI Microsoft : sintatticamente puliti, tag HTML bilanciati (verificato dopo aver anche corretto uno script di supporto
BUG: il box "Prova con CLI Microsoft 365 invece" restava visibile con il messaggio del tentativo precedente dopo un cambio di tenant attivo (v0.10.20, 27/08/2026)	corretto	Trovato durante lo stress-test esplicitamente richiesto dall'utente subito dopo la v0.10.19 ("stress test fallback (simulando un login fallito per consenso mancante, <code>NeedsAdminConsent</code> vero), poi attivato tentativo del tenant precedente. Passando per un tenant App-only in mezzo il box spariva solo visivarn qualunque tenant Delegato dopo, riappariva comunque "sporco". Causa: <code>loadConnectionStatus</code> (alias <code>refreshDelegatedLoginBox</code> , richiamata anche da <code>activ</code> senza mai toccare lo stato del box di fallback <code>#delegated-login-consent-fallback</code> annidato al s restava quindi "in eredita'" da un tentativo precedente su un tenant diverso finche' non ne partiva uno Corretto nascondendo esplicitamente <code>#delegated-login-consent-fallback</code> in <code>loadConnectio</code> raggiungibile) - cosi' ogni refresh dello stato connessioni (cambio tenant compreso) riparte sempre pu quel momento. Verificato dal vivo sul server di test reale, stessa sequenza usata per riprodurre il bug: mostrato il box tornato su un terzo tenant Delegato (box ancora correttamente nascosto in ogni punto della sequenz comportamento su tenant App-only, avvio di un nuovo login dopo un annullamento client-side) - tutt <code><script></code> inline sintatticamente validi.
Report di utilizzo Microsoft 365 Copilot agganciati all'IA in chat; 3 bug HTML pre-esistenti trovati e corretti nella guida (v0.10.21, 27/08/2026)	nuovo	Richiesto esplicitamente dall'utente dopo la chiusura del giro v0.10.19/v0.10.20 ("aggancia le funzioni ma mai raggiungibili se non da console PowerShell diretta - vedi sezione 24.2.1). Aggiunti due tool AI LETTURA con parametro opzionale <code>period</code> (default D30). Deliberatamente NON aggiunta una voce bug-hunt mirati sulle sue voci gemelle (DeferWords/frasi di continuazione/invio email in coda) - dupli beneficio marginale visto che il tool AI copre gia' lo stesso caso d'uso in linguaggio naturale. Verificat <code>get_copilot_usage_summary</code> , "dammi il dettaglio per singolo utente di chi ha usato copilot negli entrambe hanno propagato all'utente il vero 403 <code>Reports.Read.All</code> mancante (permesso non anc disponibile". Bug collaterali trovati durante la verifica di bilanciamento tag di questa stessa modifica (non leg <code>class="warn"></code> (sezioni "Passi per assegnare il ruolo directory Entra ID", "17.16 Copertura Intune", "1 "warn" davvero aperto nell'HTML, facendo si' che lo sfondo/bordo di avviso si estendesse visivamente "ancora irrisolto"); nel terzo caso il <code><div></code> si chiudeva comunque correttamente subito dopo, quindi tag mirata (stack-based, non solo conteggio open/close, per localizzare esattamente dove) eseguita co div/p/span/section risultano tutti bilanciati (0 tag non chiusi) sull'intero documento.
Pulsante "Disconnetti tutto" / "Connetti tutto" per il tenant attivo (v0.10.22, 27/08/2026)	nuovo	Richiesto esplicitamente dall'utente: un modo per "rinforzare la connessione applicativa" subito dopo riflettono i permessi presenti al momento in cui sono stati rilasciati, non quelli aggiunti dopo. Discussi pulsante "Disconnetti" piu' radicale che diventasse "Connetti" una volta verificato che nessuna connes Risposta alla domanda sui tenant Delegati: il login Delegato di questo modulo usa un'app Microsoft configurata nel profilo tenant - un permesso aggiunto all'App Registration non ha quindi alcun effetto meta "disconnetti/pulisci tutto" resta pero' utile per ENTRAMBE le modalita' (motivo diverso: non per comportamenti diversi in base alla modalita' del tenant attivo, invece di due pulsanti separati con parz

		Implementato: <code>Disconnect-M365OpsAllConnections</code> (nuova) interrompe TUTTO per il tenant att token Graph (app-only o delegato) ed un eventuale login a codice dispositivo lasciato a meta' - riusan riconnette tutto cio' che PUO' esserlo in silenzio: su App-only rifà token Graph diretto + Exchange + T per passo, mai un fallimento che blocca i fratelli - stesso principio già applicato sistematicamente in v dipende da un nuovo login interattivo) e restituisce solo un'indicazione chiara verso il pulsante "Acced GUI: un solo pulsante in cima a "Stato connessioni" (tab Tenant), che il codice etichetta "Disconnetti tu "connesso" copre anche il token Graph diretto (campo nuovo <code>AppOnlyTokenCached</code> in <code>Get-M3650 dedicato. Dopo un "Connetti tutto" su App-only, i risultati per-area (8 passi: token Graph, Exchange, Tr riassunti in un unico esito.</code> Bug collaterale corretto mentre si costruiva questa funzione: <code>Connect-M3650ps</code> (attivazione tenan Compliance/Purview (nessuna funzione dedicata esisteva) - il flag <code>ComplianceConnected</code> restava q modulo <code>ExchangeOnlineManagement</code>). Aggiunta <code>Disconnect-M3650psCompliance.ps1</code> (nuova, Verificato dal vivo end-to-end su vnsys-test (App-only) tramite la GUI reale nel browser, non solo via SharePoint, Compliance, Intune, CLI-Microsoft365, Lokka - toolCount reali confermati, es. Lokka 31 toc <code>/api/mcp-status</code> , pulsante tornato a "Connetti tutto". Testato anche su un tenant Delegato ("vnsys senza produrre errori fuorvianti. 399 file <code>.ps1</code> sintatticamente puliti, tag HTML bilanciati (div/p/span
Bug reale segnalato dal vivo dall'utente su "Connetti tutto": riquadro in alto contraddiceva quelli sotto, errore generico senza indicazioni (v0.10.23, 27/08/2026)	corretto	Segnalato subito dopo il rilascio di v0.10.22: "vedo... Riconnesso 8/8... Errore nel caricare lo stato — se connesso. se sopra scrive connesso per exch intune teams shp etc allora questo semaforo verde si dev Causa: il click handler di "Connetti tutto" chiama <code>loadConnectionStatus()</code> subito dopo aver most dal vivo (fetch di quella sola chiamata simulato fallito): se QUESTA seconda chiamata fallisce, <code>loadCo Exchange/Teams/SharePoint/Purview/Intune/CLI365 piu'</code> sotto, che restavano quindi fermi sullo stato p contraddittorio segnalato dall'utente. Il messaggio d'errore era inoltre generico ("server non raggiung Perche' quella seconda chiamata puo' fallire: non riprodotto in modo affidabile sul tenant di test (st già' in esecuzione da piu' a lungo e' il conflitto residuo tra moduli .NET già' documentato in sezione 6. progetto che concatena Exchange+Teams+SharePoint+Compliance+Intune in sequenza rapida nello S Corretto su due fronti, senza aspettare di capire con certezza assoluta la causa esatta (il sintomo e la c (e il pulsante stesso) vengono ora aggiornati SUBITO dai risultati che <code>reconnect-all / disconnect SOLO da un secondo giro di rete che potrebbe fallire; (2) <code>loadConnectionStatus</code> ora riprova UNA e al riavvio dal tab Manutenzione) invece del generico precedente.</code> Verificato dal vivo su vnsys-test: percorso normale (nessun fetch simulato fallito) confermato senza r sostituito temporaneamente per intercettare solo <code>/api/mcp-status</code>) - riquadri e pulsante ora corre eliminata la contraddizione originale; stesso test ripetuto sul ramo "Disconnetti tutto" (riquadri tornati questo giro, solo <code>Gui/index.html</code>), JS validato, tag HTML bilanciati.
Pannello "Stato del processo" nel tab Manutenzione: server up/lento visibile a colpo d'occhio, con RAM/CPU/tempo di risposta (v0.10.24, 27/08/2026)	nuovo	Richiesto esplicitamente dall'utente subito dopo un episodio reale: "ho cliccato connetti tutto è fermo risultava connesso una volta finito) - "Connetti tutto" concatena 8 passi reali (token Graph, Exchange, ha richiesto fino a ~140 secondi in un giro di verifica - senza nessun segnale di progresso, un'operazic (23/08/2026, "sembra che il server sia crashato" - vedi <code>Start-M3650psIsolatedModuleConnectAsy</code> Corretto su due fronti: (1) il messaggio "Riconnessione/Disconnessione in corso" nel tab Tenant ora i - prova visibile che la scheda del browser non è bloccata, anche se il server non può ancora rispondere mentre quel tab è aperto (fermato altrove per non sprecare risorse): mostra pallino verde/rosso, temp endpoint <code>GET /api/server-health</code> , deliberatamente senza alcuna logica oltre a <code>Get-Process \$ mai da solo) e un messaggio "non risponde da Xs" se supera quella soglia.</code> Scoperta importante durante la verifica dal vivo, documentata esplicitamente nel testo del pannell corso (verificato: interrogato <code>/api/server-health</code> via <code>curl</code> mentre "Connetti tutto" era ancora ir un difetto del pannello, è la prova stessa che il processo è occupato e non morto: se fosse davvero cra questa distinzione esplicitamente, invece di lasciar credere che un "non risponde" temporaneo durant Deliberatamente NON reso asincrono con un processo isolato separato (lo stesso meccanismo usato p gli 8 passi di "Connetti tutto" avrebbe richiesto un cambiamento architetturale molto più ampio e riscl era di percezione/trasparenza, non di un vero blocco. Verificato dal vivo su vnsys-test: pannello mostra correttamente "Server attivo — ha risposto in N ms (verificato leggendo lo stato del timer lato client); simulato un fetch che non risolve mai (per testare il pallino rosso, il contatore cresce ad ogni poll successivo, e torna da solo a "Server attivo" appena il fet
SharePoint via CLI Microsoft 365 con login a certificato; pallino di stato connessioni sempre visibile nell'header (v0.10.25, 27/08/2026)	nuovo	Richiesto esplicitamente dall'utente, scegliendo tra alcune opzioni proposte dopo una domanda apert 1) SharePoint via CLI365 a certificato: limite noto da tempo (26/08/2026) - i comandi 'spo' fallivano a certificato non era implementata perché non era mai stato verificato dal vivo se '--thumbprint' da so Verificato ora con 'm365 login --help' (CLI installata, v11.10.0): serve davvero <code>--certificateBase64</code> Verificato inoltre dal vivo (autorizzazione esplicita dell'utente, trattandosi di materiale di chiave privata produzione la marcano non esportabile. <code>Connect-M3650psCliMicrosoft365.ps1</code> ora esporta il ce lo codifica in base64 e lo passa a <code>--certificateBase64Encoded / --password</code> - stesso principio nessuna alternativa esposta dal tool MCP). Il certificato è ora PREFERITO al secret quando entrambi so prompt dell'IA (<code>Invoke-M3650psAgentTools.ps1</code>) ora distingue i due casi invece di escludere sem

		Verificato dal vivo end-to-end su vnsys-test: rimossa la connessione CLI365 salvata precedentement "authType": "certificate" sulla nuova connessione; poi in chat, "chiama cli_m365_run_comman stato restituito.") - prima falliva sempre su questo stesso tenant con solo secret configurato. 2) Pallino di stato connessioni nell'header: prima, per sapere se qualcosa era connesso bisognava a aggiornato ogni 20s - verde se almeno una connessione risulta attiva, grigio altrimenti (mai un rosso a pannello "Stato del processo" in Manutenzione, v0.10.24). Clic sul pallino apre direttamente il tab Tena dentro il pulsante Disconnetti/Connetti tutto, ora un solo posto per non farla divergere nel tempo tra Verificato dal vivo nel browser: pallino grigio a tenant scollegato, passa a verde subito dopo aver cor Tenant. 399 file .ps1 sintatticamente puliti, JS validato, tag HTML bilanciati.
Generatore di certificato in GUI; fix etichetta pulsante Intune fuorviante su App-only (v0.10.26, 27/08/2026)	nuovo	1) Bug reale segnalato dal vivo dall'utente: "vedo nella sezione app che il testo Intune è 'connetti in #intune-test-btn mostrava sempre "Connetti Intune (browser)" a prescindere dalla modalità, anc silenziosa, certificato/secret) - un'incoerenza reale tra testo esplicativo e pulsante. Corretto: l'etichetta il paragrafo. 2) Generatore di certificato: richiesto esplicitamente dall'utente ("è utile implementare in gui una fur prima di implementare: va scaricata e caricata su Entra solo la parte PUBBLICA del certificato (.cer), MA (upload del .cer su Entra), oppure upload automatico via Graph (richiederebbe Application.ReadWr l'utente fa upload"). Nuova New-M365OpsSelfSignedCertificate.ps1 - stessi identici parametri già documentati in gu SelfSignedCertificate -CertStoreLocation Cert:\CurrentUser\My -KeyExportPolicy Exp automaticamente il profilo tenant ATTIVO col nuovo thumbprint (via Set-M365OpsTenant , ripassan ometterli li avrebbe azzerati per sbaglio) e aggiorna anche il contesto già in memoria di questo proces sopra SharePoint - il certificato è condiviso da entrambi oltre a Intune/CLI365): mostra thumbprint e s server) col nome del profilo, e un promemoria del passaggio manuale rimanente (Entra ID → App Reg Verifica esplicita sulla non-pubblicazione (richiesta diretta dell'utente, "ovviamente la parte dei cert finisce il thumbprint) e Logs/ sono entrambi ignorati e MAI tracciati da git; verificato che New-M36 scrivono mai il contenuto del certificato o la password effimera in nessun log; la chiave privata non to scritto sul server). Verificato dal vivo su vnsys-test: generato un certificato reale via API diretta - Config\tenants.js confermato presente nel certificate store di Windows con HasPrivateKey: True ; poi ripetuto dal t corretto). Ripristinato subito dopo il thumbprint originale del tenant di test (quello realmente caricato HTML bilanciati.
MARATONA COMPLETA su tutto il progetto (400+ file, 12 agenti in 3 fasi): ~40 bug reali trovati e corretti, incluso uno di sicurezza (v0.11.0, 31/08/2026)	corretto	Richiesta esplicita dell'utente: "riapriamo con la maratona stress test debug etc. almeno 18 ore di mar differenza dei giri precedenti (mirati su codice toccato di recente), questo copre l'INTERO progetto se ognuno ha rivisto ogni file assegnato singolarmente; (2) integrazione orchestrata - bug raccolti, corret autonomo); (3) verifica dal vivo. Dettaglio completo di ogni bug in docs/Debug-Marathon-State.m Sicurezza (il piu' grave): Write-M365OpsWriteLog.ps1 scriveva il corpo JSON completo di una sc delle scritture (Logs\writes-*.log , mai cancellato automaticamente) - lo stesso identico buco già Protect-M365OpsSecretText.ps1 (nuova, condivisa da entrambe le destinazioni) - verificato dal v Bug sistemico: nessuna funzione Get-* Intune/Graph di questo progetto seguiva @odata.nextLink prima pagina sparivano in silenzio. Corretto in UN solo posto condiviso, Invoke-M365OpsGraphRequ funzioni chiamanti - verificato dal vivo (nessuna regressione sul caso comune a una pagina sola). Rischio di doppia esecuzione: Invoke-M365OpsWriteWithIsolationRecovery.ps1 (retry autom prima di fallire solo localmente - un retry alla cieca rischiava una scrittura duplicata (team/mailbox/gru rendendo il rischio VISIBILE: nuovo parametro -RecoveredViaIsolation , l'utente riceve ora un avv Corruzione di configurazione silenziosa: Set-/Remove-M365OpsMcpServer.ps1 riscrivevano l'IN' connettore MCP su QUALUNQUE tenant cancellava silenziosamente il login SharePoint interattivo con Worker isolati mai fermati: "Disconnetti tutto" non chiamava Disconnect-M365OpsAllIsolatedW con cmdlet proxati che avrebbero continuato a colpire il tenant "scollegato". Corretto (stessa simmetr Altri ~30 bug corretti (dettaglio completo in Debug-Marathon-State.md): 2 endpoint Graph Intune c Microsoft Learn), corpo updateDefinitionValues con un livello di annidamento in piu' del dovuto senza il wrapper di retry-su-conflitto già presente sui fratelli, 1x funzione Compliance che inghiottiva « tenant+suffisso casuale al nome), 7x voci del catalogo comandi locale con lo stesso schema di bug già (stato "sporco" su 4 elementi dopo un cambio tenant, 2x guard mancante contro il collasso di un array Verificato dal vivo dopo tutte le correzioni: 401 file .ps1 sintatticamente puliti, JS validato, tag H1 confermato senza regressioni, box "sporco" dopo cambio tenant confermato ripulito nel browser reale

16. Roadmap ed estensioni future

- **futuro** Ricerca sul registro di controllo unificato Microsoft Purview ([Search-UnifiedAuditLog](#) — chi ha fatto cosa, quando, su Exchange/SharePoint/Teams). Cmdlet reale e verificata, richiede licenza/ruolo Audit Log dedicato — non ancora costruita, un vero audit log oggi si copre solo parzialmente con i report di sezione 11 (litigation hold, inoltri, regole sospette)
- **futuro** Collegare all'AI i server MCP aggiuntivi oltre a Lokka, oggi solo salvati in configurazione
- **futuro** Ricerca automatica icone app da winget-pkgs per [New-M365OpsWin32App](#)
- **futuro** Valutare Azure SRE Agent come alleato — riguarda però l'affidabilità dell'infrastruttura Azure, non l'amministrazione M365, quindi resta fuori scope diretto
- **futuro** Server multi-threaded (oggi single-threaded per design semplice) — il login Exchange delegato non blocca più (sezione 10.6), ma resta l'unico modello possibile finché il server processa una richiesta alla volta. Finché resta così, un riavvio esterno "brutale" (kill del processo, non il pulsante/endpoint di riavvio) può ancora interrompere una richiesta bloccante in corso (sezione 12.2)
- **futuro** Storico conversazione più lungo/configurabile (oggi fisso a 8 scambi per tenant — sezione 23.2) o riassunto automatico dei turni più vecchi invece di scartarli
- **futuro** Task periodici autonomi (es. report settimanale via email senza tenere aperto M365Ops) tramite le "routine" di Azure AI Foundry Agent Service (trigger cron nativi lato Azure, GA da luglio 2026) - valutato il 17/08/2026: richiede un secondo sistema separato (Agent con propri strumenti verso Graph/Exchange, autenticazione Entra ID dedicata) e andrebbe usato SOLO per letture/report, mai per scritture non supervisionate (il gate di conferma umana vive nel processo locale di M365Ops, non si può delegare a un Agent schedulato). Deliberatamente NON si sposta la chat interattiva su un Agent: il ciclo Thread+Run è più complesso del semplice request/response usato oggi, e il catalogo strumenti/script già si ricostruisce ad ogni messaggio - non c'è nulla da guadagnare spostandolo su una configurazione Agent statica da tenere comunque aggiornata a mano.

Questo documento è pensato per essere aggiornato insieme al codice: quando una riga della tabella in sezione 15 cambia stato, aggiorna anche questa pagina e rigenera il PDF (vedi il commento in testa al file HTML sorgente).

17. Script personalizzati (Scripts\Custom)

Oltre alle cmdlet del modulo core (Public\) e ai 50+ strumenti Exchange (sezione 11), è possibile aggiungere script "home made" per casi d'uso specifici di un tenant/cliente — es. estrazione permessi OneDrive/SharePoint, un report ad hoc — e renderli usabili dall'AI in chat senza toccare nessun file del modulo.

17.1 Come diventano usabili

- 1. Copia Scripts\Custom_TEMPLATE.ps1 , rinominalo come la funzione che definisce (es. Get-M3650psOneDriveSharingReport.ps1 per una funzione con lo stesso nome).
- 2. Scrivi la funzione rispettando la convenzione sotto.
- 3. Riavvia il server (tab Manutenzione) — i file si caricano all'avvio del modulo.

Da quel momento lo script compare nel tab Manutenzione (elenco script rilevati, validi o no con motivo) e l'AI lo vede tra gli strumenti disponibili al messaggio successivo — nessuna registrazione manuale, nessuna modifica al manifest del modulo.

17.2 Convenzione obbligatoria

Requisito	Perché
Un file = una funzione, nome file = nome funzione	Stessa convenzione di Public\ — scoperta automatica senza registrazione manuale.
Blocco di help PowerShell (<# .SYNOPSIS ... #>)	È l'UNICA descrizione che l'AI riceve per decidere se/come usare lo script.
.NOTES con Mode: ReadOnly oppure Mode: Write	Obbligatorio. ReadOnly = eseguibile subito dall'AI; Write = solo proponibile, mai eseguito senza conferma umana esplicita — stesso principio non negoziabile di ogni altra scrittura. Senza questo tag lo script viene ignorato, mai esposto all'AI.
Nessun input interattivo (Read-Host , popup)	Deve essere una funzione pura: parametri in ingresso, oggetti in uscita.
Usa le funzioni di accesso dati già esistenti del modulo	Mai gestire token/credenziali proprie — usa Invoke-M3650psGraphRequest per Graph, Connect-M3650psExchange + cmdlet native per Exchange. Così lo script funziona automaticamente sia sui tenant AppOnly sia Delegated (sezione 10), senza mai assumere quale modalità è attiva.

17.3 Checklist per un buon .SYNOPSIS

L'AI vede *solo* questa riga per decidere se e come usare lo script — deve rispondere a:

- 1. **Quale dato specifico restituisce o azione specifica compie** — non "esegue una query su SharePoint", ma "elenca i link di condivisione pubblici attivi su OneDrive, con proprietario e data".
- 2. **Permessi/scope non standard richiesti**, se presenti — es. Sites.Read.All non è tra i permessi Graph di default (sezione 4.2): va detto qui, prima che l'operatore scopra un errore di permesso a metà.
- 3. **Per gli script Mode: Write**, quali effetti reali ha — l'AI usa questo testo, quasi alla lettera, per spiegare la proposta in chat prima della conferma.
- 4. **Forma dei dati restituiti**, se non ovvia.

17.4 Se uno script fallisce — diagnosi e correzione via AI, mai un crash

Un errore in uno script personalizzato non blocca il resto dell'app. Sia in lettura (custom_script_query) sia in scrittura confermata (CustomWrite), il fallimento passa dallo stesso motore di diagnosi usato per le altre scritture del modulo (Invoke-M3650psErrorTriage , sezione 14): l'errore, il contesto e il codice sorgente vengono analizzati, classificati (transitorio / serve una decisione tua / possibile bug) e, se c'è una correzione precisa da proporre, questa segue lo stesso flusso proponi → conferma → applica di ogni altra scrittura — mai una riscrittura automatica senza conferma esplicita.

Le correzioni proposte devono restare compatibili con **entrambe** le modalità di autenticazione: l'AI viene istruita esplicitamente a non introdurre gestione di token/credenziali proprie nel fix, per lo stesso motivo del punto 17.2 sull'accesso ai dati. Testato dal vivo: uno script di esempio (report link di condivisione OneDrive) ha incontrato un 404 reale su un utente senza OneDrive provisionato, ed è stato classificato correttamente come "serve una decisione tua" (verificare licenza/provisioning) — non un bug nello script.

17.5 File non caricati

Qualunque file il cui nome inizia con _ (come _TEMPLATE.ps1) viene ignorato dallo scanner — utile per esempi o bozze non ancora pronte.

17.6 L'AI può proporre la creazione di uno script nuovo

Dal 17/08/2026, quando un compito è chiaramente qualcosa che tornerà utile di nuovo (un report ricorrente, un'estrazione specifica di questo tenant) e nessuno strumento esistente lo copre, l'AI può scrivere da sola il codice completo di un nuovo script e proporlo — strumento propose_new_custom_script . Non è la prima risorsa: il system prompt le impone di provare prima graph_api_call / exo_query (con lookup_ms_docs se serve un parametro nativo non standard), e di non usarlo mai come scorciatoia per una singola domanda occasionale.

Non viene mai salvato senza conferma esplicita, e la conferma mostra il codice per intero — non un riassunto — esattamente come richiesto: creare uno script nuovo aggiunge una capacità permanente all'app, non è "solo" una scrittura su un dato del tenant, quindi la conferma lo dice esplicitamente in cima al messaggio, prima ancora del codice. Per gli script proposti in modalità Write , ogni loro esecuzione futura richiederà comunque una conferma separata — la conferma sulla creazione non "sblocca" scritture non supervisionate, si limita a far esistere lo strumento.

Validazioni automatiche prima ancora di mostrare la proposta (se una fallisce, l'AI riceve il motivo esatto e può correggere nello stesso turno, invece di proporre qualcosa che verrebbe comunque scartato dopo): il codice deve essere sintatticamente valido PowerShell (analizzato con il parser reale, non un controllo superficiale), deve definire esattamente una funzione il cui

nome coincide con il nome file proposto, deve avere un blocco `.SYNOPSIS`, e il tag `Mode`: nel codice deve corrispondere esattamente al parametro dichiarato. Il nome deve seguire la stessa convenzione Verbo-M365OpsNome di ogni cmdlet del modulo, e non viene mai sovrascritto silenziosamente uno script già esistente con lo stesso nome.

Il codice viene inoltre scansionato per una lista di comandi potenzialmente distruttivi (`Remove-Item`, `Invoke-Expression`, `Stop-Computer`, ecc.) — se presenti, non bloccano la proposta ma vengono segnalati con un'ATTENZIONE ben visibile in cima alla conferma, cosa da leggere con più attenzione prima di dire "sì" (in PowerShell puro non esiste un sandboxing reale, quindi questo resta un avviso mirato, non un blocco tecnico).

Alla conferma: il file viene scritto in `Scripts\Custom\` e il server si riavvia da solo (stessa infrastruttura sicura del pulsante "Riavvia"/ `POST /api/restart`, sezione 12.2 — agisce solo dopo aver già inviato la conferma, non può mai interrompere altro) per caricarlo. Da quel momento è uno strumento vero, scoperto automaticamente come ogni altro script in questa cartella (sezione 17.1).

Perché la revisione umana del codice resta comunque indispensabile. Testato dal vivo: la prima proposta reale generata dall'AI chiamava `Invoke-M365OpsGraphRequest -Uri $uri` — il parametro reale si chiama `-Path`, non `-Uri`. Un errore che il parser PowerShell non può rilevare (sintatticamente valido, fallisce solo in esecuzione) ma che un umano che legge il codice nota subito. Proposta rifiutata prima del salvataggio, e la descrizione dello strumento è stata rafforzata con la firma esatta e verificata di `Invoke-M365OpsGraphRequest / Connect-M365OpsExchange` per ridurre (non eliminare) questa classe di errore — motivo in più per cui la conferma mostra sempre il codice completo, mai un riassunto.

17.7 Sicurezza email — antispam, threat policy, allow/block list, quarantena, hunting

Aggiunto il 17/08/2026 su richiesta esplicita: gestione della sicurezza email (Exchange Online Protection / Defender for Office 365), oltre al message trace già esistente.

Cosa	Come
Criteri anti-spam, anti-phishing, Safe Links/Safe Attachments	lettura via <code>exo_query</code> (<code>Get-M365OpsAntiSpamPolicies</code> , <code>Get-M365OpsAntiPhishPolicies</code> , <code>Get-M365OpsThreatPolicies</code>) — nessun permesso Graph nuovo, usano la stessa connessione certificato-based di Exchange Online (sezione 5)
Tenant Allow/Block List (mittenti, URL, hash file, IP)	lettura con <code>Get-M365OpsTenantAllowBlockList</code> , scrittura proposta (conferma sempre richiesta) con <code>New-/Remove-M365OpsTenantAllowBlockListEntry</code>
Quarantena, incluso il rilascio	lettura con <code>Get-M365OpsQuarantineMessages</code> (default ultimi 7 giorni), rilascio proposto con <code>Release-M365OpsQuarantineMessage</code> (a tutti i destinatari originali, opzionalmente con il mittente aggiunto alla Allow List), eliminazione definitiva proposta con <code>Remove-M365OpsQuarantineMessage</code> — entrambe le scritture passano dalla stessa conferma esplicita di ogni altra azione del modulo
Message trace oltre i 10 giorni (es. "ultimi 30 giorni")	<code>Get-M365OpsMessageTrace</code> incatena da sola più query da 10 giorni (limite del servizio) e unisce i risultati — non serve più spezzare la richiesta a mano. Il risultato si ferma comunque a 1000 righe con un avviso esplicito se superate: una query lunga SENZA filtro mittente/destinatario su un tenant con molto traffico può generare un volume enorme (bug reale osservato: 3000 righe, ~330mila token, ha superato da sola il limite di contesto del modello e fatto fallire l'intera risposta) — specificare sempre a chi si riferisce la richiesta riduce il volume di un ordine di grandezza.
"Explorer"/Threat Hunting di security.microsoft.com	non è un'unica area: alert e incidenti già rilevati da Defender si leggono con <code>graph_api_call</code> su <code>/security/alerts_v2</code> e <code>/security/incidents</code> (permessi <code>SecurityAlert.Read.All</code> / <code>SecurityIncident.Read.All</code> , sezione 4.2); la ricerca libera con query KQL nei dati grezzi (l'uso più vicino a "Explorer" in senso stretto) usa il nuovo strumento <code>security_hunting_query</code> — richiede <code>ThreatHunting.Read.All</code> E la licenza Microsoft Defender for Endpoint Plan 2: senza quella licenza Graph risponde 403 anche a permesso concesso, e l'AI è istruita a spiegarlo come limite di licenza invece di ritentare query diverse sperando che funzionino.

Tutte le nuove cmdlet di lettura/scrittura Exchange qui sopra usano la stessa `Connect-M365OpsExchange` già esistente (certificato per AppOnly, sezione 5) — verificato dal vivo il 17/08/2026 che il modulo Exchange Online v3 espone anche i cmdlet un tempo esclusivi della Security & Compliance PowerShell (quarantena, Tenant Allow/Block List) sotto la stessa connessione, senza bisogno di una sessione separata.

17.8 Invio report via email dalla conversazione libera

`Send-M365OpsReportEmail` esisteva già (invio via Graph `/sendMail`, permesso `Mail.Send`) ma era raggiungibile solo dal catalogo deterministico con una frase fissa ("invalio per email a..."). Una richiesta composita in linguaggio libero (es. "aggiorna il report e mandalo a X") arrivava alla conversazione con l'IA, che non aveva alcuno strumento per l'invio e lo segnalava correttamente invece di inventarsi una capacità inesistente — ma il gap era reale. Aggiunto il 17/08/2026 lo strumento `propose_send_report_email`: propone l'invio dell'ultimo report generato in sessione come allegato, con conferma esplicita sempre richiesta (stesso meccanismo di ogni altra scrittura — inviare un'email è un'azione visibile al destinatario). L'IA scrive anche un testo su misura nel corpo del messaggio (breve overview del contenuto reale del report, non un messaggio generico), con la stessa regola di `generate_report`: solo dati davvero recuperati in conversazione, mai inventati.

Bug reale corretto lo stesso giorno: il percorso del file allegato veniva riletto da `$script:LastReportPath` dentro `Server.ps1`, ma quella variabile — quando scritta da `Invoke-M365OpsAgentTools` — vive nello scope del modulo, non in quello di `Server.ps1` (che non fa parte del modulo, lo consuma soltanto): il valore letto da `Server.ps1` era quindi sempre `$null`, con un crash immediato ("Cannot bind argument to parameter 'Path'"). Stessa classe di bug di scope già vista altre volte in questo progetto — risolto passando il percorso esplicitamente nell'oggetto restituito invece di affidarsi a una variabile condivisa tra scope diversi. Anche il vecchio percorso deterministico ("invalio per email a...") è stato allineato: prima inviava l'email immediatamente, senza alcuna conferma — unica scrittura dell'app con questo comportamento — ora passa dalla stessa conferma di tutte le altre.

17.9 SharePoint e OneDrive

Aggiunto il 17/08/2026 su richiesta esplicita ("tutto quello che è utile"): elenco siti, condivisione esterna, storage, permessi, utilizzo OneDrive e account orfani. Architettura diversa dal resto del modulo — vale la pena capirla prima di fidarsi ciecamente dei dati.

Cmdlet	Cosa dà
Get-M3650psSharePointSites	ogni sito: storage usato/quota, <code>SharingCapability</code> (stato condivisione esterna), template, ultima modifica — un'unica chiamata copre sia il classico report "condivisione esterna" sia lo storage per sito
Get-M3650psSharePointSitePermissions	amministratori raccolta siti + membri Proprietari/Membri/Visitori di UN sito (richiede l'URL, mai indovinato — va preso dall'elenco siti)
Get-M3650psOneDriveUsageReport	ogni OneDrive personale: proprietario, storage usato/quota, ultima attività
Get-M3650psInactiveOneDriveAccounts	OneDrive il cui proprietario è disabilitato o eliminato da Entra ID — dati orfani, rischio compliance/storage, già filtrato

Connessione e permesso diversi dal resto del modulo. Exchange, Graph e le nuove cmdlet di sicurezza email usano tutte lo stesso certificato con permessi Microsoft Graph o nativi Exchange. SharePoint invece passa da `PnP.PowerShell` (`Connect-M3650psSharePoint`), che riusa lo STESSO certificato ma richiede un consenso SEPARATO sotto l'API "SharePoint" — vedi sezione 4.3. Finché quel permesso non è concesso, ogni cmdlet qui sopra fallisce con un errore "Unauthorized" chiaro (non un bug, non va ritentato).

Verificato dal vivo il 17/08/2026, dopo la concessione del permesso di sezione 4.3: tutte e 4 le cmdlet restituiscono dati reali e corretti — 44 siti con `SharingCapability` popolata e variata (Disabled/ExternalUserSharingOnly/ ExternalUserAndGuestSharing), 13 OneDrive personali, permessi reali su un sito di test, e `Get-M3650psInactiveOneDriveAccounts` ha correttamente individuato 2 OneDrive con proprietario eliminato da Entra ID (non un dato di prova - un caso reale trovato al primo giro). Il campo `Owner` è confermato un UPN pulito (es. `nome@dominio.it`), non un formato claims-encoded - la ricerca Entra ID di `Get-M3650psInactiveOneDriveAccounts` funziona quindi come previsto senza fallback necessari.

Bug reale corretto il 17/08/2026 su un secondo tenant (Fabrikam-Prod, Delegato): l'URL del centro di amministrazione veniva derivato assumendo che il TenantId del profilo fosse sempre nel formato `xxxxx.onmicrosoft.com` (vero per contoso-test) - ma un profilo può avere il TenantId salvato come GUID puro, producendo un hostname senza senso (es. `https://a1b2c3d4-...-admin.sharepoint.com` , non risolvibile via DNS) e quindi un errore ancora prima del permesso/autenticazione. Corretto risolvendo il dominio iniziale verificato reale via Microsoft Graph `/organization` quando il TenantId non è già nel formato dominio - funziona sia in AppOnly sia in Delegato, risultato cache-ato per non ripetere la chiamata ad ogni uso. Verificato con un test mirato (non dal vivo su Fabrikam-Prod, che è Delegato e richiede un login interattivo reale): la derivazione produce l'URL corretto sia nel caso GUID sia nel caso dominio, senza chiamate di rete superflue in quest'ultimo.

17.9.1 Scrittura SharePoint

Aggiunta il 18/08/2026, su richiesta esplicita dopo un batch di test con prompt reali che segnalava l'assenza di scrittura (creazione sito, quote, sharing) come gap più visibile. Stesso meccanismo di proposta/conferma di ogni altra scrittura del modulo (mai eseguita senza conferma esplicita in chat) — nessun percorso separato o meno sorvegliato.

Cmdlet	Cosa fa
New-M3650psSharePointSite	crea un Team site (con gruppo Microsoft 365 associato) o un Communication site
Set-M3650psSharePointSiteMember	aggiunge/rimuove un utente da Owners/Members/Visitors di un sito
Set-M3650psSharePointPermissionInheritance	interrompe (permessi unici) o ripristina (torna a ereditare) l'ereditarietà permessi di un sito
Set-M3650psSharePointSiteSharing	imposta la <code>SharingCapability</code> di UN sito (stesso valore letto da <code>Get-M3650psSharePointSites</code>)
Set-M3650psSharePointSiteQuota	imposta la quota di storage di un sito, in GB

Non ancora disponibile: gestione Site Collection/colonne/template (fuori perimetro di questo giro, era una richiesta a parte del catalogo di test) e federazione cross-tenant. Se richiesto, il tool lo dichiara esplicitamente invece di tentare scorciatoie via Graph.

Verificato dal vivo il 18/08/2026 sul tenant contoso-test tramite il flusso reale di chat (proposta → conferma → esecuzione), non chiamando le funzioni a mano: creato un Communication site di test (`New-M3650psSharePointSite`), confermato e poi ritrovato correttamente in un elenco siti riletto in un secondo turno separato (owner e `SharingCapability` corretti).

17.10 Come è governata la connessione (App-only vs Delegata)

La connessione SharePoint (`Connect-M3650psSharePoint`) ha una casella dedicata nel tab **Tenant** (sezione "Stato connessioni", non MCP/Connettori), con un pulsante "Connetti / Test connessione SharePoint" e uno stato in tempo reale.

Modalità	Comportamento
App-only	Connessione silenziosa col certificato già configurato per Exchange (nessun secret/certificato nuovo) - il pulsante serve solo per un test esplicito o per riconnettersi a un URL diverso, non per un login vero e proprio. Pienamente verificato con dati reali (sezione 17.9).
Delegata	Richiede un login interattivo una tantum, fatto da un terminale normale (non dal pulsante della GUI) - vedi il riquadro sotto per il perché e il comando esatto. Dopo quel login, il pulsante della GUI funziona silenziosamente riusando la cache locale, senza bloccare mai il server.

Storia reale di questa sezione: tre bug diversi in sequenza sullo stesso sintomo, lasciata per intero perché istruttiva su come NON bloccarsi su un problema che sembra sempre "lo stesso" ma non lo è.

1. **Client OAuth indovinato.** Un primo tentativo con un flusso a codice dispositivo *custom* (per non bloccare il server) usava un `client_id` "indovinato" a memoria (il presunto "PnP Management Shell", `31359c7f-bd7e-475c-86db-fdb8c937548e`) - fallito con `AADSTS700016 "Application ... was not found in the directory"` : quel client non è mai stato consentito nel tenant, e il device code flow non può creare un primo consenso da solo.
2. **Sostituito con `-Interactive`** (lascia scegliere a PnP il proprio client corretto) - fallito con `System.NotSupportedException: Specified method is not supported.`, senza dettaglio utile nell'eccezione. Ipotesi ragionevole ma SBAGLIATA: apartment threading (il processo server gira in MTA, i controlli COM per finestre embedded richiedono STA) - aggiunto `-STA` all'avvio del processo server, verificato con `Get-CimInstance Win32_Process` che il flag fosse davvero presente nel processo in esecuzione, e l'errore è rimasto **identico**: ipotesi scartata con prove, non per sospetto.
3. **Confronto con Intune (funzionante):** Intune usa anch'esso `-Interactive` per i tenant Delegati, ma con successo - il modulo `IntuneWin32App` implementa il proprio flusso PKCE con redirect su localhost scritto da zero (nessun controllo browser embedded), mentre l'implementazione interna di `-Interactive` in `PnP.PowerShell` è evidentemente meno adatta a un processo headless. Confermato con `Start-Job` che `Connect-PnPOnline -DeviceLogin` nativo (nessun client indovinato) NON lancia l'eccezione e resta correttamente in attesa - ma un processo server headless non ha una console interattiva a cui è agganciato, quindi il codice stampato da `-DeviceLogin` non è recuperabile in modo affidabile da lì.

Soluzione: `-PersistLogin` . Questo parametro di `Connect-PnPOnline` salva il token in una cache locale su disco, riutilizzabile tra sessioni PowerShell diverse e dopo un riavvio del PC - va fornito una sola volta, dopo di che ogni nuova connessione allo stesso tenant lo riusa automaticamente, **anche da un processo headless** come il server di questa app (perché a quel punto non deve più mostrare nessuna UI, si limita a leggere il token già presente). Il login iniziale va fatto UNA TANTUM da un terminale PowerShell 7 normale (non tramite il pulsante della GUI, dove `-Interactive` continua a fallire per il motivo del punto 2 sopra):

```
Import-Module PnP.PowerShell
Connect-PnPOnline -Url "https://<prefisso-tenant>-admin.sharepoint.com" -Interactive -PersistLogin
```

Dopo aver completato il login nel browser che si apre, il pulsante "Connetti / Test connessione SharePoint" nella GUI dovrebbe funzionare silenziosamente. Se in futuro cambi i permessi dell'App Registration usata per il login, la cache va invalidata esplicitamente con `Disconnect-PnPOnline -ClearPersistedLogin` prima di un nuovo login (il token in cache non si aggiorna da solo).

17.11 Microsoft Teams

Aggiunto il 17/08/2026 subito dopo SharePoint, stessa richiesta ("tutto quello che è utile"). Architettura ibrida: una parte funziona subito col certificato esistente, un'altra parte (le policy) richiede una TERZA API separata, ancora diversa da Graph e da SharePoint.

Cmdlet	Cosa dà	Permesso
Get-M3650psTeamsList	ogni Team: visibilità, archiviato, criteri di collaborazione (chi può creare canali/aggiungere app/menzionare tutti)	nessuno in più - stesso certificato di Exchange
Get-M3650psTeamsChannels	canali di UN team (richiede GroupId, mai indovinato - da Get-M3650psTeamsList)	nessuno in più
Get-M3650psTeamsMembers	membri di UN team con ruolo owner/member/guest	nessuno in più
Get-M3650psTeamsPolicies	criteri di riunione/chiamata/messaggistica in un'unica lista (registrazione consentita, chiamate private, modifica/eliminazione messaggi, ecc.)	Skype and Teams Tenant Admin API - sezione 4.4
Get-M3650psTeamsExternalAccessConfig	federazione con organizzazioni esterne + cosa possono fare gli ospiti (chat/riunioni/chiamate) - report di sicurezza classico	Skype and Teams Tenant Admin API - sezione 4.4

Verificato dal vivo il 17/08/2026: Connect-M3650psTeams (modulo MicrosoftTeams, stesso certificato di Exchange) funziona SUBITO per le prime tre cmdlet - 14 Team reali, canali e membri (incl. un membro guest reale) confermati con dati veri. Le due cmdlet di policy invece falliscono oggi con "Access Denied. Provide different credential or request access." - il permesso di sezione 4.4 non è ancora stato concesso, i loro campi restano quindi non verificati (proprietà documentate e stabili delle cmdlet native, non indovinate, ma da riconfermare al primo test reale come già fatto per SharePoint).

Stessa governance connessione di Exchange/SharePoint: casella dedicata nel tab **Tenant** (sezione "Stato connessioni", non MCP/Connettori - vedi sezione 10.7) con pulsante "Connetti / Test connessione Teams" e stato in tempo reale.

BUG STRUTTURALE trovato dal vivo il 21/08/2026 (non un'ipotesi teorica - successo due volte all'utente): -UseDeviceAuthentication lasciava l'utente bloccato per sempre. Quel parametro stampa il codice dispositivo con Write-Host sulla CONSOLE DEL PROCESSO SERVER, non nella GUI - invisibile e irrecuperabile nell'uso normale dell'app (il launcher avvia il server con finestra nascosta). L'utente vedeva solo "login in corso" bloccato per sempre, nessun popup ne' codice da nessuna parte - comportamento CORRETTO per un device-code flow (che non apre mai un browser da solo), ma il codice stesso non arrivava mai all'utente. A differenza di Exchange/Graph (sezioni 6.5/10, che hanno un vero flusso non bloccante start/poll con codice mostrato in GUI), il modulo MicrosoftTeams non espone cmdlet separati per "inizia" e "completa" un device code - Connect-MicrosoftTeams -UseDeviceAuthentication e' un'unica chiamata atomica bloccante, non costruibile come start/poll senza intercettare la sua console (fragile).

Corretto RIMUOVENDO -UseDeviceAuthentication : senza, il modulo usa il proprio flusso interattivo di default (popup browser reale, come già visto funzionare per SharePoint via Connect-PnPOnline -Interactive). Il motivo per cui -UseDeviceAuthentication era stato scelto in origine (lasciare al modulo la scelta del proprio client_id interno corretto, per evitare un client_id indovinato a memoria - stesso bug reale già visto su SharePoint, AADSTS700016, sezione 17.10) resta valido: e' una proprietà del modulo stesso, non del parametro specifico - il modulo sceglie il client corretto in ENTRAMBI i flussi. Verificato dal vivo dopo il fix: popup browser apparso correttamente, login completato senza bloccare il server.

17.11.1 Scrittura Teams

Aggiunta il 18/08/2026, stesso motivo/richiesta di 17.9.1 (SharePoint). Stesso meccanismo di proposta/conferma di ogni altra scrittura del modulo.

Cmdlet	Cosa fa	Percorso API
New-M3650psTeam	crea un Team (e il gruppo Microsoft 365 associato)	Graph - stesso certificato di Exchange, nessun permesso in più
Set-M3650psTeam	modifica nome/descrizione/visibilità/archiviazione di un Team esistente	Graph
Remove-M3650psTeam	rimuove un Team (irreversibile oltre il cestino Entra ID a 30 giorni) - stessa conferma esplicita di ogni scrittura distruttiva del modulo, nessun percorso separato	Graph
Grant-M3650psTeamsPolicy	assegna (o resetta al default, omettendo il nome) una policy meeting/messaging/app setup/app permission ESISTENTE a un utente - non crea/modifica il contenuto della policy	Cs* legacy - stesso permesso "Skype and Teams Tenant Admin API" di Get-M3650psTeamsPolicies (sezione 4.4)

Verificato dal vivo il 18/08/2026 sul tenant contoso-test tramite il flusso reale di chat: creato un Team di test privato (New-M3650psTeam), confermato e ritrovato correttamente in un elenco riletto in un secondo turno separato.

Bug reale trovato e corretto durante il test: Grant-CsTeamsMeetingPolicy (e gli altri Grant-CsTeams*Policy) trattano -PolicyName come parametro OBBLIGATORIO - ometterlo del tutto per resettare al default falliva con "missing mandatory parameters: PolicyName" invece di funzionare. Va passato esplicitamente -PolicyName \$null (comportamento nativo dei cmdlet Microsoft, non un'assunzione ragionevole) - corretto in Grant-M3650psTeamsPolicy.ps1 .

Limite noto, non ancora risolto: dopo la correzione sopra, Grant-M3650psTeamsPolicy fallisce comunque su contoso-test con un errore di serializzazione interno del modulo MicrosoftTeams ("Dictionary...Keys must be strings", dentro Microsoft.TeamsCmdlets.PowerShell.Connect) - stesso percorso legacy Cs* di Get-M3650psTeamsPolicies , che oggi fallisce con "Access Denied" per lo stesso permesso mancante (sezione 4.4). Il sintomo è diverso (un bug di serializzazione invece di un messaggio di permesso pulito) ma la causa più probabile è la stessa: finché il permesso "Skype and Teams Tenant Admin API" non è concesso, non è possibile confermare se il codice del wrapper è corretto - da riverificare non appena il permesso è disponibile, prima di fidarsi ciecamente di questa cmdlet in produzione.

17.13 Knowledge Base per tenant (tab "Documentazione")

Aggiunta il 18/08/2026: documentazione specifica di UN cliente (peculiarità di infrastruttura, procedure di troubleshooting, note operative) caricata dall'operatore e consultata dall'AI senza doverla ripetere ogni volta in chat. Formati accettati: `.txt`, `.md`, `.docx`, `.xlsx` / `.xls`, `.pdf` (tramite il modulo `PdfLexer`, installato automaticamente al primo uso).

Come funziona

All'upload il testo viene estratto e l'AI lo cataloga con **una sola chiamata** (titolo, argomenti, riassunto) - il catalogo (leggero) resta sempre nel prompt di sistema, nessun costo aggiuntivo per le domande successive. Il testo **completo** di un documento specifico viene recuperato solo quando l'AI lo ritiene rilevante, tramite lo strumento dedicato `kb_query` - mai tutto il contenuto di tutti i documenti ad ogni domanda.

Isolamento tra tenant, verificato dal vivo (non solo per progetto): caricato un documento di prova su un tenant con un "codice segreto" fittizio, l'AI lo ha recuperato correttamente SOLO su quel tenant; passando a un secondo tenant il catalogo risultava vuoto e l'AI ha rifiutato esplicitamente di inventare una risposta invece di confondere i due clienti. `kb_query` non accetta né espone un parametro "tenant": legge sempre e solo il tenant realmente attivo in quel momento - una garanzia strutturale, non solo una convenzione rispettata dal codice.

Consigli pratici

- **Un documento per argomento** funziona meglio di un unico file enorme - il riassunto nel catalogo aiuta l'AI a capire QUANDO consultare il testo completo, e riassunti più mirati sono più utili di uno generico su tutto.
- **.doc legacy (binario, non .docx) non è supportato** - salva da Word come `.docx` prima di caricarlo, un tentativo con `.doc` viene rifiutato con un messaggio chiaro invece di produrre testo corrotto.
- **PDF scansionati come immagine** (senza testo selezionabile, es. un documento cartaceo fotografato) non vengono estratti - il tool lo segnala esplicitamente invece di catalogare un documento vuoto in silenzio.
- **Ricaricare un file con lo stesso nome sostituisce** la voce precedente - comodo per aggiornare una procedura senza dover prima rimuovere la versione vecchia.
- **Meglio per contenuti procedurali/operativi** (procedure, contatti, note tecniche specifiche del cliente) che per dati che cambiano spesso - quelli è meglio chiederli in tempo reale al tenant (es. "quanti utenti ha"), la Knowledge Base è pensata per "cose che un admin dovrebbe sapere a memoria su questo cliente", non per stato live.
- Cambiando tenant nella GUI, l'elenco documenti visibile nel tab cambia da solo - non serve alcuna azione manuale, ogni cliente ha la propria Knowledge Base, mai mescolate.

Knowledge Base GLOBALE (aggiunta il 20/08/2026)

Estensione richiesta esplicitamente dall'utente: oltre alla Knowledge Base per-tenant sopra, esiste un secondo bucket **riservato** (nome tenant interno `_global`, prefisso `_` scelto apposta - i profili tenant reali in `Config\tenants.json` non iniziano mai per convenzione con underscore, quindi nessuna collisione possibile) per documentazione che riguarda **l'app stessa**, non un cliente specifico - a partire dalla guida di configurazione che stai leggendo ora, caricata come `Guida-Configurazione.pdf`. Questo bucket è visibile **sempre**, indipendentemente da quale tenant è attivo - a differenza della Knowledge Base per-tenant sopra, che resta isolata per cliente.

Si aggiorna da sola (aggiunto il 21/08/2026): ad ogni avvio del server, `Gui\Server.ps1` confronta l'hash SHA256 di `docs\Guida-Configurazione.pdf` con l'ultimo caricato (`Config\global-kb-guide-hash.txt`) e, se è cambiato, lo ricarica automaticamente nella Knowledge Base globale - nessun passo manuale da ricordare dopo aver modificato la guida. Se il PDF non è cambiato, il riavvio non fa nulla (nessuna chiamata AI inutile). Verificato dal vivo: un primo riavvio dopo una modifica della guida ha risincronizzato correttamente (log: "Guida di configurazione cambiata..."), un secondo riavvio immediato, a guida invariata, non ha risincronizzato.

Con questo, si può chiedere in chat qualcosa come "come configuro l'app per Exchange Online?" o "quali permessi Graph servono per Intune?" a prescindere dal tenant attivo, e l'AI consulta la guida vera tramite `kb_query` invece di rispondere a memoria - verificato dal vivo il 20/08/2026 (log: chiamata reale a `kb_query`, risposta con dettagli specifici della guida come il permesso `Exchange.ManageAsApp` e l'elenco completo dei permessi Graph richiesti).

L'isolamento tra tenant resta intatto: il bucket globale è un pool **aggiuntivo e separato**, mai un modo di raggiungere il KB di un altro cliente reale - `kb_query` prova prima il tenant attivo, poi il bucket globale come fallback, mai il contrario.

Per caricare o aggiornare la guida nel bucket globale (operazione manuale, non da GUI per ora):

```
Add-M365OpsKnowledgeDocument -TenantName '_global' -FilePath 'docs\Guida-Configurazione.pdf' `
-OriginalFileName 'Guida-Configurazione.pdf' -Provider AzureOpenAI
```

Bug reale trovato caricando la guida per la prima volta (52 pagine): il modulo `PdfLexer` restituisce un array di stringhe, una per pagina, non un'unica stringa - il codice originale lo passava così com'era, quindi `.Length` contava le PAGINE (52) invece dei caratteri, scartando di fatto tutto il testo vero e mandando in errore la catalogazione AI (un array non converte a un parametro tipizzato stringa). Corretto unendo esplicitamente le pagine in `Get-M365OpsExtractedFileText.ps1` prima di restituire il testo - stessa famiglia di bug "array trattato come scalare" già vista più volte in questo progetto (sezione 20.5).

Diagnosi errori che consulta anche la Knowledge Base (aggiunta il 20/08/2026)

`Invoke-M365OpsErrorTriage` (diagnosi di un'azione fallita) e `Invoke-M365OpsWriteRecovery` (proposta di correzione immediata per una scrittura Graph fallita) ora consultano, oltre a Microsoft Learn, anche la Knowledge Base - sia quella globale sia quella del tenant attivo, combinate. Match per parola chiave semplice tra il testo dell'errore/contesto e gli argomenti/titolo dei documenti in catalogo (nessuna chiamata AI aggiuntiva per la ricerca: i cataloghi restano piccoli). Se un problema è specifico di configurazione/setup di QUESTA app (es. il limite RBAC app-only della sezione 6.4 qui sopra), la documentazione interna del progetto è una fonte più precisa e mirata di una ricerca generica su Microsoft Learn.

Verificato dal vivo il 20/08/2026: passato a `Invoke-M365OpsErrorTriage` lo stesso errore reale "term not recognized" di `New-AcceptedDomain` della sezione 6.4 - la diagnosi ha citato correttamente la finestra di propagazione RBAC documentata nella guida invece di rispondere alla cieca, confermando (anche con un controllo indipendente del solo meccanismo di match per parola chiave, senza passare dall'AI) che il documento giusto viene trovato e usato.

17.14 Aggiornamenti (tab "Manutenzione")

Aggiunta il 18/08/2026, dopo la pubblicazione del repository su GitHub: la webapp puo' controllare, scaricare e applicare da sola la versione piu' recente del codice, senza bisogno di `git` installato ne' di comandi manuali - tutto da un pulsante nel tab Manutenzione.

Come si decide quando un aggiornamento diventa disponibile

Nessun commit pubblicato su GitHub diventa mai un aggiornamento da solo. Un aggiornamento esiste solo quando viene pubblicata esplicitamente una **Release** GitHub (un tag di versione, es. `v0.2.0`, con note di rilascio) - un atto deliberato, mai automatico. Finche' non si pubblica una Release, ogni installazione resta ferma alla propria versione, indipendentemente da quanti commit nel frattempo finiscono su `main`.

Canale Stable vs Testing

La distinzione usa un meccanismo **nativo di GitHub**, non qualcosa di inventato per questo progetto: al momento di pubblicare una Release, GitHub offre la casella "Set as a pre-release".

- **Stable** (default): propone solo Release NON marcate pre-release. La webapp chiama `GET /repos/<owner>/<repo>/releases/latest` - un endpoint che GitHub stesso filtra gia' per restituire solo l'ultima Release definitiva, nessuna logica di selezione scritta qui.
- **Testing**: propone SEMPRE la Release piu' recente in assoluto, marcata pre-release o no - utile solo per chi vuole provare novita' in anteprima, sconsigliato su un tenant di produzione.

Il canale e' una preferenza **locale a questa installazione** (`Config\update-channel.txt`, escluso da git) - cambiarlo non ha alcun effetto sul repository, solo su quali Release questa copia dell'app propone.

Sicurezza dei dati del tenant, per struttura non per promessa: quando si applica un aggiornamento, il codice scarica l'archivio della Release da GitHub e sovrascrive i file dell'installazione con quelli scaricati. Quell'archivio e' **esattamente l'albero tracciato da git** a quel tag - `Config\`, `Logs\`, `Uploads\`, `Reports\`, `Testing\` non ci sono MAI dentro (esclusi da `.gitignore`, mai committati), quindi l'aggiornamento non puo' fisicamente toccare tenant, chat, Knowledge Base, secret o report esistenti - non perche' vengono esclusi "a mano", ma perche' semplicemente non esistono nel materiale scaricato. Stessa garanzia strutturale gia' usata per l'isolamento della Knowledge Base (sezione 17.13).

Limite noto

Un file rimosso in una versione piu' recente non viene ripulito automaticamente da un aggiornamento - vengono applicate solo aggiunte e modifiche. Per una pulizia completa (raro che serva) resta disponibile il percorso garantito al 100%: cancellare la cartella e riclonare il repository.

L'azione di aggiornamento e' **SOLO da GUI** (pulsante "Aggiorna ora"), mai esposta come strumento all'AI - modifica l'app stessa e richiede un riavvio, non qualcosa che una richiesta in chat deve poter innescare, stessa logica gia' applicata alla cancellazione di un documento dalla Knowledge Base.

17.15 Copertura Exchange Online: cosa c'e' e cosa manca deliberatamente

Il 18/08/2026 e' stato fatto un censimento reale (non a memoria) dei cmdlet Exchange Online ufficiali - scaricati dal repository sorgente della documentazione Microsoft su GitHub ([MicrosoftDocs/office-docs-powershell](#)), **1.438 cmdlet totali**. M365Ops non punta alla parita' 1:1 con l'intero prodotto Exchange (che include amministrazione on-premise/ ibrida, RBAC, eDiscovery, Unified Messaging - aree deliberatamente fuori scope) - punta a coprire bene le aree di amministrazione M365 cloud-only quotidiane.

Aree estese il 18/08/2026 (36 nuovi cmdlet)

- **Gruppi di distribuzione:** `Set-/Enable-/Disable-M365OpsDistributionGroup` , `Set-/Remove-M365OpsDynamicDistributionGroup` , `Get-M365OpsDynamicDistributionGroupMember` , `Update-M365OpsDistributionGroupMember` (sostituisce l'INTERA membership, non incrementale)
- **Tenant Allow/Block List - spoof:** `Get-/New-/Remove-M365OpsTenantAllowBlockListSpoofItem(s)` , `Set-M365OpsTenantAllowBlockListItem` - sotto-lista separata dalle voci normali per mittente/URL/hash/IP
- **Mail flow / connettori:** `Get-/Set-M365OpsTransportConfig` (impostazioni globali), `Get-/New-/Set-/Remove-M365OpsInboundConnector` e `...OutboundConnector` (rinominati il 21/08/2026 da `...ReceiveConnector/...SendConnector` - vedi riquadro sotto), `Get-/New-/Set-/Remove-M365OpsRemoteDomain` , `New-/Set-/Remove-M365OpsAcceptedDomain` , `Get-M365OpsMailDetailTransportRuleReport`
- **Quarantena (policy):** `Get-/New-/Set-/Remove-M365OpsQuarantinePolicy` , `Get-M365OpsQuarantineMessageHeader`

Testato dal vivo con scritture reali (creato, verificato, ripulito su vnsys-test): `Set-M365OpsDistributionGroup` (nascondi/mostra in rubrica), `New-/Remove-M365OpsQuarantinePolicy` (ciclo completo). Le letture `Get-M365OpsTransportConfig` / `RemoteDomain` / `QuarantinePolicy` sono verificate dal vivo con dati reali del tenant. Aggiunto il 21/08/2026: `Set-M365OpsRemoteDomain` (toggle di `AutoForwardEnabled` sulla voce "Default", verificato con lettura indipendente e ripristinato al valore originale) e il ciclo completo `New-/Set-/Remove-M365OpsDynamicDistributionGroup` (gruppo dinamico di test creato con un filtro OPATH reale, rinominato, poi rimosso - ogni passaggio verificato con una lettura indipendente, ambiente di test tornato pulito). **Aggiunto il 21/08/2026:** ciclo completo create/verifica/modifica/verifica/rimuovi/ verifica per `New-/Set-/Remove-M365OpsInboundConnector` e `...OutboundConnector` , IN APP-ONLY - funzionano perfettamente, nessun limite RBAC reale (vedi il riquadro sotto per la storia completa).

BUG STRUTTURALE trovato e corretto il 21/08/2026 (non un limite RBAC come ipotizzato inizialmente): `Get-/New-/Set-/Remove-ReceiveConnector` e `...SendConnector` fallivano con "term not recognized" sul tenant di test - scambiato in un primo momento per lo stesso schema RBAC già documentato per Teams Policies/Purview (sezione 6.4). Approfondendo su richiesta esplicita dell'utente ("puoi fare test in modalita' delegata?"), un login Delegated reale con un utente membro completo di "Organization Management" ha dato lo STESSO errore - impossibile se fosse stato RBAC. Causa vera, confermata su documentazione ufficiale Microsoft: questi cmdlet sono **esclusivi di Exchange on-premises**, mai esistiti in Exchange Online. Sostituiti gli 8 wrapper con i veri equivalenti Exchange Online, `Get-/New-/Set-/Remove-InboundConnector` e `...OutboundConnector` (parametri diversi: `SenderDomains` invece di `Bindings` / `RemoteIPRanges` , concetti legati a un server fisico che non esiste in cloud) - testato dal vivo con un ciclo completo per entrambi, funzionano subito in App-only, zero problemi di permessi. Vedi sezione 6.4 per la cronologia completa dell'indagine.

`New-/Set-/Remove-M365OpsAcceptedDomain` ha la sintassi verificata contro la documentazione ufficiale Microsoft. Resta NON testato con una scrittura reale, deliberatamente (azione ad alto impatto, puo' interrompere la posta su un intero dominio) - ma il limite RBAC che lo bloccava e' ora CONFERMATO e circoscritto (sezione 6.4, 21/08/2026): funziona in modalita' Delegated (`Get-AcceptedDomain` verificato dal vivo con 5 domini reali), resta bloccato in App-only indipendentemente da quanti ruoli RBAC si assegnano al service principal - un limite della piattaforma per questo specifico cmdlet, non un bug di questo progetto. Chi ha bisogno di gestirlo da M365Ops deve usare un profilo tenant Delegated.

Cosa resta fuori scope (per scelta, non dimenticanza)

Il censimento ha anche quantificato aree grandi lasciate deliberatamente fuori per ora: Retention/Compliance Purview incluso DLP ed eDiscovery (~65 cmdlet - oggi solo lettura, comunque limitata dal ruolo RBAC non assegnato), migrazione Public Folder (~40 cmdlet, sotto-mondo separato mai iniziato). Candidate per un'estensione futura, su richiesta esplicita. Anti-spam/anti-phish/ malware in scrittura (~98 cmdlet), che era in questo elenco, e' stato implementato il 21/08/2026 - vedi sezione 17.19.

17.16 Copertura Intune: perche' il conteggio cmdlet non e' la metrica giusta

Censimento reale (19/08/2026, non a memoria) del repository sorgente della documentazione Microsoft Graph PowerShell SDK (`MicrosoftDocs/microsoftgraph-docs-powershell` , via Git Trees API): i moduli genuinamente Intune (`Microsoft.Graph.DeviceManagement` , `.Administration` , `.Enrollment` , `.Functions` , `Devices.CorporateManagement` - quest'ultimo copre `deviceAppManagement` , quindi le app Win32/mobile e le policy di protezione app) totalizzano **3.693 cmdlet** (1.033 in v1.0, 2.660 in beta) tra v1.0 e beta.

Un confronto diretto "N di 3.693 coperti" sarebbe fuorviante, a differenza del censimento Exchange sopra: quei cmdlet sono generati automaticamente, uno per ogni combinazione risorsa/verbo REST del namespace `deviceManagement` / `deviceAppManagement` (es. `Get-MgDeviceManagementDeviceCompliancePolicySettingStateSummary`) - la stragrande maggioranza sono letture/scritture strette su sotto-risorse molto specifiche, non funzionalita' distinte per un amministratore. M365Ops non li wrappa 1:1 come fa per Exchange: usa `graph_api_call` / `propose_graph_write` generici, che possono gia' raggiungere QUALUNQUE endpoint sotto `deviceManagement` / `deviceAppManagement` - la vera limitazione non e' "quale cmdlet manca", ma quali permessi Graph sono concessi all'App Registration e quanto lo strato AI conosce lo schema esatto di un payload prima di scriverlo (vedi i bug reali del 19/08/2026 sopra, es. `omaSettings`).

Wrapper dedicati esistono solo dove la generalizzazione non basta: `Get-M365OpsManagedDevices` , `Get-M365OpsDeviceComplianceReasons` , `Get-M365OpsCompliancePatterns` (lettura/analisi dispositivi e compliance), `New-M365OpsWin32App` / `Set-M365OpsAppAssignment` (packaging e assegnazione Win32, dove passare dal modulo community `IntuneWin32App` - non solo Graph REST - serve davvero, per la codifica/upload del pacchetto `.intunewin`). Solo 5 radici Graph risultano referenziate da codice reale nell'intero modulo (verificato per grep, non a memoria): `deviceAppManagement/mobileApps` , `deviceManagement/assignmentFilters` , `deviceManagement/deviceCompliancePolicies` , `deviceManagement/deviceConfigurations` , `deviceManagement/managedDevices` - tutto il resto, quando funziona, funziona solo perche' l'AI costruisce la chiamata Graph al volo via `graph_api_call` / `propose_graph_write` , senza alcun codice dedicato che l'abbia mai verificata dal vivo.

Censimento reale per area funzionale (19/08/2026, aggiornato dopo l'implementazione)

Conteggio cmdlet SDK per area (non conteggio di funzionalita' distinte - vedi sopra il perche' - ma utile per capire dove si concentra la superficie Intune) e stato reale in M365Ops. Tutte le aree elencate come "non implementato" nel censimento iniziale sono state costruite lo stesso giorno (19/08/2026, ~60 nuovi cmdlet in `Public\`) su richiesta esplicita dell'utente, con schema Graph verificato dal vivo su Microsoft Learn per ognuna (mai a memoria) e wiring completo nello strato AI (`intune_query` / `propose_intune_write` , stesso meccanismo generico `query/proposta-scrittura` di `exo_query` / `propose_exo_write`). Verificate dal vivo su `vnsys-test` il giorno stesso via chat reale (non solo import del modulo):

Area	Cmdlet SDK (v1.0+beta)	Stato in M365Ops
Dispositivi gestiti (elenco, dettaglio, azioni remote)	referenziato in codice	funzionante - lettura/compliance testate dal vivo ripetutamente. Azioni remote (retire/wipe/sync/restart/fresh start/autopilot reset) PROPOSTE dal vivo con successo via AI generica (mai confermate per non toccare dispositivi reali) - nessun wrapper dedicato, nessun guardrail oltre alla conferma umana standard
Compliance Policy (criteri di conformita')	66 (v1.0) + 77 (beta) = 143	solo lettura generica - elenco/dettaglio letti dal vivo via <code>graph_api_call</code> , MAI creata/modificata una policy (nessun wrapper, nessun test)
Configuration Profile (profili di configurazione "classici")	46 (v1.0) + 85 (beta) = 131	parziale, non affidabile - <code>New-M365OpsDeviceConfigurationProfile</code> esiste ma e' orfano (nessun tool AI/GUI la richiama); il tentativo di creazione via AI generica in questo stesso giro di test e' FALLITO due volte (<code>omaSettings</code>) prima di essere abbandonato - oggi non affidabile. Superato in pratica dal Settings Catalog sotto, la via moderna consigliata da Microsoft stessa
Settings Catalog (<code>configurationPolicies</code> , il rimpiazzo moderno dei profili classici)	39 (solo beta)	funzionante - <code>Get-/New-/Set-/Remove-M365OpsConfigurationPolicy</code> , <code>Set-M365OpsConfigurationPolicyAssignment</code> , <code>Get-M365OpsConfigurationSettingDefinitions</code> (ricerca impostazioni). Lettura verificata dal vivo il 19/08/2026 su <code>vnsys-test</code> (2 criteri reali restituiti correttamente via chat)
Endpoint Security (Antivirus, Disk Encryption, Firewall, EDR, Attack Surface Reduction - <code>intents</code>)	60 (solo beta)	funzionante - stesso motore/cmdlet del Settings Catalog sopra (distinto da <code>templateReference.templateFamily</code>), piu' <code>Get-M365OpsConfigurationPolicyTemplates</code> per i modelli disponibili
Security Baseline	30 (solo beta)	funzionante - stesso motore Settings Catalog (<code>templateFamily=baseline</code>), nessun cmdlet separato necessario
Autopilot (identita' dispositivo + profili di deployment + ESP)	13 (v1.0) + 52 (beta) = 65	funzionante (codice) , non verificabile end-to-end - 9 cmdlet (import/lettura/modifica/rimozione dispositivo, profili di deployment + assegnazione). L'import di un dispositivo reale richiede un hash hardware autentico (4K HH) che non e' fabbricabile per un test - mai verificato dal vivo con un dispositivo reale, solo sintassi e schema Graph
Script PowerShell Windows (<code>deviceManagementScripts</code> - DIVERSO dal packaging Win32: esecuzione diretta di uno script, non un'app)	33 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsDeviceScript</code> , <code>Set-M365OpsDeviceScriptAssignment</code> (Windows e macOS). Lettura testata dal vivo il 19/08/2026: 403 chiaro per permesso <code>DeviceManagementScripts.ReadWrite.All</code> mancante su <code>vnsys-test</code> (vedi sezione 4.2) - codice corretto, permesso da concedere
Script shell macOS	32 (solo beta)	funzionante - stesso cmdlet di sopra con <code>-Platform macOS</code> , stesso gap di permesso
Proactive Remediation (<code>deviceHealthScripts</code> - rileva+correggi)	41 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsProactiveRemediation</code> , <code>Set-M365OpsProactiveRemediationAssignment</code> (pianificazione giornaliera/oraria). Stesso gap di permesso <code>DeviceManagementScripts.ReadWrite.All</code> dello script Windows sopra (verificato dal vivo)
Admin Template / Group Policy Analytics	20 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsAdminTemplate</code> , <code>Set-M365OpsAdminTemplateAssignment</code> , <code>Find-M365OpsAdminTemplateSetting</code> (ricerca impostazioni GP) + <code>Set-M365OpsAdminTemplateSetting</code> (azione <code>updateDefinitionValues</code>). Lettura verificata dal vivo il 19/08/2026 (nessun profilo configurato su <code>vnsys-test</code> , risposta corretta senza errori)
Windows Update rings (Feature/Quality update profiles)	22 (solo beta)	funzionante - <code>New-/Get-/Remove-M365OpsUpdateRing</code> , <code>Set-M365OpsUpdateRingAssignment</code> . Lettura verificata dal vivo il 19/08/2026 (nessun anello configurato, risposta corretta)

App Protection / MAM (Managed App Protection Android/iOS)	50+49 = 99 (v1.0, Android+iOS)	funzionante - <code>New-/Get-/Remove-M365OpsAppProtectionPolicy</code> , <code>Set-/Remove-M365OpsAppProtectionAssignment</code> , <code>Set-M365OpsAppProtectionTargetApps</code> . Lettura verificata dal vivo il 19/08/2026 (nessun criterio configurato, risposta corretta). Windows Information Protection resta fuori scope (funzionalita' in via di deprecazione da parte di Microsoft)
Scope Tag (RBAC per oggetto Intune)	14 (solo beta)	funzionante (codice) , permesso da concedere - <code>New-/Get-/Remove-M365OpsScopeTag</code> per i tag stessi (oltre alla sola assegnazione gia' esistente su <code>New-M365OpsWin32App -ScopeTagNames</code>). Creazione testata dal vivo il 19/08/2026: 403 chiaro per <code>DeviceManagementRBAC.ReadWrite.All</code> mancante (vedi sezione 4.2) - pipeline proposta→conferma→esecuzione→errore Graph confermata funzionante end-to-end, manca solo il permesso
Enrollment Configuration (restrizioni/profil di iscrizione)	12 (v1.0)	funzionante (codice) , permesso da concedere - <code>Get-M365OpsEnrollmentConfigurations</code> , <code>New-M365OpsEnrollmentLimitConfiguration</code> , <code>New-M365OpsEnrollmentPlatformRestriction</code> , <code>Set-M365OpsEnrollmentConfigurationPriority / Assignment</code> , <code>Remove-M365OpsEnrollmentConfiguration</code> . Lettura testata dal vivo il 19/08/2026: 403 per <code>DeviceManagementServiceConfig.ReadWrite.All</code> mancante
Device Category	17 (v1.0)	non implementato - area di nicchia, non richiesta esplicitamente
Notification Message Template	11 (v1.0)	funzionante - <code>New-/Get-/Remove-M365OpsNotificationTemplate</code> , <code>Set-M365OpsNotificationTemplateMessage</code> (multi-lingua), <code>Send-M365OpsNotificationTemplateTest</code> . Lettura verificata dal vivo il 19/08/2026 (nessun modello configurato, risposta corretta)
Role Definition (ruoli RBAC custom Intune)	13 (v1.0)	funzionante (codice) , permesso da concedere - <code>New-/Get-/Remove-M365OpsCustomRole</code> , <code>Get-M365OpsRoleDefinitionActions</code> (catalogo azioni di riferimento), <code>Set-/Remove-M365OpsRoleAssignment</code> . Lettura testata dal vivo il 19/08/2026: stesso 403 <code>DeviceManagementRBAC.ReadWrite.All</code> dello Scope Tag sopra
Certificati/Wifi/VPN/Email (profili di connettivita')	incluso in Configuration Profile sopra	solo generico, mai testato - sono sotto-tipi di deviceConfiguration, quindi teoricamente raggiungibili con la stessa via generica, ma zero verifica dal vivo su nessuno di questi
Connettori partner (Mobile Threat Defense, Exchange, Compliance Management Partner, Remote Assistance, DEP/ABM, VPP token)	7+6+5+5+5+6 = 34 (v1.0)	funzionante (sola lettura) - <code>Get-M365OpsPartnerConnectorsStatus</code> , sola lettura per design: l'attivazione iniziale richiede un consenso OAuth interattivo con il servizio partner, non realizzabile da chat/REST. Verificato dal vivo il 19/08/2026 (query riuscita, nessun connettore configurato)

Restano deliberatamente fuori scope: Device Category (area di nicchia), Windows Information Protection (in deprecazione), profili di connettivita' Certificati/Wifi/VPN/Email (raggiungibili solo via `propose_graph_write` generico, mai verificati). Tre aree (Scope Tag, Enrollment Configuration, Role Definition/RBAC) hanno il codice completo e verificato ma restano bloccate da permessi Graph Application non ancora concessi su `vnssys-test ( DeviceManagementRBAC.ReadWrite.All , DeviceManagementServiceConfig.ReadWrite.All )` - vedi sezione 4.2 per i nomi esatti da aggiungere nell'App Registration.

17.17 Check permessi app (pulsante "🔒 Stato permessi")

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente: un pulsante in alto nella chat (o scrivendo "verifica permessi app"/"che permessi ha l'app") interroga Microsoft Graph in tempo reale per vedere quali permessi sono DAVVERO concessi (con "Grant admin consent") all'App Registration del tenant attivo, e li traduce per area funzionale (Intune, Entra ID, Report, MFA, Sicurezza, Teams, SharePoint, Exchange) con stato "nessun accesso" / "lettura" / "lettura+scrittura" - invece di scoprirlo un errore alla volta durante l'uso normale.

Nessun elenco statico: legge le assegnazioni REALI del service principal (`GET /servicePrincipals/{id}/appRoleAssignments`) - un permesso aggiunto in Entra ID ma senza "Grant admin consent" non compare, esattamente come nella realtà non funziona. Costo zero: nessuna chiamata AI, la funzione (`Get-M365OpsAppPermissionsCheck`) e' puro codice deterministico, catalogata come comando locale (sezione 17, `CommandCatalog.ps1`).

Bug reale trovato durante lo sviluppo, corretto prima di consegnare: il nome della risorsa SharePoint restituito da Graph e' "Office 365 SharePoint Online", NON "SharePoint Online" come si potrebbe supporre dal nome "SharePoint" mostrato nel selettore del portale Entra ID (sezione 4.3) - un primo tentativo con quel nome produceva sempre "nessun accesso" anche a permesso presente (un mismatch di stringa nel codice, non un vero permesso mancante), scoperto confrontando l'output con le assegnazioni grezze reali di vnsys-test prima di fidarsi del risultato.

Secondo bug reale, trovato dal vivo dall'UTENTE il 21/08/2026 (non durante lo sviluppo, dopo la consegna): "l'app ha Sites.FullControl.All, che dovrebbe essere oltre Read.All, ma il check non se ne rende conto" - confermato: il confronto era per stringa ESATTA, quindi `Sites.FullControl.All` concesso non soddisfaceva un requisito di `Sites.Read.All`, anche se chi ha accesso completo puo' ovviamente anche solo leggere. Corretto aggiungendo il riconoscimento della gerarchia standard Microsoft Graph `Read.All < ReadWrite.All < Manage.All < FullControl.All`: un permesso piu' ampio con lo stesso prefisso ora soddisfa automaticamente un requisito piu' stretto. Verificato dal vivo dopo il fix: l'area "SharePoint (lettura diretta via Graph)" e' passata da "nessun accesso" a "lettura", coerente con il permesso realmente posseduto.

Quattro "aree di concessione" Microsoft distinte vengono controllate, ognuna con il proprio consenso separato nel portale Entra ID: Microsoft Graph, "Office 365 Exchange Online" (`Exchange.ManageAsApp`), "Office 365 SharePoint Online" (`Sites.FullControl.All`, sezione 4.3) e "Skype and Teams Tenant Admin API" (solo per i cmdlet di POLICY Teams, sezione 4.4 - nome risorsa non ancora verificato dal vivo su un tenant che abbia quel permesso concesso, l'unico effetto possibile di un nome sbagliato sarebbe un falso "nessun accesso", mai un falso positivo). Teams e SharePoint "di base" (moduli MicrosoftTeams/PnP) sono segnalati separatamente come sempre disponibili con lo stesso certificato/secret gia' usato per l'autenticazione - non dipendono da nessuno di questi permessi (verificato dal vivo il 17/08/2026, sezione 17.11).

Il livello RBAC REALE di Exchange Online (quali cmdlet funzionano davvero, non solo se la connessione riesce) dipende anche dal ruolo assegnato al service principal DENTRO Exchange (sezione 6) - non verificabile da questo check senza una connessione Exchange live, segnalato come nota esplicita nel risultato invece di dare un falso senso di completezza.

Estensione per tenant Delegati (21/08/2026)

Aggiunta richiesta esplicitamente dall'utente dopo aver notato che, su un tenant Delegato, il pulsante si limitava a un errore ("il check si applica solo ai tenant App-only"): "non sarebbe meglio listare i ruoli attivi dell'utente e dirgli cosa puo'/non puo' fare con quei ruoli?". Su un tenant Delegato non esiste un'App Registration da verificare - l'accesso dipende dai RUOLI RBAC EXCHANGE REALI dell'utente con cui si e' fatto login. `Get-M365OpsDelegatedPermissionsCheck` (nuova, Private) interroga `Get-ManagementRoleAssignment -RoleAssignee` per l'utente delegato e confronta i ruoli posseduti con una tabella area=ruolo.

Onestà sulla copertura: solo 2 righe della tabella sono VERIFICATE dal vivo con una scrittura reale in modalità Delegated ("Domini accettati" via "Remote and Accepted Domains", confermato lo stesso giorno con un login reale; "Anti-spam/ anti-phishing/malware" via "Transport Hygiene", confermato in App-only con lo stesso ruolo). Le altre righe (Connettori, Mailbox/gruppi, Regole di trasporto, Role Management) sono dedotte dal NOME del ruolo Exchange, non ancora confermate con una scrittura reale in Delegated - marcate esplicitamente come tali nel risultato (nota per riga), per non dare un falso senso di certezza. L'accesso Graph (Intune/Entra ID/Teams/SharePoint) in Delegated dipende dal ruolo DIRECTORY Entra ID dell'utente - un controllo diverso, non ancora implementato, segnalato come tale nel risultato invece di ometterlo in silenzio.

L'elenco COMPLETO e grezzo dei ruoli RBAC Exchange posseduti dall'utente e' sempre incluso nel risultato (riga "informativo" dedicata) - anche per le aree non ancora mappate, cosi' non si perde informazione reale in nome di una tabella incompleta.

Verificato dal vivo il 21/08/2026 con un login Delegato reale (`diego@vnsys.it`, membro di "Organization Management"): il pulsante "🔒 Stato permessi" mostra correttamente tutte le aree mappate come disponibili, l'elenco completo dei ~90 ruoli posseduti, e la nota sul gap Graph/directory role - nessun errore, nessuna informazione fuorviante.

Due bug reali trovati dall'utente subito dopo, corretti lo stesso giorno

- Intestazione sbagliata:** il risultato apriva sempre con "Permessi reali dell'app registrata su questo tenant" - testo copiato dal ramo App-only, senza senso su un tenant Delegato (non esiste nessuna App Registration). Corretto rendendo l'intestazione condizionale in `CommandCatalog.ps1`.
- Mancavano i ruoli DIRECTORY Entra ID:** "mi aspettavo di vedere i miei ruoli, tipo Global Admin nel mio caso" - i ruoli RBAC Exchange sopra sono un sistema COMPLETAMENTE SEPARATO dai ruoli directory Entra ID che governano Intune/Entra ID/Teams/SharePoint via Graph (gia' segnalato come gap noto nella prima versione, ora colmato). Aggiunta una chiamata a `GET /me/transitiveMemberOf/microsoft.graph.directoryRole` usando lo stesso token Graph delegato gia' ottenuto con "Accedi a Microsoft Graph col mio utente" - `Invoke-M365OpsGraphRequest` sceglie gia' da solo il token giusto in base all'AuthMode, nessun codice di autenticazione nuovo necessario. Mappa i ruoli directory standard Microsoft (Global Administrator, Intune Administrator, User Administrator, Groups Administrator, Teams Administrator, SharePoint Administrator) su Intune/Entra ID/Teams/SharePoint, oltre a mostrare sempre l'elenco completo grezzo dei ruoli directory posseduti.

17.18 Tema chiaro/scuro e pannello Upload a scomparsa

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente. Due modifiche indipendenti alla GUI:

- **Tema scuro** - pulsante 🌙/☀️ nell'header. Scelta SOLO dell'utente (mai dedotta dal tema del sistema operativo), persistita in `localStorage` così resta la stessa tra un riavvio del browser e l'altro. Tutta la GUI usa già variabili CSS per i colori (nessun colore scritto a fisso in nessun punto della pagina) - il tema scuro è quindi un solo blocco di variabili alternative, non una riscrittura dei singoli stili.
- **Pannello Upload a scomparsa** - i 3 pulsanti di caricamento (file da pacchettizzare, icona, CSV migrazione), prima sempre visibili sopra la casella di testo, ora sono dentro un pannello che si apre/chiude col pulsante "📁 Upload" nell'header - stesso pattern toggle già usato per il pannello Impostazioni.

Contestualmente, i pulsanti dell'header ("⚙️ Impostazioni tenant", "🔒 Stato permessi", "📁 Upload", tema) sono stati unificati sotto un solo stile condiviso (`.header-btn`) - prima "Impostazioni tenant" aveva una regola CSS dedicata mentre "Stato permessi" (aggiunto in v0.9.18) non ne aveva nessuna, risultando visivamente diverso (osservato dall'utente).

Un file HTML/JS come `Gui\index.html` viene tenuto in memoria dal server all'avvio (`Get-Content -Raw in Gui\Server.ps1` , mai riletto ad ogni richiesta) - a differenza di un cambio a un file `.ps1` del modulo (che serve comunque un riavvio per lo stesso motivo), un cambio SOLO a `index.html` può sembrare non applicarsi se si controlla senza riavviare: verificato dal vivo durante lo sviluppo di questa stessa funzione (un primo controllo subito dopo l'edit mostrava ancora la pagina vecchia).

17.19 Anti-spam, anti-phishing e filtro malware in scrittura

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente ("passiamo all'implementazione di Anti-spam/anti-phishing/malware in scrittura — ~98 cmdlet, oggi solo lettura"). Copre le tre aree con lo stesso schema "criterio + regola" già visto per Accepted Domain/Connettori: un CRITERIO (policy) definisce COSA succede (azioni, soglie), una REGOLA (rule) definisce A CHI si applica - un criterio senza una regola collegata non si applica a nessuno (tranne quelli predefiniti del sistema).

Area	Cmdlet nativo	Wrapper M365Ops
Anti-spam	HostedContentFilterPolicy / HostedContentFilterRule	Get-/New-/Set-/Remove-M365OpsAntiSpamPolicy , Get-/New-/Set-/Remove-M365OpsAntiSpamRule , Enable-/Disable-M365OpsAntiSpamRule
Anti-phishing	AntiPhishPolicy / AntiPhishRule	Get-/New-/Set-/Remove-M365OpsAntiPhishPolicy , Get-/New-/Set-/Remove-M365OpsAntiPhishRule , Enable-/Disable-M365OpsAntiPhishRule
Filtro malware	MalwareFilterPolicy / MalwareFilterRule	Get-/New-/Set-/Remove-M365OpsMalwareFilterPolicy , Get-/New-/Set-/Remove-M365OpsMalwareFilterRule , Enable-/Disable-M365OpsMalwareFilterRule

Nessun nuovo ruolo RBAC richiesto: il ruolo Exchange "Transport Hygiene", già assegnato al service principal di questo progetto per altre funzionalità, copre già tutti e sei i cmdlet New- . Verificato dal vivo (non assunto dalla documentazione): tutti e 24 i wrapper testati in App-only con un ciclo completo create/verifica/modifica/verifica/rimuovi/verifica per ognuna delle tre aree - nessun errore di permesso, ambiente di test tornato pulito ad ogni ciclo.

Bug reale trovato dal vivo durante il test, non dalla documentazione: NESSUNO dei tre Set-*Rule (Set-HostedContentFilterRule , Set-AntiPhishRule , Set-MalwareFilterRule) accetta un parametro -Enabled , nonostante alcune fonti (inclusi risultati di ricerca web) suggerissero il contrario - verificato leggendo la lista PARAMETRI REALE del cmdlet live con Get-Command , non la sola documentazione. Il tentativo iniziale (Set-M365OpsAntiSpamRule -ExtraParams @{Enabled=\$false}) falliva con "A parameter cannot be found that matches parameter name 'Enabled'". Corretto aggiungendo 6 cmdlet dedicati Enable-/Disable-M365OpsXRule (che chiamano i veri Enable-/Disable-HostedContentFilterRule ecc., cmdlet nativi separati che ESISTONO e funzionano) - verificato dal vivo per tutti e tre i tipi.

Sintassi di tutti i cmdlet New- verificata contro la documentazione ufficiale Microsoft prima di scrivere i wrapper (non a memoria): unico parametro davvero obbligatorio per un criterio e' Name ; per una regola sono Name + il riferimento al criterio (HostedContentFilterPolicy / AntiPhishPolicy / MalwareFilterPolicy a seconda del tipo) - un criterio può essere collegato a UNA SOLA regola, Microsoft lo impedisce direttamente al momento della creazione se si tenta di riusarlo.

17.20 La guida resta raggiungibile anche a zero configurazione ([/guida](#))

Aggiunto il 21/08/2026, richiesto esplicitamente dall'utente dopo un test end-to-end su un PC completamente pulito: "quando l'app parte e non e' configurata la sua guida deve essere raggiungibile per essere consultata, o non fa perche' manca l'IA?".

Problema reale, non solo di messaggistica: prima di questo fix, la guida era leggibile SOLO tramite lo strumento `kb_query` dentro il motore AI (`Invoke-M365OpsAgentTools`) - senza nessuna chiave AI configurata, nemmeno la domanda piu' semplice sulla guida stessa ("come genero il certificato Exchange?") trovava risposta: un problema dell'uovo e della gallina, serviva la guida per sapere come configurare l'AI, ma la guida stessa richiedeva l'AI per essere consultata.

Corretto con una route statica `GET /guida` sul server web dell'app, che serve `docs\Guida-Configurazione.pdf` direttamente come file - nessun passaggio dal motore AI, nessuna chiave richiesta, nessun tenant necessario. Raggiungibile aprendo `http://localhost:<porta>/guida` in qualunque browser, anche al primissimo avvio dell'app senza nulla configurato. I messaggi di errore per chiave AI mancante (`Invoke-M365OpsAgentTools.ps1`) e il messaggio di onboarding al primo avvio (sezione 7) ora indicano esplicitamente questo percorso. Verificato dal vivo: PDF di 8 pagine servito correttamente via una richiesta GET reale.

5. Se un tenant è in modalità Delegated, fai il login la prima volta che ti serve (tab Tenant)

19. Log delle azioni — storico e debug

Ogni messaggio ricevuto in chat, ogni strumento AI chiamato (con esito), e ogni azione confermata eseguita vengono registrati in `Logs\m365ops-YYYYMMDD.log` (un file al giorno, testo semplice, append-only) — indipendentemente dai messaggi `Write-Host` già visibili nel terminale se stai guardando la finestra del server.

19.1 Consultarli

Tab Manutenzione → sezione "Log delle azioni (oggi)" mostra le ultime 200 righe, con un pulsante "Aggiorna". Oppure apri direttamente il file di testo in `Logs\`.

```
# formato di ogni riga: data/ora TAB [livello] TAB tenant[modalità] TAB messaggio
2026-08-15 10:20:46 [Info] Fabrikam-Prod[Delegated] Messaggio ricevuto: elenca le mailbox condivise
2026-08-15 10:20:46 [Info] Fabrikam-Prod[Delegated] Avvio Invoke-M365OpsAgentTools (fallback AI)...
2026-08-15 10:20:48 [Info] Fabrikam-Prod[Delegated] Tool AI chiamato: exo_query Get-M365OpsSharedMailboxes
2026-08-15 10:20:48 [Error] Fabrikam-Prod[Delegated] Tool AI fallito: ... Sessione Exchange non attiva ...
2026-08-15 10:20:48 [Info] Fabrikam-Prod[Delegated] Tool AI completato: exo_query Get-M365OpsSharedMailboxes
```

19.2 Cosa viene registrato

- Avvio del server e tenant iniziale caricato
 - Ogni messaggio ricevuto in chat (testo completo)
 - Inizio/fine di ogni chiamata al motore AI con tool-calling, con timestamp separati — se il server si blocca, l'ultima riga "chiamato" senza il corrispondente "completato" indica esattamente quale strumento è rimasto appeso (è così che è stato diagnosticato l'incidente di sezione 10.6)
 - Ogni tool AI chiamato/completato/fallito, con l'errore esatto in caso di fallimento
 - Ogni azione confermata eseguita (creazione gruppo, scrittura Exchange, script personalizzato...)
- Uso da Claude Code (o da chiunque debugga il progetto): quando qualcosa si comporta in modo strano, il primo posto da controllare è l'ultima riga di `Logs\m365ops-*.log` di oggi — spesso basta quella a capire dove si sia fermato.

20. Stato e utilizzo del motore AI

Il tab **Motore AI** mostra, oltre alle impostazioni, un pannello "Stato e utilizzo" con: provider attivo, quale delle due chiavi (Claude/Azure OpenAI) risulta configurata, e un **contatore delle chiamate** fatte a ciascun provider in questa sessione del server (azzerato ad ogni riavvio — non persistito, è un contatore di sessione; il provider *selezionato*, invece, è persistito — sezione 8).

20.1 Chi usa davvero quale provider

Dal 15/08/2026 il provider selezionato nel tab Motore AI vale per **tutto**: sia la chat libera con strumenti (`Invoke-M365OpsAgentTools` — la stragrande maggioranza delle domande) sia le chiamate singole senza strumenti (analisi pattern, diagnosi/correzione errori). Non esiste più una parte dell'app che ignora silenziosamente la scelta fatta in GUI — vedi sezione 8.1/8.2 per come funziona il tool-calling su entrambi i provider e il bug di scoping reale trovato e corretto durante questo lavoro.

20.2 Verificare la connessione dal vivo

Il pulsante "Testa connessione (provider attivo)" fa una vera chiamata reale (pochi token, non gratuita) al provider selezionato e conferma se risponde correttamente — non si limita a controllare che la chiave sia presente. Usalo su richiesta esplicita, non in automatico: costa una chiamata reale ogni volta.

20.3 Indicatore di avanzamento onesto

Il server è a thread singolo (sezione 10.6): mentre elabora una domanda non può rispondere a nessun'altra richiesta, nemmeno a un polling di stato. Un messaggio statico "sto elaborando..." sarebbe quindi indistinguibile da un blocco reale - bug reale segnalato dal vivo: "dice che sta facendo roba quando dentro non sta facendo un cazzo". La GUI ora mostra invece un contatore dei secondi realmente trascorsi (misurato lato browser, non simulato), con messaggi che cambiano dopo le prime decine di secondi per suggerire cosa potrebbe star succedendo (molte chiamate Graph in sequenza, un report con molti dati, o - oltre i 60-90s - un login interattivo Exchange/Intune avviato e mai completato in un'altra finestra, l'unico caso di blocco genuino di lunga durata). Non è un vero progresso passo-passo (il server non può emettere eventi mentre è occupato), ma è un segnale onesto invece che uno finto.

20.4 Budget di token e risposte vuote sui modelli reasoning

Il limite di token per ogni singolo giro del ciclo di tool-calling (`max_tokens` per Claude, `max_completion_tokens` per Azure OpenAI) è 4000, non più 2000. Bug reale scoperto il 17/08/2026 testando la creazione di script da parte dell'AI (sezione 17.6): su un modello reasoning come `gpt-5-mini`, i token di ragionamento interno condividono lo stesso budget dei token restituiti come risposta — con un compito che richiede scrivere codice vero (non solo un breve JSON), il modello poteva esaurire l'intero budget nel ragionamento e restituire un contenuto **vuoto**, senza alcun errore lanciato. L'utente vedeva semplicemente un messaggio bianco in chat.

Corretto in due modi complementari: il budget è stato alzato a 4000 (riduce la probabilità che succeda), e — indipendentemente dalla causa — un contenuto vuoto non arriva più silenzioso all'utente: viene rilevato, registrato nel log con `finish_reason` e il conteggio dei token di ragionamento usati (utile per capire se serve alzare ancora il budget in futuro), e sostituito con un messaggio che spiega cosa è successo invece di una risposta bianca.

20.5 Rete di sicurezza contro un messaggio con 'role' nullo

Bug osservato due volte in questo progetto con la stessa identica firma: Azure OpenAI rifiuta l'INTERA richiesta con 400 `"Invalid type for messages[N].role... got null instead"` se anche un solo messaggio nell'array ha 'role' mancante o nullo. La prima occorrenza (storico conversazione persistito corrotto) era stata mitigata filtrando le voci non valide in `Get-M365OpsChatHistory`. Riapparso il 17/08/2026 su un tenant con storico pulito (verificato leggendo direttamente il file) - l'origine esatta di QUESTA seconda occorrenza non è stata individuata con certezza (probabile causa: una voce costruita durante il ciclo di tool-calling stesso, non nello storico persistito).

Invece di continuare a inseguire ogni possibile origine, la mitigazione è stata spostata al punto più a valle possibile: l'array `messages` viene filtrato appena prima di ogni chiamata reale (sia il ramo Azure sia il ramo Claude), scartando qualunque voce senza un 'role' valido. Una voce malformata da QUALSIASI causa, presente o futura, degrada quindi a "quel turno manca dal contesto" invece di far fallire l'intera risposta.

Causa radice trovata il 19/08/2026 (richiesto esplicitamente dall'utente dopo averlo rivisto ricomparire durante un bug-hunt sul catalogo comandi): il filtro sopra continuava a scartare 1 voce ad OGNI chiamata AI che parte da uno storico vuoto (non un caso raro - praticamente ogni "nuova conversazione"). Instrumentato temporaneamente il codice per vedere `$messages` nel momento esatto del filtro: la voce fantasma era sempre `{"role":null,"content":null}` come primo elemento. Causa, verificata empiricamente in isolamento passo per passo: `Get-M365OpsChatHistory` fa `return @()` a storico vuoto, ma `Gui\Server.ps1` assegnava il risultato SENZA avvolgerlo in `@(...)` - un array vuoto restituito così da una funzione PowerShell **collassa a \$null** in un'assegnazione scalare (verificato: `$a = FunzioneCheRestituisceArrayVuoto` rende `$a` letteralmente `$null`, non un array di lunghezza 0). `-History $null` arriva così in `Invoke-M365OpsAgentTools` (un `$null` passato esplicitamente NON attiva il default del parametro, quello scatta solo se il parametro è del tutto omissso), e `$null | ForEach-Object {...}` NON esegue zero iterazioni come ci si aspetterebbe da una collezione vuota - ne esegue UNA, con `$_ = $null`, producendo esattamente la voce fantasma osservata. Corretto in due punti: `@(...)` aggiunto attorno alla chiamata mancante in `Server.ps1` (l'altro punto che già la chiamava, `GET /api/chat/history`, ce l'aveva già), più `Where-Object { $_ }` prima del `ForEach-Object` che costruisce `$messages` da `$History` in `Invoke-M365OpsAgentTools.ps1` - rete di sicurezza simmetrica a quella già esistente sull'output, così un `$History` nullo non può più produrre la voce fantasma indipendentemente da futuri errori di chiamata. Verificato dal vivo: reset storico + messaggio → nessun "Filterate...role mancante" nei log (prima compariva sempre in questo esatto scenario).

21. Stato e reset MFA — dettagli tecnici

Vedi sezione 13.5 per l'uso quotidiano in chat. Questa sezione documenta i dettagli verificati sulla documentazione Microsoft reale (non a memoria), utili se qualcosa non torna.

21.1 Perché non c'è un singolo flag "MFA abilitata"

La pagina "Authentication states" di Microsoft Graph (vedere/impostare lo stato MFA per-utente con un valore unico) è esplicitamente **non ancora supportata in v1.0** - lo stato reale è governato da Conditional Access o dalle vecchie impostazioni per-utente dell'admin center, non da un campo Graph interrogabile. Il modo corretto, verificato su learn.microsoft.com/graph/api/authentication-list-methods e [.../authenticationmethods-overview](https://learn.microsoft.com/graph/api/authentication-methods-overview), è elencare `GET /users/{id}/authentication/methods` e guardare quali tipi risultano registrati oltre alla password: se c'è solo la password, l'utente non ha configurato nessun metodo di verifica aggiuntivo. `Get-M365OpsUserMfaStatus` esclude dal conteggio "MFA configurata" sia la password (fattore primario, non un secondo fattore) sia il metodo email (serve solo per il reset password self-service, non per l'MFA).

21.2 Cosa significa davvero "resettare l'MFA"

Non esiste un'azione atomica equivalente in Graph: la documentazione ufficiale stessa indica di eliminare ogni metodo uno per uno, tramite l'endpoint REST specifico del suo tipo (es. `DELETE /users/{id}/authentication/microsoftAuthenticatorMethods/{methodId}`, `.../phoneMethods/{methodId}`, `.../fido2Methods/{methodId}`, e così via per app OATH, Windows Hello for Business, Temporary Access Pass). `Reset-M365OpsUserMfa` fa esattamente questo, uno per uno, senza mai toccare password o email, e restituisce sia i metodi rimossi con successo sia quelli falliti (invece di fermarsi al primo errore).

21.3 Permessi e ruolo — due requisiti separati

Serve **sia** lo scope Graph `UserAuthenticationMethod.Read.All` (lettura) o `.ReadWrite.All` (reset) **sia** un ruolo Entra assegnato a chi ha fatto login: Global Reader per la sola lettura; Authentication Administrator o Privileged Authentication Administrator per il reset. Senza il ruolo, Graph risponde 403 "Request Authorization failed" anche con lo scope corretto nel token - un errore facile da scambiare per uno scope mancante, ma è invece un requisito di ruolo separato. Incontrato dal vivo il 15/08/2026 e risolto assegnando il ruolo giusto all'utente che fa login.

22. Report AI-orchestrati con grafici — dettagli tecnici

Vedi sezione 13.4 per l'uso quotidiano in chat. Architettura: l'AI raccoglie SEMPRE i dati grezzi completi (mai un riassunto pre-aggregato) tramite gli strumenti di lettura già esistenti (`graph_api_call` , `exo_query`), poi passa l'intero dataset più un elenco opzionale di campi da graficare (`chartFields`) allo strumento `generate_report` . Da lì in poi non c'è più ragionamento AI coinvolto:

- **Excel:** tutte le righe raccolte, via `Export-M365opsReport -Format xlsx` .
- **Aggregazione grafici:** `Group-Object` sul campo richiesto, conteggio per valore distinto - fatto dal codice, MAI chiesto all'AI. Un conteggio sbagliato su centinaia di righe sarebbe un errore silenzioso difficile da notare guardando solo il grafico finale.
- **Grafici:** SVG generati internamente (`New-M365OpsSvgBarChart` / `New-M365OpsSvgPieChart`), nessuna libreria JavaScript esterna, nessuna dipendenza da internet in fase di stampa - stessa filosofia di Edge headless locale già usata per ogni altro export PDF.
- **PDF:** HTML con i grafici SVG incorporati + tabella dati completa, stampato via Microsoft Edge headless, stesso meccanismo di `Export-M365opsReport` .

Entrambi i file compaiono in chat come pulsanti di download reali (`GET /api/reports/download?file=...` , protetto da path traversal: qualunque nome file con `/ o \` viene rifiutato) - il testo di risposta dell'AI non indica mai un percorso assoluto su disco, corretto dopo un bug reale in cui l'AI annunciava "genero il report" senza aver davvero chiamato lo strumento (vedi il blocco "REGOLA CRITICA" nel system prompt di `Invoke-M365opsAgentTools` , più un controllo di sicurezza lato server che segnala il sospetto se il testo promette un report senza allegati).

23. Memoria della conversazione — dettagli tecnici

Vedi sezione 13.6 per l'uso quotidiano. Prima di questa funzione (aggiunta il 15/08/2026), ogni domanda all'AI ripartiva da zero: `Invoke-M365OpsAgentTools` costruiva la conversazione con il solo messaggio corrente, mai i turni precedenti - una domanda di follow-up del tipo "e quello di prima?" non aveva alcun contesto a cui agganciarsi.

23.1 Cosa viene salvato e dove

`Config\ChatHistory-<nometenant>.json`, uno storico per tenant (mai condiviso tra profili diversi, stesso principio di isolamento multi-tenant della sezione 14). Ogni scambio (domanda utente + risposta finale, qualunque sia stato il percorso - catalogo locale o AI) viene accodato in un unico punto in `Server.ps1`, così lo storico resta coerente con ciò che l'utente ha davvero visto in chat, non solo con le risposte generate dall'AI.

Bug reale corretto il 16/08/2026. L'endpoint che legge lo storico (`GET /api/chat/history`) pipelava l'array in `ConvertTo-Json` - per un array VUOTO (tenant senza nessuno scambio ancora, es. subito dopo un riavvio), questo restituisce `$null` vero, non la stringa "[]": `GetBytes($null)` lanciava "Value cannot be null" ad ogni caricamento pagina. Corretto passando a `ConvertTo-Json -InputObject $array` (mai pipelato), che serializza correttamente 0, 1 o più elementi senza bisogno di controlli separati - stesso fix applicato a tutti gli altri punti a rischio del progetto (`/api/tenants` , `/api/setup-status` , `/api/logs` , il salvataggio dello storico stesso) dopo averli controllati uno per uno.

23.2 Limiti deliberati

- **Ultimi 8 scambi** (16 voci) per tenant: oltre, i più vecchi vengono scartati. Senza un tetto, una conversazione lunga rischierebbe di gonfiare ogni chiamata successiva in costo/tempo, fino a rischiare di sfiorare il limite di contesto del modello.
- **Ogni messaggio troncato a 3000 caratteri** prima del salvataggio: un elenco dispositivi o un dump JSON molto lungo non deve consumare da solo tutto il budget di contesto delle domande successive.
- **Password redatte prima del salvataggio** - qualunque campo `"password": "..."` nel testo (es. il corpo di una proposta di creazione utente) viene sostituito con `[REDACTED]` prima di scrivere su disco, stesso principio "zero segreti su disco" già applicato a `tenants.json` (sezione 14), esteso qui ai segreti che possono comparire dentro una conversazione stessa.

23.3 Azzerare lo storico

Pulsante "Nuova conversazione" in GUI, oppure scrivere "nuova conversazione" / "dimentica tutto" / "cancella cronologia" / "resetta la chat" in chat (riconosciuto dal catalogo comandi locale, nessun costo AI) - elimina il file per il tenant attivo e annulla anche un'eventuale proposta di scrittura ancora in sospeso, per evitare di ripartire "puliti" in chat ma con un'azione dimenticata ancora pronta a essere eseguita da un vecchio "sì".

24. Report di utilizzo Microsoft 365 Copilot (22/08/2026)

Primo passo dell'integrazione con l'amministrazione di Microsoft 365 Copilot, richiesta esplicitamente dall'utente: "dobbiamo aggiungere tutto quello che riguarda il supporto copilot M365... quello che si vede da admin center". Ambito volutamente ristretto alla parte amministrativa/di reporting - **M365Ops non chiama mai Copilot come motore AI** (usa sempre Claude/Azure OpenAI per quello, sezione 8), qui si tratta solo di leggere DATI su come Copilot viene usato nel tenant, non di usarlo.

24.1 Cosa e' disponibile, cosa no

Ricerca fatta contro la documentazione Microsoft Learn reale (non a memoria) prima di implementare, per evitare di costruire su un'API instabile o su un percorso che poi risulta Delegato-only mentre questo progetto lavora principalmente in App-only:

Area	Stato	Motivo
Report di utilizzo (per-utente e aggregato)	implementato	Endpoint Graph v1.0 (GA, stabile), permesso <code>Reports.Read.All</code> - funziona identico sia App-only sia Delegato, nessuna limitazione. Vedi 24.2.
Impostazioni Copilot (User access/Data access/Copilot actions)	non implementato	API <code>/copilot/admin/policySettings</code> - permesso <code>CopilotPolicySettings.Read/.ReadWrite</code> risulta letteralmente "Application: Not supported" nella documentazione ufficiale - SOLO Delegato, mai utilizzabile su un tenant App-only.
Catalogo agenti/app Copilot	non implementato	API <code>/copilot/admin/catalog/packages</code> ancora in preview, rollout tenant-per-tenant con errori 403 documentati anche a permessi corretti quando il feature flag non e' ancora attivo - troppo instabile per costruirci sopra ora.
Connettori federati Copilot	non implementato	<code>Set-FederatedConnectorToggle</code> (modulo <code>Connector.cmd</code>) risulta in dismissione dal 25/08/2026, sostituito da "Allowed agent types" in Agent 365 dal 5/10/2026 - costruirci sopra ora avrebbe significato shippare qualcosa gia' morto in pochi giorni.
Cronologia interazioni Copilot (contenuto reale dei prompt)	non implementato	Permesso <code>AiEnterpriseInteraction.Read.All</code> (solo Application) esisterebbe, ma espone il CONTENUTO REALE di cosa gli utenti hanno scritto a Copilot - dato di livello compliance/eDiscovery, non semplice telemetria di utilizzo. Non aggiunto senza una richiesta esplicita e consapevole dell'utente.

24.2 Get-M365OpsCopilotUsageReport / Get-M365OpsCopilotUsageSummary

`Get-M365OpsCopilotUsageReport -Period D30` (report per-utente: ultima attivita' complessiva e per singola app - Teams/Word/Excel/PowerPoint/Outlook/OneNote/Loop/Copilot Chat) e `Get-M365OpsCopilotUsageSummary -Period D30` (numero aggregato di utenti abilitati/ attivi per app) - gli stessi identici dati mostrati in "Report > Utilizzo > Microsoft 365 Copilot" nell'admin center. `-Period` accetta `D7 / D30 / D90 / D180 / ALL`.

Endpoint Graph v1.0 (`/copilot/reports/getMicrosoft365CopilotUsageUserDetail` e `getMicrosoft365CopilotUserCountSummary`) - a differenza della maggior parte delle altre API di questo progetto, la risposta v1.0 e' SEMPRE un file CSV (comportamento documentato su Microsoft Learn, non un bug), mai JSON: entrambe le cmdlet lo convertono con `ConvertFrom-Csv` e rinominano le colonne in PascalCase per coerenza con lo stile del resto del modulo.

Permesso mancante su vnsys-test, verificato dal vivo il 22/08/2026: `Reports.Read.All` (Application) non e' ancora concesso - Graph risponde 403 "S2SUnauthorized: Invalid permission." Va aggiunto in Entra ID > App Registration > API permissions > Add a permission > Microsoft Graph > Application permissions > `Reports.Read.All` > Grant admin consent (sezione 4.2) - lo stesso permesso serve gia' per altri report generali di Microsoft 365, non e' specifico di Copilot. Il parsing CSV di queste due cmdlet non e' quindi ancora stato verificato dal vivo con dati reali - solo il percorso di errore (403) e' stato confermato.

24.2.1 Raggiungibili dall'IA in chat (27/08/2026)

Le due cmdlet erano finora richiamabili solo da console PowerShell diretta - nessun tool AI, nessuna voce nel catalogo comandi locale, nessun pulsante GUI le esponeva, nonostante fossero gia' implementate dal 22/08/2026. Aggiunti due tool AI in `Invoke-M365OpsAgentTools.ps1`: `get_copilot_usage_summary` e `get_copilot_usage_report` (entrambi SOLA LETTURA, parametro opzionale `period` con default D30), cosi' una domanda in linguaggio naturale ("quanti utenti usano Copilot in Teams", "dammi il dettaglio per utente di chi ha usato Copilot negli ultimi 90 giorni") viene ora gestita senza dover passare da uno script personalizzato o dalla console. Non aggiunta una voce nel catalogo comandi locale (`CommandCatalog.ps1`): quel meccanismo di trigger deterministico per le voci gemelle (es. `MfaStatus/ListDevices`) e' il risultato di svariati giri di bug-hunt mirati su `DeferWords`/frasi di continuazione - duplicarlo da zero per una nuova voce senza lo stesso livello di test avrebbe rischiato di reintrodurre proprio le classi di bug gia' risolte altrove (richieste composte, verbi di scrittura ambigui, invio email in coda alla domanda), a fronte di un beneficio marginale (il tool AI copre gia' lo stesso identico caso d'uso). **Verificato dal vivo** su vnsys-test (App-only): entrambe le domande in chat hanno attivato il tool corretto (incluso il riconoscimento implicito del periodo, "ultimi 90 giorni" tradotto correttamente in `period=D90`) ed hanno propagato all'utente il vero 403 `Reports.Read.All` mancante, senza inventare un "nessun dato disponibile" - esattamente il comportamento richiesto nella descrizione del tool.

24.3 Knowledge Base su Copilot

Per le aree NON implementate (24.1), invece di costruire integrazioni fragili o premature, l'AI puo' comunque rispondere a domande concettuali su Microsoft 365 Copilot (cos'e' il Copilot Control System, perche' le impostazioni Policy sono solo Delegate, cosa mostra l'admin center) attingendo alla Knowledge Base globale (sezione 17.13) - stesso meccanismo gia' usato per la guida stessa, riusabile per qualunque documentazione aggiuntiva caricata con `Add-M365OpsKnowledgeDocument -TenantName '_global'`.

Appendice A — Struttura dei file

```

M365Ops\
├─ M365Ops.psd1 / M365Ops.psm1    manifest e loader del modulo (export dinamico, sezione 17)
├─ Public\
│   ├── Get-M365OpsUserMfaStatus.ps1 / Reset-M365OpsUserMfa.ps1    stato/reset MFA (sezione 21)
│   ├── Export-M365OpsDataReport.ps1                                report AI-orchestrati con grafici (sezione 22)
│   ├── Connect-M365OpsIntune.ps1                                  connessione Intune, entrambe le modalità (sezione 10.9)
│   ├── Invoke-M365OpsActionRecovery.ps1                          retry AI-assistito su scritture fallite (sezione 13.7)
│   └─ Get-M365OpsChatHistory.ps1 / Add-M365OpsChatHistoryTurn.ps1 / Clear-M365OpsChatHistory.ps1    memoria conversazione (sezione 23)
├─ Private\
│   ├── funzioni interne (token, chiamate Graph/MCP, device code flow)
│   ├── Test-M365OpsIntuneAuthValid.ps1                          verifica dal vivo del token Intune (sezione 10.9)
│   ├── New-M365OpsSvgChart.ps1                                   grafici SVG per i report (sezione 22)
│   └─ ConvertTo-M365OpsHashtable.ps1                             normalizza i parametri AI (PSCustomObject → Hashtable) prima di ogni scrittura
├─ Scripts\Custom\
│   └─ script personalizzati (sezione 17) - README.md e _TEMPLATE.ps1
├─ Config\
│   ├── tenants.json                                             profili tenant (mai segreti in chiaro)
│   ├── M365Ops-Exchange.cer                                     chiave pubblica del certificato Exchange
│   ├── ai-provider.json                                         provider AI selezionato in GUI, persistito (sezione 8)
│   ├── last-active-tenant.txt                                   ultimo tenant attivo, letto da Launch-M365Ops.ps1 (sezione 12.1)
│   └─ ChatHistory-<tenant>.json                                  storico conversazione per tenant, password redatte (sezione 23)
├─ Gui\
│   ├── Server.ps1                                              server HTTP + logica chat/conferme
│   ├── index.html                                              interfaccia web di chat
│   └─ CommandCatalog.ps1                                       comandi deterministici a costo zero
├─ Reports\
│   └─ output export CSV/XLSX/PDF (anche i grafici SVG generati, sezione 22)
├─ Uploads\
│   └─ file caricati da GUI (installer, icone, CSV migrazione)
├─ Logs\
│   └─ storico azioni, un file al giorno (sezione 19)
├─ Tools\
│   └─ IntuneWinAppUtil.exe, New-M365OpsIcon.ps1 / New-M365OpsShortcut.ps1 (sezione 3.1)
├─ Assets\
│   └─ M365Ops.ico                                              icona dell'app, usata dal collegamento sul Desktop (sezione 3.1)
├─ docs\
│   ├── Guida-Configurazione.html                               sorgente di questa guida
│   └─ Guida-Configurazione.pdf                                 versione stampabile
├─ Launch-M365Ops.ps1 / M365Ops.bat                             avvio con un click (12.1) o da terminale (12.2), resume-session, auto-install prerequisiti
├─ Show-M365OpsPrereqInstaller.ps1                             GUI di installazione prerequisiti, richiamata da M365Ops.bat solo se manca qualcosa (sezione 3.1)
├─ Bootstrap-WinGet.ps1                                         installa winget stesso se assente, richiamato da Show-M365OpsPrereqInstaller.ps1 (sezione 3.1)
├─ Kill-And-Restart-M365Ops.ps1 / .bat                          termina il server (anche se bloccato) e lo riavvia - ultima risorsa se il riavvio normale non risponde (sezione 12.3)
├─ Start-M365OpsWebApp.ps1 / Start-M365Ops.ps1                 superati da Launch-M365Ops.ps1 (mancano rilevamento porta e resume-session) - lasciati solo per compatibilità con script
└─ README.md                                                    guida rapida sviluppatore

```

Appendice B — Tutto avviene dalla GUI, non serve PowerShell

Per scelta esplicita, l'uso quotidiano di M365Ops non richiede mai di scrivere un comando PowerShell: setup del tenant, motore AI, email, connettori MCP, report, reset MFA, tracking messaggi, migrazioni, sicurezza email, SharePoint/OneDrive, Teams — ogni operazione descritta in questa guida ha un campo, un pulsante o una richiesta in chat come suo equivalente diretto in GUI (vedi in particolare le sezioni 7, 9, 11.1, 13, 17). Le cmdlet PowerShell (`Get-Command -Module M365Ops` per l'elenco completo) restano lo strato che la GUI stessa chiama dietro le quinte, e sono comunque disponibili per chi vuole scriptare qualcosa di più avanzato da un terminale — ma non sono la via prevista per l'uso normale, e questa guida non le presenta più come tale.

Le uniche eccezioni reali, dove un passaggio manuale fuori dalla GUI resta inevitabile perché riguarda strumenti esterni (Azure, Exchange, il certificate store di Windows) e non l'app in sé, sono documentate nel punto in cui servono: creazione dell'App Registration e del certificato (sezioni 4-5), assegnazione del ruolo RBAC su Exchange Online (sezione 6), e trasferimento del certificato su un PC nuovo (sezione 18.3).

M365Ops — Guida alla Configurazione — documento vivo, aggiornato ad ogni modifica rilevante